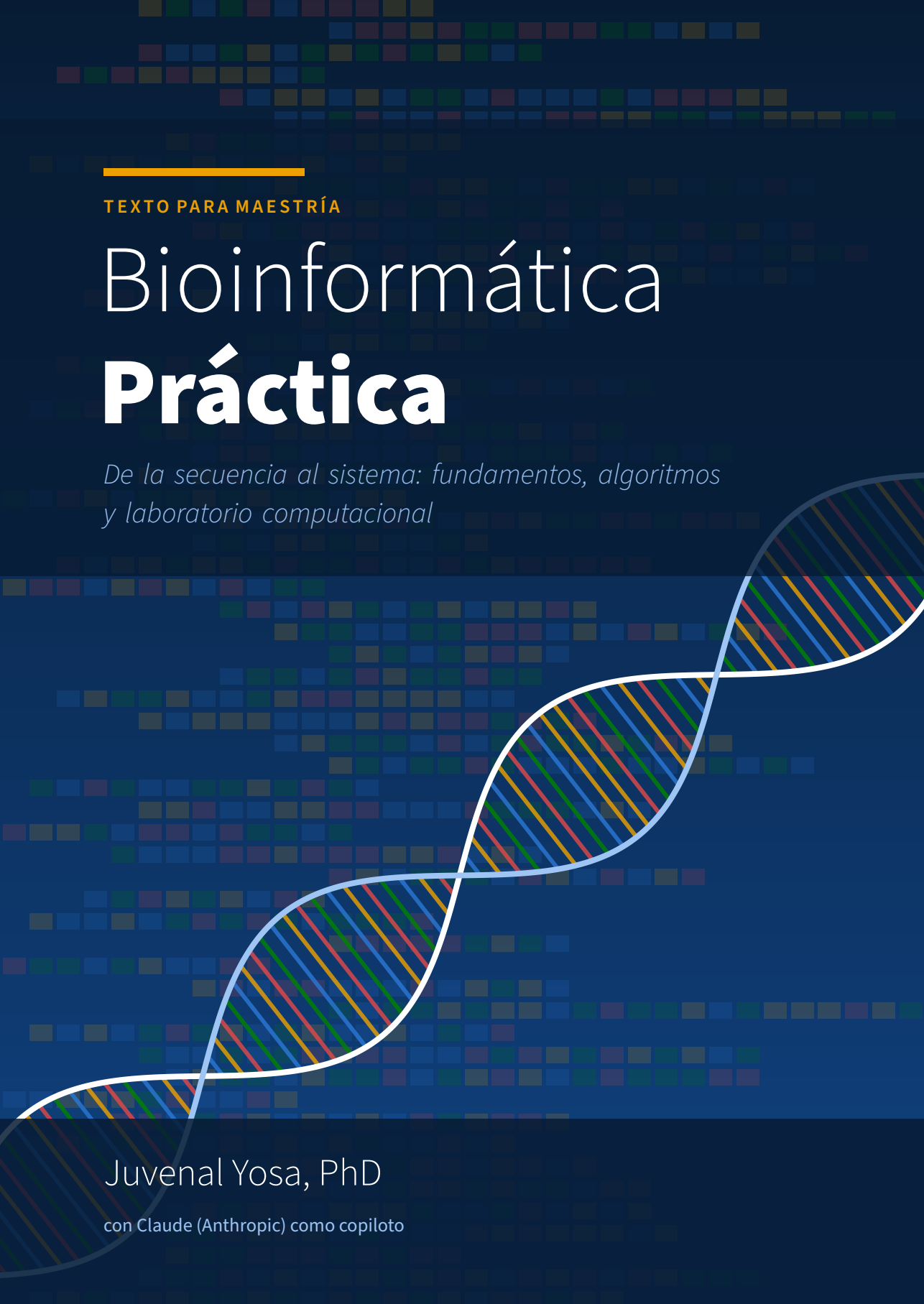




TEXTO PARA MAESTRÍA

Bioinformática **Práctica**

*De la secuencia al sistema: fundamentos, algoritmos
y laboratorio computacional*

Juvenal Yosa, PhD

con Claude (Anthropic) como copiloto

Bioinformática Práctica

De la secuencia al sistema: fundamentos, algoritmos y laboratorio computacional

Autor: Juvenal Yosa, PhD · juvenal.yosa@gmail.com

Copiloto: Claude (Anthropic), asistente de inteligencia artificial empleado en la redacción, la programación de figuras y la verificación bibliográfica, bajo la dirección y revisión del autor.

Laboratorios: cada capítulo tiene notebooks de Google Colab en github.com/juvenalyosa/bioinformatica-practica.

Primera edición: 2026.

Licencia: texto, figuras y código se distribuyen bajo la licencia MIT. Se permite usar, copiar, modificar y redistribuir este material, con o sin fines comerciales, siempre que se conserve el aviso de autoría y de licencia.

Referencias: todas las referencias bibliográficas fueron verificadas contra los registros de Crossref, DataCite u OpenLibrary (DOI o ISBN) y siguen las normas APA (7.^a edición).

Tipografía: compuesto con Lua $\text{\LaTeX}$ en Libertinus Serif, Source Sans Pro y Fira Mono. Figuras vectoriales en TikZ y PGFPlots, generadas a partir de cálculos reproducibles.



Prólogo

En 1977, secuenciar unos pocos miles de bases del bacteriófago ϕ X174 fue una hazaña que llevó años. Hoy, un solo instrumento produce en un día más secuencias de las que existían en todas las bases de datos del mundo a comienzos de este siglo. La biología se ha convertido, casi sin pedir permiso, en una ciencia de la información. Quien quiera hacer preguntas sobre genes, células, poblaciones o ecosistemas necesita hoy dos alfabetos: el de las moléculas y el de los algoritmos.

Este libro nació como acompañante de un curso de bioinformática práctica para estudiantes de maestría. Cada capítulo corresponde a un módulo del curso, y cada sección a una lección con un laboratorio ejecutable en Google Colab. Pero el libro está pensado para leerse por sí mismo: allí donde el notebook muestra *cómo* hacer algo, el texto se detiene a explicar *por qué* funciona, de dónde viene cada ecuación y qué supuestos la sostienen.

El recorrido tiene una lógica deliberada. Cada tema comienza con una situación cotidiana que ofrece un punto de apoyo intuitivo; sigue con las definiciones básicas y, paso a paso, sube hasta la teoría formal que se encuentra en la literatura especializada. Las ecuaciones no aparecen como adornos: cada una va acompañada de una tabla que explica todos sus símbolos, de un ejemplo numérico resuelto y de un fragmento de código que la pone a trabajar. Las figuras se generaron a partir de cálculos reales y reproducibles, y las referencias, que remiten siempre a las fuentes originales, fueron verificadas una a una.

El libro está organizado en cinco partes. La primera prepara el laboratorio digital y el vocabulario molecular. La segunda estudia la comparación de secuencias, desde el alineamiento hasta la filogenética. La tercera sigue a las lecturas de secuenciación desde el instrumento hasta las variantes. La cuarta aborda la genómica de poblaciones, la transcriptómica, la epigenómica y la metagenómica. La quinta sube de escala hacia la estructura de las proteínas, las redes, el aprendizaje automático y los flujos de trabajo reproducibles.

Nada de esto sustituye la práctica. La bioinformática se aprende como se aprende un instrumento: leyendo la partitura, sí, pero sobre todo tocando. Por eso le invito a leer con el notebook abierto, a cambiar los parámetros, a romper el código y a repararlo. Si al terminar un capítulo usted puede explicar a un colega por qué un algoritmo funciona, y no solo cómo se ejecuta, el libro habrá cumplido su propósito.

Juvenal Yosa

2026

Cómo leer este libro

Cada capítulo abre con una lista de objetivos y cierra con un resumen, una serie de ejercicios graduados y las referencias citadas. Entre medias, el texto se apoya en un pequeño repertorio de recuadros con funciones precisas. Conocerlos de antemano le ayudará a decidir qué leer con calma y qué puede saltarse en una primera lectura.

Recuadros de intuición

Abren cada tema con una situación cotidiana que sirve de punto de apoyo. No sustituyen la teoría: preparan el terreno para ella.

Definición. (Definiciones)

Enuncian con precisión los objetos matemáticos o biológicos con los que trabajaremos. Están numeradas por capítulo.

Teorema. (Resultados)

Recogen las ecuaciones y los resultados centrales de cada método, normalmente seguidos de una tabla de símbolos.

DONDE

x Cada ecuación importante va acompañada de una tabla como esta, que explica el significado de todos sus símbolos.

Ejemplo. (Ejemplos resueltos)

Aplican la teoría a números concretos, paso a paso, para que usted pueda reproducirlos con lápiz o con Python.

ejemplo.py

```
# Los fragmentos de código son breves, completos y ejecutables.
gc = sum(base in "GC" for base in "ATGCGC") / 6
```

Notas históricas

Cuentan quién, cuándo y por qué se desarrolló una idea. La ciencia es una actividad humana.

Advertencias

Señalan errores frecuentes, supuestos que se olvidan y trampas de interpretación.

Para profundizar

Material avanzado que puede omitirse en una primera lectura sin perder el hilo.

 **Colab** • enlace al laboratorio de cada lección

 repositorio del curso

Convenciones. Las explicaciones están en español; el código, los nombres de funciones y muchos términos técnicos se mantienen en inglés, como aparecen en la literatura y en la documentación de las herramientas; su primera aparición se marca en *cursiva*. Los decimales se escriben con coma. Las secuencias de ADN se colorean con el código de la tabla 0.1, el mismo que se usa en los notebooks del curso.

Tabla 0.1. Colores de nucleótidos usados en todo el libro.

A	adenina	C	citosa	G	guanina	T	timina	(U	uracilo)
---	---------	---	--------	---	---------	---	--------	----	----------

Ejercicios. Están marcados según su dificultad: ●●● básica, ●●● intermedia y ●●● avanzada. Los de nivel avanzado suelen requerir programación o una demostración.

Requisitos. Se asume una familiaridad básica con cálculo, probabilidad y programación en Python. El capítulo 0 repasa las herramientas computacionales y el capítulo 1, la biología molecular necesaria.

Índice general

Prólogo	iii
---------	-----

Cómo leer este libro	v
----------------------	---

I Fundamentos

0 Preparación del laboratorio digital	3
--	----------

0.1 Colab como laboratorio: Python, numpy, pandas y la visualización científica	4
0.2 Biopython y herramientas de línea de comandos	17
0.3 Reproducibilidad: semillas, versiones, hashes y Git	27
Ejercicios	39
Referencias del capítulo	40

1 Biología molecular para bioinformáticos	43
--	-----------

1.1 El dogma central como sistema de información	44
1.2 Genes, genomas y marcos abiertos de lectura	58
1.3 Composición de secuencias: contenido GC, GC skew y el origen de replicación	69
Ejercicios	79
Referencias del capítulo	80

2 Secuencias, formatos y bases de datos	83
--	-----------

2.1 El idioma de los archivos	84
2.2 Bases de datos biológicas desde Python	107
Ejercicios	119
Referencias del capítulo	120

II Comparar secuencias

3	Alineamiento de secuencias	125
3.1	Dot plots: mirar antes de calcular	126
3.2	Programación dinámica: Needleman-Wunsch y Smith-Waterman	129
3.3	Matrices de sustitución y penalización de gaps	135
3.4	BLAST y la estadística de Karlin-Altschul	138
	Ejercicios	143
	Referencias del capítulo	144
4	Alineamiento múltiple, motivos y perfiles	147
4.1	Alineamiento múltiple: el método progresivo	148
4.2	Motivos, matrices de peso posicional y logos	159
4.3	Modelos ocultos de Márkov y perfiles (HMMER)	168
	Ejercicios	183
	Referencias del capítulo	184
5	Filogenética y evolución molecular	187
5.1	Distancias evolutivas y modelos de sustitución	188
5.2	UPGMA y Neighbor-Joining	203
5.3	Máxima verosimilitud y bootstrap	211
	Ejercicios	224
	Referencias del capítulo	225
III	Del secuenciador al genoma	
6	Secuenciación de nueva generación (NGS)	229
6.1	Tecnologías de secuenciación	230
6.2	Calidad Phred, control de calidad y <i>trimming</i>	239
6.3	Cobertura y la teoría de Lander-Waterman	252
	Ejercicios	263
	Referencias del capítulo	265
7	Mapeo de lecturas	269
7.1	Transformada de Burrows-Wheeler y FM-index	270
7.2	BWA, minimap2 y samtools	284
7.3	Visualización de alineamientos y cobertura	297
	Ejercicios	305
	Referencias del capítulo	307

IV Genomas en acción

V Estructura, sistemas e inteligencia

PARTE

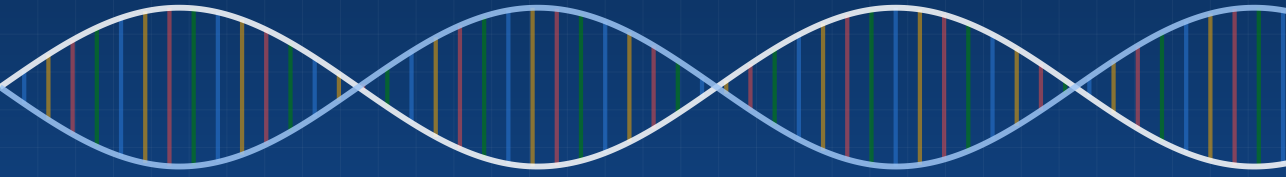


Fundamentos

El laboratorio, la molécula y el archivo

Antes de comparar genomas hay que saber qué es un genoma, cómo se guarda en un archivo y cómo se interroga con un computador. Esta parte prepara el laboratorio digital y el vocabulario molecular que usaremos durante todo el libro.

Preparación del laboratorio digital



Un artículo sobre ciencia computacional en una publicación científica no es la obra académica en sí misma; es solo la publicidad de esa obra. La verdadera obra es el entorno completo de desarrollo de software y el conjunto completo de instrucciones que generaron las figuras.

— Buckheit y Donoho (1995), resumiendo una idea de Jon Claerbout

● Al terminar este capítulo usted podrá

- Representar secuencias biológicas como arreglos en memoria, estimar el costo de un cálculo con notación asintótica y explicar por qué la vectorización con numpy es órdenes de magnitud más rápida que un bucle.
- Reconocer los límites de la aritmética de punto flotante y usar el espacio logarítmico para trabajar con probabilidades diminutas.
- Organizar datos en tablas ordenadas (*tidy*) con pandas y construir figuras que respeten los principios de la percepción visual.
- Leer, traducir y verificar un genoma real con Biopython y combinar Python con herramientas de línea de comandos como BLAST+.

- Explicar cómo funcionan un generador de números pseudoaleatorios, una función *hash* y un repositorio de Git, y usar los tres para que un análisis sea reproducible.

Un biólogo molecular no empieza su primer experimento sin saber calibrar una micropipeta, preparar un tampón o anotar un protocolo en su cuaderno. El bioinformático tampoco. Su mesa de trabajo es una computadora; sus reactivos, archivos de secuencias; sus instrumentos, bibliotecas de programación y programas de línea de comandos.

Este capítulo prepara ese laboratorio. No es un tutorial de programación: queremos entender *qué ocurre dentro de la máquina* cuando analizamos datos biológicos, porque de ello dependen la velocidad, la exactitud y la credibilidad de todo lo que haremos en el resto del libro. Avanzaremos en tres pasos. Primero convertiremos el cuaderno de Colab en un instrumento de medida: veremos cómo se representan las secuencias en memoria, cuánto cuesta un cálculo, qué precisión tienen los números de la máquina y cómo se organiza y se muestra un conjunto de datos (sección 0.1). Después incorporaremos las herramientas de la disciplina: Biopython, las bases de datos del NCBI y la línea de comandos, con un genoma real, el de SARS-CoV-2, como material de trabajo (sección 0.2). Por último, estudiaremos la maquinaria de la reproducibilidad: los números aleatorios que no lo son, las huellas digitales de los archivos, el control de versiones y los principios que la comunidad ha acordado para que un análisis pueda verificarse (sección 0.3).

0.1 Colab como laboratorio: Python, numpy, pandas y la visualización científica

 Colab · 0.1 · Colab como laboratorio

 [abrir notebook](#)

Un cuaderno que se ejecuta solo

En el laboratorio húmedo, el cuaderno contiene tres cosas mezcladas: el protocolo (lo que pensábamos hacer), las notas (lo que hicimos de verdad) y los resultados pegados con cinta adhesiva (la foto del gel, la curva de crecimiento). Si seis meses después alguien pregunta cómo se obtuvo una banda, la respuesta está en el cuaderno, junto a la banda. Ahora imagine un cuaderno en el que el

protocolo es el experimento: cada vez que alguien lo lee de principio a fin, el experimento vuelve a ocurrir y la foto del gel se regenera ante sus ojos. Eso es un *notebook* computacional.

0.1.1 La computación como instrumento de laboratorio

Un *notebook* de Jupyter, y su variante alojada en la nube, Google Colab, es un documento formado por *celdas*. Las celdas de texto contienen la narración, con ecuaciones y enlaces; las celdas de código contienen instrucciones que se envían a un proceso independiente, el *núcleo* (*kernel*), que las ejecuta y devuelve el resultado para que se muestre debajo de la celda. Kluyver et al. (2016) describen este diseño como un formato de publicación para flujos de trabajo computacionales reproducibles: el mismo archivo documenta, ejecuta y muestra.

La separación entre documento y núcleo tiene una consecuencia que conviene comprender desde el principio. El núcleo guarda en memoria un *estado*: todas las variables, funciones y módulos que se han definido hasta el momento. Ese estado depende del *orden en que se ejecutaron* las celdas, no del orden en que aparecen en la página. Si ejecuta la celda 5, luego corrige la celda 2 y la vuelve a ejecutar, el núcleo contiene una mezcla que no corresponde a ninguna lectura lineal del documento (figura 0.1). Los números entre corchetes a la izquierda de cada celda, [1], [2], ..., registran precisamente ese orden de ejecución.

Colab añade una segunda capa: el núcleo corre en una máquina virtual prestada que se destruye tras un rato de inactividad, llevándose consigo todo lo no guardado y las bibliotecas instaladas. Esa fragilidad obliga a escribir cuadernos que reconstruyan su entorno desde cero, que es justo lo que exige la reproducibilidad (sección 0.3).

El estado oculto

Antes de compartir un cuaderno o de confiar en sus resultados, use *Entorno de ejecución* → *Reiniciar y ejecutar todo*. Si el cuaderno falla o produce otros números, el resultado anterior dependía de un estado que no está escrito en ninguna parte.

0.1.2 Una secuencia es un arreglo de símbolos

La información genética de una molécula de ADN depende solo del orden de sus bases. Para la computadora, por lo tanto, una secuencia es una cadena finita de símbolos.

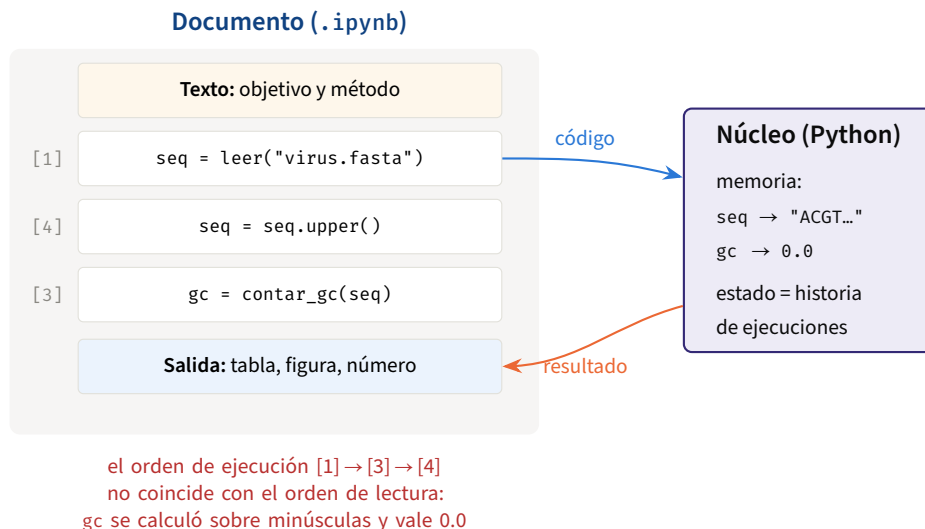


Figura 0.1. Anatomía de un *notebook*. El documento (izquierda) alterna celdas de texto, de código y de salida; el núcleo (derecha) ejecuta el código y conserva en memoria todas las variables. Los contadores [1], [3], [4] revelan un *estado oculto*: la celda que calcula gc se ejecutó antes que la que convierte la secuencia a mayúsculas, así que contó cero bases G o C en un archivo escrito en minúsculas. La memoria contiene ya la secuencia corregida, pero gc conserva el valor erróneo, distinto del que obtendría un lector que ejecutara el cuaderno de arriba abajo.

Definición 0.1. (Alfabeto, secuencia y codificación)

Un *alfabeto* Σ es un conjunto finito de símbolos; para el ADN, $\Sigma = \{A, C, G, T\}$ y para las proteínas, los 20 aminoácidos. Una *secuencia* de longitud n es una función $x : \{0, 1, \dots, n-1\} \rightarrow \Sigma$, que escribimos $x = x_0x_1 \dots x_{n-1}$. Una *codificación* es una función inyectiva $c : \Sigma \rightarrow \{0, 1\}^b$ que asigna a cada símbolo una palabra de b bits.

Hemos numerado las posiciones desde cero, como Python, y no desde uno, como los biólogos. Esta diferencia es fuente de incontables errores, y conviene fijar la regla de conversión. En Python, el corte $x[i:j]$ contiene las posiciones $i, i+1, \dots, j-1$: es un intervalo *semiabierto* $[i, j)$ cuya longitud es simplemente $j-i$. Un gen anotado en las posiciones biológicas $a..b$ (ambas incluidas, numeradas desde 1) se extrae así:

$$\text{posiciones biológicas } [a, b] \longleftrightarrow x[a-1:b], \quad \text{longitud} = b - a + 1. \quad (0.1)$$

DONDE

a, b	Primera y última posición del segmento en la numeración biológica (base 1, extremos incluidos).
$x[a-1:b]$	Corte de Python equivalente: empieza en el índice $a-1$ y se detiene <i>antes</i> del índice b .

¿Cuánta memoria ocupa una secuencia? Si cada símbolo se codifica con b bits, una secuencia de n símbolos ocupa

$$M(n) = \frac{nb}{8} \text{ bytes,} \quad b \geq \lceil \log_2 |\Sigma| \rceil. \quad (0.2)$$

DONDE

$M(n)$	Memoria necesaria, en bytes (1 byte = 8 bits).
b	Bits por símbolo de la codificación elegida.
$ \Sigma $	Tamaño del alfabeto; $\lceil \log_2 \Sigma \rceil$ es el mínimo número de bits que distingue todos sus símbolos.

Con cuatro bases bastan $\log_2 4 = 2$ bits, pero el texto ASCII usa 8. Para un genoma humano de $3,1 \times 10^9$ bases, la ecuación (0.2) da 775 MB empaquetado a 2 bits (formato `.2bit`), 3,1 GB como texto (`str` o `np.uint8`), 24,8 GB como lista de Python (un puntero de 8 bytes por base) y 99,2 GB como codificación *one-hot* en `float64`, habitual en aprendizaje automático: la misma información, con un factor de 128 de diferencia.

0.1.3 Cuánto cuesta un cálculo: complejidad asintótica

Antes de ejecutar un análisis sobre un genoma completo conviene saber si tardará un segundo o un año. La herramienta para responder sin medir es la notación asintótica, que describe cómo crece el costo de un algoritmo cuando crece el tamaño n de la entrada, sin preocuparse por las constantes que dependen de la máquina.

Definición 0.2. (Notación O grande)

Sean f y g funciones positivas de n . Decimos que $f(n) = O(g(n))$ si existen constantes $c > 0$ y n_0 tales que

$$f(n) \leq c g(n) \quad \text{para todo } n \geq n_0. \quad (0.3)$$

Si además $f(n) \geq c' g(n)$ para alguna $c' > 0$, escribimos $f(n) = \Theta(g(n))$: f y g crecen al mismo ritmo.

DONDE

$f(n)$	Costo real del algoritmo (operaciones o segundos) para una entrada de tamaño n .
$g(n)$	Función de referencia sencilla: n , $n \log n$, n^2 , $2^n \dots$
c, c', n_0	Constantes que no dependen de n ; absorben la velocidad de la máquina y los detalles de implementación.

La intuición es sencilla: $O(g)$ ignora las constantes y se queda con la *forma* del crecimiento. Contar el contenido GC recorre la secuencia una vez: es $\Theta(n)$. Comparar todas las posiciones de una secuencia con todas las de otra, como en el alineamiento del capítulo 3, es $\Theta(n^2)$. Ordenar n lecturas es $\Theta(n \log n)$. A 10^9 operaciones por segundo, un algoritmo $\Theta(n^2)$ tarda menos de un segundo sobre el genoma de SARS-CoV-2 (3×10^4 bases), unas siete horas sobre el de *E. coli* (5×10^6) y casi tres siglos sobre el humano (3×10^9), mientras que uno lineal tarda, respectivamente, microsegundos, milisegundos y segundos.

0.1.4 Vectorización: la misma operación sobre millones de datos

Python es un lenguaje *interpretado*: en un bucle `for`, el intérprete vuelve a averiguar en cada vuelta qué tipo de objeto tiene delante, dónde está su método de comparación y cómo sumar el resultado. Cada iteración cuesta decenas de nanosegundos, aunque la operación útil (comparar un byte) cueste menos de uno. Llenar una placa de 96 pozos con una micropipeta monocanal, pozo por pozo, se parece mucho a eso.

La biblioteca NumPy resuelve el problema con una estructura de datos, el arreglo `ndarray`, y un estilo de programación, la *vectorización* (Harris et al., 2020). Un arreglo es un bloque contiguo de memoria descrito por cuatro atributos: un puntero al primer byte, un tipo de dato (`dtype`) común a todos los elementos, una forma

(shape) y unos pasos (strides) que indican cuántos bytes hay que avanzar para moverse una posición en cada eje. La dirección del elemento (i, j) de una matriz es

$$\text{dir}(i, j) = \text{dir}_0 + i s_0 + j s_1. \quad (0.4)$$

DONDE

dir_0	Dirección de memoria del primer elemento.
s_0, s_1	Pasos (strides) en bytes: cuánto avanzar para cambiar de fila y de columna.
i, j	Índices de fila y columna (desde 0).

Como todos los elementos tienen el mismo tipo y están uno tras otro, una operación como `a == 71` (¿es esta base una G, cuyo código ASCII es 71?) puede ejecutarse en un bucle escrito en C, compilado, que el procesador además puede paralelizar con instrucciones vectoriales (SIMD). La ecuación (0.4) explica también un truco muy útil: transponer una matriz o tomar una de cada tres bases (`a[:,3]`, las primeras posiciones de los codones) no copia datos, solo cambia los pasos. Esas *vistas* son gratuitas.

```
gc_vectorizado.py

import numpy as np

def gc_loop(seq: str) -> float:
    n = 0
    for b in seq:
        if b == "G" or b == "C":
            n += 1
    return n / len(seq)

def gc_numpy(seq: str) -> float:
    a = np.frombuffer(seq.encode("ascii"), dtype=np.uint8)
    is_gc = (a == ord("G")) | (a == ord("C")) # máscara booleana
    return np.count_nonzero(is_gc) / a.size
```

La figura 0.2 compara ambas versiones. Extrapolada al genoma humano, la diferencia es de unos 160 segundos frente a menos de uno. Observe que ambas versiones siguen siendo $\Theta(n)$: la vectorización cambia la constante, no la complejidad.

La vectorización se completa con la *difusión* (broadcasting): cuando se combinan arreglos de formas distintas, NumPy repite virtualmente las dimensiones de tamaño 1 sin copiar datos. La expresión `x[:, None] == y[None, :]`, que compara cada base

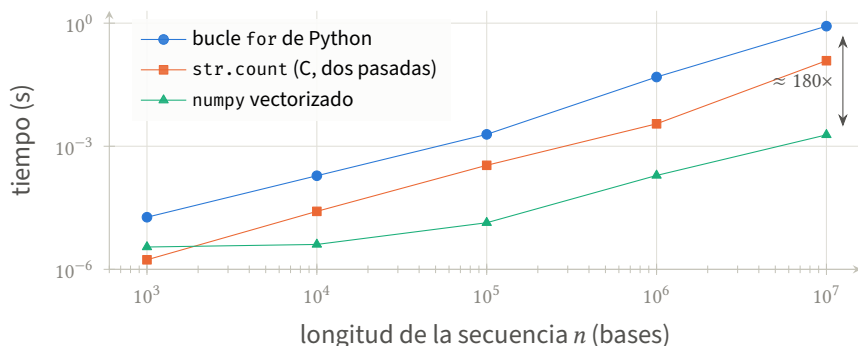


Figura 0.2. Tiempo para calcular el contenido GC de secuencias aleatorias de 10^3 a 10^7 bases (mejor de tres repeticiones, medido con `figuras/cap00/generar.py`). En escala doble logarítmica, un algoritmo $\Theta(n)$ es una recta de pendiente 1: los tres métodos son lineales, como predice el análisis asintótico. Lo que cambia es la constante c de la ecuación (0.3): el bucle interpretado gasta unos 50 ns por base y la versión vectorizada unos 0,3 ns.

de una secuencia con cada base de otra y construye la matriz de un *dot plot* en una línea, es el ejemplo que usaremos en el capítulo 3.

0.1.5 Números en la máquina: enteros y punto flotante

Los enteros de la máquina tienen un tamaño fijo: un `np.uint8` solo guarda valores de 0 a 255, y 255 + 1 da 0 sin aviso, como un cuentakilómetros que da la vuelta. Los `int` de Python crecen según haga falta, pero los arreglos de NumPy no.

Los números reales plantean un problema más sutil. Una computadora no puede almacenar π , ni siquiera 0,1, con infinitos decimales. El estándar IEEE 754, que siguen prácticamente todos los procesadores, representa un número de doble precisión (`float64`) con 64 bits: uno de signo, once de exponente y 52 de fracción (Goldberg, 1991):

$$x = (-1)^s \times \left(1 + \sum_{k=1}^{52} f_k 2^{-k} \right) \times 2^{e-1023}. \quad (0.5)$$

DONDE

s	Bit de signo (0 positivo, 1 negativo).
f_k	Bits de la fracción o mantisa; el 1 inicial está implícito (“bit escondido”).
e	Exponente almacenado, un entero de 11 bits; se le resta el sesgo 1023.

La consecuencia fundamental de la ecuación (0.5) es que los números representables no están distribuidos uniformemente: entre dos potencias de 2 consecutivas hay siempre el mismo número de ellos (2^{52}), así que su separación *absoluta* crece con la magnitud mientras su separación *relativa* se mantiene casi constante: entre 1 y 2 los números vecinos distan 2^{-52} , entre 2^{52} y 2^{53} distan exactamente 1, y $2^{53} + 1$ ya no es representable. El número que mide esa separación relativa es el *épsilon de la máquina*, $\varepsilon = 2^{-52} \approx 2,22 \times 10^{-16}$, y de él se deriva el modelo estándar del error de redondeo:

$$\text{fl}(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq u = \frac{\varepsilon}{2}. \quad (0.6)$$

DONDE

$\text{fl}(\cdot)$	Resultado que devuelve la máquina: el número representable más cercano al valor exacto.
$\circ$	Cualquiera de las operaciones $+$, $-$, $\times$, $\div$.
δ, u	Error relativo cometido y su cota, la unidad de redondeo ($\approx 1,1 \times 10^{-16}$).

La ecuación (0.6) parece inocente, porque un error de 10^{-16} rara vez importa. Hay, sin embargo, tres situaciones en las que muerde. La primera es la comparación exacta: `0.1 + 0.2 == 0.3` es `False`, porque ni 0,1 ni 0,2 tienen representación binaria finita y sus errores de redondeo no se cancelan; por eso se compara con una tolerancia (`np.isclose`). La segunda es la *cancelación catastrófica*: al restar dos números casi iguales, los dígitos correctos se anulan y queda el error, de modo que `1e16 + 1 - 1e16` devuelve 0. La tercera, la más importante en bioinformática, es el *subdesbordamiento* (*underflow*) al multiplicar muchas probabilidades.

Probabilidades diminutas y el espacio logarítmico

La probabilidad de observar una secuencia concreta de 1000 bases bajo un modelo uniforme es $0,25^{1000} \approx 10^{-602}$. El número positivo más pequeño que admite un `float64` es del orden de 5×10^{-324} , así que el producto se redondea a cero exacto,

y con él cualquier razón de verosimilitudes que dependa de él. Los modelos de Márkov, los HMM y la inferencia filogenética del resto del libro multiplican miles de probabilidades; sin un remedio, serían inutilizables.

El remedio es trabajar con logaritmos: $\log \prod_i p_i = \sum_i \log p_i$, y la suma de 1000 términos de $\ln 0,25$ es simplemente $-1386,29$, un número perfectamente ordinario. La dificultad aparece cuando hay que *sumar* probabilidades (por ejemplo, para marginalizar sobre estados ocultos) que solo conocemos por sus logaritmos. La solución es un pequeño teorema que se usa en todas partes.

Teorema 0.1. (Log-suma-exp)

Sean $a_1, \dots, a_k$ números reales (logaritmos de probabilidades) y $m = \max_i a_i$. Entonces

$$\log \sum_{i=1}^k e^{a_i} = m + \log \sum_{i=1}^k e^{a_i - m}, \quad (0.7)$$

y el lado derecho puede evaluarse sin desbordamiento ni subdesbordamiento, porque todos los exponentes son ≤ 0 y al menos uno vale exactamente 0.

DONDE

a_i	Logaritmo de la i -ésima probabilidad, $a_i = \log p_i$.
m	El mayor de los a_i ; se “factoriza” fuera de la suma.

La demostración es una línea: $\sum_i e^{a_i} = e^m \sum_i e^{a_i - m}$, y se toma el logaritmo. El truco está en que la suma de la derecha vale entre 1 y k , un número que la máquina maneja sin problemas.

Por ejemplo, con $\log p_1 = -1000$ y $\log p_2 = -1001$, la evaluación directa (`np.log` de la suma de `np.exp(-1000)` y `np.exp(-1001)`) devuelve $-\infty$, mientras que la ecuación (0.7) da $-1000 + \log(1 + e^{-1}) = -999,68674$; NumPy y SciPy la ofrecen como `np.logaddexp` y `scipy.special.logsumexp`.

0.1.6 Un primer modelo: el contenido GC y la distribución binomial

Con las herramientas anteriores podemos formular nuestra primera medida bioinformática. Los pares G–C se unen con tres puentes de hidrógeno y los pares A–T con dos, de modo que la proporción de G y C influye en la estabilidad térmica del ADN, en el diseño de cebadores y en la eficiencia de la secuenciación. Además,

varía mucho entre especies y dentro de un mismo genoma. El *contenido GC* de una secuencia x es

$$\text{GC}(x) = \frac{n_G + n_C}{n_A + n_C + n_G + n_T}. \quad (0.8)$$

DONDE

$n_A, \dots, n_T$ Número de apariciones de cada base en x ; las ambigüedades como N no cuentan en el denominador.

Al calcular el GC en ventanas deslizantes a lo largo de un genoma, los valores fluctúan. ¿Cuánto de esa fluctuación es biología y cuánto es puro azar de muestreo? Para responder necesitamos un *modelo nulo*. Supongamos que cada base es G o C con probabilidad p , independientemente de las demás. El número de bases GC en una ventana de W posiciones sigue entonces una distribución binomial, y la proporción observada tiene media y varianza

$$K \sim \text{Bin}(W, p), \quad \mathbb{E}\left[\frac{K}{W}\right] = p, \quad \text{Var}\left[\frac{K}{W}\right] = \frac{p(1-p)}{W}. \quad (0.9)$$

DONDE

K Número de bases G o C en la ventana (variable aleatoria).
 W Tamaño de la ventana, en bases.
 p Probabilidad de que una base sea G o C; se estima con el GC del genoma completo.

La desviación estándar $\sqrt{p(1-p)/W}$ decrece como $1/\sqrt{W}$: ventanas cuatro veces más grandes reducen el ruido a la mitad, a costa de perder resolución espacial. Por el teorema central del límite, alrededor del 95 % de las ventanas deberían caer dentro de $p \pm 2\sqrt{p(1-p)/W}$ si el modelo nulo fuera cierto.

Ejemplo 0.1. (El GC de SARS-CoV-2 frente a su modelo nulo)

El genoma de referencia de SARS-CoV-2 (NC_045512.2) tiene 29 903 bases, de las cuales 11 355 son G o C: $p = 0,3797$. Con ventanas de $W = 500$ bases,

$$\sigma = \sqrt{\frac{0,3797 \times 0,6203}{500}} = 0,0217,$$

de modo que el modelo binomial predice que casi todas las ventanas tendrán un GC entre 0,336 y 0,423. En el genoma real (figura 0.3), el GC por ventana

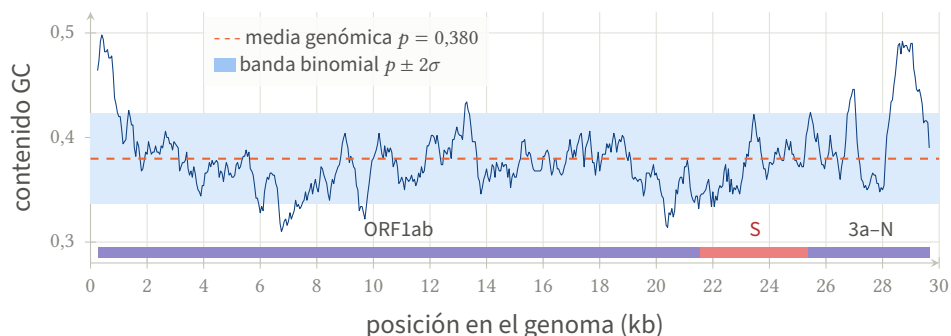


Figura 0.3. Contenido GC en ventanas de 500 bases (paso de 50) a lo largo del genoma de referencia de SARS-CoV-2. La banda azul es la región $p \pm 2\sigma$ que predice el modelo binomial (ecuación (0.9)); si el genoma fuera homogéneo, solo un 5 % de las ventanas escaparía de ella. Escapa casi el triple. Los extremos, sobre todo hacia el final del genoma (genes estructurales y regiones no traducidas), revelan heterogeneidad biológica real.

oscila entre 0,310 y 0,498, y el 14,4 % de las ventanas cae fuera de la banda de $\pm 2\sigma$, casi el triple de lo esperado. El exceso de variación (*sobredispersión*) nos dice que el genoma no es una sucesión de monedas idénticas: su composición cambia de una región a otra.

Calcular el GC de cada ventana por separado costaría $\Theta(nW)$. Un truco clásico lo reduce a $\Theta(n)$: se calcula una vez la suma acumulada $S_i = \sum_{k < i} g_k$ del indicador g_k (1 si la base k es G o C) y el GC de la ventana $[i, i + W)$ es simplemente $(S_{i+W} - S_i) / W$.

gc_ventanas.py

```
import numpy as np
from Bio import SeqIO

rec = SeqIO.read("NC_045512.2.gb", "genbank")
a = np.frombuffer(bytes(rec.seq), dtype=np.uint8)
g = ((a == ord("G")) | (a == ord("C"))).astype(float)
S = np.concatenate([[0.0], np.cumsum(g)]) # sumas acumuladas
W, step = 500, 50
starts = np.arange(0, a.size - W + 1, step)
gc = (S[starts + W] - S[starts]) / W # Theta(n), no Theta(nW)
p = g.mean(); sd = np.sqrt(p * (1 - p) / W)
print(f"p={p:.4f} fuera de 2 sd: {np.mean(abs(gc - p) > 2*sd):.1%}")
```

0.1.7 Datos tabulares: pandas y el formato ordenado

Casi todo resultado bioinformático termina en una tabla: genes con sus coordenadas, muestras con sus metadatos, resultados de BLAST con sus puntuaciones. La biblioteca pandas introdujo en Python el `DataFrame`, una tabla bidimensional cuyas columnas pueden tener tipos distintos y cuyas filas y columnas llevan etiquetas, con operaciones de agrupación, unión y remodelado inspiradas en las bases de datos relacionales y en R (McKinney, 2010).

Tener la herramienta no basta; hay que decidir la *forma* de la tabla. Una misma información puede escribirse de muchas maneras, y algunas facilitan el análisis mientras que otras lo entorpecen. Wickham (2014) formalizó la forma más conveniente con el nombre de datos *ordenados* (*tidy data*).

Definición 0.3. (Datos ordenados)

Un conjunto de datos está ordenado si (1) cada variable forma una columna, (2) cada observación forma una fila y (3) cada tipo de unidad observacional forma una tabla.

Las hojas de cálculo del laboratorio casi nunca cumplen la definición: suelen usar la forma “ancha” de la figura 0.4, con la variable *muestra* escondida en los encabezados. Para agrupar, filtrar o graficar conviene la forma “larga”, una medición por fila; en pandas se pasa de una a otra con `melt` y `pivot`.

Una advertencia: al abrir una lista de genes, Excel convierte por defecto símbolos como *SEPT2* o *MARCH1* en fechas, y Ziemann et al. (2016) hallaron que alrededor de una quinta parte de los artículos de genómica con listas de genes en Excel contenían estas conversiones erróneas. Lea y escriba las tablas con código y trate los archivos originales como de solo lectura.

0.1.8 Principios de visualización científica

Una figura es un instrumento de medida para el ojo. Como cualquier instrumento, tiene una resolución, y esa resolución no es la misma para todas las magnitudes visuales. Cleveland y McGill (1984) lo midieron experimentalmente: pidieron a observadores que estimaran proporciones codificadas de distintas maneras y ordenaron las codificaciones según la exactitud de los juicios. De más a menos precisas: posición sobre una escala común; posición en escalas no alineadas; longitud, dirección y ángulo; área; volumen y curvatura; y, por último, sombreado y saturación del color.

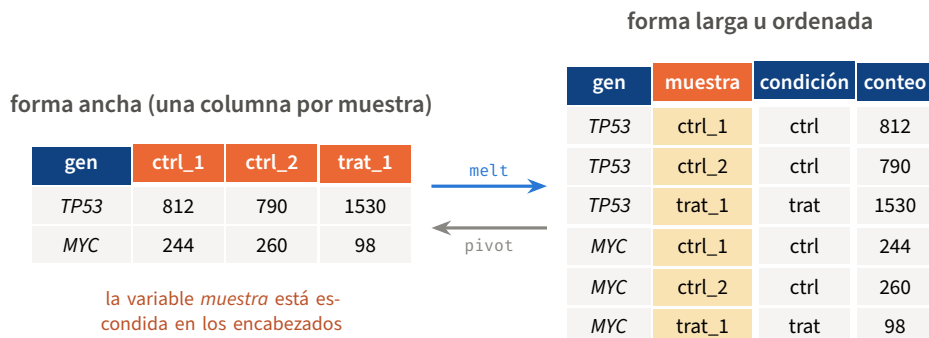


Figura 0.4. La misma tabla de conteos de expresión en dos formas. En la forma ancha (izquierda) los nombres de las muestras son encabezados; en la forma ordenada (derecha) son valores de una columna, cada fila es una observación y aparece una variable nueva, la condición, derivada de la muestra. Todas las operaciones de agrupación y de gráficas de pandas, seaborn o plotly esperan la forma ordenada.

La jerarquía justifica reglas prácticas que usaremos en todo el libro: preferir barras o puntos a gráficas de pastel (que codifican con ángulos y áreas), reservar el color para distinguir categorías o para magnitudes secundarias, y comparar lo que importa comparar sobre un eje común. Tufte (2001) añadió un principio de economía: la *razón datos-tinta*,

$$R = \frac{\text{tinta dedicada a los datos}}{\text{tinta total de la figura}}, \quad (0.10)$$

debe ser tan alta como sea razonable. Rejillas gruesas, fondos sombreados, efectos tridimensionales y leyendas redundantes consumen tinta sin aportar información; Tufte los llamó *chartjunk*. En la práctica, esto se traduce en rotular directamente las series en lugar de usar leyendas, en rejillas tenues y en títulos que enuncian la conclusión.

El color como dato

Cuando el color codifica una magnitud continua (una matriz de expresión, un mapa de contactos), el mapa de color debe ser *perceptualmente uniforme*: pasos iguales en el dato deben producir pasos iguales en la sensación visual. La luminosidad L^* del espacio CIELAB es una buena medida de esa sensación. El mapa arcoíris *jet*, durante años el predeterminado de muchos programas, viola esta condición: su luminosidad sube y baja de forma irregular, de modo que crea bordes donde el dato

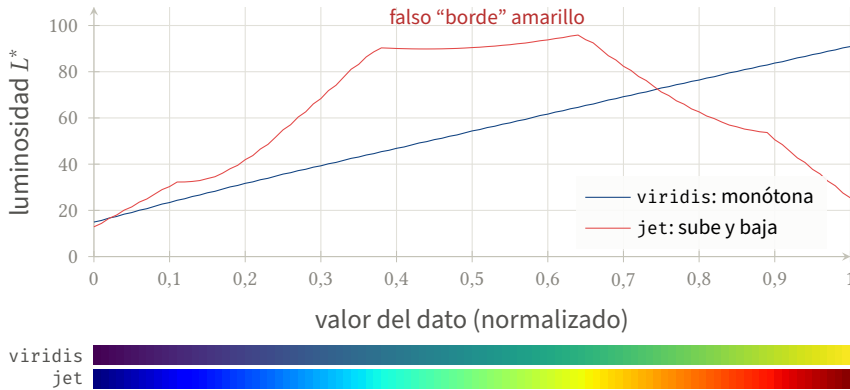


Figura 0.5. Luminosidad perceptual L^* (CIELAB) a lo largo de dos mapas de color, calculada a partir de sus valores RGB. En viridis (azul) crece de forma monótona y casi lineal: el ojo lee un orden correcto incluso en escala de grises. En jet (rojo) sube hasta el cian y el amarillo y vuelve a caer: dos datos distintos pueden verse igual de claros, y la franja amarilla central aparece como un borde que no existe en los datos.

es suave y oculta cambios donde el dato varía (figura 0.5). Además, es ilegible para las personas con deficiencias en la visión del color, que confunden sus rojos y verdes. Crameri et al. (2020) documentaron lo extendido de este mal uso y recomiendan mapas derivados científicamente, como viridis o cividis.

Todas las figuras de este libro, y las del curso en Colab, se construyen con estos criterios. En Python la herramienta básica es matplotlib (Hunter, 2007), sobre la que se apoyan bibliotecas de más alto nivel; para figuras interactivas usaremos plotly en los cuadernos.

0.2 Biopython y herramientas de línea de comandos

 Colab · 0.2 · Biopython y herramientas

 abrir notebook

Instrumentos calibrados por otros

Nadie fabrica su propia micropipeta antes de cada experimento. Se compra un instrumento calibrado, probado por miles de usuarios, y se dedica el esfuerzo a la pregunta biológica. En la sección anterior escribimos a mano

funciones para contar bases; fue útil para entender qué ocurre por dentro. A partir de aquí usaremos dos tipos de instrumentos ya calibrados: las bibliotecas de Python, que son como la gaveta de pipetas, puntas y gradillas de la mesa de trabajo, y los programas de línea de comandos, que son los equipos grandes del laboratorio (el termociclador, el secuenciador), especializados, muy rápidos y controlados mediante instrucciones escritas.

Biopython es la biblioteca de referencia para biología molecular en Python: un conjunto de herramientas libres, desarrollado por una comunidad internacional, que va de la manipulación de secuencias y la lectura de decenas de formatos al acceso programático a bases de datos (Cock et al., 2009). Aquí la usaremos para ir del ADN a la proteína, descargar y verificar un genoma real y conectar Python con programas externos.

0.2.1 El objeto Seq: complemento y complemento reverso

Un objeto Seq se comporta como una cadena de texto, pero “conoce” la biología: sabe complementar, transcribir y traducir. La operación más frecuente, y la que más errores provoca, es el complemento reverso. Las dos hebras de la doble hélice son *antiparalelas*: una corre en sentido $5' \rightarrow 3'$ y la otra en sentido $3' \rightarrow 5'$. Como por convención toda secuencia se escribe de $5'$ a $3'$, leer la hebra opuesta exige complementar cada base e invertir el orden.

Definición 0.4. (Complemento reverso)

Sea $\kappa : \Sigma \rightarrow \Sigma$ el complemento de Watson y Crick, $\kappa(A) = T$, $\kappa(T) = A$, $\kappa(C) = G$, $\kappa(G) = C$. El complemento reverso de $x = x_0 \cdots x_{n-1}$ es la secuencia $\bar{x}$ de la misma longitud con

$$\bar{x}_i = \kappa(x_{n-1-i}), \quad i = 0, \dots, n-1. \quad (0.11)$$

DONDE

κ	Función de complemento base a base.
$\bar{x}_i$	Base en la posición i del complemento reverso; se toma de la posición simétrica $n-1-i$ de x .

De la definición se sigue que el complemento reverso es una *involución*: aplicado dos veces devuelve la secuencia original, $\bar{\bar{x}} = x$, porque tanto κ como la inversión del orden lo son. Una secuencia que coincide con su complemento reverso, $\bar{x} = x$, se llama *palindrómica*; muchos sitios de restricción lo son, como GAATTC (EcoRI), porque las enzimas que los cortan actúan como dímeros simétricos.

0.2.2 El código genético como función

La traducción es una función que lee la secuencia de tres en tres. ¿Por qué tres? Porque con palabras de k letras sobre un alfabeto de cuatro se pueden escribir 4^k palabras distintas, y hacen falta al menos 21 (veinte aminoácidos y una señal de parada): $4^1 = 4$ y $4^2 = 16$ no alcanzan, mientras que $4^3 = 64$ sobran. El código genético estándar es, por lo tanto, una función sobreyectiva

$$\tau : \{A, C, G, U\}^3 \longrightarrow \{20 \text{ aminoácidos}\} \cup \{*\}, \quad |\text{dominio}| = 64, \quad |\text{imagen}| = 21. \quad (0.12)$$

DONDE

τ	Tabla de traducción; Biopython la guarda en <code>Bio.Data.CodonTable</code> (la estándar es la tabla 1 del NCBI).
$\{\dots\}^3$	Conjunto de todos los codones, palabras de tres nucleótidos.
$*$	Señal de parada (UAA, UAG, UGA).

Como hay más codones que significados, el código es *degenerado*: varios codones sinónimos codifican el mismo aminoácido. La figura 0.6 muestra que la degeneración no es aleatoria. Los sinónimos suelen diferir solo en la tercera base, de modo que muchas mutaciones en esa posición son silenciosas, y los aminoácidos químicamente parecidos ocupan casillas vecinas, lo que amortigua el efecto de los errores de lectura. Leucina, serina y arginina tienen seis codones cada una; metionina y triptófano, uno solo. AUG, el codón de la metionina, es además la señal de inicio más común.

0.2.3 Seis marcos de lectura y la estadística de los ORF

El ADN no tiene espacios entre codones. Una secuencia puede leerse empezando en la primera, la segunda o la tercera base, y lo mismo en su complemento reverso: hay seis *marcos de lectura*. En general solo uno de ellos, en cada región, corresponde a la proteína real. Un *marco abierto de lectura* (ORF, *open reading frame*) es un tramo de codones sin paradas; los genes que codifican proteínas son ORF largos, y buscar ORF largos fue durante décadas la primera estrategia de anotación de genomas.

		2.ª base					
		U	C	A	G		
1.ª base	U	UUU PheF	UCU SerS	UAU TyrY	UGU CysC	U	3.ª base
		UUC PheF	UCC SerS	UAC TyrY	UGC CysC	C	
		UUA LeuL	UCA SerS	UAA Stop*	UGA Stop*	A	
		UUG LeuL	UCG SerS	UAG Stop*	UGG TrpW	G	
	C	CUU LeuL	CCU ProP	CAU HisH	CGU ArgR	U	
		CUC LeuL	CCC ProP	CAC HisH	CGC ArgR	C	
		CUA LeuL	CCA ProP	CAA GlnQ	CGA ArgR	A	
		CUG LeuL	CCG ProP	CAG GlnQ	CGG ArgR	G	
	A	AUU IleI	ACU ThrT	AAU AsnN	AGU SerS	U	
		AUC IleI	ACC ThrT	AAC AsnN	AGC SerS	C	
		AUA IleI	ACA ThrT	AAA LysK	AGA ArgR	A	
		AUG MetM	ACG ThrT	AAG LysK	AGG ArgR	G	
	G	GUU ValV	GCU AlaA	GAU AspD	GGU GlyG	U	
		GUC ValV	GCC AlaA	GAC AspD	GGC GlyG	C	
		GUA ValV	GCA AlaA	GAA GluE	GGA GlyG	A	
		GUG ValV	GCG AlaA	GAG GluE	GGG GlyG	G	

Figura 0.6. El código genético estándar (tabla 1 del NCBI), generado a partir de Bio.Data.CodonTable. Cada bloque corresponde a una primera base (filas) y una segunda base (columnas); dentro del bloque, la tercera base recorre U, C, A, G. Colores: hidrofóbicos (amarillo), polares sin carga (verde azulado), con carga positiva (azul), con carga negativa (rojo), especiales G, P y C (violeta) y paradas (gris). Los bloques casi monocromáticos de la columna U (hidrofóbicos) y la vecindad de D y E muestran que el código agrupa los aminoácidos por química.

¿Qué significa “largo”? Una vez más, necesitamos un modelo nulo. Si las bases fueran independientes con frecuencias p_A, p_C, p_G, p_T , la probabilidad de que un codón cualquiera sea de parada sería

$$\pi = p_T p_A p_A + p_T p_A p_G + p_T p_G p_A = p_T p_A (p_A + 2p_G), \tag{0.13}$$

sumando las probabilidades de TAA, TAG y TGA. El número L de codones consecutivos sin parada sigue entonces una distribución geométrica: cada codón es un “intento” que termina el ORF con probabilidad π . Así,

$$\mathbb{P}(L \geq k) = (1 - \pi)^k, \quad \mathbb{E}[L] = \frac{1 - \pi}{\pi}. \tag{0.14}$$

DONDE

$p_A, \dots, p_T$	Frecuencias de las bases en el genoma (modelo de bases independientes).
π	Probabilidad de que un codón al azar sea de parada.
L	Número de codones de un tramo abierto antes de encontrar una parada.
k	Longitud umbral, en codones.

Ejemplo 0.2. (¿Cuántos ORF largos produce el azar?)

Con bases equiprobables, $\pi = 3/64$ y un ORF aleatorio tiene en promedio $E[L] = 61/3 \approx 20$ codones; la probabilidad de superar 100 codones es $(61/64)^{100} = 0,0082$. El genoma de SARS-CoV-2, sin embargo, es rico en A y T ($p_A = 0,299$, $p_T = 0,321$, $p_G = 0,196$), lo que hace las paradas más frecuentes: la ecuación (0.13) da $\pi = 0,0664$, es decir, unas $9968 \times 0,0664 \approx 662$ paradas esperadas por marco, y un ORF aleatorio medio de solo 14 codones. La probabilidad de que un tramo aleatorio supere 100 codones cae a $(1 - 0,0664)^{100} = 0,0010$, así que en todo un marco esperaríamos $662 \times 0,0010 \approx 0,7$ ORF así de largos por puro azar. Los más de 7000 codones de ORF1ab son inexplicables por azar.

La figura 0.7 confirma la predicción: los genes destacan como tramos largos sin paradas. Pero la hebra complementaria contiene algunos ORF de más de 100 codones que no codifican nada, porque el genoma real no está hecho de bases independientes.

0.2.4 Archivos y registros: SeqRecord y SeqIO

Una secuencia aislada rara vez basta; necesitamos su identificador, su descripción y sus anotaciones. Biopython agrupa todo ello en un objeto SeqRecord, y el módulo SeqIO traduce entre registros y archivos en decenas de formatos. Los dos formatos que usaremos desde el primer día son FASTA, mínimo (una línea de encabezado que empieza con > seguida de la secuencia), y GenBank, rico en metadatos. Estudiaremos los formatos con detalle en el capítulo 2; aquí basta con saber que el encabezado de un GenBank se convierte en los atributos .id, .description y .annotations; su tabla de características, en una lista de objetos SeqFeature (con tipo, localización y calificadores), y su sección ORIGIN, en el atributo .seq.

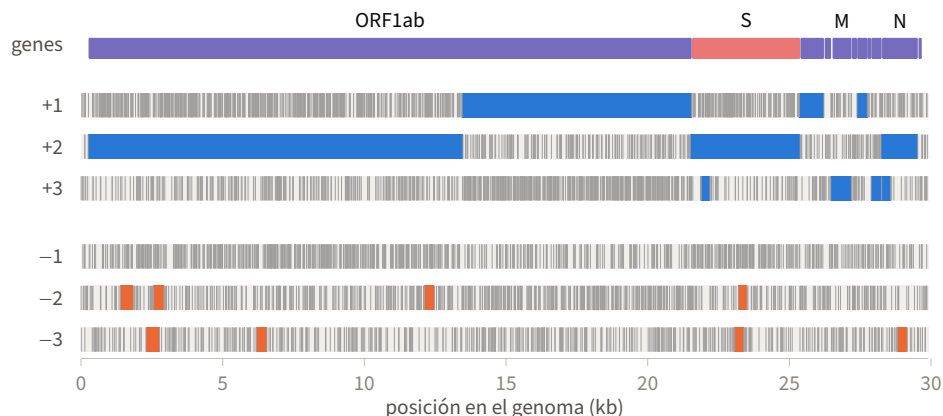


Figura 0.7. Los seis marcos de lectura del genoma de SARS-CoV-2. Cada raya gris es un codón de parada; los bloques de color son ORF de al menos 100 codones (azul, hebra directa; naranja, hebra complementaria). Arriba, los genes anotados en el GenBank. El marco +2 tiene solo 255 paradas, frente a las ~660 que predice la ecuación (0.13), porque contiene los más de 13 000 nucleótidos abiertos de ORF1a. En la hebra complementaria aparecen algunos ORF largos que no codifican nada, más de los que predice el modelo independiente: la longitud es una pista, no una prueba.

SeqIO ofrece dos funciones de lectura con una diferencia importante. `SeqIO.read` espera un archivo con *exactamente* un registro y falla si hay más, lo que protege contra errores silenciosos. `SeqIO.parse` devuelve un *iterador*: un objeto que produce los registros uno por uno, a medida que se le piden, sin cargar el archivo completo en memoria. Si un archivo contiene N registros de tamaño medio ℓ , leerlo todo en una lista necesita memoria $\Theta(N\ell)$, mientras que recorrerlo con el iterador necesita solo $\Theta(\ell)$, la del registro actual. Con un archivo FASTQ de 50 millones de lecturas, la diferencia es la que hay entre un análisis que funciona en Colab y otro que agota la memoria de la máquina.

0.2.5 Bases de datos programables: Entrez y el genoma de SARS-CoV-2

El Centro Nacional de Información Biotecnológica de Estados Unidos (NCBI) mantiene decenas de bases de datos enlazadas, desde GenBank y RefSeq hasta PubMed, y las expone mediante una interfaz programática, las E-utilities, que el sistema Entrez unifica (Sayers et al., 2022). El módulo `Bio.Entrez` traduce esas llamadas a funciones de Python: `esearch` busca identificadores que satisfacen una

consulta, `efetch` descarga los registros y `elink` navega entre bases de datos. El NCBI exige identificarse con un correo y limita las solicitudes por segundo.

```
descargar_genoma.py

from Bio import Entrez, SeqIO

Entrez.email = "su.correo@universidad.edu"      # obligatorio
acc = "NC_045512.2"
with Entrez.efetch(db="nucleotide", id=acc, rettype="gbwithparts",
                  retmode="text") as handle:
    rec = SeqIO.read(handle, "genbank")
    cds = [f for f in rec.features if f.type == "CDS"]
    print(rec.description, len(rec), "nt,", len(cds), "CDS")
```

El registro que descargamos es la secuencia de referencia del aislado Wuhan-Hu-1. Wu et al. (2020) lo obtuvieron por secuenciación metagenómica de ARN a partir del lavado broncoalveolar de un paciente hospitalizado en Wuhan en diciembre de 2019; el genoma completo mide 29 903 nucleótidos, y su análisis filogenético mostró que el nuevo virus estaba estrechamente emparentado (89,1 % de similitud nucleotídica) con un grupo de coronavirus similares al del SARS hallados en murciélagos. Es un excelente material de prácticas: cualquier resultado puede contrastarse con una literatura enorme.

Ejemplo 0.3. (Verificar una anotación y descubrir un salto de marco)

El GenBank anota 12 regiones codificantes (CDS) en 11 genes. Una prueba de consistencia natural es traducir cada CDS y comparar el resultado con la proteína que el propio archivo registra en el calificador `/translation`. Si extraemos cada región ingenuamente, desde su inicio hasta su final (con `seq[start:end]` y `translate`), 11 de las 12 traducciones coinciden. La excepción es la poliproteína ORF1ab, de 7096 aminoácidos: la traducción ingenua se detiene mucho antes. Si en cambio usamos `feature.extract(seq)`, que respeta la localización compuesta `join(266..13468,13468..21555)`, la traducción coincide residuo a residuo. La base 13 468 aparece en los dos intervalos: se lee dos veces. Es la huella de un *desplazamiento ribosómico del marco* en -1 : una estructura de ARN en forma de pseudonudo hace que, en una fracción de las traducciones, el ribosoma retroceda un nucleótido y continúe en otro marco. Así, un mismo ARN produce la poliproteína corta pp1a (sin salto, termina

en la parada del marco original) y la larga ppiab (con salto). La verificación computacional no solo detectó una “anomalía”: redescubrió un mecanismo biológico.

0.2.6 La línea de comandos y la filosofía Unix

Buena parte de las herramientas bioinformáticas más usadas (BLAST, BWA, samtools, SPAdes, IQ-TREE) son programas de línea de comandos escritos en C o C++. No tienen interfaz gráfica: se invocan escribiendo su nombre y sus opciones en una terminal, leen archivos o flujos de texto y escriben archivos o flujos de texto. Esta forma de trabajar se remonta al sistema operativo Unix. En su descripción original, Ritchie y Thompson (1974) presentaron, entre otras ideas, la *tubería (pipe)*: un canal que conecta la salida de un programa con la entrada del siguiente, de modo que programas pequeños, cada uno especializado en una tarea, se combinan para resolver problemas grandes.

Todo proceso de Unix tiene tres flujos estándar: la entrada (stdin), la salida (stdout) y el canal de errores (stderr), además de un *código de salida*, un entero que vale o si el programa terminó bien. Si vemos cada programa como una función que transforma texto en texto, una tubería es una composición de funciones:

$$p_1 | p_2 | \dots | p_k \equiv f_k \circ \dots \circ f_2 \circ f_1. \tag{0.15}$$

DONDE	
p_i	El i -ésimo programa de la tubería, con sus opciones.
$ $	Operador de tubería del intérprete de comandos: conecta stdout de la izquierda con stdin de la derecha.
f_i	Transformación de texto que realiza p_i ; la composición se lee de derecha a izquierda.

Los programas de una tubería se ejecutan *simultáneamente*: el segundo empieza a procesar las primeras líneas mientras el primero todavía está leyendo el archivo. Igual que el iterador de SeqIO.parse, una tubería procesa archivos mucho más grandes que la memoria disponible. La figura 0.8 muestra una tubería típica, que cuenta los tipos de anotación de un archivo GFF.

0.2.7 BLAST+ desde Python

BLAST es el ejemplo arquetípico de herramienta de línea de comandos: el conjunto BLAST+ reorganizó el código original en C++ y lo dividió en programas

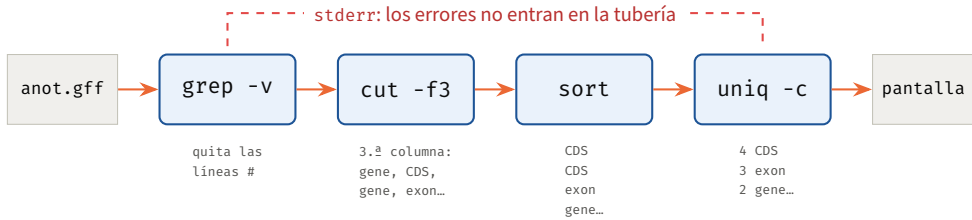


Figura 0.8. La tubería `grep -v '^#' anot.gff | cut -f3 | sort | uniq -c`. Cada programa lee el flujo de texto que le entrega el anterior (flechas naranjas, `stdout`→`stdin`), lo transforma y lo pasa al siguiente; debajo de cada uno, un fragmento de lo que produce. Los cuatro procesos corren a la vez y ninguno necesita cargar el archivo completo.

especializados, `makeblastdb` para indexar una base de datos y `blastn`, `blastp` y compañía para buscar en ella (Camacho et al., 2009). Estudiaremos su algoritmo y su estadística en el capítulo 3. Aquí nos interesa como instrumento: cómo se instala, cómo se invoca desde Python y cómo se lee lo que produce.

En Colab, BLAST+ se instala desde los repositorios del sistema (`apt-get install ncbi-blast+`); en un equipo propio conviene usar un gestor de entornos como `conda`, que veremos en la sección 0.3. Para llamarlo desde Python se usa el módulo estándar `subprocess`, pasando los argumentos como una lista y pidiendo `check=True` para que un código de salida distinto de cero se convierta en una excepción. El formato tabular `-outfmt 6` produce una fila por alineamiento, lista para `pandas`.

```

blast_subprocess.py

import subprocess
import pandas as pd

subprocess.run(["makeblastdb", "-in", "sarscov2.fasta",
               "-dbtype", "nucl", "-out", "sarscov2_db"], check=True)
cols = ("qseqid sseqid pident length mismatch gapopen "
        "qstart qend sstart send evalue bitscore").split()
subprocess.run(["blastn", "-query", "consulta.fasta",
               "-db", "sarscov2_db", "-out", "hits.tsv",
               "-outfmt", "6 " + " ".join(cols)], check=True)
hits = pd.read_csv("hits.tsv", sep="\t", names=cols)
print(hits.sort_values("bitscore", ascending=False).head())

```

¿Cuándo encuentra BLAST una secuencia mutada?

El cuaderno de la lección propone un experimento: tomar un fragmento de 300 nucleótidos del genoma, introducirle mutaciones al azar y comprobar si `blastn` lo encuentra. La respuesta depende de la estrategia de BLAST, que solo empieza a alinear donde encuentra una *semilla*: una palabra de w nucleótidos idéntica en la consulta y en la base de datos ($w = 28$ en el modo predeterminado de `blastn`, llamado megablast, y $w = 11$ con la opción `-task blastn`). Si cada posición conserva su base con probabilidad q (la identidad) de forma independiente, una palabra concreta sobrevive intacta con probabilidad q^w , y el número esperado de semillas en un fragmento de longitud ℓ es

$$E[\text{semillas}] = (\ell - w + 1) q^w. \quad (0.16)$$

DONDE

ℓ	Longitud del fragmento de consulta.
w	Longitud de palabra de la semilla (<code>-word_size</code>).
q	Identidad por posición entre la consulta y su origen, $q = 1 - \text{tasa de mutación}$.

Ejemplo 0.4. (Quince mutaciones en trescientas bases)

Con 15 mutaciones en 300 nucleótidos la identidad es $q = 0,95$. Con palabras de 11 bases, cada una sobrevive con probabilidad $0,95^{11} = 0,569$; hay $300 - 11 + 1 = 290$ palabras, así que esperamos unas $290 \times 0,569 \approx 165$ semillas. Con el modo predeterminado ($w = 28$) la supervivencia cae a $0,95^{28} = 0,238$, pero sobre 273 palabras aún quedan unas 65 semillas: BLAST encontrará el fragmento en ambos casos. El panorama cambia con $q = 0,80$ (60 mutaciones): con $w = 11$ esperamos todavía $290 \times 0,80^{11} \approx 25$ semillas, mientras que con $w = 28$ solo $273 \times 0,80^{28} \approx 0,5$. Es muy probable que megablast no encuentre nada. Para homólogos aún más lejanos, la búsqueda de ADN pierde sensibilidad incluso con palabras cortas y conviene comparar proteínas.

El ejemplo resume el compromiso: el tamaño de palabra regula el equilibrio entre velocidad y sensibilidad, y es uno de los parámetros que conviene registrar siempre junto con los resultados. Que las mutaciones se distribuyan al azar es, a su vez, un supuesto: en genes reales las mutaciones se concentran en las terceras posiciones de los codones, y eso favorece a las semillas “discontinuas” que ignoran esas posiciones, una idea que retomaremos en el capítulo 3.

0.3 Reproducibilidad: semillas, versiones, hashes y Git

 Colab · 0.3 · Reproducibilidad

 abrir notebook

La receta de la abuela

“Agregue un poco de harina, una pizca de sal y hornee hasta que esté listo”. La receta funciona para quien la escribió, que sabe cuánto es “un poco” y reconoce el punto exacto del horno, pero nadie más obtiene el mismo pastel. Un análisis bioinformático sin semillas aleatorias registradas, sin versiones de los programas y sin una huella de los datos de entrada es esa receta: funcionó una vez, en una computadora, un día concreto, y nadie, ni siquiera su autor seis meses después, puede garantizar que volverá a obtener lo mismo.

0.3.1 ¿Qué significa reproducir un análisis?

La palabra “reproducibilidad” se usa con sentidos distintos, y conviene separarlos. Hablaremos de *repetibilidad* cuando el mismo investigador, con los mismos datos, el mismo código y el mismo entorno, obtiene el mismo resultado; de *reproducibilidad computacional* cuando lo obtiene otra persona a partir de los datos y el código publicados; y de *replicabilidad* cuando un estudio independiente, con datos nuevos, llega a la misma conclusión. La reproducibilidad computacional es el requisito mínimo: sin ella, una discrepancia con otro estudio no puede atribuirse ni a la biología ni a un error.

El problema no es hipotético. En una encuesta de *Nature* a 1576 investigadores, más del 70 % declaró haber intentado sin éxito reproducir un experimento de otro grupo, y más de la mitad, uno propio (Baker, 2016). En bioinformática la evidencia es más directa. Ioannidis et al. (2009) encargaron a dos equipos independientes reproducir una tabla o figura de 18 análisis de expresión con micromatrices publicados en *Nature Genetics*: lograron reproducir dos en principio y seis de forma parcial o con discrepancias; los diez restantes no pudieron reproducirse. La causa principal fue la falta de disponibilidad de los datos, y las discrepancias se debieron sobre todo a descripciones incompletas del procesamiento.

Un análisis computacional puede dejar de ser reproducible por cuatro vías: porque depende de *números aleatorios* no registrados, porque sus *datos* cambiaron sin que

nadie lo notara, porque su *código* se modificó después de producir los resultados, o porque su *entorno* (versiones de bibliotecas, del sistema operativo) ya no es el mismo. El resto del capítulo presenta un instrumento para cada vía.

0.3.2 El azar determinista: generadores pseudoaleatorios

Simulaciones, remuestreos *bootstrap*, inicializaciones de algoritmos de agrupamiento, divisiones de datos para entrenar un modelo: la bioinformática consume cantidades enormes de números aleatorios. Pero una computadora es una máquina determinista y no puede lanzar una moneda. Lo que hace es calcular una sucesión que *parece* aleatoria.

Definición 0.5. (Generador pseudoaleatorio)

Un generador de números pseudoaleatorios es una tupla $(\mathcal{S}, f, g, s_0)$ formada por un conjunto finito de estados $\mathcal{S}$, una función de transición $f : \mathcal{S} \rightarrow \mathcal{S}$, una función de salida $g : \mathcal{S} \rightarrow [0, 1)$ y un estado inicial s_0 , la *semilla*. El generador produce $u_n = g(s_n)$ con $s_{n+1} = f(s_n)$. Como $\mathcal{S}$ es finito, la sucesión es tarde o temprano periódica; el menor P tal que $s_{n+P} = s_n$ para todo n grande es su *período*.

La definición contiene la clave de la reproducibilidad: toda la sucesión queda determinada por la semilla. Es como una baraja mezclada de antemano: la semilla dice en qué carta empezar.

El generador más antiguo y más estudiado es el *congruencial lineal* (LCG), cuyo estado es un único entero:

$$x_{n+1} = (a x_n + c) \bmod m, \quad u_n = \frac{x_n}{m}. \quad (0.17)$$

DONDE

x_n	Estado del generador, un entero entre 0 y $m - 1$; x_0 es la semilla.
a, c, m	Multiplicador, incremento y módulo: constantes que definen el generador.
u_n	Número pseudoaleatorio en $[0, 1)$ que se entrega al usuario.

El período de un LCG no puede superar m , y alcanzar ese máximo exige elegir bien las constantes. El resultado clásico, que Knuth (1997) demuestra con detalle, es el siguiente.

Teorema 0.2. (*Período completo de un LCG (Hull–Dobell)*)

Si $c \neq 0$, el generador de la ecuación (0.17) tiene período m para toda semilla s y solo si: (i) c y m son primos entre sí; (ii) $a - 1$ es divisible por todos los factores primos de m ; y (iii) $a - 1$ es divisible por 4 si m lo es.

Ejemplo 0.5. (Un generador de juguete con período completo)

Tome $m = 16$, $a = 5$, $c = 3$. Las condiciones se cumplen: $\text{mcd}(3, 16) = 1$; el único primo de 16 es 2, que divide a $a - 1 = 4$; y como $4 \mid 16$, también $4 \mid (a - 1)$. Con semilla $x_0 = 7$, la sucesión es

$$\begin{aligned} 7 \rightarrow 6 \rightarrow 1 \rightarrow 8 \rightarrow 11 \rightarrow 10 \rightarrow 5 \rightarrow 12 \rightarrow 15 \\ \rightarrow 14 \rightarrow 9 \rightarrow 0 \rightarrow 3 \rightarrow 2 \rightarrow 13 \rightarrow 4 \rightarrow 7 \rightarrow \dots \end{aligned}$$

que recorre los 16 estados antes de repetirse. Con $a = 3$, en cambio, la condición (ii) se cumple pero la (iii) no, y el período se acorta. Observe también que los bits bajos son pésimos: $x_n \bmod 2$ alterna 1, 0, 1, 0..., un defecto general de los LCG con módulo potencia de 2.

0.3.3 Cuando el azar tiene estructura: RANDU

Período largo no significa buena calidad. El caso más famoso es RANDU, el generador de la biblioteca científica de IBM en los años sesenta, con $a = 65539$, $c = 0$ y $m = 2^{31}$, usado durante años en simulaciones publicadas. Su multiplicador tiene una forma especial, $a = 2^{16} + 3$, y eso basta para arruinarlo. Elevando al cuadrado,

$$a^2 = 2^{32} + 6 \cdot 2^{16} + 9 \equiv 6 \cdot 2^{16} + 9 = 6(2^{16} + 3) - 9 = 6a - 9 \pmod{2^{31}}, \quad (0.18)$$

porque 2^{32} es múltiplo de 2^{31} . Multiplicando por x_n y usando $x_{n+1} \equiv ax_n$ y $x_{n+2} \equiv a^2x_n$, obtenemos una relación lineal entre tres valores consecutivos:

$$x_{n+2} \equiv 6x_{n+1} - 9x_n \pmod{2^{31}} \implies 9u_n - 6u_{n+1} + u_{n+2} = k \in \mathbb{Z}. \quad (0.19)$$

DONDE

u_n	Salida normalizada $x_n/2^{31} \in [0, 1)$.
k	Entero; como $9u_n - 6u_{n+1} + u_{n+2}$ está entre -6 y 10 , solo puede tomar 15 valores $(-5, \dots, 9)$.

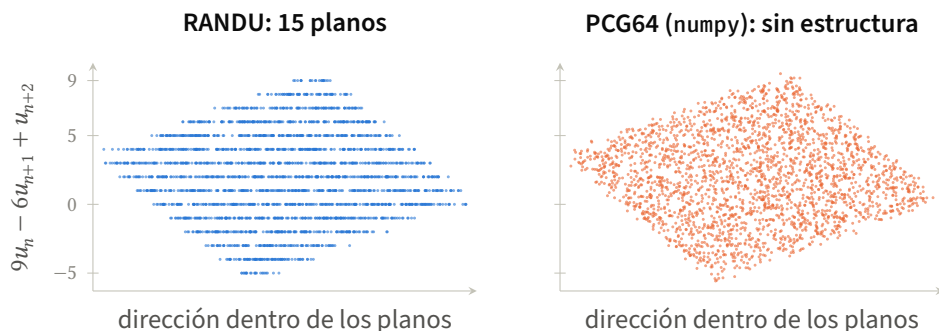


Figura 0.9. 2500 tripletes consecutivos (u_n, u_{n+1}, u_{n+2}) proyectados sobre dos direcciones: el eje vertical es la combinación $9u_n - 6u_{n+1} + u_{n+2}$ de la ecuación (0.19) y el horizontal, una dirección perpendicular a ella. Los tripletes de RANDU (izquierda) solo toman 15 valores enteros en el eje vertical: son 15 planos vistos de canto. Los de PCG64, el generador predeterminado de NumPy (derecha), llenan el espacio de forma homogénea.

La ecuación (0.19) dice que todos los tripletes (u_n, u_{n+1}, u_{n+2}) caen sobre 15 planos paralelos, con vector normal $(9, -6, 1)$ (figura 0.9). Una simulación tridimensional con RANDU explora solo 15 láminas del cubo y deja vacío todo lo demás. Marsaglia (1968) demostró que el defecto es inherente a todos los generadores congruenciales multiplicativos, aunque en grado muy distinto: los puntos de d valores consecutivos caen siempre sobre una familia de hiperplanos paralelos, cuyo número no puede superar $(d!m)^{1/d}$. Para $m = 2^{31}$ y $d = 3$ esa cota es de unos 2344 planos; RANDU se queda con 15.

0.3.4 Generadores modernos: Mersenne Twister y PCG

Los generadores actuales se diseñan con pruebas teóricas de uniformidad en muchas dimensiones y se someten a baterías de pruebas estadísticas. El Mersenne Twister (MT19937) de Matsumoto y Nishimura (1998), el generador del módulo random de Python y durante años el de NumPy, tiene un período de $2^{19937} - 1$ y está equidistribuido en 623 dimensiones: una garantía que RANDU no cumple ni en tres. Pero es lineal sobre el cuerpo de dos elementos: unas cuantas salidas consecutivas bastan para reconstruir su estado, así que sirve para simular, nunca para criptografía.

Desde la versión 1.17, NumPy usa por defecto PCG64, un generador de la familia de congruenciales permutados propuesta por Melissa O'Neill: un LCG de 128 bits cuyo estado no se entrega directamente, sino a través de una permutación que mezcla

sus bits (se combinan sus dos mitades con un o exclusivo y el resultado se rota un número de posiciones que dictan sus bits altos), lo que elimina las regularidades de los bits bajos que vimos en el ejemplo 0.5. Tiene período 2^{128} , un estado pequeño y puede “saltar” hacia adelante en la sucesión de forma eficiente, lo que permite crear flujos independientes para cálculos en paralelo (NumPy Developers, 2024).

La forma moderna de usarlo es crear un generador propio con una semilla explícita y pasarlo a las funciones que lo necesitan, en lugar de fijar el estado global con `np.random.seed`, que cualquier biblioteca importada puede alterar sin aviso.

```
semillas.py

import numpy as np

rng = np.random.default_rng(20240517)           # semilla explícita
reads = rng.choice(list("ACGT"), size=(3, 12))
print(["".join(r) for r in reads])              # idéntico en cada ejecución

# flujos independientes para 4 procesos en paralelo
children = np.random.SeedSequence(20240517).spawn(4)
streams = [np.random.default_rng(s) for s in children]
```

La misma semilla solo reproduce la misma sucesión con el mismo generador y la misma versión del código; por eso se registra junto con las versiones del software.

0.3.5 Monte Carlo y su error

Una simulación Monte Carlo estima una cantidad (una probabilidad, una media) promediando muchas repeticiones aleatorias. Si $X_1, \dots, X_N$ son repeticiones independientes de una variable con media μ y desviación estándar σ , el estimador es la media muestral $\hat{\mu}_N$, y su precisión obedece a un resultado fundamental.

Teorema 0.3. (Error estándar de Monte Carlo)

El estimador $\hat{\mu}_N = \frac{1}{N} \sum_{i=1}^N X_i$ es insesgado, $E[\hat{\mu}_N] = \mu$, y su error estándar es

$$EE(\hat{\mu}_N) = \sqrt{\text{Var}(\hat{\mu}_N)} = \frac{\sigma}{\sqrt{N}}. \quad (0.20)$$

Por el teorema central del límite, $\hat{\mu}_N \pm 2\sigma/\sqrt{N}$ contiene a μ con probabilidad cercana al 95 %.

DONDE

X_i	Resultado de la i -ésima repetición (por ejemplo, 1 si una lectura simulada no tiene errores y 0 si los tiene).
μ, σ	Media y desviación estándar verdaderas de X_i para una variable de Bernoulli, $\sigma = \sqrt{p(1-p)}$.
N	Número de repeticiones de la simulación.

La demostración es directa: por independencia, $\text{Var}(\sum X_i) = N\sigma^2$, y dividir entre N divide la varianza entre N^2 . La consecuencia práctica es la ley de la raíz cuadrada: *para ganar un decimal de precisión hay que multiplicar el trabajo por cien*. Invirtiendo la ecuación (0.20), el número de repeticiones necesario para un error estándar ϵ es $N = \sigma^2/\epsilon^2$.

Ejemplo 0.6. (Lecturas sin errores)

Un secuenciador comete un error en cada base con probabilidad $e = 0,01$, de forma independiente. ¿Qué fracción de las lecturas de 150 bases no tiene ningún error? La respuesta exacta es $p = (1 - e)^{150} = 0,99^{150} = 0,2215$; se parece a $e^{-1,5} = 0,2231$, la aproximación de Poisson. Si no conociéramos la fórmula, podríamos simular $N = 10\,000$ lecturas y contar las perfectas. El error estándar sería $\sqrt{0,2215 \times 0,7785/10\,000} = 0,0042$, y las cuatro simulaciones de la figura 0.10 terminan en 0,2195, 0,2240, 0,2204 y 0,2223 tras 10^5 lecturas: todas dentro de la banda teórica. Para obtener la tercera cifra decimal con seguridad ($\epsilon = 0,0005$) harían falta unas $0,17/0,0005^2 \approx 690\,000$ lecturas simuladas.

0.3.6 Huellas digitales: funciones hash

Los datos también cambian sin avisar. Un archivo se trunca durante una descarga, una base de datos publica una nueva versión con el mismo nombre de archivo, un colaborador “limpia” una tabla y la guarda encima de la original. Para detectar cualquier cambio, por pequeño que sea, se usa la huella digital del archivo, el resultado de una función *hash* criptográfica.

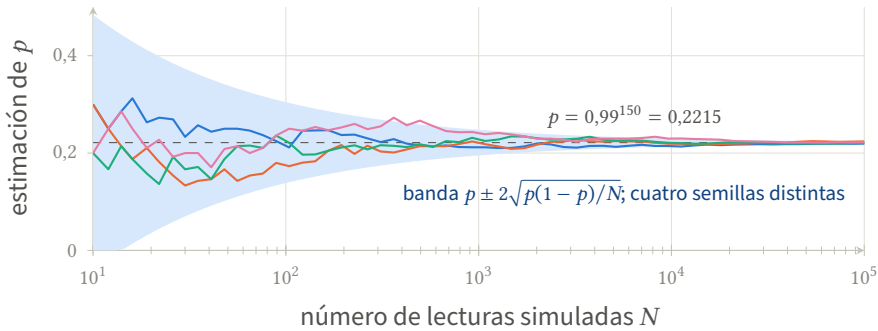


Figura 0.10. Estimación Monte Carlo de la fracción de lecturas de 150 bases sin errores de secuenciación ($e = 0,01$), con cuatro semillas distintas (líneas de color). Al principio las estimaciones difieren mucho; a medida que crece N convergen hacia el valor exacto (línea discontinua) y se mantienen dentro de la banda de ± 2 errores estándar, cuyo ancho se reduce como $1/\sqrt{N}$: en escala logarítmica, cada década de N estrecha la banda en un factor $\sqrt{10} \approx 3,2$.

Definición 0.6. (Función *hash* criptográfica)

Una función $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ que asigna a cualquier mensaje, de cualquier longitud, un resumen de n bits, y que satisface tres propiedades: (1) *resistencia a la preimagen*: dado un resumen y , es inviable encontrar un mensaje x con $h(x) = y$; (2) *resistencia a la segunda preimagen*: dado x , es inviable encontrar $x' \neq x$ con $h(x') = h(x)$; (3) *resistencia a colisiones*: es inviable encontrar cualquier par $x \neq x'$ con $h(x) = h(x')$.

SHA-256, definida en el estándar federal estadounidense de *hash* seguro (National Institute of Standards and Technology, 2015), produce resúmenes de $n = 256$ bits, que se escriben como 64 dígitos hexadecimales. Procesa el mensaje en bloques de 512 bits, cada uno mediante 64 rondas de sumas modulares, rotaciones y funciones lógicas, diseñadas para que cada bit de la salida dependa de todos los bits de la entrada. El resultado es el *efecto avalancha*: cambiar un solo bit de la entrada cambia, en promedio, la mitad de los bits de la salida, y el resumen modificado no guarda ninguna relación visible con el original.

Ejemplo 0.7. (Una base de diferencia)

Las secuencias **GATTACA** y **GATTACC** difieren en una sola base. Sus resúmenes SHA-256 son

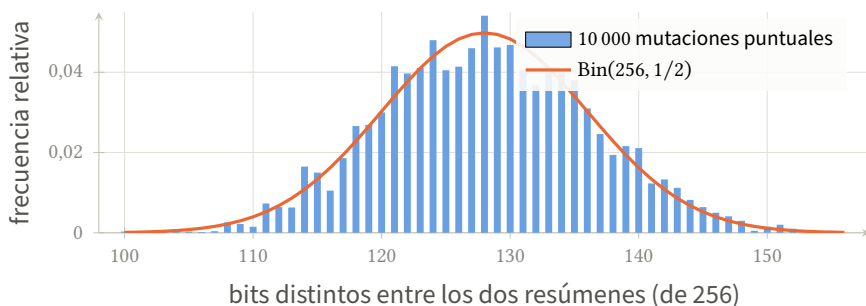


Figura 0.11. Efecto avalancha. Se tomó una secuencia aleatoria de 1000 bases y se le aplicaron 10 000 mutaciones puntuales independientes; para cada una se contaron los bits que difieren entre el SHA-256 de la secuencia original y el de la mutada (media observada 128,2; desviación estándar 8,2). La distribución coincide con la de 256 monedas justas (naranja): un cambio mínimo en los datos produce una huella completamente distinta, y nada en la huella delata dónde estuvo el cambio.

```
d74f6c423e80cbf69d76149048e458a10c96f927c896ea9ff4f44616b643eb22
60bcbfb9d965bb9c40959b91781d457c8801eca61d70fee26377d612d205eecb7
```

y difieren en 129 de sus 256 bits. Si la salida fuera una sucesión de bits aleatorios e independientes, el número de bits distintos seguiría una binomial $\text{Bin}(256, \frac{1}{2})$, con media 128 y desviación estándar 8. La figura 0.11 muestra que eso es exactamente lo que ocurre.

¿Pueden dos archivos distintos tener la misma huella?

Sí, necesariamente: hay infinitos mensajes y solo 2^{256} resúmenes. La pregunta relevante es con qué probabilidad ocurrirá *por accidente*. Es el problema del cumpleaños: si k archivos reciben resúmenes que se comportan como uniformes e independientes sobre $M = 2^n$ valores, la probabilidad de que no haya ninguna coincidencia es

$$\mathbb{P}(\text{sin colisión}) = \prod_{i=1}^{k-1} \left(1 - \frac{i}{M}\right) \approx \exp\left(-\frac{k(k-1)}{2M}\right), \quad (0.21)$$

usando $1 - x \approx e^{-x}$ para x pequeño y $\sum_{i=1}^{k-1} i = k(k-1)/2$.

DONDE

k	Número de archivos a los que se calcula la huella.
$M = 2^n$	Número de resúmenes posibles; $n = 256$ para SHA-256.

La probabilidad de colisión se vuelve apreciable solo cuando k se acerca a $\sqrt{M} = 2^{n/2}$. Para SHA-256 eso significa $2^{128} \approx 3 \times 10^{38}$ archivos. Con un billón (10^{12}) de archivos, la probabilidad de alguna colisión accidental es del orden de $10^{24}/2^{257} \approx 4 \times 10^{-54}$. Para efectos prácticos, dos archivos con el mismo SHA-256 son el mismo archivo.

Verificar la integridad de los datos

```
# calcular y guardar las huellas de los datos crudos
sha256sum datos/*.fastq.gz > datos/SHA256SUMS
# meses después, u otra persona: ¿son los mismos archivos?
sha256sum -c datos/SHA256SUMS
# lecturas_R1.fastq.gz: OK
# lecturas_R2.fastq.gz: FAILED    <- el archivo cambió
```

0.3.7 Git: el historial como grafo

La tercera vía de pérdida de reproducibilidad es el código. Carpetas llenas de `analisis_final.py`, `analisis_final_v2.py` y `analisis_final_DEFINITIVO.py` son la versión computacional del cuaderno de laboratorio con páginas arrancadas. Un sistema de control de versiones registra cada estado del proyecto, quién lo produjo, cuándo y por qué, y permite volver a cualquiera de ellos. Git, el sistema dominante, tiene un modelo interno sorprendentemente sencillo, y entenderlo evita la mayoría de los sustos.

Definición 0.7. (Historial de Git)

Un *commit* es una instantánea completa de los archivos del proyecto, acompañada de metadatos (autor, fecha, mensaje) y de la referencia a uno o más commits *padres*. Cada commit se identifica por el *hash* de todo su contenido, incluidos los identificadores de sus padres. El historial es un *grafo acíclico dirigido* (DAG) $G = (V, E)$: los vértices V son commits y hay una arista $(c \rightarrow p) \in E$ si p es padre de c .

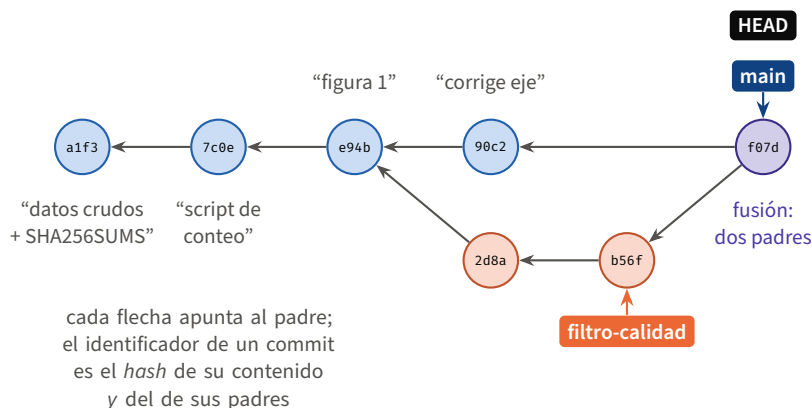


Figura 0.12. El historial de un proyecto como grafo acíclico dirigido. Cada nodo es un commit, identificado por los primeros caracteres de su *hash*; las flechas apuntan a los padres. La rama *filtro-calidad* (naranja) se separó en e94b para probar un nuevo filtro sin tocar la línea principal; al terminar, se fusionó con *main* en el commit f07d, que tiene dos padres. Cualquier resultado puede asociarse al identificador exacto del commit que lo produjo.

Que el identificador sea un *hash* del contenido tiene dos consecuencias profundas. La primera es la integridad: modificar un archivo antiguo cambia el *hash* de ese commit, que forma parte del contenido de su hijo, cuyo *hash* cambia también, y así hasta el último. Es imposible alterar el pasado en silencio: el identificador del commit más reciente certifica todo el historial, igual que una cadena de huellas digitales. La segunda es que el grafo es necesariamente acíclico: para que un commit fuera su propio antepasado, su *hash* tendría que aparecer dentro de su propio contenido, lo que exigiría resolver una ecuación que la resistencia a la preimagen hace inviable.

Sobre ese grafo, los conceptos de Git se vuelven simples. Una *rama* no es una copia del proyecto: es un puntero móvil, un nombre que apunta a un commit y avanza con cada commit nuevo. HEAD es el puntero que indica en qué rama estamos. Una *fusión* (*merge*) crea un commit con dos padres, que reúne dos líneas de trabajo (figura 0.12).

Perez-Riverol et al. (2016) resumen en diez reglas cómo aprovechar Git y la plataforma GitHub en la investigación, entre ellas usar ramas y bifurcaciones para desarrollar funciones nuevas sin romper lo que funciona, etiquetar las versiones con números semánticos, automatizar las pruebas, hacer el código citable y usar la plataforma para que la investigación sea más reproducible. Una regla práctica:

incluya el identificador del commit en cada figura o tabla generada, y guarde los datos grandes fuera de Git, referenciados por su huella.

0.3.8 Entornos, versiones y contenedores

La cuarta vía es el entorno. El mismo código puede producir resultados distintos con otra versión de NumPy, de Biopython o de BLAST, y un análisis que hoy funciona puede fallar mañana porque Colab actualizó una biblioteca. La defensa tiene varias capas, de menor a mayor aislamiento, y un análisis es tan reproducible como la capa más débil que haya quedado sin registrar.

La capa mínima es *registrar*: guardar junto con los resultados la lista exacta de versiones (`pip freeze`, `Bio.__version__`, `blastn -version`), por ejemplo en la primera celda de cada cuaderno. La siguiente es *fixar*: declarar esas versiones en un archivo (`requirements.txt` o `environment.yml`) a partir del cual cualquiera pueda recrear el entorno. Con versionado semántico (MAYOR.MENOR.PARCHE), fijar la versión mayor (`numpy>=1.26,<2`) evita rupturas, y fijar la exacta (`numpy==1.26.4`) garantiza además los mismos resultados numéricos.

La capa más robusta es el *contenedor*, una imagen que empaqueta el programa con todas las bibliotecas del sistema de las que depende y que se ejecuta igual en cualquier computadora con el motor adecuado. Grüning et al. (2018) describen cómo combinar estas capas en la práctica: gestores de paquetes como conda y su canal Bioconda para instalar software bioinformático con versiones exactas, contenedores generados automáticamente a partir de esos paquetes, y sistemas de gestión de flujos de trabajo que registran cada paso.

0.3.9 Reglas y principios: de Sandve a FAIR

Las herramientas de esta sección responden a un conjunto de principios que la comunidad ha ido formulando. Sandve et al. (2013) los condensaron en diez reglas simples para la investigación computacional reproducible, que la tabla 0.1 relaciona con los instrumentos de este capítulo. En paralelo, Wilson et al. (2014) reunieron las *mejores prácticas* de la computación científica (escribir programas para personas, automatizar lo repetitivo, hacer cambios pequeños y versionarlos, planear para los errores), y Wilson et al. (2017) propusieron un conjunto “suficientemente bueno” al alcance de cualquier grupo: guardar los datos crudos sin modificar, dar a cada proyecto una estructura predecible, versionar el código y documentar cómo se ejecuta.

Los principios FAIR amplían la mirada de los análisis a los *datos*. Wilkinson et al. (2016) propusieron que los datos científicos sean *localizables* (*Findable*: con

Tabla 0.1. Las diez reglas de Sandve et al. (2013) para la investigación computacional reproducible (traducción libre) y el instrumento del capítulo que ayuda a cumplirlas.

Regla	Instrumento
1 Registre cómo se produjo cada resultado.	cuaderno, commit
2 Evite los pasos manuales de manipulación de datos.	código, pandas
3 Archive las versiones exactas de los programas externos.	pip freeze, conda
4 Ponga bajo control de versiones todos sus <i>scripts</i> .	Git
5 Registre los resultados intermedios, en formatos estándar si es posible.	CSV, FASTA
6 En los análisis con aleatoriedad, anote las semillas.	default_rng
7 Guarde siempre los datos crudos detrás de cada gráfica.	.dat, .csv
8 Genere salidas jerárquicas, que permitan inspeccionar niveles de detalle.	tablas y figuras
9 Conecte las afirmaciones del texto con los resultados que las sustentan.	cuaderno
10 Dé acceso público a los <i>scripts</i> , las ejecuciones y los resultados.	GitHub

identificadores persistentes y metadatos ricos, registrados en recursos donde se puedan buscar), *accesibles* (*Accessible*: recuperables mediante protocolos abiertos y estandarizados, con metadatos que persisten aunque los datos dejen de estar disponibles), *interoperables* (*Interoperable*: descritos con vocabularios y formatos compartidos) y *reutilizables* (*Reusable*: con licencia clara, procedencia documentada y conformes a los estándares de su comunidad). Una idea central del artículo es que los datos deben ser legibles no solo por personas sino también por máquinas. Un número de acceso como NC_045512.2, con su versión explícita, es un buen ejemplo: el sufijo de versión .2 garantiza que cualquier cambio futuro recibirá un identificador nuevo.

Semilla, huella, commit y entorno:
cuatro registros baratos que convierten una receta en un protocolo.

En resumen

- Un *notebook* separa el documento del núcleo que lo ejecuta; el estado del núcleo depende del orden de ejecución, y solo “reiniciar y ejecutar todo” garantiza que el documento y sus resultados coinciden.
- Una secuencia es un arreglo de símbolos; su costo en memoria depende de la codificación ($nb/8$ bytes) y el costo de un algoritmo se resume con la notación O . La vectorización con NumPy no cambia la complejidad, pero reduce la constante en dos órdenes de magnitud.
- El punto flotante tiene error relativo $\approx 10^{-16}$ y se desborda por abajo al multiplicar probabilidades: se trabaja con logaritmos y log-suma-exp.
- Los datos ordenados y las figuras diseñadas según la percepción forman parte del método, no de la estética.
- Biopython representa secuencias (Seq), registros (SeqRecord) y archivos (SeqIO); Entrez descarga datos del NCBI; las herramientas de línea de comandos se combinan con tuberías y se llaman desde Python con subprocess.
- Un generador pseudoaleatorio es una máquina determinista cuya salida fija la semilla; RANDU muestra que un mal generador introduce estructura (15 planos), y los modernos (Mersenne Twister, PCG64) se diseñan para evitarla. El error Monte Carlo decrece como $\sigma/\sqrt{N}$.
- SHA-256 da a cada archivo una huella de 256 bits con efecto avalancha y colisiones accidentales prácticamente imposibles; Git encadena huellas en un grafo acíclico dirigido de commits.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 0.1** ●●● Un gen está anotado en las posiciones 21 563..25 384 de un genoma (numeración de GenBank). Escriba el corte de Python que lo extrae y calcule su longitud en nucleótidos y en codones. ¿Qué error se comete si se escribe `seq[21563:25384]`?
- 0.2** ●●● Calcule cuánta memoria ocupa el genoma de *Escherichia coli* ($4,6 \times 10^6$ pb) en cada una de las representaciones descritas tras la ecuación (0.2). ¿Cuál elegiría para calcular un *dot plot* contra otro genoma del mismo tamaño?

- 0.3** ●●● Implemente `logsumexp(a)` a partir de la ecuación (0.7) y compárela con `scipy.special.logsumexp` para $a = [-1000, -1001, -1002]$ y para $a = [1000, 1001]$. ¿Por qué falla la evaluación directa en cada caso?
- 0.4** ●●● Repita el ejemplo 0.1 con ventanas de 100, 500 y 2000 bases. ¿Cómo cambia la fracción de ventanas fuera de la banda binomial? Interprete el resultado en términos de ruido de muestreo y de heterogeneidad biológica.
- 0.5** ●●● Demuestre la aproximación de la ecuación (0.21) y úsela para calcular cuántos archivos harían falta para tener un 50 % de probabilidad de colisión con MD5 (128 bits) y con SHA-256. Compare con el número estimado de archivos almacenados en el mundo (del orden de 10^{15}).
- 0.6** ●●● Escriba una función que reciba una semilla, simule N lecturas de 150 bases con una tasa de error por base e y devuelva la estimación de la fracción de lecturas sin errores con su error estándar. Determine empíricamente el N necesario para que el intervalo de ± 2 errores estándar tenga un ancho menor que 0,001 y compárelo con la predicción de la ecuación (0.20).
- 0.7** ●●● Tome un análisis propio (o el cuaderno de la sección 0.2) y evalúelo frente a las diez reglas de la tabla 0.1. Para cada regla incumplida, proponga el cambio concreto (un archivo, una línea de código, un comando) que la haría cumplir.

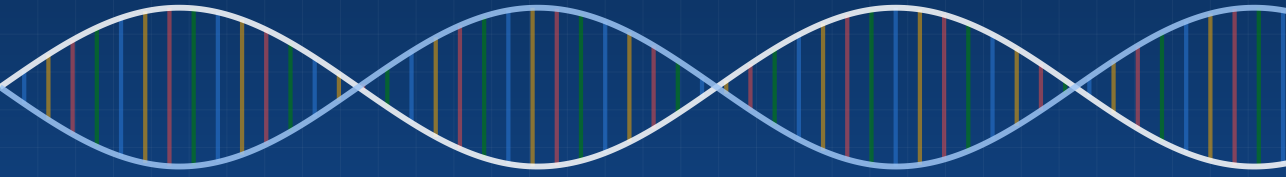
Referencias del capítulo

- Baker, M. (2016). 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604), 452-454. <https://doi.org/10.1038/533452a>
- Buckheit, J. B., & Donoho, D. L. (1995). WaveLab and Reproducible Research. En *Wavelets and Statistics* (pp. 55-81). Springer New York. https://doi.org/10.1007/978-1-4612-2544-7_5
- Camacho, C., Coulouris, G., Avagyan, V., Ma, N., Papadopoulos, J., Bealer, K., & Madden, T. L. (2009). BLAST+: architecture and applications. *BMC Bioinformatics*, 10(1), Artículo 421. <https://doi.org/10.1186/1471-2105-10-421>
- Cleveland, W. S., & McGill, R. (1984). Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods. *Journal of the American Statistical Association*, 79(387), 531-554. <https://doi.org/10.1080/01621459.1984.10478080>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Cramer, F., Shephard, G. E., & Heron, P. J. (2020). The misuse of colour in science communication. *Nature Communications*, 11(1), Artículo 5444. <https://doi.org/10.1038/s41467-020-19160-7>
- Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1), 5-48. <https://doi.org/10.1145/103162.103163>
- Grüning, B., Chilton, J., Köster, J., Dale, R., Soranzo, N., van den Beek, M., Goecks, J., Backofen, R., Nekrutenko, A., & Taylor, J. (2018). Practical Computational Reproducibility in the Life Sciences. *Cell Systems*, 6(6), 631-635. <https://doi.org/10.1016/j.cels.2018.03.014>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- Ioannidis, J. P. A., Allison, D. B., Ball, C. A., Coulibaly, I., Cui, X., Culhane, A. C., Falchi, M., Furlanello, C., Game, L., Jurman, G., Mangion, J., Mehta, T., Nitzberg, M., Page, G. P., Petretto, E., & van Noort, V. (2009). Repeatability of published microarray gene expression analyses. *Nature Genetics*, 41(2), 149-155. <https://doi.org/10.1038/ng.295>
- Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter Development Team. (2016). Jupyter Notebooks – a publishing format for reproducible computational workflows. En *Positioning and Power in Academic Publishing: Players, Agents and Agendas* (pp. 87-90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>
- Knuth, D. E. (1997). *The Art of Computer Programming* (3.^a ed., Vol. 2). Addison-Wesley.
- Marsaglia, G. (1968). Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, 61(1), 25-28. <https://doi.org/10.1073/pnas.61.1.25>
- Matsumoto, M., & Nishimura, T. (1998). Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, 8(1), 3-30. <https://doi.org/10.1145/272991.272995>
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- National Institute of Standards and Technology. (2015). *Secure Hash Standard* (Federal Information Processing Standards Publication N.º 180-4). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.FIPS.180-4>

- NumPy Developers. (2024). *Permuted Congruential Generator (64-bit, PCG64)*. NumPy documentation. https://numpy.org/doc/stable/reference/random/bit_generators/pcg64.html
- Perez-Riverol, Y., Gatto, L., Wang, R., Sachsenberg, T., Uszkoreit, J., Leprevost, F. d. V., Fufezan, C., Ternent, T., Eglen, S. J., Katz, D. S., Pollard, T. J., Kononov, A., Flight, R. M., Blin, K., & Vizcaino, J. A. (2016). Ten Simple Rules for Taking Advantage of Git and GitHub. *PLOS Computational Biology*, 12(7), Artículo e1004947. <https://doi.org/10.1371/journal.pcbi.1004947>
- Ritchie, D. M., & Thompson, K. (1974). The UNIX time-sharing system. *Communications of the ACM*, 17(7), 365-375. <https://doi.org/10.1145/361011.361061>
- Sandve, G. K., Nekrutenko, A., Taylor, J., & Hovig, E. (2013). Ten Simple Rules for Reproducible Computational Research. *PLoS Computational Biology*, 9(10), Artículo e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>
- Sayers, E. W., Bolton, E. E., Brister, J. R., Canese, K., Chan, J., Comeau, D. C., Connor, R., Funk, K., Kelly, C., Kim, S., Madej, T., Marchler-Bauer, A., Lanczycki, C., Lathrop, S., Lu, Z., Thibaud-Nissen, F., Murphy, T., Phan, L., Skripchenko, Y., ... Sherry, S. T. (2022). Database resources of the National Center for Biotechnology Information. *Nucleic Acids Research*, 50(D1), D20-D26. <https://doi.org/10.1093/nar/gkab1112>
- Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2.^a ed.). Graphics Press.
- Wickham, H. (2014). Tidy Data. *Journal of Statistical Software*, 59(10), 1-23. <https://doi.org/10.18637/jss.v059.i10>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, Artículo 160018. <https://doi.org/10.1038/sdata.2016.18>
- Wilson, G., Aruliah, D. A., Brown, C. T., Chue Hong, N. P., Davis, M., Guy, R. T., Haddock, S. H. D., Huff, K. D., Mitchell, I. M., Plumbley, M. D., Waugh, B., White, E. P., & Wilson, P. (2014). Best Practices for Scientific Computing. *PLoS Biology*, 12(1), Artículo e1001745. <https://doi.org/10.1371/journal.pbio.1001745>
- Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), Artículo e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>
- Wu, F., Zhao, S., Yu, B., Chen, Y.-M., Wang, W., Song, Z.-G., Hu, Y., Tao, Z.-W., Tian, J.-H., Pei, Y.-Y., Yuan, M.-L., Zhang, Y.-L., Dai, F.-H., Liu, Y., Wang, Q.-M., Zheng, J.-J., Xu, L., Holmes, E. C., & Zhang, Y.-Z. (2020). A new coronavirus associated with human respiratory disease in China. *Nature*, 579(7798), 265-269. <https://doi.org/10.1038/s41586-020-2008-3>
- Ziemann, M., Eren, Y., & El-Osta, A. (2016). Gene name errors are widespread in the scientific literature. *Genome Biology*, 17(1), Artículo 177. <https://doi.org/10.1186/s13059-016-1044-7>

Biología molecular para bioinformáticos



No ha escapado a nuestra atención que el apareamiento específico que hemos postulado sugiere de inmediato un posible mecanismo de copia del material genético.

— Watson y Crick (1953)

● Al terminar este capítulo usted podrá

- Describir el dogma central como un sistema que almacena, copia y ejecuta información, y distinguir las transferencias generales, especiales y prohibidas de Crick (1970).
- Medir la información de una secuencia con la entropía de Shannon, relacionarla con la compresión y explicar por qué el ADN real está cerca de 2 bits por base.
- Cuantificar la robustez del código genético frente a mutaciones puntuales y compararlo con códigos alternativos generados al azar.
- Implementar un buscador de marcos abiertos de lectura (ORFs) en los seis marcos, construir su modelo nulo geométrico y evaluarlo contra la anotación real de *Mycoplasma genitalium*.

- Calcular el contenido GC, el sesgo GC (*GC skew*) y su versión acumulada, y usarlos para localizar el origen de replicación de *Escherichia coli* a partir de la secuencia sola.

Un bioinformático no necesita saber pipetear, pero sí necesita saber qué significa cada letra que procesa. Una secuencia de ADN no es una cadena arbitraria: es el producto de miles de millones de años de copias imperfectas, filtradas por la selección natural y marcadas por la química de sus propias bases. Cada rareza estadística que encontremos en un genoma (una región pobre en codones de parada, un exceso de guaninas en una mitad del cromosoma, un dinucleótido que casi no aparece) tiene una explicación molecular, y cada explicación molecular deja una huella estadística. Este capítulo recorre ese puente en ambas direcciones.

Avanzaremos en tres pasos. Primero trataremos la célula como un sistema de información: veremos cómo el ADN almacena un mensaje, cómo se copia y se traduce, cuánta información cabe en cada base y por qué el código genético parece diseñado para tolerar errores (sección 1.1). Después buscaremos genes en un genoma real, el de la bacteria con uno de los genomas más pequeños conocidos, y aprenderemos a separar las señales biológicas del ruido con un modelo nulo explícito (sección 1.2). Por último, contaremos letras a lo largo de un cromosoma completo y descubriremos que la replicación deja una cicatriz tan clara que permite localizar el origen de replicación de *E. coli* con un error de unos pocos cientos de bases entre 4,6 millones (sección 1.3).

1.1 El dogma central como sistema de información

 Colab · 1.1 · El dogma central

 abrir notebook

La bóveda y la fotocopia

Piense en el archivo de planos de una gran fábrica. Los originales se guardan en una bóveda; nunca salen de allí y solo se copian, con muchísimo cuidado, cuando la fábrica abre una sucursal. Cuando un taller necesita construir una pieza, no se lleva el original: pide una fotocopia de la página que le interesa, la lleva al taller y allí una máquina lee la fotocopia y ensambla la pieza. La fotocopia se tira a las pocas horas; si tenía una mancha, se estropean unas

cuantas piezas, pero el original sigue intacto. Esa división del trabajo (un archivo permanente, copias de trabajo desechables y máquinas que ejecutan) es exactamente la que usa la célula.

1.1.1 La doble hélice: un medio de almacenamiento

El ADN es un polímero de cuatro nucleótidos, que se distinguen por su base nitrogenada: adenina (A), citosina (C), guanina (G) y timina (T). Cada nucleótido se une al siguiente mediante un enlace fosfodiéster entre el carbono 3' de un azúcar y el carbono 5' del siguiente, de modo que cada hebra tiene una dirección química: un extremo 5' y un extremo 3'. Por convención, las secuencias se escriben siempre de 5' a 3'.

Watson y Crick (1953) propusieron que dos hebras se enrollan en una doble hélice *antiparalela* (una corre 5' → 3' y la otra 3' → 5') unidas por puentes de hidrógeno entre pares de bases complementarias: A con T y G con C. La frase que abre este capítulo resume la consecuencia: si cada hebra determina por completo a la otra, cualquiera de las dos sirve de molde para reconstruir la molécula entera. El ADN no es solo un almacén; es un almacén *autocopiabile*.

Desde el punto de vista computacional, una molécula de ADN bicatenario es una cadena sobre un alfabeto de cuatro letras, más una regla determinista para obtener su pareja.

Definición 1.1. (Secuencia y complemento reverso)

Una secuencia de ADN es una palabra $x = x_1 x_2 \cdots x_n$ sobre el alfabeto $\mathcal{A} = \{A, C, G, T\}$, leída de 5' a 3'. Su *complemento reverso* es

$$\bar{x} = \kappa(x_n) \kappa(x_{n-1}) \cdots \kappa(x_1), \quad \kappa : A \leftrightarrow T, C \leftrightarrow G. \quad (1.1)$$

La hebra complementaria de x , leída también de 5' a 3', es exactamente $\bar{x}$.

DONDE

x_i	Base en la posición i de la hebra, contando desde el extremo 5'.
n	Longitud de la secuencia, en nucleótidos (nt) o pares de bases (pb).
κ	Función de complementariedad de Watson-Crick, que es su propia inversa: $\kappa(\kappa(b)) = b$.
$\bar{x}$	Complemento reverso: la otra hebra, escrita en la dirección convencional.

Dos propiedades de la ecuación (1.1) nos acompañarán todo el capítulo. Primero, $\bar{\bar{x}} = x$: aplicar la operación dos veces devuelve la secuencia original. Segundo, la complementariedad *intercambia* los conteos de bases: el número de **G** de una hebra es el número de **C** de la otra. Esta simetría parece trivial, pero es la razón por la que el sesgo GC de la sección 1.3 cambia de signo cuando se lee la hebra opuesta.

1.1.2 Tres procesos, dos alfabetos: el dogma central

La información guardada en el ADN fluye por tres procesos principales:

- **Replicación** (ADN→ADN): la ADN polimerasa copia cada hebra por complementariedad antes de la división celular.
- **Transcripción** (ADN→ARN): la ARN polimerasa lee una hebra *molde* de 3' a 5' y sintetiza un ARN de 5' a 3'. El ARN resultante es idéntico a la otra hebra, la *codificante*, salvo que lleva uracilo (**U**) en lugar de timina.
- **Traducción** (ARN→proteína): el ribosoma lee el ARN mensajero (ARNm) de tres en tres nucleótidos (*codones*) y, con la ayuda de los ARN de transferencia, une el aminoácido que corresponde a cada codón.

Francis Crick bautizó este esquema como *dogma central* en 1958 y lo reformuló con precisión en 1970. En su versión madura, el dogma no afirma que la información fluya siempre de ADN a ARN y de ARN a proteína; afirma algo más sutil: que la información *secuencial*, residuo a residuo, no puede transferirse desde una proteína a otra proteína ni a un ácido nucleico (Crick, 1970). Crick clasificó las nueve transferencias posibles entre los tres tipos de polímero en tres grupos (figura 1.1): *generales*, que ocurren en todas las células; *especiales*, que ocurren solo en circunstancias particulares, como en ciertos virus; y *desconocidas*, que el dogma postula que no ocurren.

La palabra *dogma* ha causado confusión: Crick la empleó para una hipótesis general todavía sin prueba directa, no para una creencia incuestionable. Por eso,

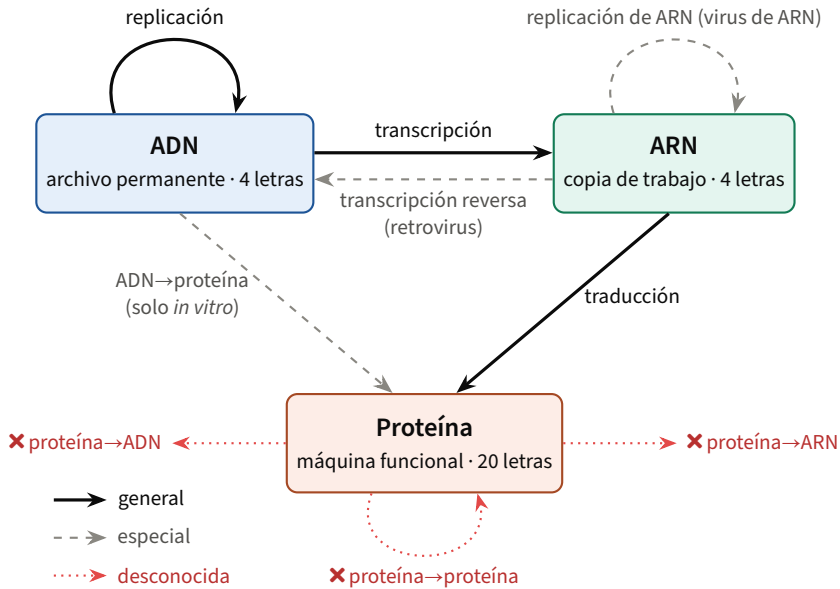


Figura 1.1. El dogma central en la formulación de Crick (1970). Las flechas continuas son las transferencias generales de información secuencial, presentes en toda célula; las discontinuas, las transferencias especiales (la transcripción reversa y la replicación de ARN ocurren en virus; la traducción directa de ADN solo se ha logrado en sistemas acelulares). Las flechas punteadas rojas son las transferencias que el dogma declara inexistentes: una vez que la información llega a una proteína, no puede volver a salir como secuencia. Observe la asimetría de alfabetos: dos cajas usan 4 letras y una usa 20.

cuando Temin y Mizutani (1970) y, de forma independiente, David Baltimore descubrieron en 1970 la transcriptasa reversa, que sintetiza ADN a partir de ARN, no refutaron el dogma: Crick (1970) ya contemplaba ARN→ADN como transferencia especial. Lo que el dogma prohíbe es la salida de información secuencial desde las proteínas, y eso sigue sin observarse.

¿Por qué la célula necesita una copia intermedia? Porque separar el archivo de la copia de trabajo permite tratarlos de manera distinta. El ADN se replica con una fidelidad extraordinaria, del orden de un error por cada 10^9 nucleótidos copiados tras la corrección de pruebas y la reparación de apareamientos erróneos (Alberts et al., 2022), porque cada error se hereda. El ARN, en cambio, se sintetiza con un error por cada 10^4 nucleótidos aproximadamente: sale barato, se produce en muchas copias y se degrada en minutos. Un error en un ARNm estropea unas cuantas proteínas; un error en el ADN se transmite a todos los descendientes. La arquitectura del dogma

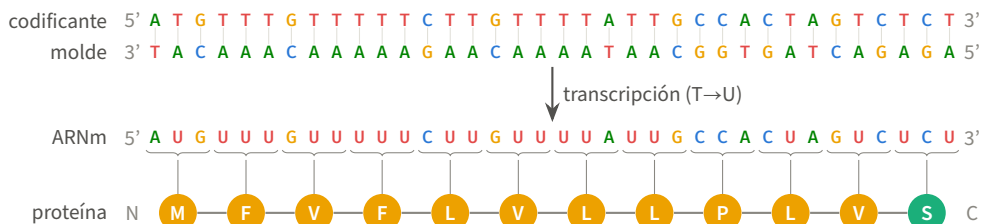


Figura 1.2. Transcripción y traducción de los primeros 36 nucleótidos del gen S de SARS-CoV-2 (NC_045512.2). La hebra molde, que la ARN polimerasa lee de 3' a 5', es la complementaria de la codificante; el ARNm reproduce la codificante con **U** en lugar de **T**. Cada llave agrupa un codón, y el círculo inferior es el aminoácido correspondiente, coloreado por clase fisicoquímica (amarillo: hidrofóbico; azul: básico; rojo: ácido). El tramo MFVFLVLLPLVS, casi todo hidrofóbico, es el péptido señal que dirige la proteína a la membrana. Figura generada con `figuras/cap01/generar.py`.

es, en términos de ingeniería, una forma de concentrar el presupuesto de fidelidad donde más importa.

1.1.3 Transcripción y traducción, paso a paso

Tomemos un ejemplo real: los primeros 36 nucleótidos del gen S del coronavirus SARS-CoV-2, que codifica la proteína de la espiga (*Spike*). La figura 1.2 muestra el flujo completo. La hebra codificante y la hebra molde se aparean base a base; la ARN polimerasa lee la molde y produce un ARNm que reproduce la codificante con **U** en lugar de **T**. El ribosoma lee después el ARNm en codones consecutivos, sin solapamiento y sin separadores, y cada codón aporta un aminoácido.

La implementación computacional de ambos procesos es corta, y escribirla uno mismo es el mejor antídoto contra los errores de convención (¿qué hebra?, ¿qué dirección?) que aparecen una y otra vez en los análisis reales.

`dogma.py`

```
BASES = "TCAG"
AA = ("FFLLSSSSYY**CC*WLLLLPPPHHHQRRRR"
      "IIIMTTTTNNKKSSRRVVVVAAAADDEEGGG") # orden clásico TCAG
CODIGO = {a + b + c: AA[16 * i + 4 * j + k]
          for i, a in enumerate(BASES) for j, b in enumerate(BASES)
          for k, c in enumerate(BASES)}
```

```

COMP = str.maketrans("ACGT", "TGCA")

def complemento_reverso(x: str) -> str:
    return x.translate(COMP)[::-1]          # ecuación (1.1)

def transcribir(codificante: str) -> str:
    return codificante.replace("T", "U")

def traducir(adn: str, hasta_parada: bool = True) -> str:
    prot = []
    for i in range(0, len(adn) - 2, 3):
        aa = CODIGO[adn[i:i + 3]]
        if aa == "*" and hasta_parada:
            break
        prot.append(aa)
    return "".join(prot)

```

Aplicado al gen *S* completo (3822 nt, incluida la parada), traducir produce $3822/3 - 1 = 1273$ aminoácidos que coinciden letra a letra con la traducción depositada en el GenBank. Traducir por error la hebra molde, o el complemento sin invertir, da una secuencia sin relación alguna, casi siempre interrumpida por paradas a las pocas decenas de residuos.

1.1.4 ¿Cuánta información cabe en una base?

Si el ADN es un medio de almacenamiento, podemos preguntarnos por su capacidad, igual que con un disco duro. La herramienta para responder la creó Shannon (1948) para las líneas telefónicas, y se aplica sin cambios a las secuencias biológicas.

Empecemos por un juego. Alguien elige una base al azar y usted debe adivinarla con preguntas de sí o no. La estrategia óptima es partir el conjunto por la mitad: “¿es una purina (A o G)?”, y después “¿es A?”. Siempre bastan dos preguntas, porque $\log_2 4 = 2$. Esa es la capacidad máxima de una posición: *2 bits por base*. Un genoma humano haploide de unos $3,1 \times 10^9$ pares de bases cabe, por lo tanto, en $3,1 \times 10^9 \times 2/8 \approx 775$ megabytes, algo menos que un CD.

Pero la capacidad es solo un techo. Si las bases no fueran equiprobables (por ejemplo, si el 70 % fueran A), usted podría empezar preguntando “¿es A?” y acertar a la primera la mayoría de las veces. El número *medio* de preguntas bajaría, y con él la información que realmente aporta cada base. Shannon definió esa cantidad media como la entropía.

Definición 1.2. (Entropía de Shannon)

Sea X una variable aleatoria que toma valores en un alfabeto finito $\mathcal{A}$ con probabilidades $p_a = \mathbb{P}(X = a)$. Su entropía, en bits, es

$$H(X) = - \sum_{a \in \mathcal{A}} p_a \log_2 p_a, \quad (1.2)$$

con el convenio $0 \log_2 0 = 0$. La cantidad $-\log_2 p_a$ se llama *sorpresa* (o autoinformación) del símbolo a , y H es la sorpresa promedio.

DONDE

$H(X)$	Entropía: información media, en bits, que aporta observar un símbolo.
$\mathcal{A}$	Alfabeto: {A, C, G, T} para ADN, los 20 aminoácidos para proteínas.
p_a	Probabilidad (en la práctica, frecuencia observada) del símbolo a .
$-\log_2 p_a$	Sorpresa de a : un símbolo raro sorprende mucho; uno casi seguro, casi nada.

La definición no es arbitraria. Shannon demostró que H es, salvo una constante multiplicativa, la *única* función que es continua en las probabilidades, crece con el número de alternativas equiprobables y se descompone de forma aditiva cuando una elección se hace en dos etapas, como en nuestro juego de preguntas (Shannon, 1948). Sus cotas fundamentales se derivan en pocas líneas.

Teorema 1.1. (Cotas de la entropía)

Para cualquier distribución sobre un alfabeto de K símbolos,

$$0 \leq H(X) \leq \log_2 K, \quad (1.3)$$

con $H = 0$ si y solo si un símbolo tiene probabilidad 1, y $H = \log_2 K$ si y solo si todos son equiprobables.

La cota inferior es inmediata: cada término $-p_a \log_2 p_a$ es no negativo porque $0 \leq p_a \leq 1$. Para la superior usamos la desigualdad $\ln y \leq y - 1$, válida para todo $y > 0$ con igualdad solo en $y = 1$. Tomando $y = 1/(K p_a)$,

$$H(X) - \log_2 K = \sum_a p_a \log_2 \frac{1}{K p_a} \leq \frac{1}{\ln 2} \sum_a p_a \left(\frac{1}{K p_a} - 1 \right) = \frac{1}{\ln 2} \left(\sum_a \frac{1}{K} - 1 \right) = 0,$$

y la igualdad exige $K p_a = 1$ para todo a . Para el ADN, $K = 4$ y el techo es 2 bits; para las proteínas, $K = 20$ y el techo es $\log_2 20 \approx 4,32$ bits.

Ejemplo 1.1. (La entropía de SARS-CoV-2)

El genoma de SARS-CoV-2 tiene 29 903 nucleótidos con composición $p_A = 0,2994$, $p_C = 0,1837$, $p_G = 0,1961$ y $p_T = 0,3208$: es rico en **A** y **T**. Aplicando la ecuación (1.2),

$$\begin{aligned} H &= -(0,2994 \log_2 0,2994 + 0,1837 \log_2 0,1837 \\ &\quad + 0,1961 \log_2 0,1961 + 0,3208 \log_2 0,3208) \\ &= 0,5210 + 0,4491 + 0,4608 + 0,5262 = 1,957 \text{ bits por base.} \end{aligned}$$

El sesgo de composición le resta apenas 0,043 bits por base al máximo teórico. Para comparar, una secuencia en la que una base aparece el 70 % de las veces y las otras tres el 10 % tendría $H = 1,357$ bits, y la repetición **AAAAAAAA** tendría $H = 0$.

La entropía de bases sueltas ignora el orden. La repetición **CAGCAGCAG**, asociada a la enfermedad de Huntington cuando se expande, tiene tres letras equiprobables y por tanto $H = \log_2 3 \approx 1,585$ bits por base; sin embargo, conocida una letra, la siguiente es completamente predecible. Para capturar esa estructura se calcula la entropía de *palabras* de longitud k (k -mers) y se divide entre k :

$$h_k = \frac{H_k}{k}, \quad H_k = - \sum_{w \in \mathcal{A}^k} p_w \log_2 p_w, \quad h = \lim_{k \rightarrow \infty} h_k. \quad (1.4)$$

donde w recorre las 4^k palabras posibles, p_w es su frecuencia entre todas las ventanas de longitud k , H_k es la *entropía de bloque* y su límite h es la *tasa de entropía*.

Como las dependencias entre vecinos solo pueden reducir la incertidumbre, h_k es no creciente en k y converge a la tasa de entropía h , que mide la información genuinamente nueva que aporta cada base. En la práctica, k no puede crecer mucho: con 4^k palabras posibles y una secuencia finita, la mayoría de las palabras largas no

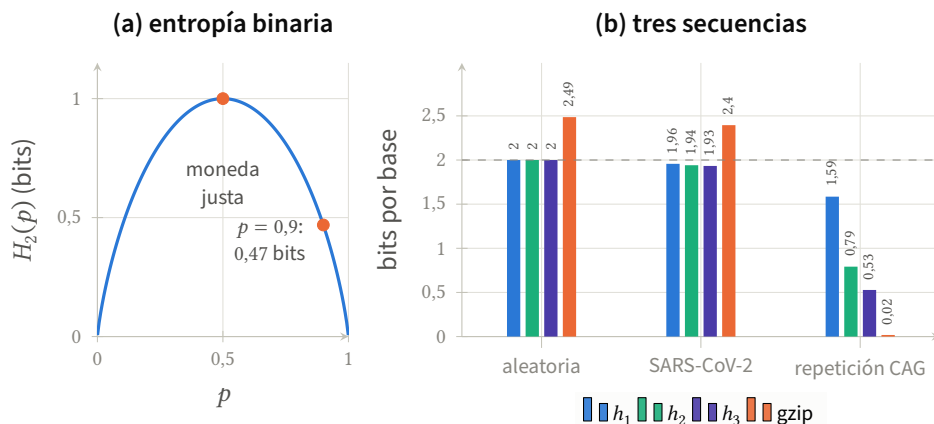


Figura 1.3. Medir la información de una secuencia. (a) Entropía de un alfabeto de dos símbolos en función de la probabilidad p de uno de ellos: es máxima (1 bit) cuando ambos son equiprobables y cae rápidamente cuando uno domina. (b) Entropía por base con palabras de $k = 1, 2, 3$ letras (ecuación (1.4)) y bits por base tras comprimir con **gzip** (nivel 9) para tres secuencias de 29 903 nt: una aleatoria, el genoma de SARS-CoV-2 y una repetición CAG. La línea discontinua marca el techo de 2 bits. El genoma real se comporta casi como una secuencia aleatoria; la repetición tiene 1,58 bits con $k = 1$ pero solo 0,53 con $k = 3$. El compresor genérico ni siquiera alcanza los 2 bits en las dos primeras secuencias.

aparecen nunca y la estimación se sesga hacia abajo. Para un genoma viral de 30 kb, $k \leq 3$ es razonable; para uno bacteriano de 5 Mb, $k \leq 8$.

La figura 1.3b aplica estas ideas a tres secuencias de la misma longitud. La secuencia aleatoria tiene 2 bits por base con cualquier k . El genoma de SARS-CoV-2 queda muy cerca ($h_1 = 1,957$, $h_3 = 1,933$): su composición está sesgada y hay algo de dependencia entre vecinos, pero a simple vista el genoma es casi indistinguible del azar. La repetición, en cambio, se desploma de 1,585 a 0,528 bits cuando se tienen en cuenta los vecinos.

⚠ Entropía no es significado

Que un genoma tenga casi 2 bits por base no quiere decir que “contenga mucha información biológica”, ni que una región de baja entropía sea inútil. La entropía de Shannon mide la imprevisibilidad *estadística* de las letras, no lo que hacen. Un texto en español y el mismo texto con las letras barajadas

tienen idéntica h_1 . En bioinformática, la baja entropía es sobre todo una *alarma*: las regiones de baja complejidad (colas poli-A, microsatélites) producen alineamientos espurios y se enmascaran antes de buscar homólogos, como vimos con BLAST en el capítulo 3.

1.1.5 Compresión: la entropía medida a la fuerza

La entropía tiene una interpretación operativa que la vuelve muy concreta: es el límite de la compresión sin pérdida. El *teorema de codificación de la fuente* de Shannon (1948) establece que ningún código binario decodificable sin ambigüedad puede usar, en promedio, menos de H bits por símbolo, $\bar{\ell} = \sum_a p_a \ell_a \geq H$ (donde ℓ_a es la longitud de la palabra de código del símbolo a), y que codificando bloques cada vez más largos se puede llegar tan cerca de H como se quiera.

Para el ADN, el código trivial $A \mapsto 00$, $C \mapsto 01$, $G \mapsto 10$, $T \mapsto 11$ usa 2 bits por base y es óptimo si las bases son equiprobables. Un archivo FASTA, en cambio, gasta un byte (8 bits) por base: cuatro veces más de lo necesario. La figura 1.3b muestra que *gzip*, un compresor genérico que busca repeticiones pero no “sabe” que solo hay cuatro letras, se queda en 2,4–2,5 bits por base con la secuencia aleatoria y con el genoma viral (peor que el código trivial), mientras reduce la repetición CAG a 0,02 bits por base. Por eso existen formatos especializados como CRAM.

1.1.6 El código genético: un diccionario de 64 entradas

Tres letras de un alfabeto de cuatro producen $4^3 = 64$ codones, que deben codificar 20 aminoácidos y una señal de parada. En términos de información, un codón podría transportar $\log_2 64 = 6$ bits, pero el mensaje que transporta (uno de 21 significados) necesita como máximo $\log_2 21 \approx 4,39$ bits. El código es, por lo tanto, *redundante*: la mayoría de los aminoácidos tienen varios codones sinónimos. Esa redundancia, lejos de ser un desperdicio, resultará ser la clave de su robustez.

Descifrar el código en cinco años

En 1961, Marshall Nirenberg y Heinrich Matthaei añadieron a un extracto de *E. coli* sin células un ARN sintético formado solo por uracilos (poli-U) y obtuvieron un polipéptido formado solo por fenilalanina: UUU codificaba Phe (Nirenberg & Matthaei, 1961). Ese mismo año, Crick et al. (1961) demostraron con mutantes de desplazamiento de marco en el bacteriófago T4 que el código se lee en tripletes, sin solapamiento y desde un punto de inicio fijo. Nirenberg y Leder (1964) idearon después un ensayo

de unión de trinucleótidos al ribosoma que permitió asignar codones uno por uno, y el grupo de Har Gobind Khorana confirmó las asignaciones con ARN sintéticos de secuencia repetitiva. Hacia 1966 la tabla estaba completa. Nirenberg, Khorana y Robert Holley compartieron el Nobel de 1968.

La figura 1.4 presenta el código estándar como una rueda que se lee desde el centro: primera base en el anillo interno, segunda en el intermedio y tercera en el externo. La disposición revela de inmediato un patrón que la tabla rectangular oculta: los codones sinónimos se agrupan en sectores contiguos que casi siempre comparten las dos primeras bases y difieren solo en la tercera. Crick (1966) explicó el patrón con la hipótesis del *bamboleo* (*wobble*): el anticodón del ARN de transferencia se aparea de forma estricta con las dos primeras bases del codón pero de forma relajada con la tercera, de modo que un mismo ARNt puede leer varios codones.

El código es casi universal, pero no del todo. Las mitocondrias de muchos organismos y algunas bacterias usan variantes: en los micoplasmas, por ejemplo, UGA no es de parada sino que codifica triptófano (Yamao et al., 1985). El NCBI numera estas variantes como “tablas de traducción” (la 1 es la estándar, la 11 la de bacterias y la 4 la de micoplasmas y mitocondrias de hongos), y en la sección 1.2 veremos que ignorarlas puede arruinar la anotación de un genoma completo.

1.1.7 Robustez ante errores: ¿es óptimo el código?

Cuando la ADN polimerasa se equivoca, cambia una base por otra. En una región codificante, la consecuencia depende del codón y de la posición:

- una mutación **sinónima** (o silenciosa) cambia el codón pero no el aminoácido;
- una mutación **de sentido erróneo** (*missense*) cambia el aminoácido;
- una mutación **sin sentido** (*nonsense*) crea un codón de parada y trunca la proteína.

Contemos todas las posibilidades. Hay 61 codones con sentido y cada uno tiene $3 \times 3 = 9$ vecinos que difieren en una sola base, de modo que existen $61 \times 9 = 549$ mutaciones puntuales. Un recuento directo sobre la tabla estándar (CODIGO en `dogma.py`) da 134 sinónimas (24,4 %), 23 sin sentido y 392 de sentido erróneo. Pero el promedio esconde una enorme diferencia entre posiciones.

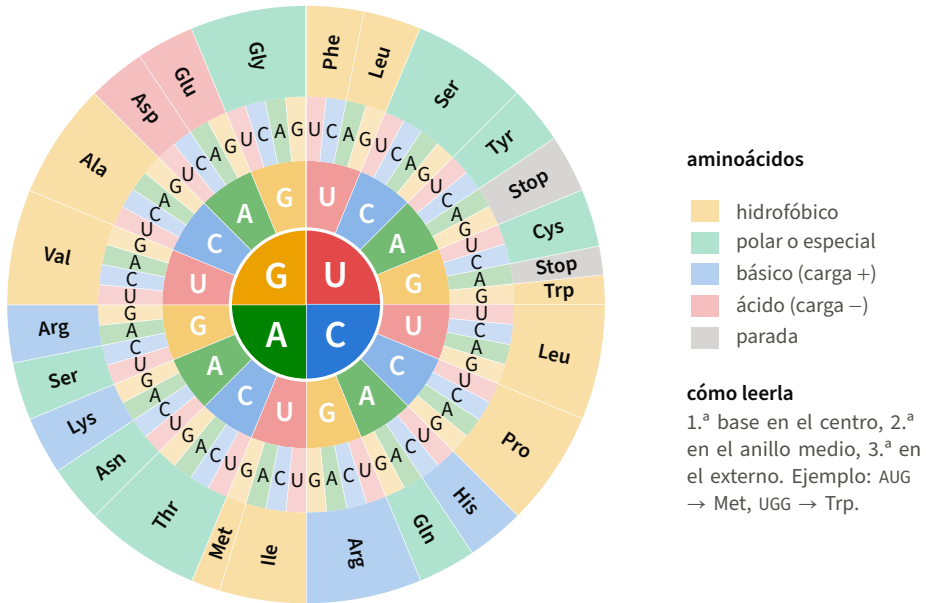


Figura 1.4. El código genético estándar como rueda. Cada sector externo agrupa los codones que comparten las dos primeras bases y codifican el mismo aminoácido. Tres regularidades saltan a la vista: la tercera base casi nunca cambia el aminoácido dentro de un bloque de cuatro; los codones con U en la segunda posición (cuadrante superior de cada primera base) codifican casi siempre aminoácidos hidrofóbicos (amarillo); y los aminoácidos cargados (azul y rojo) se concentran en los codones con A o G en la segunda posición. Los codones de parada UAA, UAG y UGA aparecen en gris.

Ejemplo 1.2. (Mutaciones en el gen de Spike)

Tomemos los 1273 codones con sentido del gen *S* de SARS-CoV-2 y apliquemos, a cada uno, las nueve mutaciones puntuales posibles: 11 457 mutaciones en total. Clasificándolas por posición del codón se obtiene:

Posición	Sinónimas	Sentido erróneo	Sin sentido
1. ^a	2,4 %	92,1 %	5,4 %
2. ^a	0,0 %	96,3 %	3,7 %
3. ^a	64,1 %	31,7 %	4,2 %

En la segunda posición *ninguna* mutación es silenciosa; en la tercera, casi dos de cada tres lo son. La primera posición tiene unas pocas sinónimas gracias a leucina y arginina, que tienen seis codones repartidos entre dos bloques (CUN/UUR y CGN/AGR). Esta asimetría es la base del cociente d_N/d_S entre tasas de sustitución no sinónima y sinónima, que usaremos para detectar selección natural en genómica evolutiva.

La pregunta profunda es otra: cuando una mutación *sí* cambia el aminoácido, ¿lo cambia por uno parecido? Si el código se hubiera fijado al azar, un error cambiaría con la misma facilidad un aminoácido hidrofóbico, que suele ir enterrado en el interior de la proteína, por uno cargado, que prefiere el agua; y eso suele destruir el plegamiento. Haig y Hurst (1991) convirtieron esta intuición en una prueba estadística. La idea es definir un *costo* que mida el daño medio que causan las mutaciones puntuales y comparar el código real con una población de códigos alternativos.

Definición 1.3. (Costo de error de un código)

Sea g un código genético (una función de los 61 codones con sentido en los 20 aminoácidos) y $h(\cdot)$ una propiedad fisicoquímica de los aminoácidos. Sea M el conjunto de pares (c, c') de codones con sentido que difieren en una sola base. El costo de g es

$$\Phi(g) = \frac{1}{|M|} \sum_{(c, c') \in M} \left(h(g(c)) - h(g(c')) \right)^2. \quad (1.5)$$

DONDE

$g(c)$	Aminoácido que el código g asigna al codón c .
M	Pares de codones con sentido separados por una mutación puntual; en el código estándar, $ M = 526$ (las 549 mutaciones menos las 23 que crean una parada).
$h(a)$	Propiedad del aminoácido a ; aquí, la hidropatía de Kyte y Doolittle (1982), positiva para los hidrofóbicos y negativa para los hidrofílicos.
$\Phi(g)$	Cambio cuadrático medio de la propiedad por mutación: cuanto menor, más robusto es el código. Las sinónimas contribuyen con 0.

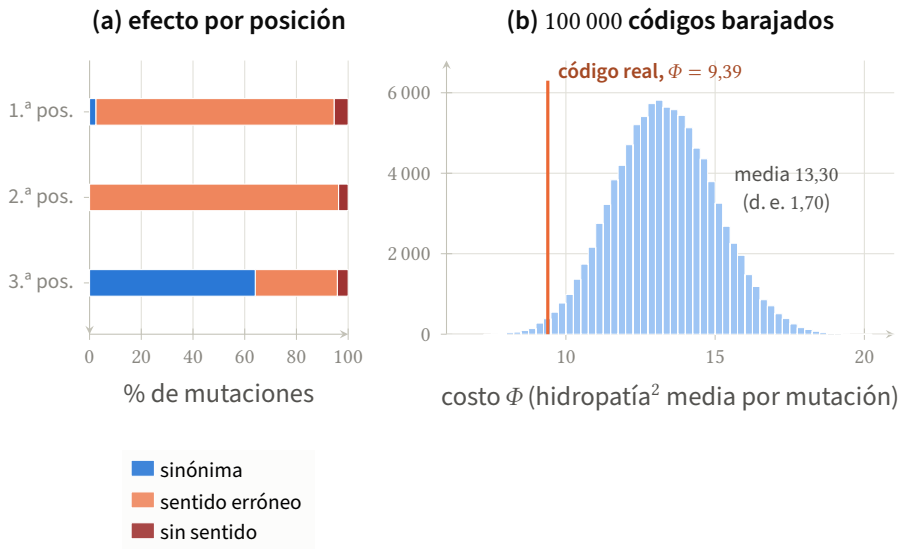


Figura 1.5. El código genético amortigua las mutaciones. (a) Efecto de las 11 457 mutaciones puntuales posibles en los codones del gen S de SARS-CoV-2, según la posición del codón: la tercera posición absorbe la mayoría de los cambios sin alterar la proteína. (b) Distribución del costo Φ (ecuación (1.5)), medido con la hidropatía de Kyte-Doolittle, para 100 000 códigos que conservan los bloques de codones del código real pero barajan los aminoácidos. El código real (línea naranja) queda en la cola izquierda: solo el 0,82 % de los códigos barajados es tan robusto o más. Con medidas más refinadas, que ponderan los tipos de error, esa fracción cae a una en un millón (Freeland & Hurst, 1998).

¿Con qué códigos alternativos comparamos? El procedimiento de Haig y Hurst (1991) conserva la estructura de bloques del código real (qué codones van juntos y dónde están las paradas) y *baraja* los 20 aminoácidos entre los 20 bloques. Hay $20! \approx 2,4 \times 10^{18}$ códigos así, demasiados para enumerarlos, pero basta con muestrear al azar unos cuantos miles para estimar la distribución de Φ . Con este diseño, cualquier diferencia entre el código real y los barajados se debe a *qué aminoácido* se asignó a cada bloque, no a la redundancia en sí.

La figura 1.5b muestra el resultado con 100 000 códigos barajados. El código estándar tiene $\Phi = 9,39$, frente a una media de 13,30 para los barajados, y solo 824 de los 100 000 (el 0,82 %) igualan o mejoran su robustez. Haig y Hurst (1991), usando el “requerimiento polar” de los aminoácidos como propiedad, obtuvieron un resultado del mismo tipo. Freeland y Hurst (1998) refinaron el análisis teniendo en cuenta dos hechos biológicos: las transiciones (purina ↔ purina, pirimidina ↔ pirimidina)

son mucho más frecuentes que las transversiones, y los errores de traducción son más frecuentes en la primera y tercera posiciones que en la segunda. Con esa ponderación, solo uno de cada millón de códigos alternativos resultó mejor que el natural, de ahí el título de su artículo.

⚠ Lo que el experimento demuestra, y lo que no

Nuestra cifra (0,82 %) y la de Freeland y Hurst (una en un millón) no son contradictorias: dependen de la propiedad elegida y de cómo se ponderan los errores, y la versión con hidropatía sin ponderar es una aproximación didáctica. Además, que el código sea robusto no demuestra que la selección natural lo haya *optimizado*. Crick (1968) sostuvo que el código podría ser un “accidente congelado”: una vez que muchas proteínas dependen de él, cualquier cambio de asignación sería letal, y por eso ya no puede evolucionar. Las hipótesis estereoquímica, de coevolución con las rutas biosintéticas de los aminoácidos y adaptativa siguen en discusión; Koonin y Novozhilov (2009) ofrecen una revisión equilibrada.

El código genético guarda la información con redundancia, y la reparte de forma que los errores más probables sean los menos dañinos.

1.2 Genes, genomas y marcos abiertos de lectura

 Colab · 1.2 · Buscador de ORFs

 abrir notebook

Un recetario sin espacios

Imagine un libro de cocina de 580 000 letras escrito sin espacios, sin puntos y sin títulos: solo A, C, G y T, una detrás de otra, y además impreso dos veces, una al derecho y otra al revés y cambiando cada letra por su pareja. En alguna parte hay unas 500 recetas, pero nadie las subrayó. Usted sabe, eso sí, que toda receta empieza con la palabra ATG y termina con TAA, TAG o TGA, y que las

palabras tienen siempre tres letras. ¿Cómo encontraría las recetas? ¿Y cómo sabría que un tramo que “parece” receta no es una casualidad del alfabeto?

1.2.1 Anatomía de un gen: procariotas y eucariotas

Un gen que codifica una proteína es un tramo de ADN rodeado de señales que la maquinaria celular sabe leer. En bacterias, la estructura es compacta (figura 1.6, arriba):

- Un **promotor**, con dos cajas conservadas a unas 10 y 35 bases antes del inicio de la transcripción, donde se une la ARN polimerasa.
- Un **sitio de unión al ribosoma (RBS)**, con la secuencia de **Shine-Dalgarno**, parecida a **AGGAGG**, unas 8 bases antes del codón de inicio. Shine y Dalgarno (1974) observaron que el extremo 3' del ARN ribosómico 16S de *E. coli* es complementario de esa secuencia: el ribosoma “encuentra” el inicio apareándose con ella.
- Una **región codificante (CDS)** continua, desde un codón de inicio (**ATG** casi siempre; **GTG** o **TTG** en una minoría de genes bacterianos) hasta el primer codón de parada en el mismo marco.
- Un **terminador**, a menudo una horquilla de ARN rica en GC seguida de uracilos.

Además, los genes bacterianos suelen organizarse en *operones*: varios CDS consecutivos transcritos en un solo ARNm. En los eucariotas la situación es muy distinta. Berget et al. (1977), estudiando ARNm del adenovirus, descubrieron que la secuencia del mensajero maduro no es colineal con el ADN: faltan segmentos internos que se eliminan durante el procesamiento. Gilbert (1978) propuso llamar *intrones* a los segmentos eliminados y *exones* a los conservados, y sugirió que esta organización en piezas facilita la recombinación de dominios proteicos durante la evolución. El *corte y empalme (splicing)* reconoce señales en los bordes del intrón, casi siempre **G** **T** en el extremo 5' y **AG** en el 3', y el ARNm maduro recibe además una caperuza de 7-metilguanosina en su extremo 5' y una cola de poliadenina en el 3'.

1.2.2 Marcos de lectura

Como los codones se leen de tres en tres sin separadores, una misma secuencia puede cortarse en codones de tres maneras en cada hebra, según la letra en que empezemos. Con las dos hebras, hay seis *marcos de lectura* (figura 1.7).

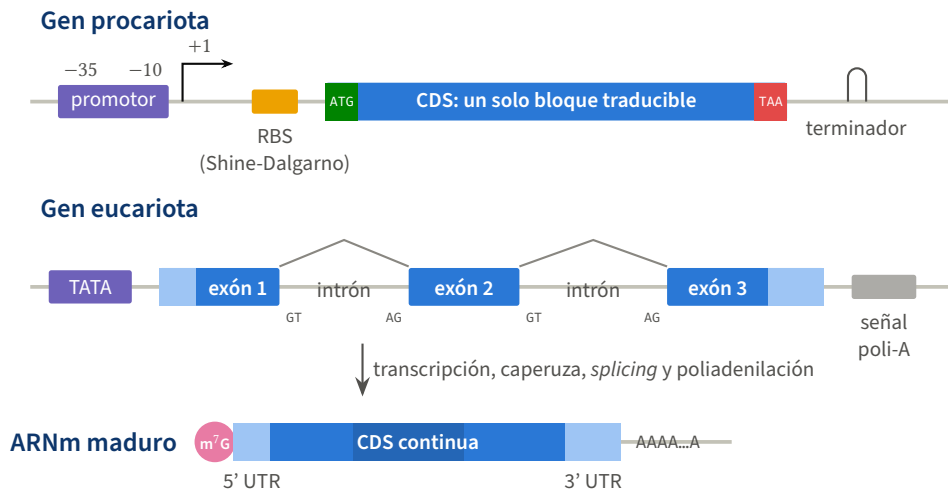


Figura 1.6. Anatomía de un gen. **Arriba:** en una bacteria, la región codificante (azul) es un bloque continuo entre un codón de inicio (verde) y el primer codón de parada del mismo marco (rojo); el ribosoma se posiciona gracias a la secuencia de Shine-Dalgarno. **Abajo:** en un eucariota la información codificante está repartida en exones separados por intrones, que se eliminan por *splicing* (bordes GT...AG); solo el ARNm maduro contiene una CDS continua, flanqueada por regiones no traducidas (UTR, azul claro), la caperuza 5' y la cola poli-A. Un buscador de ORFs como el de esta sección funciona sobre el genoma bacteriano, pero no sobre el ADN genómico eucariota.

Definición 1.4. (Marco abierto de lectura (ORF))

Sea s una de las dos hebras de un genoma (la directa x o su complemento reverso $\bar{x}$) y $f \in \{0, 1, 2\}$ un desplazamiento. El marco (s, f) es la sucesión de codones $c_k = s_{f+3k+1} s_{f+3k+2} s_{f+3k+3}$, $k = 0, 1, \dots$. Dados un conjunto de codones de inicio $\mathcal{I}$ y de parada $\mathcal{T}$, un *marco abierto de lectura* es un tramo de codones $c_a, \dots, c_b$ de un mismo marco tal que

$$c_a \in \mathcal{I}, \quad c_b \in \mathcal{T}, \quad c_k \notin \mathcal{T} \text{ para } a \leq k < b, \quad (1.6)$$

y c_a es el primer codón de inicio posterior a la parada anterior del marco. Su longitud es $\ell = b - a$ codones (sin contar la parada).

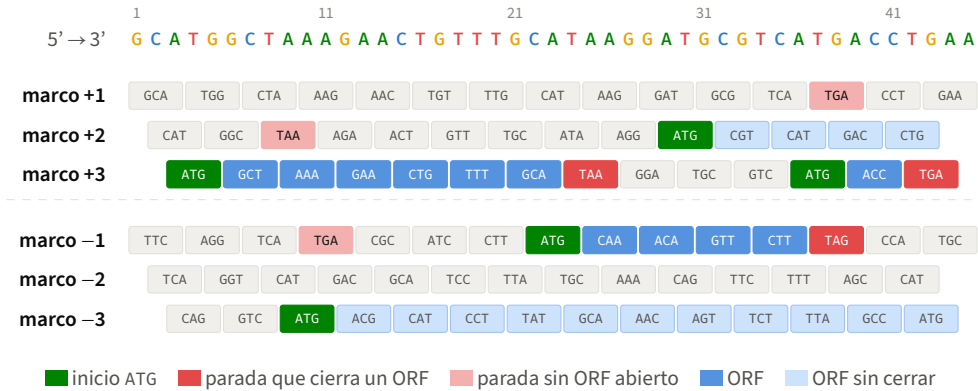


Figura 1.7. Una secuencia de 45 nt leída en sus seis marcos. Los tres marcos superiores cortan la hebra directa empezando en la primera, segunda o tercera base; los tres inferiores hacen lo mismo con el complemento reverse, de modo que todas las filas se leen de 5' a 3'. Un ORF se abre con un ATG y se cierra con la primera parada *del mismo marco*: en el marco +3, el ATG inicial abre un ORF que la parada TAA cierra seis codones después, aunque en el marco +1 haya paradas intercaladas. En el marco -3 el ORF llega al final de la secuencia sin cerrarse. Figura generada con `figuras/cap01/generar.py`.

DONDE

s, f	Hebra y desplazamiento que definen uno de los seis marcos: +1, +2, +3 en x y -1, -2, -3 en $\bar{x}$.
c_k	k -ésimo codón del marco.
$\mathcal{I}, \mathcal{T}$	Codones de inicio ($\{\text{ATG}\}$, o $\{\text{ATG}, \text{GTG}, \text{TTG}\}$ en bacterias) y de parada (dependen de la tabla de traducción).
ℓ	Longitud del ORF en codones, igual al número de aminoácidos de la proteína que codificaría.

Elegir el *primer* inicio tras la parada anterior da el ORF más largo posible en ese tramo; el inicio biológico real puede estar más adentro, y predecirlo con precisión es un problema aparte que los programas modernos resuelven combinando la señal de Shine-Dalgarno con el uso de codones (Hyatt et al., 2010). El final, en cambio, es inequívoco: el primer codón de parada del marco. Por eso, cuando comparemos predicciones con la anotación, usaremos el extremo 3' como criterio de acierto.

1.2.3 Un buscador de ORFs

La ecuación (1.6) se traduce directamente en un algoritmo de una sola pasada por marco: recorrer los codones de izquierda a derecha con una variable de estado que indica si hay un ORF abierto. Al ver un inicio con el ORF cerrado, se abre; al ver una parada con el ORF abierto, se cierra y se registra. Las coordenadas de los ORFs de la hebra inversa se convierten a coordenadas de la hebra directa al final.

```

buscar_orfs.py

PARADAS_11 = {"TAA", "TAG", "TGA"}           # tabla estándar/bacteriana
PARADAS_4 = {"TAA", "TAG"}                   # Mycoplasma: TGA = Trp

def buscar_orfs(x, min_cod=100, inicios=("ATG",), paradas=PARADAS_11):
    """ORFs en los 6 marcos; coordenadas 0-based [ini, fin) en x."""
    L, orfs = len(x), []
    for hebra, s in ((+1, x), (-1, complemento_reverso(x))):
        for f in range(3):
            abierto = None
            for k in range(f, len(s) - 2, 3):
                c = s[k:k + 3]
                if abierto is None and c in inicios:
                    abierto = k                     # se abre
                elif abierto is not None and c in paradas:
                    n = (k - abierto) // 3          # codones
                    if n >= min_cod:
                        a, b = abierto, k + 3
                        if hebra == -1:             # a hebra +
                            a, b = L - b, L - a
                        orfs.append((a, b, hebra, f + 1, n))
                    abierto = None                 # se cierra
            return orfs

```

El algoritmo recorre cada base una vez por marco, así que su costo es $O(L)$ en tiempo. En Python puro tarda unos segundos para un genoma bacteriano; la versión vectorizada del notebook convierte cada codón en un entero entre 0 y 63 y localiza, para cada parada, el primer inicio posterior a la parada anterior con una búsqueda binaria (`np.searchsorted`), lo que la hace unas cien veces más rápida con resultados idénticos.

1.2.4 El modelo nulo: ¿cuán largo puede ser un ORF por azar?

Aquí está el corazón estadístico de la lección. Si escribimos letras al azar también aparecen ORFs, pero cortos: tarde o temprano tropezamos con una parada. Para decidir si un ORF largo es un gen necesitamos saber qué longitudes produce el azar.

Supongamos que las bases son independientes, con frecuencias $\pi_A, \pi_C, \pi_G, \pi_T$. La probabilidad de que un codón cualquiera sea de parada es

$$p = \sum_{c \in \mathcal{T}} \pi_{c_1} \pi_{c_2} \pi_{c_3}, \quad (1.7)$$

que con bases equiprobables vale $p = 3/64 \approx 0,0469$. Recorriendo un marco codón a codón, cada codón es un ensayo de Bernoulli con probabilidad de “éxito” (parada) p , independiente de los demás. La longitud de un tramo entre dos paradas consecutivas es el número de fracasos antes del primer éxito.

Teorema 1.2. (Longitud de un ORF aleatorio)

Bajo el modelo de bases independientes, la longitud L (en codones) de un tramo sin paradas sigue una distribución geométrica:

$$\mathbb{P}(L = \ell) = (1 - p)^\ell p, \quad \mathbb{P}(L \geq \ell) = (1 - p)^\ell, \quad \mathbb{E}[L] = \frac{1 - p}{p}. \quad (1.8)$$

En escala logarítmica, la función de supervivencia $\mathbb{P}(L \geq \ell)$ es una recta de pendiente $\ln(1 - p)$.

DONDE

π_b	Frecuencia de la base b en el genoma.
p	Probabilidad de que un codón al azar sea de parada (ecuación (1.7)).
L	Número de codones consecutivos sin parada en un marco.
$\mathbb{P}(L \geq \ell)$	Supervivencia: probabilidad de que un tramo alcance al menos ℓ codones.

La supervivencia se deduce sin cálculo: $L \geq \ell$ ocurre exactamente cuando los primeros ℓ codones no son de parada, y como son independientes, la probabilidad es el producto $(1 - p)^\ell$. La media se obtiene de la serie $\mathbb{E}[L] = \sum_{\ell \geq 1} \mathbb{P}(L \geq \ell) = \sum_{\ell \geq 1} (1 - p)^\ell = (1 - p)/p$. Con bases equiprobables, $\mathbb{E}[L] = 61/3 \approx 20,3$ codones, y el 1 % de los tramos supera $\ell_{1\%} = \ln 0,01 / \ln(61/64) \approx 96$ codones. Un tramo de 300 codones sin paradas tiene probabilidad $(61/64)^{300} \approx 5,6 \times 10^{-7}$.

⚠ La composición manda

Los codones de parada son ricos en **A** y **T**. Con 20 % de GC, $p = 0,096$ y el 1 % de los tramos al azar supera solo 46 codones; con 72 % de GC, $p = 0,017$ y ese percentil sube a 271. En genomas ricos en GC, como los de *Streptomyces*, el azar fabrica ORFs muy largos: nunca use un umbral “universal” sin recalcular p con la composición de su genoma.

El teorema también nos dice cómo elegir un umbral. Si el genoma tiene L_g bases, cada uno de los seis marcos contiene unos $L_g/3$ codones y, por lo tanto, unas $pL_g/3$ paradas; el número total de tramos entre paradas es aproximadamente $N \approx 2pL_g$. El número esperado de tramos aleatorios de longitud al menos ℓ es $N(1 - p)^\ell$, y exigir que sea menor que un número tolerable α de falsos positivos da

$$N(1 - p)^{\ell^*} \leq \alpha \iff \ell^* \geq \frac{\ln(\alpha/N)}{\ln(1 - p)}. \quad (1.9)$$

DONDE

L_g	Longitud del genoma en pares de bases.
N	Número esperado de tramos entre paradas en los seis marcos, $\approx 2pL_g$.
α	Número de ORFs espurios que estamos dispuestos a tolerar en todo el genoma.
ℓ^*	Longitud mínima que garantiza, en promedio, no más de α falsos positivos.

Esta es una corrección por comparaciones múltiples disfrazada: cuanto mayor es el genoma, más “intentos” tiene el azar y más largo debe ser el umbral. El umbral es conservador por una segunda razón: exigir además un codón de inicio elimina muchos tramos.

1.2.5 Un genoma mínimo real: *Mycoplasma genitalium*

Mycoplasma genitalium es una bacteria parásita del tracto urogenital humano, sin pared celular, con uno de los genomas más pequeños conocidos entre los organismos capaces de crecer en cultivo. Fraser et al. (1995) lo secuenciaron completo, el segundo genoma bacteriano de la historia, y describieron unos 580 kb con alrededor de 470 regiones codificantes predichas; su título, “el complemento génico mínimo”, anunciaba la pregunta que ha guiado desde entonces la investigación sobre

este organismo: ¿cuántos genes necesita una célula para vivir? Glass et al. (2006) respondieron en parte con mutagénesis por transposones, concluyendo que 382 de sus 482 genes codificantes de proteínas son esenciales. Hutchison et al. (2016) llevaron la pregunta a la síntesis: diseñaron y construyeron JCVI-syn3.0, una célula de *Mycoplasma mycoides* con un genoma sintético de 531 kb y 473 genes, de los cuales 149 tenían una función biológica desconocida.

La versión actual del genoma de referencia (NC_000908.2) tiene 580 076 pb, un contenido GC de solo 31,7 % y 504 CDS anotadas, más 20 pseudogenes. La mediana de longitud de sus proteínas es de 287 aminoácidos, y solo 35 tienen menos de 100.

🔗 El codón que no para

En los micoplasmas, UGA no es un codón de parada sino que codifica triptófano. Yamao et al. (1985) lo demostraron en *Mycoplasma capricolum* y la variante se comparte en el linaje, hasta el punto de que el NCBI le asigna una tabla de traducción propia (la 4). Se cree que la presión hacia genomas pobres en GC favoreció el cambio: UGG, el codón habitual del triptófano, es rico en G, y UGA fue “capturado” para el mismo aminoácido.

El detalle tiene consecuencias dramáticas. Si traducimos las 504 CDS de *M. genitalium* con la tabla 4, las 504 proteínas coinciden con la anotación. Con la tabla bacteriana estándar (11), solo 145 (el 28,8 %) salen completas: las otras 359 contienen al menos un TGA interno y se truncan en él. Un anotador automático que ignorara la tabla de traducción predeciría cientos de genes fragmentados.

Ejemplo 1.3. (El umbral de longitud para *M. genitalium*)

Con la composición real del genoma ($\pi_A = 0,3457$, $\pi_C = 0,1578$, $\pi_G = 0,1591$, $\pi_T = 0,3374$) y las dos paradas de la tabla 4, la ecuación (1.7) da

$$p = \pi_T \pi_A \pi_A + \pi_T \pi_A \pi_G = 0,3374 \times 0,3457 \times (0,3457 + 0,1591) = 0,0589.$$

La longitud media de un tramo al azar es $E[L] = 0,9411/0,0589 \approx 16,0$ codones. El número de tramos en los seis marcos es $N \approx 2 \times 0,0589 \times 580\,076 \approx 68\,300$, y la ecuación (1.9) con $\alpha = 1$ da

$$\ell^* \geq \frac{\ln(1/68\,300)}{\ln 0,9411} = \frac{-11,13}{-0,0607} \approx 183 \text{ codones.}$$

Por debajo de 100 codones, en cambio, el azar produce unos $N(1-p)^{100} \approx 158$ tramos: demasiados para confiar en la longitud sola. Observe que la mayoría

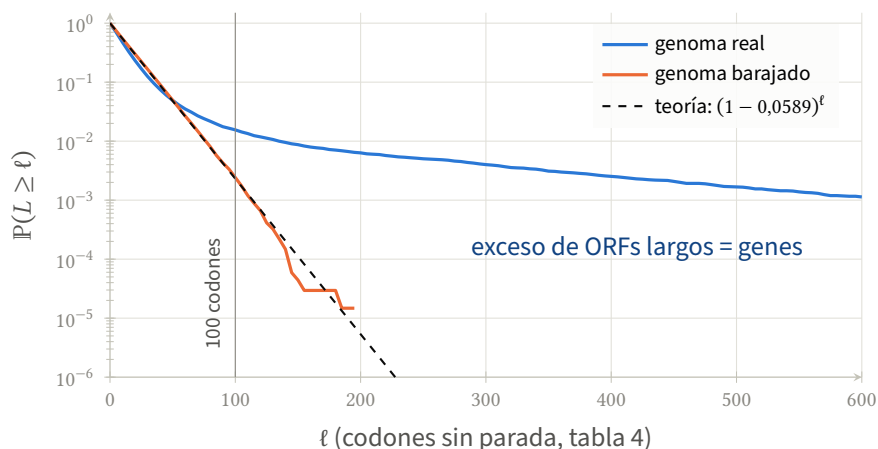


Figura 1.8. Longitud de los tramos sin codones de parada en los seis marcos de *M. genitalium* (tabla 4). El genoma barajado (naranja), que conserva la composición pero no el orden, sigue casi exactamente la recta geométrica del teorema 1.2 (discontinua) y no produce ningún tramo de más de 200 codones. El genoma real (azul) coincide con el azar para tramos cortos, pero a partir de unos 60 codones se separa y mantiene una cola pesada que llega a cientos de codones: ahí viven los genes. A 100 codones, la cola real es 6,4 veces más frecuente que la barajada; a 200, la barajada ya es cero.

de los genes reales del genoma (mediana de 287 aa) están por encima del umbral estricto, pero no todos.

1.2.6 Señal contra ruido

¿Cómo comprobamos que los ORFs largos del genoma real no son casualidad? Construimos un *control negativo*: barajamos las letras del genoma. La secuencia barajada tiene exactamente la misma composición, y por tanto la misma p , pero ha perdido todo orden biológico. La figura 1.8 compara las funciones de supervivencia del genoma real, del barajado y de la predicción geométrica.

Para longitudes cortas, las tres curvas coinciden: $P(L \geq 50)$ vale 0,049 en el genoma real, 0,050 en el barajado y 0,048 en la teoría. La mayoría de los tramos del genoma real son, estadísticamente, ruido: están en los marcos que no se traducen. Pero a 100 codones la frecuencia real es de $1,55 \times 10^{-2}$ frente a $2,4 \times 10^{-3}$ en el barajado, y a 200 codones el barajado no produce ningún tramo mientras que el real conserva un 0,63 %. El genoma barajado es, además, la mejor validación del

modelo: sus puntos caen sobre la recta teórica, lo que confirma que la dependencia entre bases vecinas apenas afecta a la distribución de las paradas.

El barajado permite también estimar directamente la proporción de falsos descubrimientos que cometería un umbral $\ell_{\min}$:

$$\widehat{\text{FDR}}(\ell_{\min}) \approx \frac{N_{\text{barajado}}(\ell_{\min})}{N_{\text{real}}(\ell_{\min})}, \quad (1.10)$$

DONDE

$N_{\text{real}}(\ell_{\min})$	Número de ORFs (con inicio ATG, GTG o TTG) de al menos $\ell_{\min}$ codones en el genoma real.
$N_{\text{barajado}}(\ell_{\min})$	Lo mismo en el genoma barajado, que estima cuántos de los anteriores esperaríamos por azar.

Con $\ell_{\min} = 40$, el genoma barajado produce 2861 ORFs frente a 2909 en el real ($\widehat{\text{FDR}} \approx 0,98$): casi todo es ruido. Con 60 codones la proporción baja a 0,55; con 100, a 0,083 (67 frente a 809); con 150, a 0,004. No existe un umbral perfecto, solo un compromiso entre falsos positivos y genes cortos perdidos.

1.2.7 Sensibilidad, precisión y ORFs sombra

La anotación del GenBank nos da una “respuesta correcta” contra la que evaluar el buscador. Decimos que un ORF predicho es un *verdadero positivo* (VP) si termina en el mismo codón de parada y la misma hebra que una CDS anotada; si no, es un *falso positivo* (FP). Las CDS anotadas que ningún ORF alcanza son *falsos negativos* (FN).

$$\text{sensibilidad} = \frac{\text{VP}}{\text{VP} + \text{FN}}, \quad \text{precisión} = \frac{\text{VP}}{\text{VP} + \text{FP}}. \quad (1.11)$$

La sensibilidad responde “¿qué fracción de los genes reales encontré?”; la precisión, “¿qué fracción de mis predicciones son genes?”.

Con un umbral de 100 codones y los tres inicios bacterianos, el buscador predice 809 ORFs, de los cuales 473 coinciden con CDS anotadas: sensibilidad de 93,8 % pero precisión de solo 58,5 %. ¿De dónde salen los 336 falsos positivos? Al examinarlos, 289 (el 86 %) se solapan en más de la mitad de su longitud con un gen real situado en *otro marco o en la otra hebra*, y 29 caen en pseudogenes. Son *ORFs sombra*: una región codificante está depurada de paradas en su marco, y como efecto colateral de su composición y su uso de codones, los marcos solapados también tienen menos paradas de lo que el azar predice.

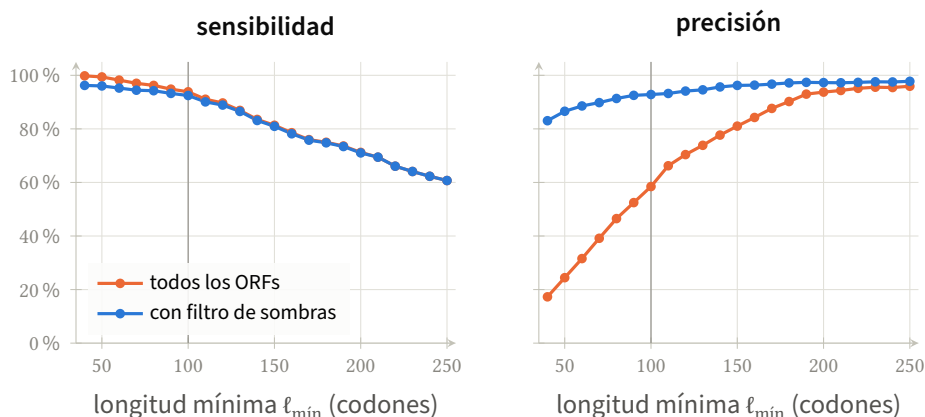


Figura 1.9. Evaluación del buscador de ORFs contra las 504 CDS anotadas de *M. genitalium* (acierto: mismo codón de parada y misma hebra). Al subir el umbral, la sensibilidad cae porque se pierden los genes cortos, y la precisión sube porque desaparece el ruido. El filtro de sombras (azul) dispara la precisión a cualquier umbral casi sin coste en sensibilidad: con 100 codones (línea gris) pasa del 58 % al 93 % manteniendo el 92 % de los genes.

Como dos genes bacterianos rara vez se solapan mucho, un filtro sencillo elimina la mayoría de las sombras: se ordenan los ORFs de más largo a más corto y se descarta cualquier ORF que se solape más de un 50 % con otro más largo ya aceptado. Con este filtro, a 100 codones quedan 502 predicciones, la sensibilidad apenas baja a 92,5 % y la precisión sube a 92,8 %. La figura 1.9 muestra el compromiso para todos los umbrales.

Los 38 genes que el buscador filtrado no encuentra a 100 codones tienen algo en común: 31 de ellos codifican proteínas de menos de 100 aminoácidos, por debajo del umbral. Los demás empiezan con un codón de inicio poco habitual o se solapan con un gen vecino más largo. Detectar genes pequeños sigue siendo un problema abierto; muchas proteínas de menos de 50 aminoácidos se han descubierto apenas en la última década.

🧪 Más allá de la longitud: contenido codificante

Las regiones codificantes tienen una *periodicidad de tres*: la composición de la primera, segunda y tercera posición del codón es distinta, porque la tercera tolera más cambios. Fickett (1982) convirtió esa asimetría en un estadístico (TESTCODE) que reconoce regiones codificantes sin buscar ORFs. Los buscadores modernos combinan varias

señales en modelos probabilísticos: Prodigal (Hyatt et al., 2010) modela el uso de codones, la señal de Shine-Dalgarno y el contexto del inicio, y supera el 95 % de sensibilidad en genomas bacterianos. Todos parten, sin embargo, del esqueleto que hemos construido: enumerar ORFs y compararlos con un modelo nulo.

1.3 Composición de secuencias: contenido GC, GC skew y el origen de replicación

 Colab · 1.3 · GC skew y oriC

 abrir notebook

Un saldo bancario que se lee en el ADN

Imagine que lleva la cuenta de un cajero que, a lo largo del día, recibe billetes de dos colores: suma uno por cada billete verde y resta uno por cada billete rojo. Si los billetes llegan al azar, el saldo oscila sin rumbo alrededor de cero. Pero si durante la mañana llega un pequeño exceso de verdes y durante la tarde un pequeño exceso de rojos, el saldo subirá toda la mañana y bajará toda la tarde, y su máximo marcará, con bastante precisión, la hora del almuerzo, aunque ningún billete individual lo revele. En esta sección haremos exactamente eso con las guaninas y las citosinas de un cromosoma bacteriano, y la “hora del almuerzo” será el punto donde la célula empieza a copiar su ADN.

1.3.1 Contenido GC

La medida de composición más sencilla es la proporción de bases que son **G** o **C**.

Definición 1.5. (Contenido GC)

Para una secuencia (o una ventana) con n_A , n_C , n_G y n_T bases de cada tipo,

$$GC = \frac{n_G + n_C}{n_A + n_C + n_G + n_T}. \quad (1.12)$$

Como cada **G** de una hebra está apareada con una **C** de la otra, el contenido GC es el mismo en ambas hebras: es una propiedad de la *molécula*, no de una hebra concreta. Varía enormemente entre organismos: 31,7 % en *M. genitalium*, 38,0 % en

SARS-CoV-2, 50,8 % en *E. coli* y más de 70 % en algunas actinobacterias. Controla la estabilidad térmica del ADN (un par G–C tiene tres puentes de hidrógeno, uno A–T dos) y, como vimos en la sección 1.2, la probabilidad de encontrar paradas por azar. Variaciones locales del contenido GC dentro de un genoma bacteriano delatan a menudo *islas genómicas* adquiridas por transferencia horizontal desde otros organismos.

1.3.2 Un cromosoma circular que se copia en dos direcciones

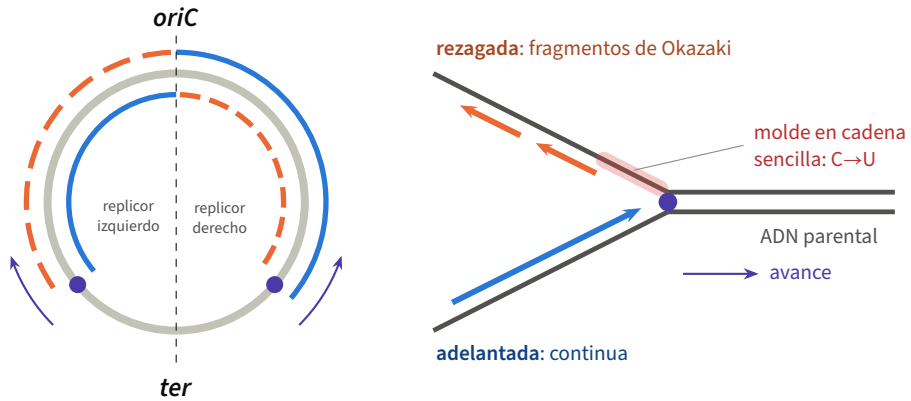
El cromosoma de *Escherichia coli* K-12, secuenciado por Blattner et al. (1997), es una sola molécula circular de unos 4,64 Mb. Su replicación sigue un plan muy ordenado. Comienza siempre en el mismo lugar, el *origen de replicación* (*oriC*), donde la proteína DnaA abre la doble hélice. Desde allí parten dos *horquillas de replicación* en direcciones opuestas, que se encuentran en el lado contrario del círculo, en la *región de terminación* (*ter*), donde la proteína Tus, unida a los sitios *Ter*, las detiene. El cromosoma queda dividido en dos mitades, los *replicores*, cada una copiada por una sola horquilla.

Dentro de cada horquilla, las dos hebras hijas no se sintetizan igual. Las ADN polimerasas solo alargan cadenas en dirección 5'→3', y las dos hebras molde son antiparalelas. Una de las hebras hijas, la *adelantada* (*leading*), puede crecer de corrido en el mismo sentido en que avanza la horquilla. La otra, la *rezagada* (*lagging*), debe sintetizarse hacia atrás, en fragmentos cortos que luego se unen: los fragmentos que Okazaki et al. (1968) detectaron marcando con radiactividad el ADN recién sintetizado. Mientras la horquilla avanza y el siguiente fragmento aún no ha empezado, el molde de la hebra rezagada queda expuesto como *cadena sencilla* (figura 1.10).

Ahora viene la observación clave. Fijemos *una* hebra del cromosoma, por ejemplo la que aparece escrita en el archivo FASTA. En un replicor, esa hebra es el molde de la hija adelantada; en el otro, el de la rezagada. El cambio de papel ocurre exactamente en *oriC* y en *ter*. Si la replicación trata de forma distinta a los dos moldes, la hebra del archivo debería mostrar un sesgo de composición que cambia de signo en esos dos puntos.

1.3.3 La cicatriz de la replicación

La citosina tiene un punto débil químico: puede perder su grupo amino y convertirse en uracilo, que en la siguiente replicación se aparea con adenina. El resultado neto es una mutación C→T. Esta *desaminación* es una de las fuentes más importantes de mutaciones espontáneas, y es mucho más rápida en ADN de cadena sencilla que en la doble hélice, donde la citosina está protegida por su pareja: Frederico et al.



las dos horquillas avanzan en sentidos opuestos

Figura 1.10. **Izquierda:** el cromosoma bacteriano se replica desde *oriC* con dos horquillas (violeta) que avanzan en sentidos opuestos hasta *ter*. En cada replicor se dibujan la hebra hija continua (azul) y la discontinua (naranja); observe que sus papeles se *intercambian* al pasar de un replicor al otro. **Derecha:** detalle de una horquilla. La hebra adelantada crece de forma continua hacia la horquilla; la rezagada se sintetiza en fragmentos de Okazaki que crecen en sentido contrario, de modo que su molde queda temporalmente como cadena sencilla (sombreado rojo), donde la desaminación de la citosina es mucho más rápida.

(1990) midieron una constante de velocidad más de cien veces mayor en cadena sencilla. Juntemos las piezas: el molde de la hebra rezagada pasa más tiempo como cadena sencilla, así que acumula más $C \rightarrow T$; esa hebra pierde C y, por complementariedad, la hebra opuesta pierde G . Lobry (1996) mostró que en varias bacterias la hebra adelantada es, en efecto, más rica en G que en C , y que el sesgo cambia de signo en el origen y el término de la replicación.

Un modelo mínimo permite ver cuánto sesgo produce una asimetría pequeña. Consideremos una hebra dentro de un replicor y fijémonos solo en las G y las C . Sea c la tasa de mutación $C \rightarrow T$ en esa hebra y a la tasa de $G \rightarrow A$ (que no es otra cosa que $C \rightarrow T$ en la hebra complementaria). Supongamos que las G y las C se reponen a partir de A y T con una misma tasa u , y que la hebra tiene aproximadamente tantas A como T .

Teorema 1.3. (Sesgo de equilibrio)

En el modelo anterior, las frecuencias de equilibrio satisfacen $u\pi_A = a\pi_G$ y $u\pi_T = c\pi_C$, de modo que, si $\pi_A \approx \pi_T$,

$$\frac{\pi_G}{\pi_C} = \frac{c}{a}, \quad S^* = \frac{\pi_G - \pi_C}{\pi_G + \pi_C} = \frac{c - a}{c + a}. \quad (1.13)$$

DONDE

c, a	Tasas de las mutaciones $C \rightarrow T$ y $G \rightarrow A$ leídas en la hebra considerada.
u	Tasa (simétrica) de las mutaciones que crean G o C a partir de A o T .
π_b	Frecuencia de equilibrio de la base b en esa hebra.
S^*	Sesgo GC de equilibrio: positivo si la hebra pierde más C que G .

La derivación es un balance de flujos: en equilibrio, las G que se pierden por unidad de tiempo ($a\pi_G$) deben igualar a las que se crean ($u\pi_A$), y lo mismo con las C . Dividiendo ambas igualdades se obtiene la ecuación (1.13). El resultado es revelador: el sesgo no depende de la magnitud absoluta de las tasas, sino solo de su *cociente*. Una asimetría minúscula, sostenida durante millones de generaciones, basta para dejar una huella medible en todo el genoma. Frank y Lobry (1999) revisan los mecanismos propuestos, que no se reducen a la desaminación: también contribuyen diferencias en la fidelidad de las polimerasas de cada hebra y efectos selectivos ligados a la orientación de los genes.

1.3.4 El sesgo GC y su versión acumulada

Para medir la huella necesitamos un número que, a diferencia del contenido GC, distinga entre las dos hebras.

Definición 1.6. (Sesgo GC y sesgo acumulado)

El *sesgo GC* (*GC skew*) de una ventana de la hebra leída es

$$S = \frac{n_G - n_C}{n_G + n_C} \in [-1, 1]. \quad (1.14)$$

El *sesgo acumulado* en la posición j es el saldo

$$C(j) = \sum_{i=1}^j (\mathbb{1}[x_i = G] - \mathbb{1}[x_i = C]). \quad (1.15)$$

DONDE

n_G, n_C	Número de guaninas y citosinas en la ventana, en la hebra que estamos leyendo.
S	Sesgo GC: +1 si solo hay G , -1 si solo hay C , 0 si están equilibradas.
$C(j)$	Saldo acumulado: suma +1 por cada G y -1 por cada C desde el inicio hasta la posición j .

Por la complementariedad, leer la otra hebra invierte el signo de S en cada ventana: el sesgo es una propiedad de la *hebra*. El sesgo acumulado, introducido como herramienta sistemática por Grigoriev (1998), tiene una ventaja decisiva sobre las ventanas: no obliga a elegir un tamaño de ventana y suma la señal a lo largo de todo el replicor, de modo que un sesgo diminuto pero sostenido produce una pendiente clara. Si en un replicor el sesgo medio es s y el contenido GC es g , cada base aporta en promedio

$$\mathbb{E}[C(j+1) - C(j)] = \frac{g}{2}(1+s) - \frac{g}{2}(1-s) = gs, \quad (1.16)$$

así que C sube con pendiente gs en el replicor de sesgo positivo y baja con pendiente $g|s|$ en el otro. El resultado es una curva en forma de V (o de V invertida) con el *mínimo en oriC* y el *máximo en ter*.

Ejemplo 1.4. (Sesgo a mano)

Tomemos la ventana **GGCAGCTGGA**. Las **G** están en las posiciones 1, 2, 5, 8 y 9, así que $n_G = 5$; las **C**, en las posiciones 3 y 6, así que $n_C = 2$. Por la ecuación (1.14),

$$S = \frac{5 - 2}{5 + 2} = \frac{3}{7} \approx 0,43.$$

El saldo acumulado, base a base, es 1, 2, 1, 1, 2, 1, 1, 2, 3, 3: sube con cada **G**, baja con cada **C** y se queda quieto con **A** y **T**. Su valor final, $C(10) = 3$, es exactamente $n_G - n_C$. En la hebra complementaria, **TCCAGCTGCC**, se obtiene $n_G = 2$, $n_C = 5$ y $S = -3/7$.

gc_skew.py

```
import numpy as np

def sesgo_ventanas(x: str, w: int = 10_000) -> np.ndarray:
    """Sesgo (G-C)/(G+C) en ventanas contiguas de w bases."""
    a = np.frombuffer(x.encode("ascii"), dtype=np.uint8)
    n = len(a) // w
    a = a[: n * w].reshape(n, w)
    g = (a == ord("G")).sum(axis=1)
    c = (a == ord("C")).sum(axis=1)
    return (g - c) / (g + c)

def sesgo_acumulado(x: str) -> np.ndarray:
    """Saldo C(j): +1 por cada G, -1 por cada C (ecuación 1.15)."""
    a = np.frombuffer(x.encode("ascii"), dtype=np.uint8)
    paso = (a == ord("G")).astype(np.int32) - (a == ord("C"))
    return np.cumsum(paso)

C = sesgo_acumulado(ecoli)          # ecoli: str con 4 641 652 pb
ori_pred = int(np.argmin(C)) + 1    # coordenada 1-based
ter_pred = int(np.argmax(C)) + 1
```

1.3.5 Encontrar *oriC* en *E. coli*

Apliquemos el método al genoma de referencia de *E. coli* K-12 MG1655 (NC_000913.3, 4 641 652 pb). Según su propia anotación, el origen de replicación (*feature rep_origin*) ocupa las posiciones 3 925 744–3 925 975. No hay una anotación de *ter* como tal, pero sí del gen *tus* (1 684 259–1 685 188), cuyo producto se une a los sitios de terminación: es un buen marcador de esa región.

La figura 1.11 muestra el resultado. El sesgo por ventana es pequeño (en promedio +0,032 en un replicor y −0,033 en el otro) y ruidoso: 224 de las 464 ventanas son positivas, casi la mitad, pero no están mezcladas al azar sino agrupadas en dos bloques. El sesgo acumulado convierte ese patrón en una curva inequívoca. Su mínimo global está en la posición 3 925 597: a solo 262 pb del centro del *oriC* anotado, un error del 0,006 % del genoma. Su máximo está en 1 552 389, a 132 kb del gen

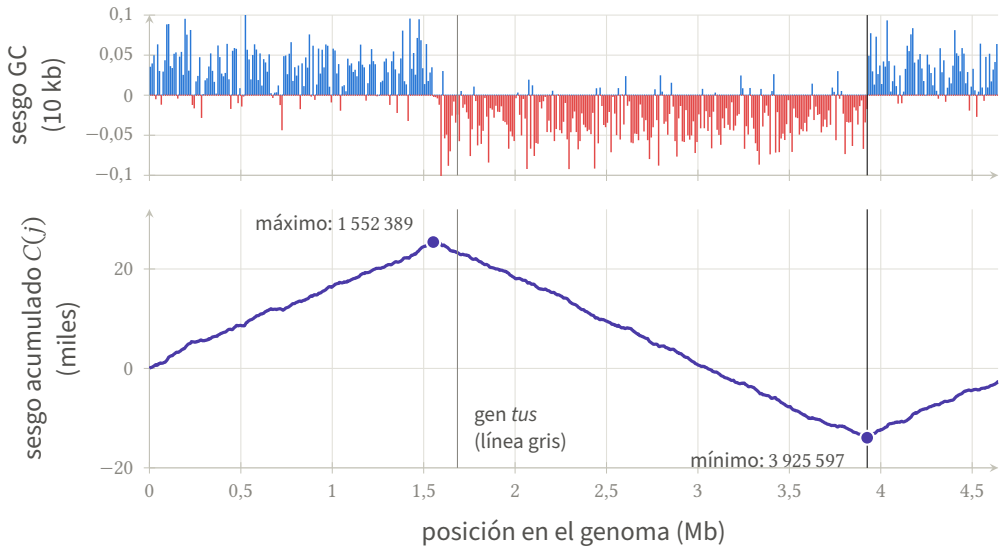


Figura 1.11. El sesgo GC de *E. coli* K-12 (NC_000913.3). **Arriba:** sesgo en 464 ventanas de 10 kb; es positivo en el replicor que va de *oriC* a *ter* en el sentido de la numeración (pasando por la posición 0) y negativo en el otro, aunque cada ventana individual es ruidosa. **Abajo:** el sesgo acumulado de la ecuación (1.15), que integra la señal y dibuja una V invertida limpia. Su mínimo (3 925 597) cae a 262 pb del centro del *oriC* anotado (línea negra); su máximo (1 552 389) está a 132 kb del gen *tus*, en la región de terminación.

tus; los dos replicores miden 2,37 y 2,27 Mb, casi iguales, como corresponde a dos horquillas que avanzan a velocidades parecidas. Con un cálculo de una línea sobre la secuencia, sin pipetas ni geles, hemos localizado el origen de replicación.

Ejemplo 1.5. (¿Cuadra la amplitud de la V?)

La ecuación (1.16) permite predecir cuánto desciende el saldo entre el máximo y el mínimo. El arco que va de *ter* ($\approx 1\,552\,389$) a *oriC* ($\approx 3\,925\,597$) mide 2 373 208 pb, el contenido GC es $g = 0,5079$ y el sesgo medio de ese replicor es $s = -0,0327$. El descenso esperado es

$$|\Delta C| \approx g |s| \Delta j = 0,5079 \times 0,0327 \times 2\,373\,208 \approx 39\,400.$$

El observado es $C_{\text{máx}} - C_{\text{mín}} = 25\,418 - (-13\,943) = 39\,361$. La coincidencia, mejor que el 0,2 %, confirma que la V es la suma de un sesgo pequeño y casi

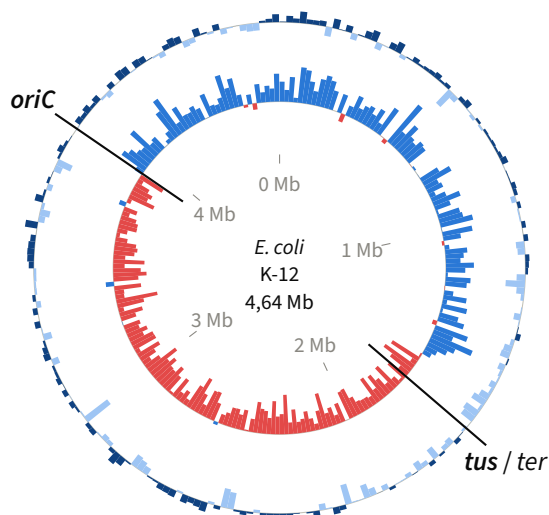


Figura 1.12. El cromosoma de *E. coli* K-12 como un reloj, en ventanas de 20 kb. **Anillo interno:** sesgo GC (azul, $G > C$, hacia fuera; rojo, $C > G$, hacia dentro). **Anillo externo:** desviación del contenido GC de cada ventana respecto a la media del genoma (azul oscuro hacia fuera, más GC; azul claro hacia dentro, menos GC). El sesgo divide el círculo en dos mitades de signo opuesto cuyas fronteras coinciden con *oriC* y con la región de *tus*. El contenido GC no muestra ese patrón: es una propiedad de la molécula, no de las hebras. Sus caídas bruscas (ventanas con hasta un 41 % de GC) señalan regiones de composición anómala, como las islas genómicas.

constante a lo largo de todo el replicor, y no el efecto de unas pocas regiones extremas.

1.3.6 El genoma como un reloj

Como el cromosoma es circular, la forma natural de representarlo es como la esfera de un reloj, con la posición o arriba y avanzando en el sentido de las agujas (figura 1.12). Este tipo de mapa, con anillos concéntricos para distintas pistas, es el formato estándar para presentar genomas bacterianos.

1.3.7 Señal contra ruido: el tamaño de la ventana

¿Por qué el sesgo por ventana es tan ruidoso, y cuánto hay que agrandar la ventana para ver la señal? Supongamos que una ventana contiene m bases que son G o C y que no hay ningún sesgo real: cada una es G con probabilidad $1/2$, indepen-

dientemente de las demás. Entonces el número de guaninas es $K \sim \text{Bin}(m, 1/2)$, y el sesgo es $S = (K - (m - K))/m = (2K - m)/m$.

Teorema 1.4. (*Ruido del sesgo en una ventana*)

Sin sesgo real, $E[S] = 0$ y

$$\text{DE}(S) = \frac{2}{m} \text{DE}(K) = \frac{2}{m} \sqrt{m \cdot \frac{1}{2} \cdot \frac{1}{2}} = \frac{1}{\sqrt{m}}, \quad m \approx w \cdot g. \quad (1.17)$$

Si además existe un sesgo real que varía entre ventanas con desviación típica σ_s , las varianzas se suman: $\text{DE}_{\text{obs}}^2 \approx 1/m + \sigma_s^2$.

DONDE

m	Número de bases G o C en la ventana.
w, g	Tamaño de la ventana (pb) y contenido GC del genoma.
K	Número de guaninas entre esas m bases.
$\text{DE}(S)$	Desviación estándar del sesgo debida solo al azar del muestreo.
σ_s	Variabilidad real del sesgo entre ventanas (señal biológica).

Con ventanas de 1 kb en *E. coli*, $m \approx 508$ y el ruido es $\pm 0,044$, mayor que la señal de $\pm 0,03$: la señal queda enterrada. Con ventanas de 100 kb el ruido baja a $\pm 0,0044$ y la señal destaca con claridad. La figura 1.13 compara la dispersión observada con la predicción del teorema 1.4.

La lección es general y reaparecerá a lo largo del libro: todo promedio en ventana es un compromiso entre resolución y ruido, y el tamaño de ventana adecuado es aquel en el que el ruido de muestreo, $1/\sqrt{m}$, cae por debajo de la magnitud de la señal que se quiere ver. En *E. coli*, eso exige $m \gtrsim 1/0,03^2 \approx 1100$ bases G o C, es decir, ventanas de al menos unos 2–3 kb, y en la práctica bastante más. El sesgo en la otra pareja de bases, $(n_A - n_T)/(n_A + n_T)$, también existe pero es unas tres veces más débil: en ventanas de 100 kb su magnitud media es 0,011 frente a 0,031 del sesgo GC.

 La replicación organiza todo el genoma

El sesgo de composición es solo una de las huellas de la replicación. Rocha (2004) revisa las demás: los genes, sobre todo los esenciales y los muy expresados, tienden

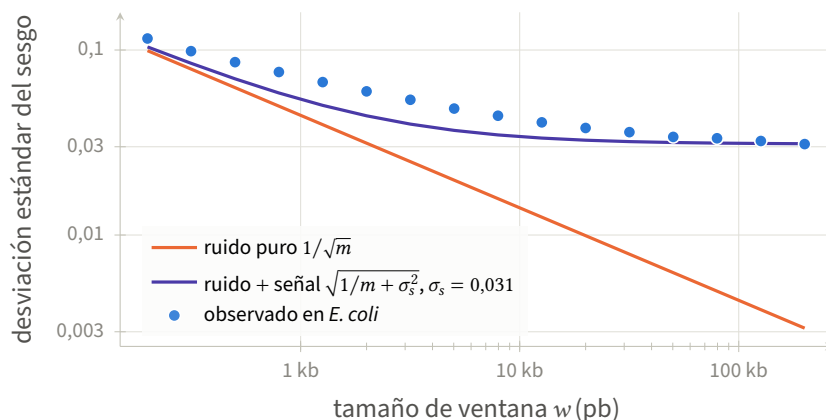


Figura 1.13. Dispersión del sesgo GC entre ventanas de *E. coli* en función del tamaño de ventana (escala log-log). Para ventanas pequeñas, la dispersión observada (puntos) está cerca del ruido binomial puro (naranja), que decrece como $1/\sqrt{m}$: casi toda la variación es azar de muestreo. Para ventanas grandes, el ruido sigue bajando pero la dispersión observada se estabiliza en torno a 0,03: es la señal de los dos replicores, que no depende del tamaño de ventana. El modelo que suma ambas varianzas (violeta) describe los extremos; los puntos intermedios quedan algo por encima porque existe además variación local real (genes de orientación particular, operones ribosomales, islas genómicas).

a estar en la hebra adelantada, probablemente para evitar choques frontales entre la ARN polimerasa y la horquilla; y en bacterias de crecimiento rápido los genes cercanos a *oriC* están en más copias que los cercanos a *ter*. Como parte del sesgo GC se debe a esa orientación de los genes, separar la contribución mutacional de la selectiva no es trivial (Frank & Lobry, 1999).

En resumen

- El dogma central describe un sistema de información: el ADN guarda, el ARN es la copia de trabajo y la proteína ejecuta. La información secuencial no sale de las proteínas; la transcripción reversa y la replicación de ARN son transferencias especiales (Crick, 1970).
- La entropía de Shannon $H = -\sum p \log_2 p$ mide la sorpresa media por símbolo: como máximo 2 bits por base en ADN. Los genomas reales están cerca del máxi-

mo; las repeticiones, muy por debajo. La entropía es el límite de la compresión sin pérdida.

- El código genético es redundante y robusto: la tercera posición del codón absorbe la mayoría de las mutaciones, y las sustituciones de aminoácido que produce una mutación puntual tienden a conservar la hidropatía mejor que en casi cualquier código alternativo.
- Un ORF va de un codón de inicio a la primera parada del mismo marco; toda secuencia tiene seis marcos. Por azar, la longitud de un tramo sin paradas es geométrica, $P(L \geq \ell) = (1 - p)^\ell$, con p determinada por la composición y el código genético del organismo.
- En *M. genitalium* (tabla 4, TGA=Trp), los genes forman una cola de ORFs largos muy por encima del modelo nulo. Un umbral de 100 codones con filtro de sombras recupera el 92 % de los genes con 93 % de precisión; los genes cortos siguen siendo difíciles.
- La replicación bidireccional desde *oriC* y la desaminación de la citosina en el molde de la hebra rezagada dejan un sesgo $S = (G - C)/(G + C)$ que cambia de signo en *oriC* y *ter*. El mínimo del sesgo acumulado localiza el *oriC* de *E. coli* a 262 pb de su posición anotada; el ruido en una ventana, $1/\sqrt{m}$, dicta el tamaño de ventana.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 1.1 ●●● Escriba a mano el complemento reverso de **ATGGCCTTAGC** y compruebe con Biopython que `Seq(x).reverse_complement()` da lo mismo. ¿Cuál es el complemento reverso del complemento reverso?
- 1.2 ●●● Calcule la entropía de Shannon de **ATATATAT** con $k = 1$ y con $k = 2$. ¿Por qué $h_2 < h_1$? ¿Qué valor tomaría la tasa de entropía h para una repetición infinita de AT?
- 1.3 ●●● Usando la ecuación (1.7), calcule p , la longitud media al azar y el percentil 99 de un ORF para un genoma con 20 %, 50 % y 72 % de GC (tabla 11, $\pi_G = \pi_C = GC/2$). Compare con los valores de la sección 1.2.
- 1.4 ●●● Repita el cálculo de robustez del código del cuaderno 1.1 usando la carga de los aminoácidos (+1 para K, R; -1 para D, E; o para el resto) en lugar

de la hidropatía. ¿Sigue siendo el código real excepcional? ¿Por qué cree que la respuesta cambia?

- 1.5 ●●● Modifique `buscar_orfs` para que, además de las coordenadas, devuelva la proteína traducida con la tabla que se le indique. Aplíquelo al genoma de SARS-CoV-2 (NC_045512.2) con $\ell_{\min} = 50$ y compare sus cinco ORFs más largos con los genes anotados. ¿Qué ocurre con *ORF1ab*, que requiere un desplazamiento ribosomal de marco?
- 1.6 ●●● Demuestre que, bajo el modelo de bases independientes, la longitud esperada de un ORF que empieza en ATG y se detiene en la primera parada es la misma que la de un tramo entre paradas. Después, verifique con una simulación que el número esperado de ORFs de al menos ℓ codones en una secuencia aleatoria de longitud L_g es aproximadamente $2L_g p q (1-p)^\ell / (p+q)$, donde q es la probabilidad de un codón de inicio. (Pista: una parada solo cierra un ORF si antes hubo un inicio.)
- 1.7 ●●● Descargue el genoma de *Bacillus subtilis* 168 (NC_000964.3) y localice su origen de replicación con el sesgo acumulado. ¿Dónde cae el mínimo? Rote la secuencia para centrar la V y estime la longitud de cada replicor. Compare la amplitud observada con la predicha por la ecuación (1.16).

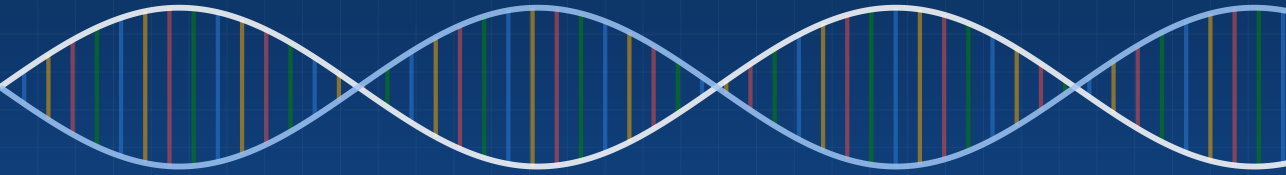
Referencias del capítulo

- Alberts, B., Heald, R., Johnson, A., Morgan, D., Raff, M., Roberts, K., & Walter, P. (2022). *Molecular Biology of the Cell* (7.^a ed.). W. W. Norton & Company.
- Berget, S. M., Moore, C., & Sharp, P. A. (1977). Spliced segments at the 5' terminus of adenovirus 2 late mRNA. *Proceedings of the National Academy of Sciences*, 74(8), 3171-3175. <https://doi.org/10.1073/pnas.74.8.3171>
- Blattner, F. R., Plunkett, G., Bloch, C. A., Perna, N. T., Burland, V., Riley, M., Collado-Vides, J., Glasner, J. D., Rode, C. K., Mayhew, G. F., et al. (1997). The Complete Genome Sequence of *Escherichia coli* K-12. *Science*, 277(5331), 1453-1462. <https://doi.org/10.1126/science.277.5331.1453>
- Crick, F. H. C. (1966). Codon-anticodon pairing: The wobble hypothesis. *Journal of Molecular Biology*, 19(2), 548-555. [https://doi.org/10.1016/S0022-2836\(66\)80022-0](https://doi.org/10.1016/S0022-2836(66)80022-0)
- Crick, F. H. C. (1968). The origin of the genetic code. *Journal of Molecular Biology*, 38(3), 367-379. [https://doi.org/10.1016/0022-2836\(68\)90392-6](https://doi.org/10.1016/0022-2836(68)90392-6)

- Crick, F. H. C., Barnett, L., Brenner, S., & Watts-Tobin, R. J. (1961). General Nature of the Genetic Code for Proteins. *Nature*, 192(4809), 1227-1232. <https://doi.org/10.1038/1921227a0>
- Crick, F. (1970). Central Dogma of Molecular Biology. *Nature*, 227(5258), 561-563. <https://doi.org/10.1038/227561a0>
- Fickett, J. W. (1982). Recognition of protein coding regions in DNA sequences. *Nucleic Acids Research*, 10(17), 5303-5318. <https://doi.org/10.1093/nar/10.17.5303>
- Frank, A. C., & Lobry, J. R. (1999). Asymmetric substitution patterns: a review of possible underlying mutational or selective mechanisms. *Gene*, 238(1), 65-77. [https://doi.org/10.1016/S0378-1119\(99\)00297-8](https://doi.org/10.1016/S0378-1119(99)00297-8)
- Fraser, C. M., Gocayne, J. D., White, O., Adams, M. D., Clayton, R. A., Fleischmann, R. D., Bult, C. J., Kerlavage, A. R., Sutton, G., Kelley, J. M., et al. (1995). The Minimal Gene Complement of *Mycoplasma genitalium*. *Science*, 270(5235), 397-404. <https://doi.org/10.1126/science.270.5235.397>
- Frederico, L. A., Kunkel, T. A., & Shaw, B. R. (1990). A sensitive genetic assay for the detection of cytosine deamination: determination of rate constants and the activation energy. *Biochemistry*, 29(10), 2532-2537. <https://doi.org/10.1021/bi00462a015>
- Freeland, S. J., & Hurst, L. D. (1998). The Genetic Code Is One in a Million. *Journal of Molecular Evolution*, 47(3), 238-248. <https://doi.org/10.1007/PL00006381>
- Gilbert, W. (1978). Why genes in pieces? *Nature*, 271(5645), 501. <https://doi.org/10.1038/271501a0>
- Glass, J. I., Assad-Garcia, N., Alperovich, N., Yooseph, S., Lewis, M. R., Maruf, M., Hutchison, C. A., Smith, H. O., & Venter, J. C. (2006). Essential genes of a minimal bacterium. *Proceedings of the National Academy of Sciences*, 103(2), 425-430. <https://doi.org/10.1073/pnas.0510013103>
- Grigoriev, A. (1998). Analyzing genomes with cumulative skew diagrams. *Nucleic Acids Research*, 26(10), 2286-2290. <https://doi.org/10.1093/nar/26.10.2286>
- Haig, D., & Hurst, L. D. (1991). A quantitative measure of error minimization in the genetic code. *Journal of Molecular Evolution*, 33(5), 412-417. <https://doi.org/10.1007/BF02103132>
- Hutchison, C. A., Chuang, R.-Y., Noskov, V. N., Assad-Garcia, N., Deerinck, T. J., Ellisman, M. H., Gill, J., Kannan, K., Karas, B. J., Ma, L., et al. (2016). Design and synthesis of a minimal bacterial genome. *Science*, 351(6280), Artículo aad6253. <https://doi.org/10.1126/science.aad6253>
- Hyatt, D., Chen, G.-L., LoCascio, P. F., Land, M. L., Larimer, F. W., & Hauser, L. J. (2010). Prodigal: prokaryotic gene recognition and translation initiation site identification. *BMC Bioinformatics*, 11, Artículo 119. <https://doi.org/10.1186/1471-2105-11-119>
- Koonin, E. V., & Novozhilov, A. S. (2009). Origin and evolution of the genetic code: The universal enigma. *IUBMB Life*, 61(2), 99-111. <https://doi.org/10.1002/iub.146>

- Kyte, J., & Doolittle, R. F. (1982). A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1), 105-132. [https://doi.org/10.1016/0022-2836\(82\)90515-0](https://doi.org/10.1016/0022-2836(82)90515-0)
- Lobry, J. R. (1996). Asymmetric substitution patterns in the two DNA strands of bacteria. *Molecular Biology and Evolution*, 13(5), 660-665. <https://doi.org/10.1093/oxfordjournals.molbev.a025626>
- Nirenberg, M., & Leder, P. (1964). RNA Codewords and Protein Synthesis. *Science*, 145(3639), 1399-1407. <https://doi.org/10.1126/science.145.3639.1399>
- Nirenberg, M. W., & Matthaei, J. H. (1961). The dependence of cell-free protein synthesis in *E. coli* upon naturally occurring or synthetic polyribonucleotides. *Proceedings of the National Academy of Sciences*, 47(10), 1588-1602. <https://doi.org/10.1073/pnas.47.10.1588>
- Okazaki, R., Okazaki, T., Sakabe, K., Sugimoto, K., & Sugino, A. (1968). Mechanism of DNA chain growth. I. Possible discontinuity and unusual secondary structure of newly synthesized chains. *Proceedings of the National Academy of Sciences*, 59(2), 598-605. <https://doi.org/10.1073/pnas.59.2.598>
- Rocha, E. P. C. (2004). The replication-related organization of bacterial genomes. *Microbiology*, 150(6), 1609-1627. <https://doi.org/10.1099/mic.0.26974-0>
- Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Shine, J., & Dalgarno, L. (1974). The 3'-Terminal Sequence of *Escherichia coli* 16S Ribosomal RNA: Complementarity to Nonsense Triplets and Ribosome Binding Sites. *Proceedings of the National Academy of Sciences*, 71(4), 1342-1346. <https://doi.org/10.1073/pnas.71.4.1342>
- Temin, H. M., & Mizutani, S. (1970). Viral RNA-dependent DNA Polymerase: RNA-dependent DNA Polymerase in Virions of Rous Sarcoma Virus. *Nature*, 226(5252), 1211-1213. <https://doi.org/10.1038/2261211a0>
- Watson, J. D., & Crick, F. H. C. (1953). Molecular Structure of Nucleic Acids: A Structure for Deoxyribose Nucleic Acid. *Nature*, 171(4356), 737-738. <https://doi.org/10.1038/171737a0>
- Yamao, F., Muto, A., Kawauchi, Y., Iwami, M., Iwagami, S., Azumi, Y., & Osawa, S. (1985). UGA is read as tryptophan in *Mycoplasma capricolum*. *Proceedings of the National Academy of Sciences*, 82(8), 2306-2309. <https://doi.org/10.1073/pnas.82.8.2306>

Secuencias, formatos y bases de datos



La buena gestión de los datos no es un fin en sí misma, sino el conducto clave que lleva al descubrimiento y la innovación, y a la posterior integración y reutilización de datos y conocimiento por parte de la comunidad.

— Wilkinson et al. (2016)

● Al terminar este capítulo usted podrá

- Reconocer la pregunta que responde cada formato (FASTA, FASTQ, GenBank, GFF3/GTF, BED, SAM/BAM/CRAM y VCF) y ubicarlo en el flujo de un análisis genómico.
- Convertir entre calidades Phred, probabilidades de error y caracteres ASCII, y calcular los errores esperados de una lectura.
- Traducir sin errores entre coordenadas 1-based cerradas y 0-based semiabiertas, en ambas hebras.
- Interpretar una CIGAR, descomponer una FLAG bit a bit y explicar cómo BGZF y los índices permiten el acceso aleatorio a archivos de cientos de gigabytes.

- Consultar NCBI, Ensembl, UniProt y PDB desde Python mediante APIs REST, respetando límites de tasa, registrando versiones e identificadores estables y aplicando los principios FAIR.

Una secuencia de ADN, por sí sola, es apenas una cadena de letras. Lo que la convierte en un *dato científico* es todo lo que la acompaña: de qué organismo proviene, con qué confianza se leyó cada base, en qué lugar del genoma cae, qué genes la atraviesan y en qué se diferencia de la referencia. Cada una de esas capas de información se guarda en un formato de archivo distinto, y cada formato se deposita en alguna de las grandes bases de datos públicas que, desde los años ochenta, sostienen la biología moderna. Quien no domina ese ecosistema pierde horas (o semanas) persiguiendo errores que no son biológicos sino tipográficos: una coordenada desplazada en una posición, una codificación de calidad mal interpretada, un identificador sin versión.

Este capítulo tiene dos partes. En la primera (sección 2.1) estudiaremos los formatos como lo que realmente son: *contratos* entre programas, con una gramática precisa y, detrás de ella, un poco de matemática (logaritmos para la calidad, álgebra de intervalos para las coordenadas, aritmética binaria para las banderas, un índice jerárquico para el acceso aleatorio). En la segunda (sección 2.2) saldremos a buscar datos reales: seguiremos al gen humano *TP53*, el llamado “guardián del genoma”, a través de NCBI, Ensembl, UniProt y PDB, y aprenderemos a hacerlo de forma reproducible, respetuosa con los servidores y trazable.

2.1 El idioma de los archivos

 Colab · 2.1 · Formatos de archivo

 [abrir notebook](#)

Una cocina profesional

En la cocina de un buen restaurante, la receta está escrita en un cuaderno, los ingredientes llegan en cajas etiquetadas con fecha de vencimiento y grado de frescura, el plan de la noche está en una pizarra y los comentarios de los comensales se anotan en un libro aparte. Nadie mezclaría todo en una sola hoja: cada soporte responde a una pregunta distinta y lo leen personas distintas. Pero todos comparten un vocabulario (el nombre del plato, el número

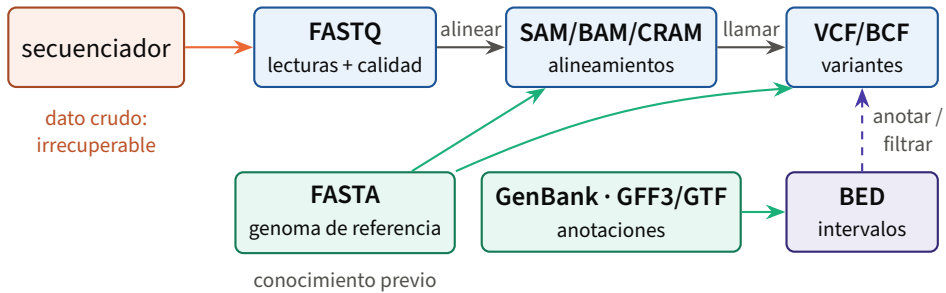


Figura 2.1. La vida de los datos en un análisis de resecuenciación. En azul, los archivos que el propio análisis produce, de izquierda a derecha cada vez más resumidos: de cientos de gigabytes de lecturas a unos pocos megabytes de variantes. En verde, el conocimiento previo (la referencia y sus anotaciones), que se descarga de las bases de datos de la sección 2.2. BED es el formato “comodín” para describir regiones con las que filtrar o anotar. Todo el sistema depende de que los nombres de cromosoma y las coordenadas signifiquen lo mismo en cada archivo.

de mesa) que permite cruzarlos. Los formatos bioinformáticos funcionan igual: uno guarda la receta (la secuencia de referencia), otro los ingredientes con su frescura (las lecturas y su calidad), otro el plan (las anotaciones), otro dónde terminó cada ingrediente (los alineamientos) y otro las quejas (las variantes). El vocabulario común son los nombres de cromosomas y las coordenadas.

2.1.1 Por qué hay tantos formatos

La figura 2.1 resume la vida de los datos en un análisis típico de resecuenciación. El único archivo que “sale de la máquina” es el FASTQ; todo lo demás es el producto de un cálculo (alinear, comparar, anotar) o conocimiento previo (el genoma de referencia y sus genes). De esa observación se desprende una regla práctica: los archivos intermedios se pueden regenerar, los FASTQ crudos no. Por eso se archivan en repositorios públicos como el *Sequence Read Archive* (Leinonen et al., 2011).

Casi todos estos formatos son *de texto delimitado*: una línea por registro, campos separados por tabuladores, encabezados que empiezan con un carácter especial. Esa elección, heredada de la filosofía de Unix, tiene un precio (archivos grandes) y una enorme ventaja: se pueden inspeccionar con `head`, filtrar con `grep` y procesar con `awk` sin ningún programa especializado, como defiende Buffalo (2015). Cuando el tamaño se vuelve inmanejable, cada formato de texto tiene un gemelo binario

comprimido e indexado (BAM para SAM, BCF para VCF), cuya lectura y escritura están implementadas en una biblioteca común, HTSlib (Bonfield et al., 2021).

2.1.2 FASTA: la secuencia desnuda

El formato FASTA nació como el formato de entrada del programa de búsqueda de similitud del mismo nombre (Lipman & Pearson, 1985; Pearson & Lipman, 1988), y sobrevivió al programa porque es casi imposible de simplificar más:

genoma.fasta

```
>NC_045512.2 Severe acute respiratory syndrome coronavirus 2
ATTAAAGGTTTATACCTTCCCAGGTAACAAACCAACCAACTTTCGATCTCTTGATGATCT
GTTCTCTAAACGAACCTTTAAATCTGTGTGGCTGTCACTCGGCTGCATGCTTAGTGCACT
```

Definición 2.1. (Registro FASTA)

Un archivo FASTA es una sucesión de registros. Cada registro comienza con una línea de encabezado que empieza por >; la primera palabra tras el > (hasta el primer espacio) es el *identificador* y el resto es una descripción libre. Las líneas siguientes, hasta el próximo > o el fin del archivo, se concatenan para formar la secuencia, escrita con el alfabeto IUPAC de nucleótidos o de aminoácidos.

Dos detalles de esta definición causan más errores de los que parece. El primero es que muchas herramientas cortan el identificador en el primer espacio: si el encabezado es >chr1 Homo sapiens, el cromosoma se llamará chr1 en el BAM, y un archivo de anotaciones que diga 1 (la convención de Ensembl) no casará con él. El segundo es que el FASTA no tiene un ancho de línea obligatorio, pero los índices (.fai) que permiten extraer una región sin leer el archivo completo sí exigen que todas las líneas de un registro tengan la misma longitud. Con líneas de w caracteres, la base x (0-based) de un registro cuya secuencia empieza en el byte o se encuentra en el byte

$$b(x) = o + \left\lfloor \frac{x}{w} \right\rfloor (w + 1) + (x \bmod w), \quad (2.1)$$

DONDE

$b(x)$	Posición en bytes, dentro del archivo, del carácter que codifica la base x .
o	Desplazamiento (en bytes) donde empieza la primera línea de secuencia del registro; se guarda en el .fa.i.
w	Número de bases por línea; el +1 cuenta el salto de línea \n.
x	Posición de la base dentro de la secuencia, contando desde o .

Es decir: saltamos $\lfloor x/w \rfloor$ líneas completas, cada una de $w + 1$ bytes, y avanzamos $x \bmod w$ caracteres dentro de la última. Si una sola línea tiene otra longitud, la fórmula apunta al lugar equivocado; por eso `samtools faidx` se niega a indexar archivos irregulares. El genoma de SARS-CoV-2 (29 903 nt) escrito con $w = 60$ ocupa $\lceil 29\,903/60 \rceil = 499$ líneas de secuencia.

2.1.3 FASTQ: la secuencia con su nivel de confianza

Un secuenciador no solo dice *qué* base leyó, sino *cuán seguro está*. Esa segunda información es la que nos permite, más adelante, distinguir una mutación real de un error de la máquina. El formato FASTQ, documentado formalmente por Cock et al. (2010) mucho después de volverse ubicuo, guarda cada lectura en exactamente cuatro líneas (figura 2.2): el nombre precedido de @, la secuencia, un separador + y una cadena de calidades con *un carácter por base*.

La escala Phred

La calidad no se guarda como probabilidad, sino en una escala logarítmica introducida por el programa `phred`, que asignaba bases y calidades a las trazas de los secuenciadores capilares del Proyecto Genoma Humano (Ewing & Green, 1998; Ewing et al., 1998).

Definición 2.2. (Calidad Phred)

Si p es la probabilidad de que la base asignada sea incorrecta, su calidad Phred es

$$Q = -10 \log_{10} p \quad \Longleftrightarrow \quad p = 10^{-Q/10}. \quad (2.2)$$

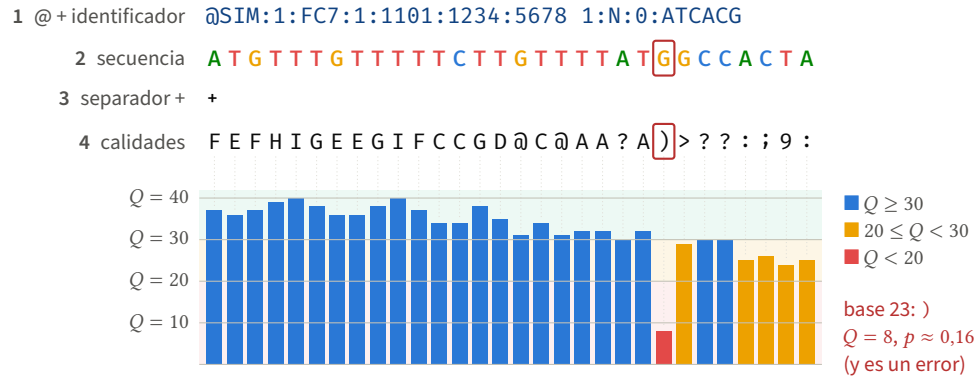


Figura 2.2. Anatomía de un registro FASTQ: las primeras 30 bases del gen S de SARS-CoV-2 leídas por un secuenciador simulado. Cada carácter de la línea 4 codifica la calidad de la base que tiene encima; las barras muestran el valor Phred decodificado ($Q = \text{código ASCII} - 33$). La calidad cae lentamente a lo largo de la lectura, como en un Illumina real. La base 23 tiene $Q = 8$ (carácter `)`: el secuenciador “avisa” que se equivoca con probabilidad $10^{-0,8} \approx 0,16$, y en efecto leyó G donde la referencia tiene T. Generada con `figuras/cap02/generar.py`.

DONDE	
p	Probabilidad estimada de error de la base (entre 0 y 1).
Q	Calidad Phred; se redondea a un entero. Más alta significa más confiable.
-10	Factor de escala: 10 unidades de Q equivalen a un factor 10 en p (como los decibelios).

¿Por qué un logaritmo? Porque lo que importa de una probabilidad de error es su *orden de magnitud*. Pasar de $p = 10^{-2}$ a $p = 10^{-3}$ es una mejora tan importante como pasar de 10^{-3} a 10^{-4} , y la escala Phred convierte ambas en el mismo salto de 10 puntos. Así, $Q = 20$ significa un error en 100 bases, $Q = 30$ uno en mil y $Q = 40$ uno en diez mil. Hay además una razón estadística más profunda: $-\log p$ es la *sorpresa* (en unidades de información) de observar un error, de modo que las calidades de bases independientes se combinan sumando logaritmos, como las puntuaciones de alineamiento del capítulo 3.

Ewing y Green (1998) no se limitaron a definir la escala: la *calibraron*. Compararon millones de bases asignadas por *phred* con secuencias de referencia de alta calidad y verificaron que, entre todas las bases con calidad Q , la fracción de errores observada

era, en efecto, cercana a $10^{-Q/10}$. Esta calibración es la que da sentido a todo lo que sigue: una calidad Phred no es un “puntaje” arbitrario, sino una predicción probabilística verificable.

Solexa: la otra escala.. Los primeros secuenciadores Solexa (luego Illumina) usaron una escala distinta, basada en la razón de probabilidades (*odds*) en lugar de la probabilidad (Cock et al., 2010):

$$Q_{\text{Sol}} = -10 \log_{10} \frac{p}{1-p}. \quad (2.3)$$

donde Q_{Sol} puede ser negativa (hasta -5 en la práctica). Despejando p en ambas definiciones y eliminándola se obtiene la conversión exacta

$$Q_{\text{Phred}} = 10 \log_{10} (10^{Q_{\text{Sol}}/10} + 1). \quad (2.4)$$

DONDE

Q_{Sol} Calidad Solexa; $p/(1-p)$ es la razón de probabilidades de error frente a acierto (*odds*).

Q_{Phred} Calidad Phred equivalente (ecuación (2.2)); siempre ≥ 0 .

Para calidades altas, $p \ll 1$ y $p/(1-p) \approx p$, de modo que ambas escalas coinciden: $Q_{\text{Phred}} = 20$ corresponde a $Q_{\text{Sol}} = 10 \log_{10} 99 = 19,96$. Solo divergen cuando la base es muy dudosa (figura 2.3): $Q_{\text{Phred}} = 3$ equivale a $Q_{\text{Sol}} \approx 0$, y el mínimo Solexa, -5 , corresponde a $p = 10^{0,5}/(1 + 10^{0,5}) \approx 0,76$, una base peor que una elección al azar entre cuatro.

De número a carácter

Para que la línea de calidades tenga la misma longitud que la secuencia, cada Q se escribe como un único carácter ASCII. Los primeros 32 códigos ASCII son invisibles (tabuladores, saltos de línea), así que se suma un desplazamiento: el estándar actual, heredado del Instituto Sanger, es *Phred*+33, de modo que $Q = 0$ se escribe ! (código 33), $Q = 30$ se escribe ? (63) y $Q = 40$ se escribe I (73). La figura 2.4 muestra la tabla completa para el rango usual.

Adivinar la codificación

El FASTQ no declara su codificación. Los archivos de Illumina anteriores a 2011 (versiones 1.3 a 1.7 de su programa) usan *Phred*+64, y todavía circulan en

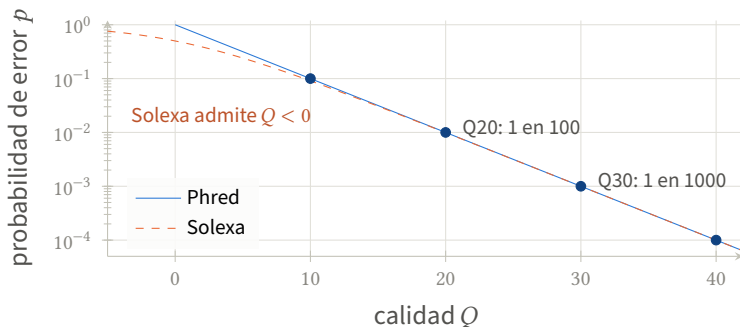


Figura 2.3. La ecuación (2.2) es una recta en escala logarítmica; cada 10 puntos de Q la probabilidad de error se divide entre 10. La escala Solexa (ecuación (2.3)) se superpone a Phred para calidades altas ($Q \geq 15$) y solo diverge cuando p se acerca a 1, donde admite valores negativos. Por eso confundir las escalas apenas afecta a las bases buenas, pero sí a las dudosas, precisamente las que un filtro de calidad debe descartar.

repositorios. Si se lee un archivo Phred+64 como Phred+33, todas las calidades aparecen infladas en 31 puntos: una base con $p = 0,1$ parece tener $p \approx 10^{-4}$, y ningún filtro de calidad la eliminará. Antes de analizar datos antiguos, compruebe el rango de caracteres presentes (figura 2.4); herramientas como FastQC lo hacen automáticamente.

Errores esperados de una lectura

¿Cuántos errores tiene una lectura? No lo sabemos, pero podemos calcular cuántos *esperamos*. Sea X_i la variable indicadora de que la base i es errónea; entonces $E[X_i] = p_i$ y, por la linealidad de la esperanza (que no exige independencia),

$$EE = E\left[\sum_{i=1}^{\ell} X_i\right] = \sum_{i=1}^{\ell} p_i = \sum_{i=1}^{\ell} 10^{-Q_i/10}. \quad (2.5)$$

Si además suponemos que los errores ocurren de forma independiente, la probabilidad de que la lectura esté completamente libre de errores es

$$\mathbb{P}(\text{sin errores}) = \prod_{i=1}^{\ell} (1 - p_i) \approx e^{-EE}, \quad (2.6)$$

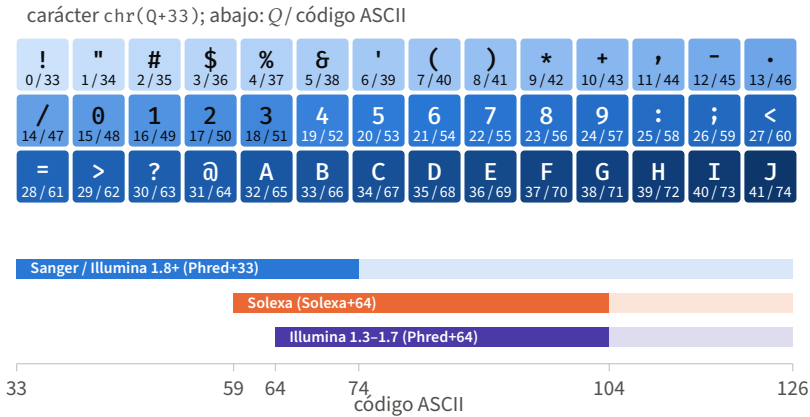


Figura 2.4. Arriba: la codificación Phred+33, carácter por carácter, coloreada desde calidades bajas (claro) hasta altas (oscuro). Abajo: los tres “dialectos” históricos de FASTQ (Cock et al., 2010). La banda clara es el rango permitido por cada codificación; la banda oscura, el rango que aparece en la práctica. Los rangos típicos se solapan entre los códigos 64 y 74 (@–J): un archivo cuyas calidades caen todas ahí es ambiguo, pero basta un solo carácter por debajo de ; (código 59) para concluir que es Phred+33.

DONDE

ℓ	Longitud de la lectura (número de bases).
X_i	Indicadora del evento “la base i es incorrecta” (vale 0 o 1).
p_i, Q_i	Probabilidad de error y calidad Phred de la base i .
EE	Errores esperados (<i>expected errors</i>) de la lectura.

donde la aproximación usa $\ln(1 - p) \approx -p$ para p pequeño: el número de errores es aproximadamente una variable de Poisson de media EE. Esta cantidad resume una lectura mucho mejor que la calidad *media*, porque la media de Q es la media de logaritmos y esconde las pocas bases malas que dominan la suma de probabilidades.

Ejemplo 2.1. (Errores esperados, a mano y en la figura 2.2)

Una lectura corta tiene calidades $Q = (40, 30, 20, 20, 10)$. Sus probabilidades de error son $(10^{-4}, 10^{-3}, 10^{-2}, 10^{-2}, 10^{-1})$ y

$$EE = 0,0001 + 0,001 + 0,01 + 0,01 + 0,1 = 0,1211.$$

Casi todo el valor procede de la única base con $Q = 10$. Su calidad media es 24, que “parece” buena, y sin embargo la probabilidad de que la lectura sea perfecta es solo $\prod(1 - p_i) = 0,8811$ (la aproximación $e^{-0,1211} = 0,8859$ es buena).

En la lectura de 30 bases de la figura 2.2 la calidad media es 32,5, pero $EE = 0,183$ y $P(\text{sin errores}) = 0,821$: la base 23 ($Q = 8$, $p = 0,158$) aporta por sí sola el 87 % de los errores esperados.

La figura 2.5 aplica estas ideas a una corrida simulada de 4000 lecturas de 150 nt, generada con el mismo modelo del notebook: una calidad media que cae de 36,9 en el primer ciclo a 24,1 en el último, más un 6 % de *clusters* defectuosos cuya calidad se desploma en la cola. Los errores se introdujeron exactamente con la probabilidad que dicta su calidad, así que conocemos la verdad: la tasa real de error resultó ser de 0,20 % (0,304 errores por lectura) y la media de EE, 0,296. El panel derecho muestra que los errores esperados predicen los observados, lo que solo ocurre si las calidades están bien calibradas. En datos reales no siempre es así, y por eso existen métodos de recalibración de calidades que estudiaremos junto con la llamada de variantes.

```
fastq_ee.py

import math
from Bio import SeqIO                                     # Biopython >= 1.83

def expected_errors(quals):
    return sum(10 ** (-q / 10) for q in quals)

kept = []
for rec in SeqIO.parse("lecturas.fastq", "fastq"): # Phred+33
    q = rec.letter_annotations["phred_quality"]
    ee = expected_errors(q)
    if ee <= 1.0:                                         # filtro por EE
        kept.append(rec)
    p0 = math.prod(1 - 10 ** (-x / 10) for x in q)
print(len(kept), "lecturas con EE <= 1")
SeqIO.write(kept, "filtradas.fastq", "fastq")
```

Biopython (Cock et al., 2009) decodifica las calidades según el nombre del formato: "fastq" (o "fastq-sanger") para Phred+33, "fastq-illumina" para Phred+64 y "fastq-solexa" para Solexa+64, y convierte entre ellos con la ecuación (2.4) al escribir.

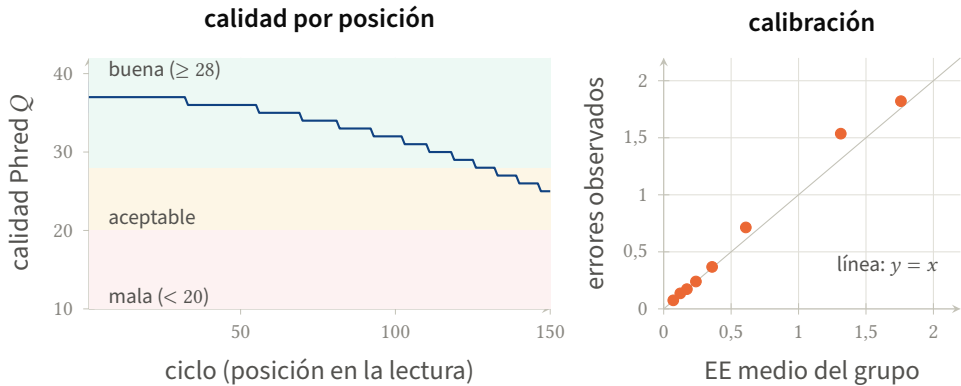


Figura 2.5. Izquierda: calidad por ciclo de 4000 lecturas simuladas, al estilo de FastQC. La línea es la mediana; la banda oscura contiene el 50 % central de las lecturas y la clara el 80 %. La caída hacia el final refleja el desfase progresivo de las moléculas dentro de cada *cluster*; el ensanchamiento de la banda inferior se debe a los *clusters* defectuosos. Esta forma es la que justifica recortar las colas antes de alinear. Derecha: las lecturas se agrupan por su EE (ecuación (2.5)) y se compara el EE medio de cada grupo con el número medio de errores realmente introducidos. Los puntos caen sobre la diagonal: con calidades calibradas, EE es un predictor insesgado del número de errores.

2.1.4 GenBank y el sistema INSDC

Un FASTA dice *qué* es la secuencia; un registro GenBank dice además *qué hay en ella*. El formato, con su encabezado (LOCUS, DEFINITION, ACCESSION, VERSION, SOURCE, referencias bibliográficas), su tabla de características (FEATURES: genes, CDS, UTR, péptidos maduros, con sus coordenadas y calificadores) y la secuencia al final (ORIGIN), es el formato nativo de GenBank, la base de datos pública de secuencias del NCBI (Sayers, Cavanaugh et al., 2025). GenBank intercambia sus datos diariamente con el *European Nucleotide Archive* (ENA) y con el *DNA Data Bank of Japan* (DDBJ): las tres forman la *International Nucleotide Sequence Database Collaboration* (INSDC), de modo que una secuencia depositada en cualquiera de ellas aparece en las tres con el mismo número de acceso (Arita et al., 2021).

fragmento de NC_045512.2.gb

```
gene          21563..25384
              /gene="S"
CDS           21563..25384
```

```

/gene="S"
/codon_start=1
/product="surface glycoprotein"
/protein_id="YP_009724390.1"

```

Las coordenadas de GenBank son *1-based e inclusivas*: el gen S ocupa las bases 21 563 a 25 384, ambas incluidas, es decir, $25\,384 - 21\,563 + 1 = 3822$ nucleótidos (1273 codones más el de terminación). Las localizaciones admiten una pequeña gramática: `complement(100..200)` para la hebra negativa, `join(1..50,80..120)` para exones separados, `<1..300` para un extremo incompleto. Biopython la traduce a objetos `SimpleLocation` y `CompoundLocation`, que, como veremos enseguida, usan internamente coordenadas *o-based*.

2.1.5 Coordenadas: el error de uno

Aquí está la fuente de errores más famosa de la bioinformática. Existen dos convenciones para numerar posiciones en una secuencia, y ambas son correctas a su manera (figura 2.6).

Definición 2.3. (Sistemas de coordenadas)

En el sistema *1-based cerrado*, las posiciones numeran *bases*: la primera base es la 1 y un intervalo $[s_1, e_1]$ incluye ambos extremos. Lo usan GenBank, GFF3/GTF, SAM (columna POS), VCF y los navegadores genómicos. En el sistema *o-based semiabierto*, las posiciones numeran los *bordes entre bases*, como las marcas de una regla: el primer borde es el 0 y un intervalo $[s_0, e_0]$ va del borde s_0 al borde e_0 , incluyendo la base que sigue a s_0 pero no la que sigue a e_0 . Lo usan BED, el formato binario BAM, Python (`seq[s:e]`) y la mayoría de las bibliotecas.

La conversión entre ambos sistemas se deduce de la figura. La base número s_1 (1-based) está a la derecha del borde $s_1 - 1$, y la última base del intervalo, e_1 , termina en el borde e_1 :

$$s_0 = s_1 - 1, \quad e_0 = e_1, \quad \text{longitud} = e_0 - s_0 = e_1 - s_1 + 1. \quad (2.7)$$

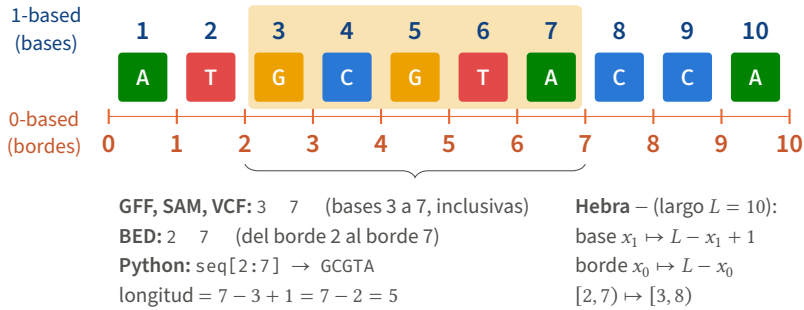


Figura 2.6. Las dos convenciones de coordenadas sobre la misma secuencia. Arriba, en azul, se numeran las bases (1-based); abajo, en naranja, los bordes entre bases, como en una regla (0-based). El intervalo resaltado, GCGTA, se escribe 3 7 en GFF y 2 7 en BED: el inicio cambia en una unidad, el final no. A la derecha, la regla para pasar a la hebra complementaria: en el sistema de bordes basta reflejar ($x_0 \mapsto L - x_0$) e intercambiar los extremos.

DONDE

s_1, e_1	Primera y última base del intervalo en el sistema 1-based cerrado (GFF, VCF, SAM).
s_0, e_0	Bordes inicial y final del intervalo en el sistema 0-based semiabierto (BED, Python).

¿Por qué los programadores prefieren el sistema semiabierto? Porque su álgebra es más limpia. La longitud es una resta directa, sin el +1 que tantas veces se olvida; dos intervalos contiguos comparten el borde ($[0, 5)$ y $[5, 10)$ ni se solapan ni dejan hueco); un intervalo vacío se representa naturalmente como $[s, s)$, lo que permite describir el *punto* donde ocurre una inserción; y la condición de solapamiento es simétrica:

$$[a, b) \cap [c, d) \neq \emptyset \iff \max(a, c) < \min(b, d), \quad (2.8)$$

$$|[a, b) \cap [c, d)| = \max(0, \min(b, d) - \max(a, c)). \quad (2.9)$$

DONDE

$[a, b), [c, d)$	Dos intervalos semiabiertos en el mismo cromosoma.
$ \cdot $	Longitud (número de bases) de la intersección.

Esta es la operación elemental de herramientas como BEDTools (Quinlan & Hall, 2010), que responden preguntas como “¿qué variantes caen en exones?” o “¿qué

picos de ChIP-seq solapan promotores?”. Con un índice ordenado por inicio, la búsqueda de todos los intervalos que solapan una consulta cuesta $O(\log n + k)$ en lugar de $O(n)$, donde k es el número de resultados.

Finalmente, la hebra negativa. Si una secuencia tiene longitud L , el borde x_0 de la hebra positiva corresponde al borde $L - x_0$ de su reverso complementario, de modo que $[s_0, e_0) \mapsto [L - e_0, L - s_0)$. En el sistema 1-based, la base x_1 corresponde a la base $L - x_1 + 1$. Los formatos de anotación *no* aplican esa transformación: GFF, BED y VCF expresan siempre las coordenadas sobre la hebra positiva y guardan la hebra en una columna aparte (+/-). Para obtener la secuencia de un gen en la hebra negativa se extrae el intervalo de la hebra positiva y luego se toma su reverso complementario.

Ejemplo 2.2. (El gen S en tres sistemas)

GenBank dice que el gen S de SARS-CoV-2 ocupa 21563..25384. En BED se escribe NC_045512.2 21562 25384, y en Python la secuencia es `ref[21562:25384]`, cuyas tres primeras letras son **ATG**. Si por descuido escribimos `ref[21563:25384]`, obtenemos una secuencia que empieza en **TGT**: el marco de lectura se desplaza y la traducción es basura. La longitud es $25384 - 21562 = 3822$ en ambos sistemas.

En la otra dirección, el servicio esummary del NCBI informa para *TP53* humano (hebra negativa del cromosoma 17) `chrstart = 7687489` y `chrstop = 7668420`, en coordenadas 0-based y en el sentido de lectura del gen. Sumando 1 a cada extremo y ordenándolos se obtiene 7 668 421–7 687 490, que es exactamente lo que reporta Ensembl, en 1-based, para el transcrito canónico (sección 2.2).

⚠ Tres síntomas del error de uno

1. *Traducciones sin sentido*: una proteína llena de codones de parada suele indicar un marco desplazado en una base.
2. *Alelos de referencia que no coinciden*: en un VCF correcto, la columna REF debe ser igual a la base de la referencia en POS. Si casi ninguna coincide, alguien convirtió mal las coordenadas. Es la prueba más barata y eficaz que existe.

3. *Intervalos de longitud cero o negativa*: un BED con `start = end` tras una conversión suele venir de restar 1 al final en lugar de al inicio.

2.1.6 GFF3, GTF y BED: anotaciones e intervalos

El GenBank es riquísimo, pero su estructura depende de sangrías y es incómodo de procesar. Para intercambiar anotaciones se usa el *General Feature Format*, cuya versión 3 (GFF3) es un archivo tabular de nueve columnas: `seqid`, `source`, `type`, `start`, `end`, `score`, `strand`, `phase` y `attributes`. La columna `type` no es texto libre: debe ser un término de la *Sequence Ontology* (`gene`, `mRNA`, `exon`, `CDS`, `five_prime_UTR`...), un vocabulario controlado creado precisamente para que las anotaciones de distintos proyectos significaran lo mismo (Eilbeck et al., 2005). Los atributos ID y Parent enlazan las filas y forman un árbol: un gen es padre de sus transcritos, cada transcrito es padre de sus exones y CDS.

sars.gff3

```
##gff-version 3
NC_045512.2 RefSeq gene 21563 25384 . + . ID=gene-S;Name=S
NC_045512.2 RefSeq CDS 21563 25384 . + 0 ID=cds-S;Parent=gene-S
```

El GTF (*Gene Transfer Format*, a veces llamado GFF2.5) usa las mismas nueve columnas pero con atributos obligatorios `gene_id` y `transcript_id` y una sintaxis distinta (`gene_id "TP53";`). Ensembl y GENCODE lo distribuyen porque es la entrada estándar de los programas de cuantificación de RNA-seq.

La columna `phase` merece una explicación, porque es la menos intuitiva. Un CDS partido en exones no tiene por qué empezar cada fragmento al inicio de un codón. La fase de un fragmento es el número de bases que hay que saltar desde su inicio para llegar al primer codón completo. Si antes de ese fragmento ya se recorrieron d bases codificantes,

$$\phi = (3 - d \bmod 3) \bmod 3 \in \{0, 1, 2\}. \quad (2.10)$$

DONDE

ϕ	Fase del fragmento de CDS (columna 8 del GFF3).
d	Número de bases codificantes contenidas en los fragmentos anteriores, en el sentido de la traducción.

Por ejemplo, si el primer exón codificante tiene 100 bases, $d = 100$, $100 \bmod 3 = 1$ y el segundo fragmento tiene fase $\phi = 2$: sus dos primeras bases completan el codón que quedó abierto.

El formato BED, creado para el navegador genómico de la Universidad de California en Santa Cruz (Kent et al., 2002), es aún más sencillo: tres columnas obligatorias (chrom, chromStart, chromEnd) en coordenadas o-based semiabiertas, y hasta nueve opcionales (nombre, puntuación, hebra, colores, bloques). Es el formato de las *regiones*: exones de un panel de captura, picos de ChIP-seq, regiones de baja complejidad que se deben excluir. Convertir de GFF3 a BED consiste, sencillamente, en aplicar la ecuación (2.7): restar 1 al inicio y dejar el final.

2.1.7 SAM, BAM y CRAM: dónde cayó cada lectura

Tras la secuenciación, un alineador (capítulo 3) busca en qué lugar de la referencia encaja cada lectura. El resultado se escribe en el formato *Sequence Alignment/Map* (SAM), propuesto por Li et al. (2009) en el contexto del Proyecto 1000 Genomas para poner fin a la proliferación de formatos propios de cada alineador. Un SAM tiene un encabezado (líneas que empiezan con @: @SQ para cada secuencia de referencia con su longitud, @RG para los grupos de lecturas, @PG para los programas que lo procesaron) y una fila por alineamiento con once campos obligatorios, resumidos en la tabla 2.1.

La columna MAPQ reutiliza la escala Phred para otra probabilidad, la de que la *posición* sea incorrecta: $\text{MAPQ} = -10 \log_{10} P(\text{posición errónea})$. Un MAPQ de 60 corresponde a una probabilidad de error de 10^{-6} ; un MAPQ de 0 significa que la lectura encaja igual de bien en al menos otro lugar del genoma (una repetición), y no debe usarse para llamar variantes. Cada alineador estima esta probabilidad a su manera, por lo que los valores de MAPQ no son comparables entre programas.

La CIGAR: instrucciones para colocar la lectura

La CIGAR (*Compact Idiosyncratic Gapped Alignment Report*) codifica el alineamiento como una sucesión de pares (n_k, o_k) : una longitud y una operación. La clave para leerla es saber qué operaciones *consumen* bases de la lectura, de la referencia o de ambas (tabla 2.2).

Tabla 2.1. Los once campos obligatorios de una línea de alineamiento SAM.

#	Campo	Tipo	Ejemplo	Significado
1	QNAME	texto	r2	nombre de la lectura (el mismo para las dos del par)
2	FLAG	entero	99	doce banderas codificadas en bits (figura 2.8)
3	RNAME	texto	NC_045512.2	secuencia de referencia
4	POS	entero	21608	primera base alineada, <i>1-based</i>
5	MAPQ	entero	60	confianza en la posición, en escala Phred
6	CIGAR	texto	3S9M2I8M	cómo encaja la lectura (figura 2.7)
7-9	RNEXT, PNEXT, TLEN		= 21790 332	posición de la pareja y longitud del fragmento
10	SEQ	texto	TGTTT...	bases, en la orientación de la referencia
11	QUAL	texto	FEFHI...	calidades Phred+33

Teorema 2.1. (Longitudes implícitas en una CIGAR)

Sea una CIGAR $C = (n_1, o_1) \cdots (n_K, o_K)$. La longitud de la lectura almacenada en SEQ, el tramo de referencia cubierto y la última posición alineada son

$$\ell_q = \sum_{k: o_k \in \{M, I, S, =, X\}} n_k, \quad (2.11)$$

$$\ell_r = \sum_{k: o_k \in \{M, D, N, =, X\}} n_k, \quad (2.12)$$

$$\text{END} = \text{POS} + \ell_r - 1. \quad (2.13)$$

Tabla 2.2. Operaciones de la CIGAR y qué secuencia consume cada una.

Op.	Nombre	Lect.	Ref.	Uso típico
M	alineada (coincida o no)	sí	sí	base frente a base
= / X	coincidencia / sustitución	sí	sí	versión explícita de M
I	inserción	sí	no	bases que la referencia no tiene
D	delección	no	sí	bases de la referencia ausentes en la lectura
N	salto	no	sí	intrón en RNA-seq
S	recorte suave	sí	no	extremo no alineado, presente en SEQ
H	recorte duro	no	no	extremo no alineado, ausente de SEQ

DONDE

n_k, o_k	Longitud y operación del k -ésimo elemento de la CIGAR.
ℓ_q	Número de bases de la lectura guardadas en SEQ; debe coincidir con la longitud de SEQ.
ℓ_r	Número de bases de la referencia que abarca el alineamiento.
POS, END	Primera y última posición de la referencia cubiertas (1-based, inclusivas).

La demostración es inmediata: al recorrer la CIGAR, un puntero avanza sobre la lectura y otro sobre la referencia, cada uno según la tabla. Lo importante es la consecuencia práctica: los recortes suaves *no* desplazan POS, que siempre apunta a la primera base *alineada*. En o-based semiabierto, el alineamiento ocupa $[\text{POS} - 1, \text{POS} - 1 + \ell_r]$, que es lo que devuelve `pysam` como `reference_start` y `reference_end`.

Ejemplo 2.3. (Leer una CIGAR)

Una lectura tiene $\text{POS} = 1000$ y $\text{CIGAR} = 3\text{S}40\text{M}1\text{I}20\text{M}5\text{D}30\text{M}2\text{S}$. Según las ecuaciones (2.11) y (2.12),

$$\ell_q = 3 + 40 + 1 + 20 + 30 + 2 = 96, \quad \ell_r = 40 + 20 + 5 + 30 = 95,$$

de modo que SEQ debe tener 96 caracteres y el alineamiento termina en $\text{END} = 1000 + 95 - 1 = 1094$. Las bases 1 a 3 de la lectura (recorte suave) no están alineadas; la base 4 de la lectura está frente a la posición 1000 de la

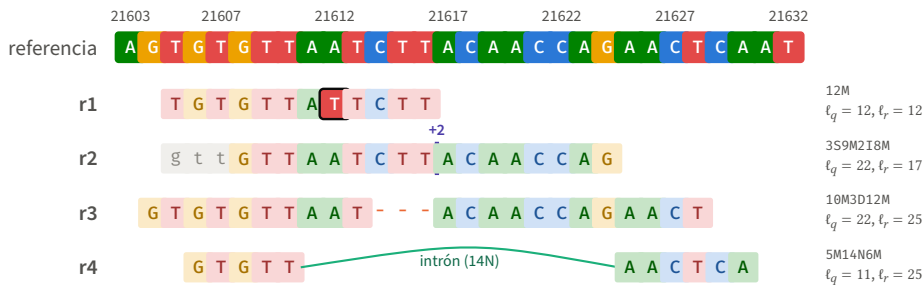


Figura 2.7. Cuatro lecturas alineadas contra el inicio del gen S de SARS-CoV-2 (posiciones 1-based arriba). **r1**: coincidencia completa con una sustitución (recuadro), que la CIGAR M no distingue. **r2**: tres bases recortadas (en minúscula, a la izquierda de POS), luego una inserción de dos bases, marcada entre dos posiciones porque no ocupa lugar en la referencia; la lectura mide 22 bases pero cubre solo 17 de la referencia. **r3**: una delección de tres bases, que sí ocupa referencia. **r4**: una lectura de RNA-seq que salta un intrón de 14 bases (N). A la derecha, ℓ_q y ℓ_r según las ecuaciones (2.11) y (2.12).

referencia; la inserción aparece entre las posiciones 1039 y 1040; la delección elimina las posiciones 1060 a 1064.

La FLAG: doce preguntas en un número

La columna FLAG guarda doce respuestas de sí o no en un único entero. Cada respuesta es un bit $b_k \in \{0, 1\}$ y el número es

$$F = \sum_{k=0}^{11} b_k 2^k, \quad b_k = \left\lfloor F/2^k \right\rfloor \text{ mód } 2. \tag{2.14}$$

DONDE	
F	Valor de la columna FLAG (entero entre 0 y 4095).
b_k	Bit k : 1 si la condición k se cumple (p. ej., b_4 : la lectura está en la hebra reversa).
2^k	Valor decimal del bit k : 1, 2, 4, ..., 2048.

Como la representación binaria de un entero es única, la descomposición también lo es: cada FLAG corresponde exactamente a una combinación de condiciones. En código, la pregunta “¿está encendido el bit de valor 16?” se formula con el Y binario: `flag & 16 != 0`. La figura 2.8 muestra las banderas más frecuentes.

	suplement. 2048	duplicado 1024	falla QC 512	secundaria 256	R2 128	R1 64	pareja – 32	hebra – 16	pareja no al. 8	no alineada 4	par correcto 2	pareada 1	
99	0	0	0	0	0	1	1	0	0	0	1	1	1 + 2 + 32 + 64 R1 de un par típico
147	0	0	0	0	1	0	0	1	0	0	1	1	1 + 2 + 16 + 128 R2 de ese par
83	0	0	0	0	0	1	0	1	0	0	1	1	1 + 2 + 16 + 64 R1 en hebra –
163	0	0	0	0	1	0	1	0	0	0	1	1	1 + 2 + 32 + 128 R2 en hebra +
4	0	0	0	0	0	0	0	0	0	1	0	0	4 no alineada
1171	0	1	0	0	1	0	0	1	0	0	1	1	1 + 2 + 16 + 128 + 1024 R2 duplicado
2064	1	0	0	0	0	0	0	1	0	0	0	0	16 + 2048 suplementaria, hebra –

Figura 2.8. Siete valores de FLAG frecuentes, descompuestos en sus doce bits (el de menor valor a la derecha, como en la escritura binaria). 99 y 147 son la pareja canónica de un par bien alineado: R1 en la hebra directa con su pareja en la reversa, y viceversa. 83/163 es la configuración espejo. 4 es una lectura sin alinear. 1171 es la R2 anterior marcada como duplicado de PCR. 2064 es un alineamiento suplementario: un trozo de una lectura quimérica que cae en otro lugar del genoma, señal típica de reordenamientos estructurales.

Ejemplo 2.4. (Construir y descomponer una FLAG)

¿Qué FLAG tiene la segunda lectura de un par, bien emparejada, alineada en la hebra reversa y marcada como duplicado? Encendemos los bits de valor 1 (pareada), 2 (par correcto), 16 (hebra reversa), 128 (R2) y 1024 (duplicado): $F = 1 + 2 + 16 + 128 + 1024 = 1171$.

A la inversa, $F = 83$: la mayor potencia de 2 que cabe es 64 (resta 19), luego 16 (resta 3), luego 2 (resta 1) y 1. Así, $83 = 64 + 16 + 2 + 1$: primera lectura del par, en la hebra reversa, par correcto. Su pareja típica es $163 = 128 + 32 + 2 + 1$.

BAM, BGZF e índices: comprimir sin perder el acceso

Un SAM de un genoma humano a 30× pesa cientos de gigabytes. El formato BAM, su versión binaria, resuelve dos problemas a la vez. El primero es el tamaño: los campos numéricos se guardan como enteros binarios, las bases se empaquetan en 4 bits y el resultado se comprime. El segundo, más sutil, es el *acceso aleatorio*. Un archivo gzip ordinario es un único flujo comprimido: para leer el último megabyte hay que descomprimir todo lo anterior. BAM usa en cambio BGZF (*Blocked GNU Zip*

Format): una concatenación de bloques gzip independientes, cada uno con a lo sumo 64 KiB de datos sin comprimir (Li et al., 2009). Cualquier programa que entienda gzip puede leerlo, pero uno que conozca la estructura puede saltar directamente a un bloque.

Para ello, cada posición del archivo se describe con un *desplazamiento virtual* de 64 bits:

$$v = c \cdot 2^{16} + u, \quad 0 \leq u < 2^{16}, \quad (2.15)$$

DONDE

v	Desplazamiento virtual, el número que se guarda en el índice.
c	Desplazamiento en bytes, dentro del archivo <i>comprimido</i> , del inicio del bloque BGZF (48 bits).
u	Desplazamiento dentro del bloque, una vez <i>descomprimido</i> (16 bits, pues el bloque mide a lo sumo 64 KiB).

Leer en v significa: saltar al byte c , descomprimir un solo bloque y avanzar u bytes. Como los desplazamientos virtuales conservan el orden del archivo, se pueden comparar como enteros ordinarios.

Falta saber *qué* v corresponde a una región del genoma. Si el BAM está ordenado por coordenada, el índice .bai lo responde con un esquema de *casillas* (*bins*) heredado del navegador de UCSC (Kent et al., 2002): el cromosoma se divide en una jerarquía de seis niveles, y cada alineamiento se asigna a la casilla más pequeña que lo contiene por completo (figura 2.9).

Teorema 2.2. (*Esquema de casillas del índice BAI*)

En el nivel $l \in \{0, \dots, 5\}$ hay 8^l casillas, cada una de 2^{29-3l} pares de bases, numeradas consecutivamente a partir de

$$\text{off}(l) = \frac{8^l - 1}{7} \in \{0, 1, 9, 73, 585, 4681\}. \quad (2.16)$$

Un intervalo $[s, e)$ se asigna a la casilla $\text{off}(l^*) + \lfloor s/2^{29-3l^*} \rfloor$, donde l^* es el nivel más profundo en el que $\lfloor s/2^{29-3l} \rfloor = \lfloor (e-1)/2^{29-3l} \rfloor$. En total hay $\sum_{l=0}^5 8^l = 37\,449$ casillas.

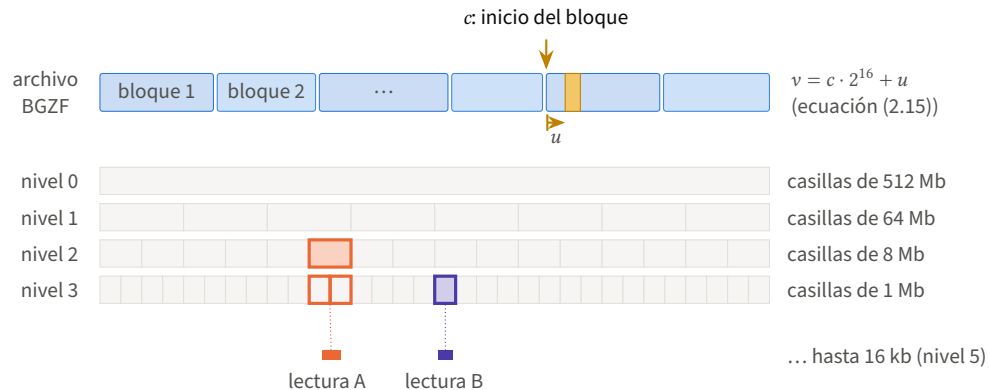


Figura 2.9. Arriba: un archivo BGZF es una cadena de bloques gzip independientes. Un desplazamiento virtual combina el byte c donde empieza un bloque comprimido con la posición u dentro del bloque descomprimido. Abajo: los primeros niveles de la jerarquía de casillas (dibujados a escala solo en su número relativo). La lectura B cabe dentro de una casilla de 1 Mb y se asigna a ella; la lectura A cruza la frontera entre dos casillas de nivel 3 y sube a la casilla de 8 Mb que las contiene. Consultar una región implica leer unas pocas casillas por nivel, nunca el archivo completo.

DONDE	
l	Nivel de la jerarquía: 0 es una casilla de 512 Mb; 5 son casillas de 16 kb.
2^{29-3l}	Tamaño de cada casilla del nivel l ($2^{29} \approx 5,4 \times 10^8$ pb en el nivel 0).
$\text{off}(l)$	Número de la primera casilla del nivel l (suma de las casillas de los niveles anteriores).
$[s, e)$	Intervalo o-based semiabierto que ocupa el alineamiento.

Para cada casilla, el índice guarda los intervalos de desplazamientos virtuales (*chunks*) donde se encuentran sus alineamientos. Para consultar una región se calculan las casillas que la solapan (a lo sumo una por nivel más las vecinas; unas pocas decenas en total), se leen solo los bloques correspondientes y se descartan los alineamientos que no solapan de verdad. Un índice lineal adicional, con una entrada por ventana de 16 kb, permite saltarse los *chunks* de casillas grandes que terminan antes de la región. El formato tabix generaliza exactamente este mecanismo a cualquier archivo de texto tabular ordenado y comprimido con BGZF, como VCF, BED o GFF (Li, 2011).

Ejemplo 2.5. (¿En qué casilla cae *TP53*?)

El transcrito canónico de *TP53* ocupa [7 668 420, 7 687 490) del cromosoma 17. En el nivel 5 sus extremos caen en casillas distintas ($\lfloor s/2^{14} \rfloor = 468$, $\lfloor (e-1)/2^{14} \rfloor = 469$); en el nivel 4 ambos dan 58, así que su casilla es $\text{off}(4) + 58 = 585 + 58 = 643$. Una lectura de 150 nt en [7 674 219, 7 674 369) cabe en la casilla de nivel 5 número $4681 + 468 = 5149$.

La última vuelta de tuerca es CRAM (Hsi-Yang Fritz et al., 2011). Su idea es que la inmensa mayoría de las bases de una lectura coinciden con la referencia, así que basta guardar la posición y las *diferencias*: una lectura perfecta de 150 nt se reduce a unos pocos números. Combinado con modelos estadísticos para las calidades y los nombres, CRAM ocupa típicamente entre un 40 y un 60 % menos que BAM; a cambio, descomprimirlo exige disponer *exactamente* de la misma referencia con la que se creó (de ahí la importancia de versionar los ensamblajes, sección 2.2). HTSlib lee y escribe SAM, BAM y CRAM de forma transparente (Bonfield et al., 2021), y *samtools* expone esas capacidades en la línea de órdenes (Danecek et al., 2021).

2.1.8 VCF: las diferencias con la referencia

Si el BAM contiene todas las lecturas, el *Variant Call Format* (Danecek et al., 2011) contiene solo sus conclusiones: las posiciones donde una o más muestras difieren de la referencia. Un VCF tiene metadatos (##) que declaran cada campo, una línea de encabezado (#CHROM) y una fila por variante con ocho columnas fijas (CHROM, POS, ID, REF, ALT, QUAL, FILTER, INFO), seguidas de una columna FORMAT y una columna por muestra.

variantes.vcf

```
##fileformat=VCFv4.2
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotipo">
##FORMAT=<ID=PL,Number=G,Type=Integer,Description="Verosimilitudes">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT S1 S2
chr17 101 . C T 50 PASS AC=3 GT:PL 0/1:50,0,90 1/1:.
chr17 102 . CAG C 38 PASS AC=1 GT:PL 0/1:. 0/0:.
```

POS es 1-based. El genotipo GT de un diploide lista dos alelos (0 = REF, 1 = primer ALT, 2 = segundo ALT...): 0/1 es heterocigoto y 1/1 homocigoto alternativo; con

| en lugar de / el genotipo está *fasado*, es decir, se sabe qué alelo está en cada cromosoma homólogo.

Verosimilitudes de genotipo

El campo PL resume la evidencia que llevó al genotipo, y es otra aplicación de la escala Phred. Supongamos que en una posición cubierta por n lecturas observamos a con el alelo alternativo y $n - a$ con el de referencia, y que cada base se lee mal con probabilidad ε (su calidad Phred). Bajo cada genotipo diploide G , cada lectura muestrea uno de los dos cromosomas al azar, de modo que

$$P(D | G) = \prod_{j=1}^n \left[\frac{1}{2} P(d_j | A_1) + \frac{1}{2} P(d_j | A_2) \right], \quad PL_G = -10 \log_{10} \frac{P(D | G)}{\max_{G'} P(D | G')}, \quad (2.17)$$

DONDE

$D = (d_1, \dots, d_n)$ Bases observadas en las n lecturas que cubren la posición.

$G = A_1/A_2$ Genotipo diploide candidato (o/o, o/1 o 1/1).

$P(d_j | A)$ Probabilidad de leer d_j si el alelo verdadero es A : $1 - \varepsilon$ si coinciden, ε si no.

PL_G Verosimilitud del genotipo en escala Phred, normalizada para que el más verosímil valga 0.

Ejemplo 2.6. (Del recuento de lecturas al campo PL)

Diez lecturas cubren la posición 101: seis dicen **C** (REF) y cuatro **T** (ALT), con $\varepsilon = 0,01$ ($Q = 20$). Entonces

$$P(D | 0/0) = 0,99^6 0,01^4 = 9,41 \times 10^{-9},$$

$$P(D | 0/1) = 0,5^{10} = 9,77 \times 10^{-4},$$

$$P(D | 1/1) = 0,01^6 0,99^4 = 9,61 \times 10^{-13}.$$

El heterocigoto es el más verosímil ($PL = 0$); para los otros, $-10 \log_{10}(9,41 \times 10^{-9}/9,77 \times 10^{-4}) = 50$ y $-10 \log_{10}(9,61 \times 10^{-13}/9,77 \times 10^{-4}) = 90$. El VCF guarda $PL=50, 0, 90$: el homocigoto de referencia es 10^5 veces menos verosímil que el heterocigoto.

A partir de los genotipos de N individuos diploides se estima la frecuencia del alelo alternativo, $\widehat{AF} = \sum_i g_i / (2N)$, donde $g_i \in \{0, 1, 2\}$ es el número de copias del alelo alternativo del individuo i ; los campos AC y AN de INFO guardan precisamente

el numerador y el denominador. Volveremos a esta cantidad al estudiar genética de poblaciones.

Una variante, muchas escrituras

Las inserciones y deleciones se escriben con una *base ancla*: la base anterior al evento, para que ni REF ni ALT queden vacíos. Pero en regiones repetitivas la misma variante admite varias escrituras. Si la referencia es **GCAGAGT** (posiciones 101 a 107) y la muestra perdió una copia de **AG**, tanto POS=102 REF=CAG ALT=C como POS=104 REF=GAG ALT=G producen la misma secuencia **GCAGT**. Dos VCF que describen la misma variante de maneras distintas no “casan” al compararlos. Tan et al. (2015) demostraron que toda variante tiene una única representación *normalizada*: la más parsimoniosa (sin bases redundantes a ambos lados) y alineada a la izquierda, en este caso la de POS=102. Normalizar (`bcftools norm`) antes de comparar o anotar variantes es una práctica obligatoria.

2.2 Bases de datos biológicas desde Python

 Colab · 2.2 · Bases de datos

 abrir notebook

Un sistema de bibliotecas

Una ciudad con buenas bibliotecas no guarda cada libro en un solo edificio. Hay un archivo nacional que conserva, sin tocarlos, todos los manuscritos depositados; hay bibliotecas especializadas cuyos bibliotecarios escogen la mejor edición de cada obra y le añaden notas; y hay catálogos que no guardan libros, sino fichas que dicen dónde está cada uno y qué otros libros lo citan. Todo el sistema funciona porque cada libro tiene un número de catálogo que nunca cambia, y porque las nuevas ediciones reciben un número de edición en lugar de borrar la anterior. Las bases de datos biológicas están organizadas exactamente así.

2.2.1 Primarias, curadas e integradoras

Cada día, laboratorios de todo el mundo producen secuencias, estructuras y mediciones. Desde los años ochenta, la comunidad acordó depositarlas en repositorios públicos, gratuitos y sincronizados. Conviene distinguir tres niveles (figura 2.10):

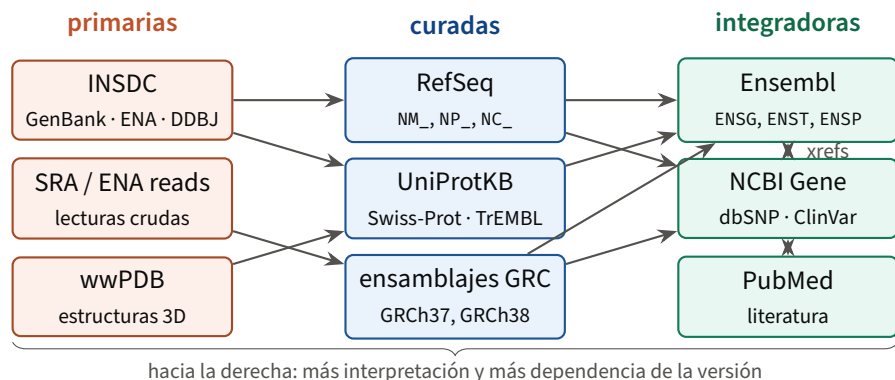


Figura 2.10. El ecosistema de bases de datos organizado por nivel de interpretación. Las flechas indican de dónde toma cada recurso sus datos; las líneas discontinuas, referencias cruzadas (*xrefs*) mantenidas entre recursos. Los archivos primarios nunca reescriben lo depositado; las bases curadas eligen y anotan una secuencia por entidad; los recursos integradores la sitúan en un genoma y la conectan con variantes, literatura y otras especies. Un mismo gen tiene un identificador distinto en cada caja.

- **Archivos primarios:** guardan lo que los autores depositaron, tal cual, con su número de acceso. Son GenBank, ENA y DDBJ para secuencias (Arita et al., 2021), el SRA para lecturas crudas (Leinonen et al., 2011) y el *Protein Data Bank* para estructuras. Nadie “corrige” un registro ajeno: si hay diez depósitos de la misma proteína, hay diez registros.
- **Bases curadas:** expertos (o procesos automáticos con reglas explícitas) eligen una secuencia representativa por entidad biológica y la anotan con evidencia. Es el caso de RefSeq, del NCBI (O’Leary et al., 2016), y de UniProtKB/Swiss-Prot (The UniProt Consortium, 2025).
- **Recursos integradores:** combinan ensamblajes, anotaciones, variantes y comparaciones entre especies, y enlazan con todo lo demás. Ensembl (Dyer et al., 2025) y el conjunto de bases del NCBI (Gene, dbSNP, ClinVar, PubMed...) (Sayers, Beck et al., 2025) son los ejemplos más usados.

2.2.2 Identificadores estables y versiones

Si hay una lección que llevarse de esta sección es esta: *un nombre de gen no es un identificador*. “*TP53*” designa, según el contexto, un locus del cromosoma 17, cualquiera de sus decenas de transcritos, varias isoformas proteicas, el gen ortólogo

de otra especie o la proteína p53 en general. Los identificadores de las bases de datos, en cambio, tienen dos propiedades que los hacen útiles:

Definición 2.4. (Identificador estable versionado)

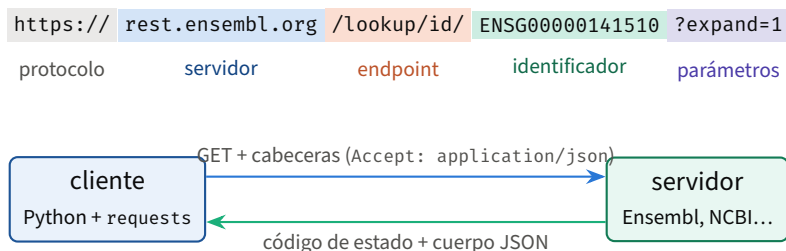
Un identificador estable es una cadena que designa siempre la misma entidad, que nunca se reasigna a otra y que se sigue resolviendo aunque la entidad quede obsoleta. Cuando el contenido de la entidad cambia, el identificador conserva su parte estable y se incrementa un número de versión: `acceso.versión`.

Así, NM_000546 es el ARNm de referencia de *TP53* en RefSeq, y NM_000546.6 es su sexta versión; cualquier cambio en la secuencia, por pequeño que sea, genera la .7. Ensembl sigue la misma lógica (ENST00000269305.9). UniProt separa dos contadores: la entrada P04637 va por la versión 315 de su *anotación*, pero por la versión 4 de su *secuencia*, que no cambia desde el 24 de noviembre de 2009. En un artículo o en un cuaderno de laboratorio hay que registrar el identificador *con* su versión, y la versión de la base de datos consultada.

Los identificadores tienen además una sintaxis reconocible: ENSG, ENST y ENSP seguidos de once dígitos en Ensembl; NM_, NP_ o NC_ en RefSeq; seis o diez caracteres alfanuméricos en UniProt; cuatro caracteres en el PDB. Esa regularidad permite validarlos con expresiones regulares antes de enviarlos a un servidor. Los de *TP53* se reúnen en la figura 2.15.

El ensamblaje también tiene versión

Las coordenadas genómicas solo tienen sentido respecto de un ensamblaje. Entre GRCh37 (2009) y GRCh38 (2013) se corrigieron errores, se cerraron huecos y se añadieron secuencias, de modo que casi todas las coordenadas cambiaron. *TP53* está en 17:7565097-7590856 en GRCh37 y en 17:7661779-7687546 en GRCh38, unos 96 682 pb más adelante; su identificador de gen es el mismo, pero su versión pasó de 11 a 21, y la del transcrito canónico, de 4 a 9. Mezclar un VCF en GRCh37 con una anotación en GRCh38 no produce ningún error de ejecución: produce resultados silenciosamente equivocados. La traslación de coordenadas entre ensamblajes (*liftover*) es posible, pero no es trivial, porque algunas regiones no tienen equivalente.



200 OK: leer el cuerpo. **400/404** pregunta mal formada o recurso inexistente: no reintentar. **429** demasiadas peticiones: esperar (Retry-After) y reintentar. **5xx** fallo del servidor: reintentar con espera creciente.

Figura 2.11. Arriba: anatomía de una URL de la API REST de Ensembl (Yates et al., 2015), que pide el gen *TP53* con sus transcritos y exones (`expand=1`). Abajo: el ciclo petición-respuesta y la interpretación de los códigos de estado más frecuentes. La distinción importante es entre errores del cliente (4xx: repetir la petición no servirá, salvo el 429) y errores transitorios (429 y 5xx: vale la pena esperar y reintentar).

De un libro de proteínas a millones de secuencias

El primer “banco de datos” de secuencias fue un libro: el *Atlas of Protein Sequence and Structure* que Margaret Dayhoff publicó desde 1965 y que, en el capítulo 3, dio origen a las matrices PAM. GenBank se fundó en 1982 en Los Álamos y pasó al NCBI a finales de esa década; el *Protein Data Bank* se creó en 1971 con apenas siete estructuras (Berman et al., 2000). En 2003 los centros que lo gestionaban en Estados Unidos, Europa y Japón formalizaron el *worldwide PDB*, un único archivo global mantenido en colaboración (Berman et al., 2003), siguiendo el modelo que la INSDC ya aplicaba a las secuencias.

2.2.3 APIs REST: preguntarle a un servidor

Casi todas las bases de datos modernas ofrecen, además de su página web, una *API REST*: una forma de consultar sus datos mediante direcciones web que devuelven texto estructurado (JSON, XML, FASTA) en lugar de páginas para humanos. Una petición tiene cuatro partes (figura 2.11): el protocolo y el servidor, el *endpoint* (qué tipo de pregunta hacemos), el recurso (sobre qué entidad) y los parámetros (cómo queremos la respuesta). El servidor responde con un *código de estado* de tres dígitos y un cuerpo.

La respuesta JSON se convierte directamente en diccionarios y listas de Python. En el caso de Ensembl, un gen contiene una lista `Transcript`; cada transcrito, una lista `Exon` y, si codifica proteína, un diccionario `Translation`.

Límites de tasa

Los servidores públicos atienden a millones de usuarios y protegen su capacidad con *límites de tasa*. Las E-utilities del NCBI admiten 3 peticiones por segundo sin registro y 10 con una clave de API gratuita; Ensembl admite en promedio unas 15 por segundo. Quien los excede recibe respuestas 429 y, si insiste, puede ver bloqueada su dirección IP (y la de todo su laboratorio, que comparte la misma).

Un límite de r peticiones por segundo impone una cota inferior al tiempo de una descarga de N peticiones:

$$T \geq \frac{N}{r}. \quad (2.18)$$

DONDE

T	Tiempo total de la descarga, en segundos.
N	Número de peticiones necesarias.
r	Tasa máxima admitida por el servidor (peticiones por segundo).

La consecuencia práctica es que el único modo de acelerar una descarga grande es reducir N : pedir muchos registros por petición. Las E-utilities lo permiten a través de su *History server*: `esearch` con `usehistory=y` guarda la lista de resultados en el servidor y devuelve dos claves (`WebEnv` y `query_key`), con las que `efetch` descarga los registros en lotes de hasta varios cientos.

Ejemplo 2.7. (Descargar 5000 proteínas)

Queremos las secuencias FASTA de 5000 proteínas del NCBI. Pidiéndolas de una en una ($N = 5000$) sin clave de API ($r = 3$), la ecuación (2.18) da $T \geq 1667$ s, casi media hora; con clave ($r = 10$), 500 s. Si en cambio usamos el *History server* y lotes de 500 registros, bastan una petición `esearch` y $\lceil 5000/500 \rceil = 10$ peticiones `efetch`: $N = 11$ y $T \geq 3,7$ s. El cuello de botella deja de ser el límite de tasa y pasa a ser el volumen de datos transferido.

Reintentos con espera exponencial

Los errores transitorios (429, 5xx, cortes de red) se tratan reintentando, pero no inmediatamente: si mil clientes reintentan a la vez tras una caída, la provocan de nuevo. La estrategia estándar es la *espera exponencial con variación aleatoria*:

$$t_k = \min(t_{\text{máx}}, t_0 2^k) + U_k, \quad U_k \sim \text{Uniforme}(0, t_0), \quad (2.19)$$

DONDE

t_k	Tiempo de espera antes del reintento k ($k = 0, 1, 2, \dots$).
t_0	Espera inicial (típicamente 1 s o el valor de la cabecera Retry-After).
$t_{\text{máx}}$	Espera máxima, para no esperar indefinidamente.
U_k	Variación aleatoria (<i>jitter</i>) que desincroniza a los clientes.

Con $t_0 = 1$ s y cinco reintentos, la espera determinista total es $1 + 2 + 4 + 8 + 16 = 31$ s. Si cada intento falla de forma independiente con probabilidad $q = 0,2$, la probabilidad de que fallen los seis (el original y los cinco reintentos) es $q^6 = 6,4 \times 10^{-5}$: la espera exponencial convierte un servicio poco fiable en uno casi perfecto a cambio de latencia ocasional.

Un cliente robusto combina estas ideas con una *caché* (no preguntar dos veces lo mismo) y un *registro de procedencia* (qué se pidió, a quién, cuándo, y si la respuesta vino del servidor o de la caché). El notebook de esta lección construye la función `fetch()` que usaremos en el resto del curso; su núcleo es el siguiente:

```
cliente.py

import time, random, requests

def get_json(url, params=None, t0=1.0, tmax=30, tries=6):
    for k in range(tries):
        r = requests.get(url, params=params, timeout=30,
                        headers={"Accept": "application/json"})
        if r.status_code == 200:
            return r.json()
        if r.status_code not in (429, 500, 502, 503, 504):
            r.raise_for_status() # error del cliente
        wait = float(r.headers.get("Retry-After", 0))
        wait = max(wait, min(tmax, t0 * 2 ** k))
        time.sleep(wait + random.uniform(0, t0))
    raise RuntimeError(f"sin respuesta tras {tries} intentos")
```

2.2.4 NCBI y las E-utilities

El NCBI agrupa más de treinta bases de datos (Nucleotide, Protein, Gene, PubMed, SRA, dbSNP, ClinVar, Taxonomy...) bajo un sistema de búsqueda común llamado Entrez (Sayers, Beck et al., 2025). Sus *Entrez Programming Utilities* son una familia de servicios que se combinan como piezas:

Utilidad	Pregunta que responde	Devuelve
esearch	¿qué registros cumplen esta búsqueda?	identificadores (UID) y recuento
esummary	¿qué dice el resumen de estos registros?	metadatos
elink	¿qué registros de otra base están ligados a estos?	UID relacionados
efetch	dame el registro completo	FASTA, GenBank, XML...

El flujo típico es esearch → esummary/elink → efetch: primero se encuentra, luego se mira, al final se descarga. Para *TP53*, esearch en la base Gene con la consulta `TP53[sym] AND human[orgn]` devuelve el UID 7157; esummary devuelve su localización (ejemplo 2.2); elink con `linkname=gene_protein_refseq` lleva a sus proteínas RefSeq, y efetch descarga `NP_000537.3` en FASTA, una proteína de 393 aminoácidos. Biopython envuelve estas llamadas en el módulo `Bio.Entrez`, que además respeta automáticamente el límite de tasa (Cock et al., 2009).

```

entrez_tp53.py

from Bio import Entrez, SeqIO

Entrez.email = "usted@universidad.edu"          # obligatorio
# Entrez.api_key = "..."                       # opcional: 10/s
h = Entrez.esearch(db="gene", term="TP53[sym] AND human[orgn]")
gene_id = Entrez.read(h)["IdList"][0]           # '7157'
h = Entrez.elink(dbfrom="gene", db="protein", id=gene_id,
                 linkname="gene_protein_refseq")
links = Entrez.read(h)[0]["LinkSetDb"][0]["Link"]
h = Entrez.efetch(db="protein", id="NP_000537.3",
                  rettype="fasta", retmode="text")
p53 = SeqIO.read(h, "fasta")
print(gene_id, len(links), p53.id, len(p53))

```

Con el parámetro `rettype=count`, esearch devuelve solo el número de registros que cumplen una búsqueda, lo que permite medir, por ejemplo, cuántos artículos por año indexa PubMed sobre un tema; el notebook de la lección usa esa consulta

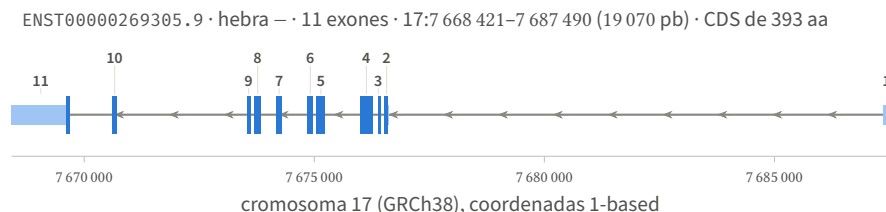


Figura 2.12. Modelo del transcrito canónico de *TP53* dibujado directamente a partir de la respuesta de la API de Ensembl. Los bloques gruesos son región codificante (CDS), los delgados son regiones no traducidas (UTR) y la línea con flechas son intrones; el gen se transcribe de derecha a izquierda porque está en la hebra negativa, así que el exón 1 es el de la derecha. El exón 1 es completamente no codificante: la proteína empieza en el exón 2. Los 11 exones suman 2512 nt, apenas un 13 % de los 19 070 pb del transcrito; el resto son intrones.

para estimar el tiempo de duplicación de la literatura sobre CRISPR (menos de dos años entre 2013 y 2020).

2.2.5 Ensembl: el gen en su genoma

Ensembl anota automáticamente los genomas de cientos de especies de vertebrados y, en su versión actual, integra además los resultados del proyecto GENCODE para el ser humano y el ratón (Dyer et al., 2025). Su API REST (Yates et al., 2015) expone casi todo su contenido; el endpoint `lookup/id` con `expand=1` devuelve el gen con todos sus transcritos y exones.

Para *TP53* en GRCh38, Ensembl informa un gen de 25 768 pb en la hebra negativa del cromosoma 17, con 40 transcritos anotados: distintos sitios de inicio, combinaciones de exones por *splicing* alternativo y transcritos que no codifican proteína. Entre ellos, uno se marca como *canónico*: ENST00000269305.9, que corresponde al ARNm de referencia NM_000546.6 de RefSeq. La figura 2.12 lo dibuja a partir de la respuesta de la API.

⚠ «La proteína de *TP53*» no existe

Con 40 transcritos anotados, la frase “la secuencia de *TP53*” es ambigua. Hay que especificar el transcrito (ENST00000269305.9 o NM_000546.6) y el ensamblaje. Las herramientas de anotación de variantes pueden describir la misma mutación con posiciones proteicas distintas según el transcrito

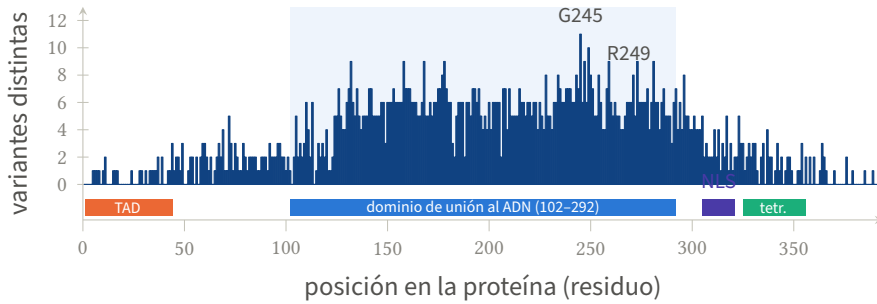


Figura 2.13. Número de variantes de un aminoácido distintas registradas en UniProtKB (P04637) en cada posición de p53. Abajo, las regiones funcionales anotadas en la misma entrada: dominio de transactivación (TAD, 1–44), dominio de unión al ADN (102–292), señal de localización nuclear bipartita (NLS) y dominio de oligomerización o tetramerización (325–356). Las variantes se concentran de forma abrumadora en el dominio de unión al ADN (fondo azul), donde están las posiciones con más variantes distintas, como G245 y R249.

elegido; por eso las recomendaciones clínicas exigen el transcrito de referencia versionado.

2.2.6 UniProt: la proteína y su conocimiento

UniProt es el recurso central para las proteínas (The UniProt Consortium, 2025). Su base de conocimiento, UniProtKB, tiene dos secciones: Swiss-Prot, revisada manualmente por curadores que resumen la literatura con evidencia explícita, y TrEMBL, anotada automáticamente y órdenes de magnitud mayor. La entrada revisada P04637 (p53 humana, 393 aminoácidos) describe sus dominios, sitios de modificación, interacciones, isoformas, estructuras y más de mil variantes naturales, casi todas mutaciones somáticas halladas en tumores. La API REST las devuelve en JSON desde <https://rest.uniprot.org/uniprotkb/P04637.json>, como una lista features con su tipo y localización.

La figura 2.13 sugiere que las variantes no están repartidas al azar. Podemos cuantificarlo con una prueba binomial, un buen ejemplo de cómo los datos de una base de datos se convierten en una afirmación estadística.

Ejemplo 2.8. (¿Se concentran las variantes en el dominio de unión al ADN?)

La entrada registra $N = 1363$ variantes naturales en 332 posiciones distintas. El dominio de unión al ADN abarca 191 de los 393 residuos, una fracción $\pi_0 = 191/393 = 0,486$. Si las variantes cayeran al azar a lo largo de la proteína, el número dentro del dominio, K , seguiría una distribución $\text{Bin}(N, \pi_0)$, con media $N\pi_0 = 662,4$ y desviación típica $\sqrt{N\pi_0(1 - \pi_0)} = 18,5$. Se observan $K = 1052$ (el 77 %), lo que da

$$z = \frac{1052 - 662,4}{18,5} = 21,1,$$

y una probabilidad exacta $\mathbb{P}(K \geq 1052) \approx 8 \times 10^{-104}$. La concentración es incuestionable. Pero cuidado con la interpretación: el resultado mezcla biología (las mutaciones que destruyen la unión al ADN son las seleccionadas en los tumores) con un *sesgo de verificación*, porque durante años muchos estudios secuenciaron solo los exones 5 a 8, que codifican ese dominio. Los datos de una base de datos reflejan también cómo se recolectaron.

2.2.7 PDB: la estructura

El *Protein Data Bank* archiva las estructuras tridimensionales de macromoléculas determinadas experimentalmente (Berman et al., 2000). Cada entrada tiene un identificador de cuatro caracteres, las coordenadas atómicas (en los formatos PDB clásico o mmCIF) y los metadatos del experimento: método, resolución, cadenas, ligandos. El portal estadounidense, RCSB PDB, ofrece además APIs de búsqueda y de datos (esta última en GraphQL), y desde 2022 muestra, junto a las estructuras experimentales, modelos computacionales predichos por inteligencia artificial, claramente separados (Burley et al., 2025).

Una búsqueda de todas las entradas cuyas cadenas remiten al acceso UniProt Po4637 devuelve 323 estructuras de p53 humana. La figura 2.14 resume su historia: 275 por difracción de rayos X, 30 por resonancia magnética nuclear y 16 por criomicroscopía electrónica. La resolución mediana es de 1,8 Å para rayos X y de 3,83 Å para criomicroscopía; la mejor alcanza 1,01 Å.

La entrada 1TSR (publicada en el PDB en enero de 1996, 2,2 Å) contiene tres copias del dominio de unión al ADN de p53 (cadenas A, B y C) unidas a una doble hélice de ADN (cadenas E y F). Su publicación (Cho et al., 1994) fue un hito, porque mostró por qué los “puntos calientes” de mutación en cáncer son los que son: residuos como R248 y R273 contactan directamente el ADN, y otros como R175, G245 y R249 sostienen la superficie de contacto. Una sola sustitución en esos lugares basta para

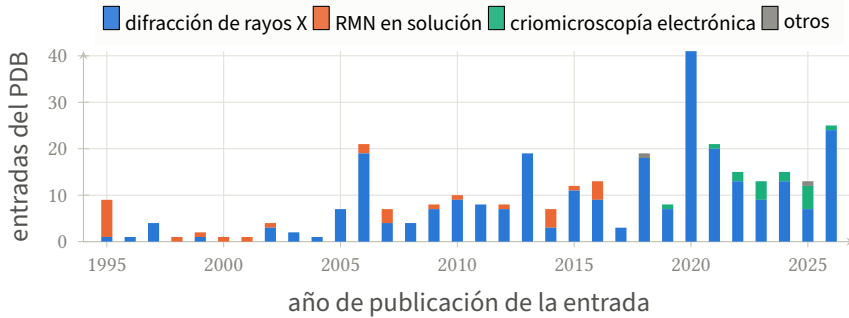


Figura 2.14. Entradas del PDB que contienen p53 humana (acceso UniProt P04637), por año de publicación y método experimental, obtenidas de la API de RCSB PDB en septiembre de 2026 (el último año está incompleto). Las primeras estructuras de mediados de los noventa, como 1TSR, resolvieron el dominio de unión al ADN; la mayoría de las posteriores son variantes de ese dominio con fármacos candidatos a “rescatar” la p53 mutada. La criomicroscopía aparece solo en los últimos años, típicamente con complejos grandes y resoluciones más modestas.

que p53 suelte el ADN o pierda su forma. Acabamos de encadenar tres bases de datos (la variante en UniProt, la posición en el genoma de Ensembl y la estructura en el PDB) para explicar un fenotipo.

2.2.8 El pasaporte de un gen y los principios FAIR

Los recursos que hemos visitado no son islas: cada uno mantiene referencias cruzadas hacia los demás. El endpoint `xrefs/id` de Ensembl, por ejemplo, devuelve para ENSG00000141510 su identificador HGNC, su UID de NCBI Gene y su número OMIM, y para la proteína ENSP00000269305 su acceso de UniProt, su proteína RefSeq y la lista de estructuras del PDB. Con esas referencias se construye el “pasaporte” de la figura 2.15.

Esta capacidad de pasar de un recurso a otro sin intervención humana no es casual. Wilkinson et al. (2016) la formularon como cuatro principios que deberían cumplir todos los datos científicos, resumidos en el acrónimo FAIR:

- **F** (*Findable*, localizables): los datos tienen identificadores globales y persistentes, y metadatos ricos indexados en un recurso consultable.
- **A** (*Accessible*, accesibles): se recuperan por su identificador mediante un protocolo abierto y estandarizado (como HTTP), y los metadatos siguen disponibles aunque los datos dejen de estarlo.

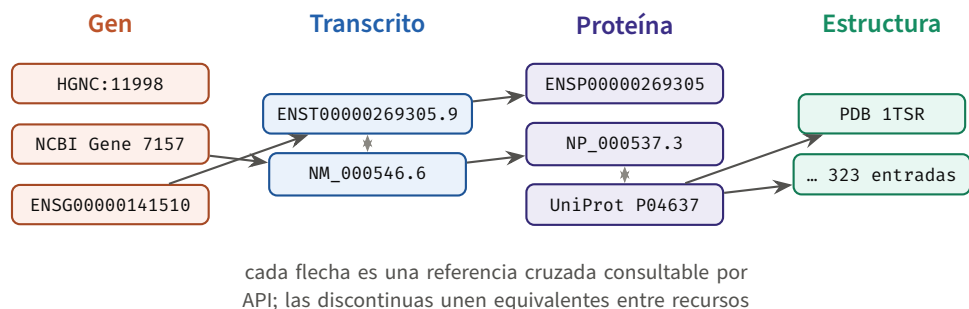


Figura 2.15. El “pasaporte” de *TP53*, construido con los identificadores obtenidos en esta sección. El nivel de detalle crece de izquierda a derecha, como en el dogma central: un gen, uno de sus transcritos, la proteína que codifica y una de sus estructuras. Cada identificador de transcrito y proteína lleva su versión; los de gen y estructura designan entidades cuyo contenido puede actualizarse sin cambiar de nombre, por lo que conviene registrar también la versión de la base de datos consultada.

- **I** (*Interoperable*, interoperables): usan lenguajes y vocabularios formales y compartidos (como la *Sequence Ontology*) e incluyen referencias a otros datos.
- **R** (*Reusable*, reutilizables): tienen una licencia clara, una procedencia detallada y cumplen los estándares de su comunidad.

Todo lo que hemos visto en este capítulo es una implementación concreta de esos principios: identificadores versionados (F), APIs REST (A), formatos y ontologías estándar (I). El cuarto principio, la reutilización, depende de nosotros: de que registremos *qué* pedimos, *a quién*, *cuándo* y en *qué versión*.

Identificador + versión + fecha + versión de la base.
Sin esos cuatro datos, un análisis no es reproducible.

☰ En resumen

- FASTA guarda secuencias; FASTQ añade una calidad Phred por base, $Q = -10 \log_{10} p$, codificada como $\text{chr}(Q+33)$. Los errores esperados, $EE = \sum 10^{-Q_i/10}$, resumen una lectura mejor que su calidad media.

- GenBank, GFF3, GTF y SAM/VCF usan coordenadas 1-based cerradas; BED, BAM y Python, 0-based semiabiertas. La conversión es $s_0 = s_1 - 1$, $e_0 = e_1$, y el REF de un VCF es la prueba de fuego.
- En SAM, la CIGAR determina la longitud de la lectura y el tramo de referencia según qué operaciones consumen cada secuencia; la FLAG codifica doce condiciones como suma de potencias de 2; MAPQ es la calidad Phred de la posición.
- BAM y CRAM comprimen en bloques BGZF; un índice jerárquico de 37 449 casillas y desplazamientos virtuales $v = c \cdot 2^{16} + u$ permite leer una región sin descomprimir todo el archivo. VCF describe variantes con genotipos y verosimilitudes (PL) en escala Phred, y debe normalizarse.
- Las bases de datos se organizan en primarias (INSDC, SRA, PDB), curadas (RefSeq, Swiss-Prot) e integradoras (Ensembl, NCBI Gene). Se consultan mediante APIs REST, respetando límites de tasa ($T \geq N/r$), reintentando con espera exponencial y guardando caché y procedencia.
- Un gen se identifica con identificadores estables *versionados* y en un ensamblaje concreto; las referencias cruzadas entre recursos materializan los principios FAIR.

Ejercicios

Dificultad:    básica    intermedia    avanzada

- 2.1**    (a) ¿Qué calidad Phred corresponde a una probabilidad de error de 0,0005? (b) ¿Con qué carácter se escribe en Phred+33? (c) Decodifique la cadena 5?I+ a calidades y probabilidades de error.
- 2.2**    Descomponga las FLAG 83, 2064 y 3844. ¿Qué filtro -F de `samtools view` excluye a la vez lecturas no alineadas, secundarias, duplicadas y suplementarias?
- 2.3**    Para la CIGAR 3S40M1I20M5D30M2S con POS=1000: (a) ¿qué posición de la referencia queda frente a la base 50 de la lectura? (b) ¿Qué intervalo 0-based semiabierto devuelve `reference_start`, `reference_end` en `pysam`? Escriba una función que, dada una CIGAR y POS, devuelva el par de listas de posiciones alineadas.

- 2.4 ●●● Usando la ecuación (2.6), calcule la calidad Phred uniforme mínima que debe tener una lectura de 150 nt para que la probabilidad de estar libre de errores sea al menos 0,9. Compare la aproximación de Poisson con el cálculo exacto.
- 2.5 ●●● Implemente la función `reg2bin(beg, end)` del teorema 2.2 y verifique los resultados del ejemplo 2.5. ¿Cuántas casillas hay que examinar como máximo para una región de 100 kb?
- 2.6 ●●● Escriba un cliente que obtenga para *BRCA1* humano su UID de NCBI Gene (`esearch`), su identificador de Ensembl (`endpoint lookup/symbol`), su transcrito canónico y su acceso de UniProt (`xrefs`), respetando el límite de tasa de cada servicio y registrando la fecha y la versión de cada respuesta.
- 2.7 ●●● Repita el ejemplo 2.6 para 20 lecturas con 3 alternativas y $\varepsilon = 0,01$, y luego con $\varepsilon = 0,05$. ¿Cómo cambia la decisión entre o/o y o/1? Explique por qué una calidad mal calibrada (figura 2.5) produce falsos heterocigotos.

Referencias del capítulo

- Arita, M., Karsch-Mizrachi, I., & Cochrane, G. (2021). The international nucleotide sequence database collaboration. *Nucleic Acids Research*, 49(D1), D121-D124. <https://doi.org/10.1093/nar/gkaa967>
- Berman, H., Henrick, K., & Nakamura, H. (2003). Announcing the worldwide Protein Data Bank. *Nature Structural & Molecular Biology*, 10(12), 980. <https://doi.org/10.1038/nsb1203-980>
- Berman, H. M., Westbrook, J., Feng, Z., Gilliland, G., Bhat, T. N., Weissig, H., Shindyalov, I. N., & Bourne, P. E. (2000). The Protein Data Bank. *Nucleic Acids Research*, 28(1), 235-242. <https://doi.org/10.1093/nar/28.1.235>
- Bonfield, J. K., Marshall, J., Danecek, P., Li, H., Ohan, V., Whitwham, A., Keane, T., & Davies, R. M. (2021). HTSlib: C library for reading/writing high-throughput sequencing data. *GigaScience*, 10(2), giaboo7. <https://doi.org/10.1093/gigascience/giab007>
- Buffalo, V. (2015). *Bioinformatics data skills: Reproducible and robust research with open source tools*. O'Reilly Media.
- Burley, S. K., Bhatt, R., Bhikadiya, C., Bi, C., Biester, A., Biswas, P., Bittrich, S., Blaumann, S., Brown, R., Chao, H., et al. (2025). Updated resources for exploring experimentally-determined PDB structures and Computed Structure Models at the RCSB Protein Data Bank. *Nucleic Acids Research*, 53(D1), D564-D574. <https://doi.org/10.1093/nar/gkae1091>

- Cho, Y., Gorina, S., Jeffrey, P. D., & Pavletich, N. P. (1994). Crystal structure of a p53 tumor suppressor-DNA complex: Understanding tumorigenic mutations. *Science*, 265(5170), 346-355. <https://doi.org/10.1126/science.8023157>
- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Cock, P. J. A., Fields, C. J., Goto, N., Heuer, M. L., & Rice, P. M. (2010). The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 38(6), 1767-1771. <https://doi.org/10.1093/nar/gkp1137>
- Danecek, P., Auton, A., Abecasis, G., Albers, C. A., Banks, E., DePristo, M. A., Handsaker, R. E., Lunter, G., Marth, G. T., Sherry, S. T., McVean, G., Durbin, R., & 1000 Genomes Project Analysis Group. (2011). The variant call format and VCFtools. *Bioinformatics*, 27(15), 2156-2158. <https://doi.org/10.1093/bioinformatics/btr330>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giaboo8. <https://doi.org/10.1093/gigascience/giab008>
- Dyer, S. C., Austine-Orimoloye, O., Azov, A. G., Barba, M., Barnes, I., Barrera-Enriquez, V. P., Becker, A., Bennett, R., Beracochea, M., Berry, A., et al. (2025). Ensembl 2025. *Nucleic Acids Research*, 53(D1), D948-D957. <https://doi.org/10.1093/nar/gkae1071>
- Eilbeck, K., Lewis, S. E., Mungall, C. J., Yandell, M., Stein, L., Durbin, R., & Ashburner, M. (2005). The Sequence Ontology: A tool for the unification of genome annotations. *Genome Biology*, 6(5), R44. <https://doi.org/10.1186/gb-2005-6-5-r44>
- Ewing, B., & Green, P. (1998). Base-calling of automated sequencer traces using *Phred*. II. Error probabilities. *Genome Research*, 8(3), 186-194. <https://doi.org/10.1101/gr.8.3.186>
- Ewing, B., Hillier, L., Wendl, M. C., & Green, P. (1998). Base-calling of automated sequencer traces using *Phred*. I. Accuracy assessment. *Genome Research*, 8(3), 175-185. <https://doi.org/10.1101/gr.8.3.175>
- Hsi-Yang Fritz, M., Leinonen, R., Cochrane, G., & Birney, E. (2011). Efficient storage of high throughput DNA sequencing data using reference-based compression. *Genome Research*, 21(5), 734-740. <https://doi.org/10.1101/gr.114819.110>
- Kent, W. J., Sugnet, C. W., Furey, T. S., Roskin, K. M., Pringle, T. H., Zahler, A. M., & Haussler, D. (2002). The human genome browser at UCSC. *Genome Research*, 12(6), 996-1006. <https://doi.org/10.1101/gr.229102>
- Leinonen, R., Sugawara, H., & Shumway, M. (2011). The Sequence Read Archive. *Nucleic Acids Research*, 39(Database), D19-D21. <https://doi.org/10.1093/nar/gkq1019>
- Li, H. (2011). Tabix: Fast retrieval of sequence features from generic TAB-delimited files. *Bioinformatics*, 27(5), 718-719. <https://doi.org/10.1093/bioinformatics/btq671>

- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, 25(16), 2078-2079. <https://doi.org/10.1093/bioinformatics/btp352>
- Lipman, D. J., & Pearson, W. R. (1985). Rapid and sensitive protein similarity searches. *Science*, 227(4693), 1435-1441. <https://doi.org/10.1126/science.2983426>
- O'Leary, N. A., Wright, M. W., Brister, J. R., Ciufu, S., Haddad, D., McVeigh, R., Rajput, B., Robbertse, B., Smith-White, B., Ako-Adjei, D., et al. (2016). Reference sequence (RefSeq) database at NCBI: Current status, taxonomic expansion, and functional annotation. *Nucleic Acids Research*, 44(D1), D733-D745. <https://doi.org/10.1093/nar/gkv1189>
- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444-2448. <https://doi.org/10.1073/pnas.85.8.2444>
- Quinlan, A. R., & Hall, I. M. (2010). BEDTools: A flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6), 841-842. <https://doi.org/10.1093/bioinformatics/btq033>
- Sayers, E. W., Beck, J., Bolton, E. E., Brister, J. R., Chan, J., Connor, R., Feldgarden, M., Fine, A. M., Funk, K., Hoffman, J., Kannan, S., Kelly, C., Klimke, W., Kim, S., Lathrop, S., Marchler-Bauer, A., Murphy, T. D., et al. (2025). Database resources of the National Center for Biotechnology Information in 2025. *Nucleic Acids Research*, 53(D1), D20-D29. <https://doi.org/10.1093/nar/gkae979>
- Sayers, E. W., Cavanaugh, M., Frisse, L., Pruitt, K. D., Schneider, V. A., Underwood, B. A., Yankie, L., & Karsch-Mizrachi, I. (2025). GenBank 2025 update. *Nucleic Acids Research*, 53(D1), D56-D61. <https://doi.org/10.1093/nar/gkae1114>
- Tan, A., Abecasis, G. R., & Kang, H. M. (2015). Unified representation of genetic variants. *Bioinformatics*, 31(13), 2202-2204. <https://doi.org/10.1093/bioinformatics/btv112>
- The UniProt Consortium. (2025). UniProt: The Universal Protein Knowledgebase in 2025. *Nucleic Acids Research*, 53(D1), D609-D617. <https://doi.org/10.1093/nar/gkae1010>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- Yates, A., Beal, K., Keenan, S., McLaren, W., Pignatelli, M., Ritchie, G. R. S., Ruffier, M., Taylor, K., Vullo, A., & Flicek, P. (2015). The Ensembl REST API: Ensembl data for any language. *Bioinformatics*, 31(1), 143-145. <https://doi.org/10.1093/bioinformatics/btu613>

PARTE

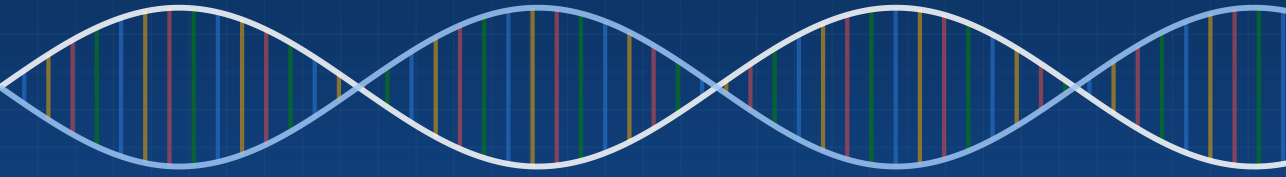


Comparar secuencias

Alineamiento, perfiles y árboles

La comparación es la operación fundamental de la bioinformática. Aquí convertimos la intuición del parecido en algoritmos exactos, en modelos probabilísticos y, finalmente, en la historia evolutiva que conecta a las especies.

Alineamiento de secuencias



Nada tiene sentido en biología si no es a la luz de la evolución.

— Dobzhansky (1973)

● Al terminar este capítulo usted podrá

- Leer un *dot plot* e identificar en él identidad, repeticiones, inversiones e inserciones.
- Formular el alineamiento global y local como problemas de optimización y resolverlos por programación dinámica (Needleman-Wunsch y Smith-Waterman).
- Explicar de dónde salen las matrices PAM y BLOSUM como razones de verosimilitud logarítmicas, y justificar las penalizaciones afines de *gaps*.
- Interpretar un resultado de BLAST: puntuación en bits, valor *E* y los supuestos de la estadística de Karlin-Altschul.

Comparar es la operación más antigua de la biología. Linneo comparó flores; Darwin comparó picos de pinzones; nosotros comparamos cadenas de letras. Cuando dos secuencias se parecen más de lo que el azar permitiría, la explicación más sencilla es que comparten un ancestro, y de esa inferencia (la *homología*) depende casi todo lo que haremos en el resto del libro: anotar genes, predecir funciones, reconstruir

árboles, mapear lecturas o llamar variantes. Este capítulo construye, desde cero, la herramienta que hace posible esa comparación: el *alineamiento*.

Avanzaremos en cuatro pasos. Primero miraremos las secuencias con los ojos, mediante una matriz de puntos (sección 3.1). Luego convertiremos esa intuición visual en un algoritmo exacto, la programación dinámica (sección 3.2). Después nos preguntaremos cuánto debe valer cada coincidencia, cada sustitución y cada hueco, lo que nos llevará a las matrices de sustitución (sección 3.3). Por último, escalaremos el problema a bases de datos con millones de secuencias y aprenderemos a distinguir un parecido significativo de uno casual con BLAST y la estadística de Karlin-Altschul (sección 3.4).

3.1 Dot plots: mirar antes de calcular

 Colab · 3.1 · Dot plots

 repositorio del curso

Dos manuscritos sobre la mesa

Imagine que recibe dos copias manuscritas de un mismo poema, copiadas por escribas distintos con siglos de diferencia. Para compararlas, podría escribir una copia a lo largo del borde superior de una hoja cuadrículada y la otra a lo largo del borde izquierdo, y marcar con un punto cada casilla donde las dos letras coinciden. Allí donde los escribas copiaron fielmente, aparecerá una línea diagonal limpia; donde uno de ellos saltó un verso, la diagonal se desplazará; donde repitió una estrofa, aparecerá una segunda diagonal paralela. Sin hacer una sola cuenta, la hoja le habrá contado la historia de las copias.

Esa hoja cuadrículada es exactamente un *dot plot* (matriz de puntos), la representación propuesta por Gibbs y McIntyre (1970) cuando las secuencias de proteínas empezaban a acumularse y aún no existían algoritmos para compararlas. Su gran virtud es que no impone ninguna hipótesis: muestra *todas* las coincidencias a la vez, y deja al ojo humano, un detector de patrones extraordinario, la tarea de interpretarlas.

3.1.1 Definición formal

Sean $x = x_1x_2 \cdots x_n$ e $y = y_1y_2 \cdots y_m$ dos secuencias sobre un alfabeto $\mathcal{A}$ (por ejemplo, $\mathcal{A} = \{A, C, G, T\}$).

Definición 3.1. (Dot plot con ventana)

Dada una ventana de longitud impar w y un umbral $t \leq w$, el *dot plot* de x contra y es la matriz binaria $D \in \{0, 1\}^{n \times m}$ con

$$D_{ij} = \mathbb{1} \left[\sum_{k=-h}^h \mathbb{1}[x_{i+k} = y_{j+k}] \geq t \right], \quad h = \frac{w-1}{2}. \quad (3.1)$$

DONDE

D_{ij}	Celda de la fila i (posición en x) y la columna j (posición en y); vale 1 si se dibuja un punto.
$\mathbb{1}[\cdot]$	Función indicadora: 1 si la condición es verdadera, o si no.
w, h	Longitud de la ventana deslizante y su semiancho.
t	Número mínimo de coincidencias dentro de la ventana para marcar la celda.

Con $w = 1$ y $t = 1$ recuperamos la versión más ingenua: un punto por cada par de letras idénticas. En ADN esto es casi inútil, porque con cuatro letras la probabilidad de que dos posiciones al azar coincidan es $1/4$: una cuarta parte de la matriz se llenaría de ruido. La ventana resuelve el problema. Si las bases fueran independientes y equiprobables, el número de coincidencias en una ventana seguiría una distribución binomial $\text{Bin}(w, 1/4)$, y la probabilidad de un punto espurio sería

$$p_{\text{ruido}} = \sum_{k=t}^w \binom{w}{k} \left(\frac{1}{4}\right)^k \left(\frac{3}{4}\right)^{w-k}. \quad (3.2)$$

Para $w = 11$, $t = 7$, esta probabilidad es de apenas 0,0076: menos de un punto espurio por cada cien celdas. Esta es la idea de filtrado que Maizel y Lenk (1981) popularizaron y que, como veremos, reaparece en el corazón de BLAST.

3.1.2 Cómo se lee un dot plot

La figura 3.1 muestra un ejemplo construido a propósito: la secuencia y es una copia de x en la que un segmento central fue invertido y complementado. Aparecen los motivos que conviene aprender a reconocer:

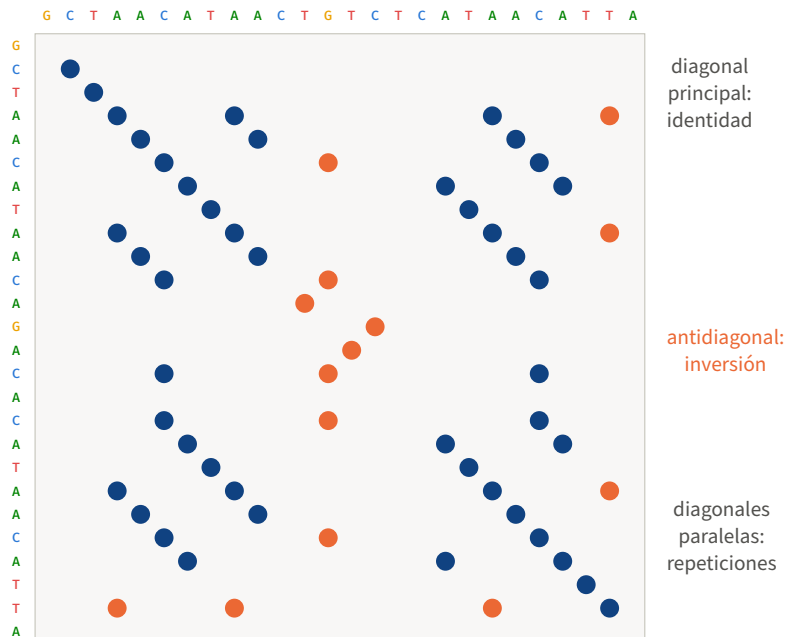


Figura 3.1. Anatomía de un *dot plot* ($w = 3, t = 3$). Las filas corresponden a x y las columnas a y . En azul, coincidencias directas: la diagonal principal y dos diagonales paralelas producidas por un segmento repetido. En naranja, coincidencias con el reverso complementario, que revelan el segmento invertido como una antidiagonal. Figura generada con `figuras/cap03/generar.py`.

- **Diagonal principal:** regiones idénticas u homólogas en el mismo orden.
- **Diagonales paralelas desplazadas:** repeticiones, pues un mismo segmento de x se parece a dos lugares de y .
- **Salto lateral de la diagonal:** inserciones o deleciones (*indels*); la diagonal continúa, pero desplazada una fila o una columna.
- **Antidiagonales:** inversiones; se detectan comparando x con el *reverso complementario* de y (puntos naranjas).
- **Bloques cuadrados:** regiones de baja complejidad (p. ej., **ATATATAT**), que coinciden consigo mismas en cualquier desfase.

Ejemplo 3.1. (Un dot plot en diez líneas)

El siguiente fragmento calcula la matriz de la ecuación (3.1) de forma vectorizada: en lugar de recorrer ventanas, se suman diagonales desplazadas de la matriz de identidad.

```
dotplot.py

import numpy as np

def dotplot(x: str, y: str, w: int = 11, t: int = 7) -> np.ndarray:
    """Matriz binaria D (n x m) con ventana w y umbral t."""
    X = np.frombuffer(x.encode(), dtype=np.uint8)
    Y = np.frombuffer(y.encode(), dtype=np.uint8)
    eq = (X[:, None] == Y[None, :]).astype(np.int16) # identidad
    ↪ letra a letra
    h = w // 2
    acc = np.zeros_like(eq)
    for k in range(-h, h + 1): # suma a lo
        ↪ largo de la diagonal
        acc += np.roll(np.roll(eq, k, axis=0), k, axis=1)
    return acc >= t
```

Con dos genomas de 30 kb la matriz tiene 9×10^8 celdas; por eso las herramientas reales, como `dotter` o `minimap2 -X`, calculan solo semillas de *k*-mers exactos en lugar de la matriz completa.

⚠ Lo que un dot plot no hace

Un *dot plot* no produce un alineamiento ni una puntuación. Es una herramienta de *exploración*: sugiere hipótesis (“aquí hay una duplicación en tándem”) que luego deben cuantificarse. Tampoco escala: su coste en memoria es $O(nm)$.

3.2 Programación dinámica: Needleman-Wunsch y Smith-Waterman

3.2.1 ¿Qué es un alineamiento?

Un *alineamiento* de x e y es una forma de escribir una secuencia encima de la otra, insertando huecos (*gaps*, representados con “-”) para que las posiciones que creemos que descienden de un mismo ancestro queden en la misma columna. Cada columna es de uno de tres tipos:

Columna	Ejemplo	Interpretación evolutiva
Coincidencia (<i>match</i>)	$\begin{pmatrix} A \\ A \end{pmatrix}$	la posición se conservó
Sustitución (<i>mismatch</i>)	$\begin{pmatrix} A \\ G \end{pmatrix}$	hubo una mutación puntual
Hueco (<i>gap</i>)	$\begin{pmatrix} A \\ - \end{pmatrix}, \begin{pmatrix} - \\ A \end{pmatrix}$	inserción en una rama o delección en la otra

Un alineamiento es, por lo tanto, una *hipótesis evolutiva* escrita en forma de tabla. Y como toda hipótesis, puede ser mejor o peor. Para compararlas asignamos a cada columna una puntuación: $s(a, b)$ para un par de letras y $-d$ por cada posición con hueco. La puntuación del alineamiento es la suma de sus columnas, y el *alineamiento óptimo* es el de puntuación máxima.

El problema de contar caminos

¿Por qué no probar todos los alineamientos y quedarnos con el mejor? Porque son astronómicamente muchos. El número de alineamientos globales de dos secuencias de longitud n crece aproximadamente como

$$\binom{2n}{n} \approx \frac{2^{2n}}{\sqrt{\pi n}},$$

de modo que para dos proteínas modestas de 300 residuos hay unos 10^{179} alineamientos, muchísimos más que átomos en el universo observable (unos 10^{80}). Necesitamos una idea más astuta que la fuerza bruta.

3.2.2 El principio de optimalidad

La idea astuta proviene de la investigación de operaciones. Bellman (1954) observó que muchos problemas de decisión secuencial satisfacen un *principio de optimalidad*: la mejor solución global contiene, como piezas, las mejores soluciones de subproblemas más pequeños. En el alineamiento, esto se traduce en una observación casi evidente una vez que se enuncia: la última columna de un alineamiento

óptimo solo puede ser de tres tipos, y lo que queda a su izquierda debe ser, a su vez, un alineamiento óptimo de los prefijos correspondientes. Si no lo fuera, podríamos sustituirlo por uno mejor y mejorar el total, lo que contradice la optimalidad. Eddy (2004a) ofrece una introducción breve y muy clara a esta forma de pensar.

3.2.3 Needleman-Wunsch: alineamiento global

Sea $F(i, j)$ la puntuación del mejor alineamiento entre los prefijos $x_{1..i}$ e $y_{1..j}$. Needleman y Wunsch (1970) mostraron que F se puede llenar como una tabla, fila por fila, con la recurrencia

Teorema 3.1. (Recurrencia de Needleman-Wunsch)

Con penalización lineal $d > 0$ por posición de hueco,

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) & (\text{alinear } x_i \text{ con } y_j) \\ F(i-1, j) - d & (x_i \text{ frente a un hueco}) \\ F(i, j-1) - d & (y_j \text{ frente a un hueco}) \end{cases} \quad (3.3)$$

con condiciones de frontera $F(0, 0) = 0$, $F(i, 0) = -id$ y $F(0, j) = -jd$. La puntuación óptima es $F(n, m)$.

DONDE

$F(i, j)$	Mejor puntuación posible alineando los primeros i símbolos de x con los primeros j de y .
$s(a, b)$	Puntuación de sustitución de la letra a por la b (positiva si se parecen).
d	Costo de cada posición alineada contra un hueco.
n, m	Longitudes de x e y .

La tabla se llena en $O(nm)$ operaciones, ya que cada celda consulta tres vecinas. Para recuperar el alineamiento, y no solo su puntuación, se recorre la tabla en sentido inverso (*traceback*) desde (n, m) , preguntando en cada celda cuál de los tres casos produjo el máximo. Una diagonal equivale a una columna de pares; un paso vertical, a una letra de x frente a un hueco; uno horizontal, a una letra de y frente a un hueco.

	ϵ	G	T	T	A	G	C	A
ϵ	0	-2	-4	-6	-8	-10	-12	-14
G	-2	1	-1	-3	-5	-7	-9	-11
A	-4	-1	0	-2	-2	-4	-6	-8
T	-6	-3	0	1	-1	-3	-5	-7
T	-8	-5	-2	1	0	-2	-4	-6
A	-10	-7	-4	-1	2	0	-2	-3
C	-12	-9	-6	-3	0	1	1	-1
A	-14	-11	-8	-5	-2	-1	0	2

Figura 3.2. Matriz de programación dinámica para $x = \text{GATTACA}$ (filas) e $y = \text{GTTAGCA}$ (columnas), con $s = +1$ para coincidencias, -1 para sustituciones y $d = 2$. Las celdas resaltadas y las flechas naranjas marcan el *traceback* desde la esquina inferior derecha. Los valores negativos aparecen en rojo.

Ejemplo 3.2. (Leyendo la matriz de la figura 3.2)

La puntuación óptima es $F(7, 7) = 2$. Siguiendo las flechas desde la esquina se obtiene

G	A	T	T	A	-	C	A
G	-	T	T	A	G	C	A

Seis coincidencias (+6) y dos huecos (-4) dan $6 - 4 = 2$, lo que confirma el valor de la esquina. Observe el primer paso vertical tras la G: el algoritmo “decidió” que era mejor pagar un hueco que forzar una A contra una T, porque el hueco permite que el bloque TTA quede perfectamente alineado.

needleman_wunsch.py

```
def needleman_wunsch(x, y, match=1, mismatch=-1, gap=2):
    n, m = len(x), len(y)
```



```

F = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1): F[i][0] = -i * gap          # frontera: solo
↳ huecos
for j in range(1, m + 1): F[0][j] = -j * gap
for i in range(1, n + 1):
    for j in range(1, m + 1):
        s = match if x[i - 1] == y[j - 1] else mismatch
        F[i][j] = max(F[i - 1][j - 1] + s,          # diagonal
                      F[i - 1][j] - gap,            # hueco en y
                      F[i][j - 1] - gap)           # hueco en x
return F[n][m], F

```

3.2.4 Smith-Waterman: alineamiento local

El alineamiento global asume que las dos secuencias son homólogas de extremo a extremo. Muchas veces no es así: dos proteínas pueden compartir un único dominio en medio de regiones sin relación, o una lectura corta de secuenciación puede provenir de un pequeño trozo de un cromosoma. En esos casos queremos el *par de subcadenas* más parecido. Smith y Waterman (1981) lo consiguieron con un cambio mínimo pero profundo: permitir que un alineamiento “vuelva a empezar” en cualquier celda.

Teorema 3.2. (Recurrencia de Smith-Waterman)

$$H(i, j) = \max \begin{cases} 0 & (\text{empezar de nuevo}) \\ H(i - 1, j - 1) + s(x_i, y_j) \\ H(i - 1, j) - d \\ H(i, j - 1) - d \end{cases} \quad (3.4)$$

con $H(i, 0) = H(0, j) = 0$. La puntuación local óptima es $\max_{i,j} H(i, j)$ y el traceback termina al encontrar un 0.

El cero cumple un papel decisivo. Un prefijo que acumula puntuación negativa nunca ayuda a una extensión posterior, así que la recurrencia lo “olvida”. Para que esto funcione, la puntuación esperada de un par aleatorio debe ser negativa, $\sum_{a,b} p_a p_b s(a, b) < 0$; de lo contrario, los alineamientos locales crecerían indefinidamente y se volverían, en la práctica, globales. Esta condición reaparecerá como supuesto central de la estadística de BLAST.

🔗 Sellers, Waterman y la métrica evolutiva

Entre Needleman-Wunsch (1970) y Smith-Waterman (1981) media una década de formalización matemática. Sellers (1974) demostró que, bajo ciertas condiciones, la distancia de edición entre secuencias es una métrica genuina, lo que dio al alineamiento un fundamento geométrico. Temple Smith y Michael Waterman, un físico y un matemático, trabajaban en Los Álamos cuando publicaron su algoritmo en un artículo de apenas tres páginas, hoy uno de los más citados de la biología computacional.

3.2.5 Huecos afines y el algoritmo de Gotoh

Con una penalización lineal, un hueco de 10 posiciones cuesta lo mismo que diez huecos de 1 posición dispersos. Biológicamente esto es absurdo: un único evento de inserción o deleción suele afectar a varias bases contiguas, y abrir un hueco es mucho más raro que alargarlo. La solución estándar es la *penalización afín*:

$$\gamma(g) = -(d_o + (g - 1) d_e), \quad d_o > d_e > 0, \quad (3.5)$$

DONDE

g	Longitud del hueco (número de posiciones consecutivas).
d_o	Costo de <i>abrir</i> el hueco (primer símbolo).
d_e	Costo de <i>extender</i> el hueco en cada posición adicional.

Una función afín rompe la recurrencia simple, porque el costo de una celda depende de si el hueco ya estaba abierto. Gotoh (1982) resolvió el problema usando tres matrices acopladas, una por cada “estado” en el que puede terminar un alineamiento:

$$M(i, j) = s(x_i, y_j) + \max\{M, I_x, I_y\}(i-1, j-1), \quad (3.6)$$

$$I_x(i, j) = \max\{M(i-1, j) - d_o, I_x(i-1, j) - d_e\}, \quad (3.7)$$

$$I_y(i, j) = \max\{M(i, j-1) - d_o, I_y(i, j-1) - d_e\}. \quad (3.8)$$

El coste sigue siendo $O(nm)$ en tiempo. Esta arquitectura de tres estados no es un truco aislado: es exactamente un *modelo oculto de Márkov de pares*, como mostraron Durbin et al. (1998), y la retomaremos en el capítulo 4 al estudiar perfiles y HMMER.

Memoria lineal: el truco de Hirschberg

Para alinear dos cromosomas bacterianos de 5 Mb, una matriz $n \times m$ necesitaría $2,5 \times 10^{13}$ celdas. Hirschberg (1975) mostró que el alineamiento óptimo puede recuperarse en espacio $O(n + m)$: se calcula la fila central de la matriz hacia adelante y hacia atrás, se localiza la celda por la que pasa el camino óptimo y se resuelven recursivamente las dos mitades. El tiempo apenas se duplica, pero la memoria se reduce en órdenes de magnitud.

3.3 Matrices de sustitución y penalización de gaps



Colab · 3.3 · Matrices de sustitución



repositorio del curso

Hasta ahora hemos usado puntuaciones arbitrarias: +1 para coincidencias, −1 para sustituciones. En ADN esa simplificación es tolerable; en proteínas, no. Cambiar una leucina por una isoleucina (ambas hidrofóbicas, casi del mismo tamaño) apenas altera una proteína, mientras que cambiar un triptófano por una glicina puede destruir su plegamiento. Necesitamos puntuaciones que reflejen la química y la evolución.

3.3.1 Puntuaciones como razones de verosimilitud

La pregunta correcta es estadística. Frente a un par alineado (a, b) , comparamos dos hipótesis:

- $\mathcal{H}_1$ (*homología*): a y b descienden de un ancestro común y aparecen juntos con probabilidad q_{ab} , estimada a partir de alineamientos confiables.
- $\mathcal{H}_0$ (*azar*): a y b se eligieron de forma independiente, con probabilidad $p_a p_b$ según sus frecuencias de fondo.

La puntuación natural es el logaritmo de la razón de verosimilitudes:

$$s(a, b) = \frac{1}{\lambda} \log \frac{q_{ab}}{p_a p_b}. \quad (3.9)$$

DONDE

q_{ab}	Frecuencia con que a y b aparecen alineados en secuencias homólogas.
p_a, p_b	Frecuencias de fondo de cada aminoácido en las proteínas.
λ	Factor de escala; con $\lambda = \ln 2/2$ las puntuaciones quedan en medios bits.

Esta forma tiene una consecuencia elegante: como el logaritmo convierte productos en sumas, sumar las puntuaciones de las columnas equivale a multiplicar probabilidades, siempre que las columnas se consideren independientes. La puntuación de un alineamiento completo es, por lo tanto, el logaritmo de la razón de verosimilitudes de la hipótesis de homología frente a la de azar. Altschul (1991) demostró además que *cualquier* matriz de puntuación con esperanza negativa y al menos un valor positivo es implícitamente una matriz de este tipo, para algún conjunto de frecuencias q_{ab} “objetivo”.

3.3.2 PAM: extrapolar la evolución

Margaret Dayhoff y sus colaboradores construyeron las primeras matrices a partir de 71 familias de proteínas muy similares, con al menos 85 % de identidad (Dayhoff et al., 1978). Contaron las sustituciones observadas en árboles filogenéticos y las normalizaron para obtener una matriz de transición $P^{(1)}$ que representa 1 PAM: la divergencia en la que, en promedio, ha cambiado un 1 % de los aminoácidos (*point accepted mutation*). Suponiendo que la evolución es un proceso de Márkov, la matriz para distancias mayores se obtiene elevando a potencias:

$$P^{(k)} = \left(P^{(1)}\right)^k, \quad s_k(a, b) = \frac{1}{\lambda} \log \frac{p_a P_{ab}^{(k)}}{p_a p_b} = \frac{1}{\lambda} \log \frac{P_{ab}^{(k)}}{p_b}. \quad (3.10)$$

La PAM250, muy usada en su época, corresponde a una distancia en la que sobreviven solo alrededor del 20 % de las identidades. El punto débil del método es precisamente la extrapolación: los errores de estimación de $P^{(1)}$ se amplifican al elevarla a la potencia 250.

3.3.3 BLOSUM: observar directamente

Henikoff y Henikoff (1992) tomaron el camino opuesto. En lugar de extrapolar desde secuencias cercanas, usaron la base de datos BLOCKS: unos 2000 segmentos conservados, sin huecos, de más de 500 familias de proteínas. Para construir la

	C	S	T	A	G	P	D	E	Q	N	H	R	K	M	I	L	V	W	Y	F	
C	9	-1	-1	0	-3	-3	-3	-4	-3	-3	-3	-3	-3	-1	-1	-1	-1	-2	-2	-2	
S	-1	4	1	1	0	-1	0	0	0	1	-1	-1	0	-1	-2	-2	-2	-3	-2	-2	
T	-1	1	5	0	-2	-1	-1	-1	-1	0	-2	-1	-1	-1	-1	-1	0	-2	-2	-2	pequeños
A	0	1	0	4	0	-1	-2	-1	-1	-2	-2	-1	-1	-1	-1	-1	0	-3	-2	-2	
G	-3	0	-2	0	6	-2	-1	-2	-2	0	-2	-2	-2	-3	-4	-4	-3	-2	-3	-3	
P	-3	-1	-1	-1	-2	7	-1	-1	-1	-2	-2	-2	-1	-2	-3	-3	-2	-4	-3	-4	
D	-3	0	-1	-2	-1	-1	6	2	0	1	-1	-2	-1	-3	-3	-4	-3	-4	-3	-3	ácidos/amidas
E	-4	0	-1	-1	-2	-1	2	5	2	0	0	0	1	-2	-3	-3	-2	-3	-2	-3	
Q	-3	0	-1	-1	-2	-1	0	2	5	0	0	1	1	0	-3	-2	-2	-2	-1	-3	
N	-3	1	0	-2	0	-2	1	0	0	6	1	0	0	0	-2	-3	-3	-3	-4	-2	-3
H	-3	-1	-2	-2	-2	-2	-1	0	0	1	8	0	-1	-2	-3	-3	-3	-2	2	-1	básicos
R	-3	-1	-1	-1	-2	-2	-2	0	1	0	0	5	2	-1	-3	-2	-3	-3	-2	-3	
K	-3	0	-1	-1	-2	-1	-1	1	1	0	-1	2	5	-1	-3	-2	-2	-3	-2	-3	
M	-1	-1	-1	-1	-3	-2	-3	-2	0	-2	-2	-1	-1	5	1	2	1	-1	-1	0	alifáticos
I	-1	-2	-1	-1	-4	-3	-3	-3	-3	-3	-3	-3	-3	1	4	2	3	-3	-1	0	
L	-1	-2	-1	-1	-4	-3	-4	-3	-2	-3	-3	-2	-2	2	2	4	1	-2	-1	0	
V	-1	-2	0	0	-3	-2	-3	-2	-2	-3	-3	-3	-2	1	3	1	4	-3	-1	-1	
W	-2	-3	-2	-3	-2	-4	-4	-3	-2	-4	-2	-3	-3	-1	-3	-2	-3	11	2	1	aromáticos
Y	-2	-2	-2	-2	-3	-3	-3	-2	-1	-2	2	-2	-2	-1	-1	-1	-1	2	7	3	
F	-2	-2	-2	-2	-3	-4	-3	-3	-3	-3	-1	-3	-3	0	0	0	-1	1	3	6	

Figura 3.3. La matriz BLOSUM62, en medios bits, con los aminoácidos ordenados por propiedades fisicoquímicas. Los bloques recuadrados en la diagonal muestran que las sustituciones *dentro* de un grupo (p. ej., $I \leftrightarrow L \leftrightarrow V$) reciben puntuaciones positivas o cercanas a cero, mientras que las sustituciones entre grupos son penalizadas. El valor de W–W (+11) refleja que el triptófano es raro y muy conservado. Valores tomados de Biopython (Cock et al., 2009).

BLOSUM L agruparon las secuencias de cada bloque con más de L % de identidad, con el fin de que las familias muy representadas no dominaran los conteos, y estimaron q_{ab} directamente de los pares observados. Así, la BLOSUM62 se basa en secuencias con a lo sumo 62 % de identidad, y la BLOSUM45 en otras aún más divergentes.

Número alto en PAM $\Rightarrow$ secuencias más divergentes.
Número alto en BLOSUM $\Rightarrow$ secuencias más cercanas.

Un error afortunado

Una década y media después de su publicación, Styczynski et al. (2008) descubrieron que el programa original que generó las BLOSUM contenía un error en el agrupamiento de secuencias. Paradójicamente, la matriz “incorrecta” encuentra homólogos remotos *mejor* que la versión corregida, y por eso la BLOSUM62 original sigue siendo la matriz por defecto de BLAST. Eddy (2004b) explica, en dos páginas, cómo reconstruirla desde los datos.

3.3.4 Elegir la matriz y los huecos

No existe una matriz universalmente óptima. Cada una está calibrada para un nivel de divergencia: una matriz “profunda” (BLOSUM45, PAM250) detecta homólogos lejanos pero puntúa en exceso los parecidos casuales en alineamientos cortos, mientras que una matriz “superficial” (BLOSUM80, PAM30) hace lo contrario. En la práctica, BLOSUM62 con apertura $d_o = 11$ y extensión $d_e = 1$ es el punto de partida de `blastp`; para péptidos cortos se recomienda PAM30 (Pearson, 2013).

Parámetros que no se pueden mezclar

Las penalizaciones de hueco no son independientes de la matriz: ambas deben estar en la misma escala y, para búsquedas en bases de datos, calibradas conjuntamente, porque de ellas dependen los parámetros λ y K de la sección siguiente. Cambiar d_o sin recalibrar invalida los valores E .

3.4 BLAST y la estadística de Karlin-Altschul

 Colab · 3.4 · BLAST y valores E

 repositorio del curso

3.4.1 El problema de escala

Smith-Waterman es exacto, pero comparar una proteína de 300 residuos contra UniProt (unos 10^{11} residuos) requeriría 3×10^{13} celdas por consulta. A finales de los ochenta, FASTA (Pearson & Lipman, 1988) mostró que se podía ganar velocidad buscando primero palabras cortas idénticas. BLAST (Altschul et al., 1990) llevó esa idea más lejos con una estrategia de tres pasos, conocida como *seed-and-extend*:

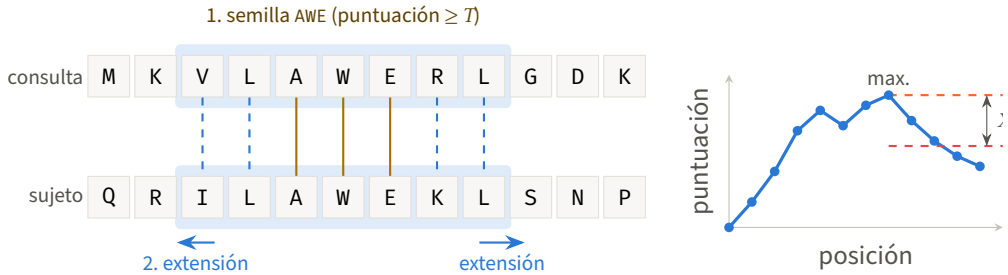


Figura 3.4. La estrategia *seed-and-extend*. **Izquierda:** una palabra de la consulta (AWE) encuentra una semilla en el sujeto; la extensión sin huecos añade pares vecinos mientras la puntuación acumulada lo justifique. **Derecha:** perfil de puntuación durante la extensión; se abandona cuando la puntuación cae X unidades por debajo del máximo, y el HSP se recorta en ese máximo.

- 1. Siembra.** Se divide la consulta en palabras de longitud w (3 para proteínas, 11 o más para ADN) y, para cada una, se genera la *vecindad* de palabras que puntúan al menos T contra ella con la matriz de sustitución.
- 2. Búsqueda.** Un índice precalculado de la base de datos localiza en tiempo casi constante todas las apariciones de esas palabras (*hits*).
- 3. Extensión.** Cada *hit* se extiende en ambas direcciones y se detiene cuando la puntuación cae más de X unidades por debajo del máximo alcanzado (*X-drop*). Los segmentos resultantes son los *high-scoring segment pairs* (HSP).

La versión de 1997 (Altschul et al., 1997) añadió dos mejoras decisivas: la heurística de “dos *hits*”, que exige dos semillas cercanas en la misma diagonal antes de extender y reduce el trabajo a la mitad, y la extensión *con huecos* mediante programación dinámica restringida a una banda. La implementación moderna, BLAST+ (Camacho et al., 2009), reorganizó el código en C++ y es la que usaremos en el laboratorio.

3.4.2 ¿Cuándo un parecido es significativo?

Toda búsqueda produce un mejor resultado, incluso si la consulta es una secuencia aleatoria. La pregunta crucial es: *¿qué puntuación esperaríamos por puro azar?* La respuesta, obtenida por Karlin y Altschul (1990), es uno de los resultados más bellos de la bioinformática.

Consideremos la puntuación máxima S de un alineamiento local *sin huecos* entre dos secuencias aleatorias de longitudes m y n . La suma de puntuaciones a lo largo de una diagonal es un paseo aleatorio con deriva negativa (porque $E[s] < 0$). Los

segmentos de alta puntuación son *excursiones* excepcionales de ese paseo, y la teoría de valores extremos predice que su máximo sigue una distribución de Gumbel.

Teorema 3.3. (Karlin-Altschul)

Si $\sum_{a,b} p_a p_b s(a,b) < \infty$ y existe al menos un par con $s(a,b) > 0$, el número esperado de HSP distintos con puntuación $\geq S$ es, asintóticamente,

$$E = K m n e^{-\lambda S}, \quad (3.11)$$

donde $\lambda > 0$ es la única raíz positiva de

$$\sum_{a,b} p_a p_b e^{\lambda s(a,b)} = 1. \quad (3.12)$$

El número de HSP con puntuación $\geq S$ sigue una distribución de Poisson de media E , de modo que $\mathbb{P}(\text{al menos uno}) = 1 - e^{-E}$.

DONDE

E	Valor E (<i>expect value</i>): cuántos resultados tan buenos esperaríamos por azar en esta búsqueda.
m, n	Longitud de la consulta y tamaño total de la base de datos, en residuos.
S	Puntuación bruta del alineamiento, en las unidades de la matriz.
λ	Escala natural de la matriz; convierte puntuaciones en unidades de información.
K	Constante que corrige por la dependencia entre HSP solapados (típicamente $\approx 0,1$).

La ecuación (3.12) merece una pausa. Si comparamos con la ecuación (3.9), vemos que $e^{\lambda s(a,b)} = q_{ab}/(p_a p_b)$, de modo que la condición dice simplemente que las frecuencias objetivo q_{ab} suman 1. El mismo λ que define la matriz como razón de verosimilitudes gobierna la estadística de sus máximos. Teoría de la información y teoría de valores extremos se encuentran en una sola ecuación.

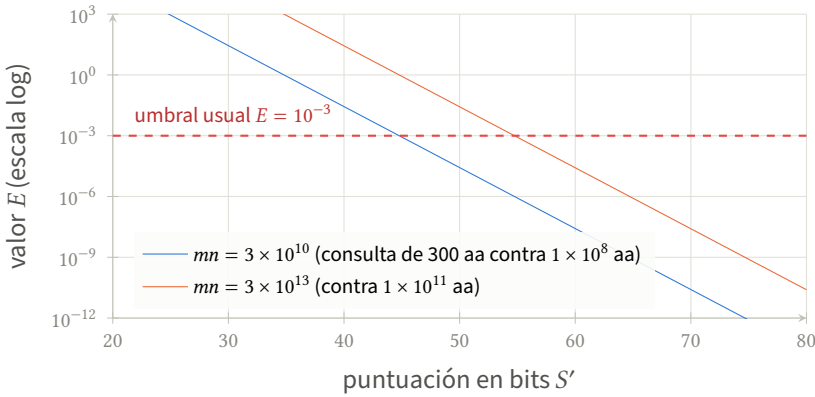


Figura 3.5. El valor E decrece exponencialmente con la puntuación en bits (ecuación (3.13)). La misma puntuación que es muy significativa contra una base de datos pequeña (azul) puede no serlo contra una mil veces mayor (naranja): el tamaño del espacio de búsqueda forma parte de la evidencia.

3.4.3 Puntuación en bits

Como λ y K dependen de la matriz, las puntuaciones brutas no son comparables entre búsquedas. BLAST las normaliza a *bits*:

$$S' = \frac{\lambda S - \ln K}{\ln 2}, \quad \text{de modo que} \quad E = mn 2^{-S'}. \quad (3.13)$$

Esta forma es muy práctica: cada bit adicional de puntuación reduce a la mitad el número esperado de falsos positivos, y para saber si un resultado es significativo basta comparar S' con $\log_2(mn)$, que es el número de bits necesarios para “destacar” en un espacio de búsqueda de tamaño mn .

Ejemplo 3.3. (Interpretar un resultado real)

Una búsqueda `blastp` de una proteína de 350 aminoácidos contra una base de datos de 2×10^{11} residuos devuelve un resultado con puntuación de 85 bits. El valor esperado es

$$E = 350 \times 2 \times 10^{11} \times 2^{-85} \approx 7 \times 10^{13} \times 2,6 \times 10^{-26} \approx 1,8 \times 10^{-12}.$$

Esperaríamos encontrar por azar un resultado así una vez en cada medio billón de búsquedas: la homología es muy probable. En cambio, un resultado de 40 bits en la misma búsqueda tendría $E \approx 64$: decenas de resultados tan

buenos aparecerían por azar, así que no debe interpretarse como evidencia de homología.

```
blast_local.py

import subprocess
from Bio import Blast                                # Biopython >=
↳ 1.83

subprocess.run(["blastp", "-query", "consulta.fasta", "-db", "swissprot",
               "-evalue", "1e-3", "-matrix", "BLOSUM62",
               "-gapopen", "11", "-gapextend", "1",
               "-outfmt", "5", "-out", "resultado.xml"], check=True)
with open("resultado.xml", "rb") as fh:
    for hit in Blast.read(fh):                        # un Hit por
↳ secuencia sujeto
        hsp = hit[0]                                  # mejor HSP
        print(f"{hit.target.description[:40]:40s} "
              f"{hsp.annotations['bit score']:6.1f} bits  "
              f"E={hsp.annotations['evalue']:.1e}")
```

3.4.4 Los supuestos, y cuándo fallan

La elegancia de la ecuación (3.11) no debe ocultar sus supuestos. La teoría es exacta solo para alineamientos *sin huecos* entre secuencias *largas* de composición *aleatoria e independiente*. Para alineamientos con huecos no existe una fórmula cerrada: λ y K se estiman por simulación para cada combinación de matriz y penalizaciones, y las secuencias cortas requieren una corrección de “longitud efectiva” (Altschul et al., 1997; Karlin et al., 1990). Además, las regiones de baja complejidad y los sesgos de composición violan la independencia; por eso BLAST enmascara esas regiones (con SEG o DUST) y aplica ajustes de composición a la matriz.

⚠ Tres errores frecuentes al leer un valor E

1. *Tratarlo como una probabilidad.* E es un número esperado y puede ser mayor que 1. Solo cuando $E < 0,01$ se cumple que $E \approx p$.
2. *Compararlo entre bases de datos.* Crece linealmente con n : la misma búsqueda contra una base de datos diez veces mayor produce un E diez veces mayor.

3. *Confundir significancia con homología global.* Un E minúsculo indica que *algún segmento* es homólogo; puede tratarse de un único dominio compartido entre proteínas por lo demás distintas.

☰ En resumen

- Un *dot plot* muestra todas las similitudes entre dos secuencias; las ventanas con umbral filtran el ruido y permiten ver identidades, repeticiones, inversiones e *indels*.
- El principio de optimalidad de Bellman reduce la búsqueda del mejor alineamiento, entre $\sim 10^{179}$ posibles, a llenar una tabla en $O(nm)$. Needleman-Wunsch alinea globalmente; Smith-Waterman, localmente, gracias al cero de la recurrencia.
- Los huecos afines modelan *indels* de varias bases y requieren tres matrices (Gotoh), equivalentes a un HMM de pares.
- Las matrices PAM y BLOSUM son razones de verosimilitud logarítmicas: PAM extrapola desde secuencias cercanas; BLOSUM observa directamente bloques divergentes.
- BLAST acelera la búsqueda con semillas y extensiones; su valor $E = Kmn e^{-\lambda S}$ cuantifica cuántos resultados así esperaríamos por azar, y la puntuación en bits lo hace comparable entre búsquedas.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 3.1 ●●● Dibuje a mano el *dot plot* ($w = 1$) de **ACGTACGT** contra sí misma. ¿Cuántas diagonales aparecen y qué representa cada una?
- 3.2 ●●● Usando la ecuación (3.2), calcule la probabilidad de un punto espurio con $w = 15$, $t = 10$ para ADN y para proteínas (suponga 20 aminoácidos equiprobables). ¿Por qué los *dot plots* de proteínas pueden usar ventanas más cortas?

- 3.3 ●●● Complete a mano la matriz de Needleman-Wunsch para GAATTC contra GATTA con $+2/-1$ y $d = 2$. Encuentre *todos* los alineamientos óptimos. ¿Por qué puede haber más de uno?
- 3.4 ●●● Modifique la función `needleman_wunsch` para implementar Smith-Waterman, devolviendo la puntuación máxima y la posición donde se alcanza.
- 3.5 ●●● Demuestre que, si $E[s] > 0$, la puntuación de Smith-Waterman entre dos secuencias aleatorias crece linealmente con su longitud. ¿Qué implica esto para el uso de matrices de puntuación de identidad pura ($+1/0$)?
- 3.6 ●●● Para ADN con $s = +1$ (coincidencia) y $s = -1$ (sustitución), y bases equiprobables, resuelva la ecuación (3.12) para λ . Compruebe su resultado numéricamente con `scipy.optimize.brentq`.
- 3.7 ●●● Simule 10 000 pares de secuencias aleatorias de 200 aminoácidos, calcule la puntuación local máxima sin huecos con BLOSUM62 y ajuste una distribución de Gumbel. Compare el λ estimado con el teórico (0,318 en unidades naturales).

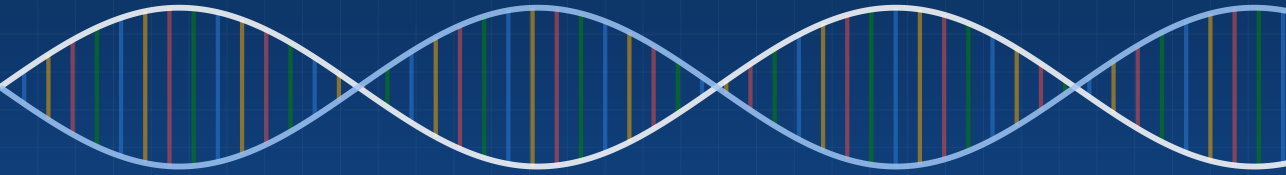
Referencias del capítulo

- Altschul, S. F. (1991). Amino acid substitution matrices from an information theoretic perspective. *Journal of Molecular Biology*, 219(3), 555-565. [https://doi.org/10.1016/0022-2836\(91\)90193-A](https://doi.org/10.1016/0022-2836(91)90193-A)
- Altschul, S. F., Gish, W., Miller, W., Myers, E. W., & Lipman, D. J. (1990). Basic local alignment search tool. *Journal of Molecular Biology*, 215(3), 403-410. [https://doi.org/10.1016/S0022-2836\(05\)80360-2](https://doi.org/10.1016/S0022-2836(05)80360-2)
- Altschul, S. F., Madden, T. L., Schäffer, A. A., Zhang, J., Zhang, Z., Miller, W., & Lipman, D. J. (1997). Gapped BLAST and PSI-BLAST: A new generation of protein database search programs. *Nucleic Acids Research*, 25(17), 3389-3402. <https://doi.org/10.1093/nar/25.17.3389>
- Bellman, R. (1954). The theory of dynamic programming. *Bulletin of the American Mathematical Society*, 60(6), 503-515. <https://doi.org/10.1090/S0002-9904-1954-09848-8>
- Camacho, C., Coulouris, G., Avagyan, V., Ma, N., Papadopoulos, J., Bealer, K., & Madden, T. L. (2009). BLAST+: Architecture and applications. *BMC Bioinformatics*, 10, 421. <https://doi.org/10.1186/1471-2105-10-421>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Dayhoff, M. O., Schwartz, R. M., & Orcutt, B. C. (1978). A model of evolutionary change in proteins. En M. O. Dayhoff (Ed.), *Atlas of protein sequence and structure* (pp. 345-352, Vol. 5). National Biomedical Research Foundation.
- Dobzhansky, T. (1973). Nothing in biology makes sense except in the light of evolution. *The American Biology Teacher*, 35(3), 125-129. <https://doi.org/10.2307/4444260>
- Durbin, R., Eddy, S. R., Krogh, A., & Mitchison, G. (1998). *Biological sequence analysis: Probabilistic models of proteins and nucleic acids*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790492>
- Eddy, S. R. (2004a). What is dynamic programming? *Nature Biotechnology*, 22(7), 909-910. <https://doi.org/10.1038/nbt0704-909>
- Eddy, S. R. (2004b). Where did the BLOSUM62 alignment score matrix come from? *Nature Biotechnology*, 22(8), 1035-1036. <https://doi.org/10.1038/nbt0804-1035>
- Gibbs, A. J., & McIntyre, G. A. (1970). The diagram, a method for comparing sequences: Its use with amino acid and nucleotide sequences. *European Journal of Biochemistry*, 16(1), 1-11. <https://doi.org/10.1111/j.1432-1033.1970.tb01046.x>
- Gotoh, O. (1982). An improved algorithm for matching biological sequences. *Journal of Molecular Biology*, 162(3), 705-708. [https://doi.org/10.1016/0022-2836\(82\)90398-9](https://doi.org/10.1016/0022-2836(82)90398-9)
- Henikoff, S., & Henikoff, J. G. (1992). Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22), 10915-10919. <https://doi.org/10.1073/pnas.89.22.10915>
- Hirschberg, D. S. (1975). A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6), 341-343. <https://doi.org/10.1145/360825.360861>
- Karlin, S., & Altschul, S. F. (1990). Methods for assessing the statistical significance of molecular sequence features by using general scoring schemes. *Proceedings of the National Academy of Sciences*, 87(6), 2264-2268. <https://doi.org/10.1073/pnas.87.6.2264>
- Karlin, S., Dembo, A., & Kawabata, T. (1990). Statistical composition of high-scoring segments from molecular sequences. *The Annals of Statistics*, 18(2), 571-581. <https://doi.org/10.1214/aos/1176347616>
- Maizel, J. V., & Lenk, R. P. (1981). Enhanced graphic matrix analysis of nucleic acid and protein sequences. *Proceedings of the National Academy of Sciences*, 78(12), 7665-7669. <https://doi.org/10.1073/pnas.78.12.7665>
- Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3), 443-453. [https://doi.org/10.1016/0022-2836\(70\)90057-4](https://doi.org/10.1016/0022-2836(70)90057-4)

- Pearson, W. R. (2013). An introduction to sequence similarity (“homology”) searching. *Current Protocols in Bioinformatics*, 42(1), 3.1.1-3.1.8. <https://doi.org/10.1002/0471250953.bi0301s42>
- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444-2448. <https://doi.org/10.1073/pnas.85.8.2444>
- Sellers, P. H. (1974). On the theory and computation of evolutionary distances. *SIAM Journal on Applied Mathematics*, 26(4), 787-793. <https://doi.org/10.1137/0126070>
- Smith, T. F., & Waterman, M. S. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195-197. [https://doi.org/10.1016/0022-2836\(81\)90087-5](https://doi.org/10.1016/0022-2836(81)90087-5)
- Styczynski, M. P., Jensen, K. L., Rigoutsos, I., & Stephanopoulos, G. (2008). BLOSUM62 miscalculations improve search performance. *Nature Biotechnology*, 26(3), 274-275. <https://doi.org/10.1038/nbt0308-274>

Alineamiento múltiple, motivos y perfiles



Los HMM de perfil convierten un alineamiento múltiple de secuencias en un sistema de puntuación específico de cada posición, adecuado para buscar en bases de datos secuencias remotamente homólogas.

— Eddy (1998)

● Al terminar este capítulo usted podrá

- Definir un alineamiento múltiple de secuencias (MSA), puntuarlo con la suma de pares y explicar por qué su solución exacta es computacionalmente intratable.
- Reproducir paso a paso el alineamiento progresivo: distancias, árbol guía y alineamiento de perfiles; reconocer sus limitaciones y cómo las corrigen MUSCLE, MAFFT, T-Coffee y Clustal Omega.
- Construir una matriz de peso posicional (PWM) a partir de sitios de unión, con pseudoconteos, y usarla para buscar sitios en una secuencia.
- Calcular la entropía y el contenido de información de cada posición de un motivo e interpretar un *sequence logo*.
- Formular un modelo oculto de Márkov y resolver sus tres problemas clásicos con los algoritmos *forward*, de Viterbi y de Baum-Welch.

- Describir la arquitectura de un HMM de perfil (estados de coincidencia, inserción y delección) y explicar cómo HMMER y Pfam lo usan para anotar dominios de proteínas.

En el capítulo 3 aprendimos a comparar *dos* secuencias. Pero la evolución rara vez nos entrega pares aislados: una familia de proteínas puede tener cientos o miles de miembros repartidos por todo el árbol de la vida, y lo que realmente queremos saber es qué tienen en común *todos* ellos. Una posición que se conserva en la hemoglobina humana, en la del ratón y en la mioglobina de un cachalote, separadas por cientos de millones de años de evolución, no se conserva por casualidad: probablemente sostiene el grupo hemo, estabiliza una hélice o participa en la función. El alineamiento de dos secuencias no puede distinguir ese residuo de uno que simplemente no ha tenido tiempo de cambiar; el alineamiento de muchas, sí.

Este capítulo recorre el camino que va de un conjunto de secuencias emparentadas a un *modelo estadístico* de la familia. Primero construiremos el alineamiento múltiple y veremos por qué la programación dinámica de la sección 3.2 deja de ser practicable, lo que obliga a recurrir a heurísticas como el alineamiento progresivo (sección 4.1). Después resumiremos un conjunto de sitios alineados, sin huecos, en una matriz de peso posicional, y mediremos con la teoría de la información cuánta “señal” contiene cada posición (sección 4.2). Por último, generalizaremos esas matrices a modelos que admiten inserciones y delecciones, los modelos ocultos de Márkov de perfil, que son el motor de HMMER y de la base de datos Pfam (sección 4.3). Al final del capítulo, el HMM de pares que se anunció con el algoritmo de Gotoh (ecuación (3.6)) habrá encontrado su lugar natural.

4.1 Alineamiento múltiple: el método progresivo

 Colab · 4.1 · MSA: alineamiento progresivo

 repositorio del curso

La receta de la abuela

Cinco primos conservan, cada uno, una copia manuscrita de la receta de pan de la abuela. Con los años, cada copia ha acumulado cambios: alguien sustituyó la manteca por aceite, otro añadió una pizca de anís, otro olvidó copiar una línea entera. Si quisiera reconstruir la receta original, no empezaría

comparando las copias más distintas. Empezaría por las de las dos hermanas que viven juntas, casi idénticas, y fijaría entre ambas una versión común; luego la compararía con la del primo que vive en la misma ciudad; y solo al final se enfrentaría a la copia del pariente lejano. Cada decisión que tome en los primeros pasos (“esta línea se perdió aquí”) condicionará todas las siguientes. Ese procedimiento, con sus virtudes y su punto débil, es exactamente el alineamiento progresivo.

4.1.1 De dos secuencias a muchas

Un alineamiento múltiple extiende la idea del alineamiento por pares: se escriben k secuencias una debajo de otra, insertando huecos, de modo que los residuos que descienden de un mismo residuo ancestral queden en la misma columna.

Definición 4.1. (Alineamiento múltiple de secuencias)

Sean $x^{(1)}, \dots, x^{(k)}$ secuencias sobre un alfabeto $\mathcal{A}$. Un *alineamiento múltiple* es una matriz $\mathbf{M}$ de k filas y L columnas con entradas en $\mathcal{A} \cup \{-\}$ tal que (i) al eliminar los huecos de la fila p se recupera exactamente $x^{(p)}$ y (ii) ninguna columna está formada solo por huecos. Cada columna es una hipótesis de homología posicional.

Los MSA son la materia prima de buena parte de la bioinformática: de ellos se infieren árboles filogenéticos, se identifican residuos catalíticos, se predicen estructuras secundarias y terciarias, se construyen los perfiles y HMM de la sección 4.3 y se detectan sitios bajo selección. Por eso importa tanto hacerlos bien. Y, como en el caso de dos secuencias, “hacerlos bien” exige primero decidir cómo se puntúan.

4.1.2 La puntuación de suma de pares

La función objetivo más usada es la *suma de pares* (SP, *sum of pairs*): puntuar cada columna como la suma de las puntuaciones de todos los pares de residuos que contiene, con la misma matriz de sustitución que usamos para dos secuencias.

$$S_{\text{SP}}(\mathbf{M}) = \sum_{c=1}^L \sum_{p < r} w_p w_r \sigma(M_{pc}, M_{rc}), \quad \sigma(a, b) = \begin{cases} s(a, b) & a, b \in \mathcal{A} \\ -d & \text{uno de los dos es } - \\ 0 & a = b = - \end{cases} \quad (4.1)$$

DONDE	
M, M_{pc}	Alineamiento múltiple y su símbolo en la fila p , columna c .
L	Número de columnas del alineamiento.
$p < r$	Recorre los $\binom{k}{2}$ pares de filas distintas.
w_p	Peso de la secuencia p (todos iguales a 1 en la versión sin ponderar).
$s(a, b)$	Puntuación de sustitución (p. ej., BLOSUM62, sección 3.3).
d	Penalización por cada par residuo–hueco; en la práctica se usan huecos afines por par.

La SP es natural, pero tiene un defecto conocido: cuenta todos los pares por igual. Si un conjunto contiene diez variantes casi idénticas de una proteína de mamífero y una sola de levadura, la puntuación estará dominada por las comparaciones entre mamíferos, y el alineamiento se ajustará a ellos a costa de la secuencia divergente. Los pesos w_p corrigen ese sesgo. Una receta sencilla y eficaz es la de Henikoff y Henikoff (1994), que asigna más peso a las secuencias que aportan residuos poco comunes en cada columna:

$$w_p \propto \sum_{c=1}^L \frac{1}{r_c n_c(M_{pc})}, \tag{4.2}$$

DONDE	
r_c	Número de símbolos distintos presentes en la columna c .
$n_c(a)$	Número de veces que aparece el símbolo a en la columna c .

Una secuencia que comparte cada residuo con otras nueve recibe, en esa columna, una décima parte del crédito que recibe una que tiene un residuo único. Los pesos se normalizan después para que sumen uno.

Ejemplo 4.1. (Suma de pares de cuatro columnas de globinas)

Las cuatro primeras columnas del alineamiento de cinco globinas que construiremos en esta sección (figura 4.3) son

MYG_HUMAN	M	G	-	L
MYG_PHYMC	M	V	-	L
HBA_HUMAN	M	V	-	L
HBB_HUMAN	M	V	H	L
HBB1_MOUSE	M	V	H	L

Con BLOSUM62, $d = 8$ y pesos unitarios, hay $\binom{5}{2} = 10$ pares por columna:

- Columna 1: diez pares M-M, $10 \times 5 = 50$.
- Columna 2: cuatro pares G-V (-3) y seis V-V (+4): $-12 + 24 = 12$.
- Columna 3: tres pares hueco-hueco (o), seis pares hueco-H (-8) y un par H-H (+8): $-48 + 8 = -40$.
- Columna 4: diez pares L-L, $10 \times 4 = 40$.

El total es $50 + 12 - 40 + 40 = 62$. Los pesos de Henikoff de las cinco filas (tratando el hueco como un símbolo más) resultan 0,267, 0,173, 0,173, 0,194 y 0,194: la mioglobina humana recibe más peso porque su G es única en la columna 2.

4.1.3 Programación dinámica en k dimensiones

La recurrencia de Needleman-Wunsch (ecuación (3.3)) se generaliza sin dificultad conceptual. En lugar de una tabla bidimensional, llenamos un hipercubo con una celda por cada tupla de prefijos $(i_1, \dots, i_k)$. La última columna de un alineamiento óptimo de esos prefijos contiene, en cada fila, o bien el último residuo del prefijo, o bien un hueco; hay por tanto $2^k - 1$ posibilidades (se excluye la columna de solo huecos):

$$F(\mathbf{i}) = \max_{\substack{\delta \in \{0,1\}^k \\ \delta \neq \mathbf{0}}} \left\{ F(\mathbf{i} - \delta) + \text{SP}(\text{columna definida por } \delta) \right\}, \quad (4.3)$$

DONDE

$\mathbf{i} = (i_1, \dots, i_k)$	Vector de longitudes de los prefijos considerados.
δ	Vector binario: $\delta_p = 1$ si la fila p aporta su residuo $x_p^{(p)}$ a la columna; $\delta_p = 0$ si aporta un hueco.
$F(\mathbf{i})$	Mejor puntuación SP de un alineamiento de los k prefijos.

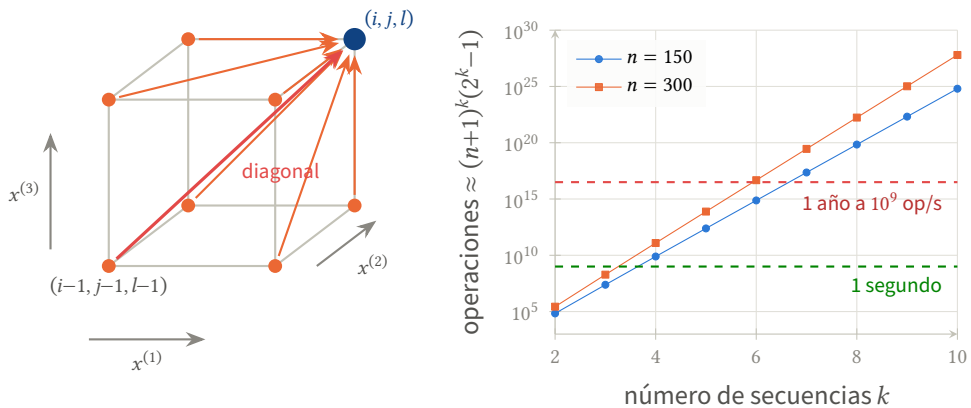


Figura 4.1. El costo del alineamiento múltiple exacto. **Izquierda:** con tres secuencias, cada celda (i, j, l) del cubo de programación dinámica depende de $2^3 - 1 = 7$ vecinas (puntos naranjas): la diagonal principal (rojo) coloca un residuo de cada secuencia en la columna; las otras seis combinan residuos y huecos. **Derecha:** el número de operaciones crece exponencialmente con k . Con proteínas de 150 residuos, la línea de un año de cómputo se cruza ya con siete secuencias; ninguna mejora de hardware alcanzará a una curva así.

Si todas las secuencias miden n , el hipercubo tiene $(n + 1)^k$ celdas, cada una consulta $2^k - 1$ vecinas y cada columna candidata requiere $\binom{k}{2}$ sumas. El coste total es

$$T(n, k) = O(k^2 2^k n^k) \text{ en tiempo,} \quad O(n^k) \text{ en memoria.} \quad (4.4)$$

Para las cinco globinas de este capítulo ($n \approx 150$) eso son unas $151^5 \approx 7,9 \times 10^{10}$ celdas, más de 300 gigabytes de memoria con enteros de 4 bytes, y $2,4 \times 10^{12}$ transiciones. Con diez secuencias, el número de celdas ($\approx 6 \times 10^{21}$) supera la memoria de todos los ordenadores del planeta juntos (figura 4.1).

¿Se trata solo de que no hemos encontrado un algoritmo más astuto? Probablemente no.

Teorema 4.1. (Intratabilidad del MSA)

Encontrar el alineamiento múltiple de puntuación SP óptima es un problema NP-difícil; su versión de decisión es NP-completa aun con matrices de puntuación sencillas (Wang & Jiang, 1994).

En consecuencia, salvo que $P = NP$, no existe un algoritmo que resuelva el MSA exacto en tiempo polinómico en k . Wang y Jiang (1994) demostraron además la

NP-completitud del *alineamiento sobre un árbol*, una variante que puntúa el MSA a lo largo de una filogenia en lugar de sumar todos los pares.

Podar el hipercubo: la cota de Carrillo-Lipman

No todo está perdido para valores pequeños de k . Carrillo y Lipman (1988) observaron que la proyección del MSA óptimo sobre cada par de secuencias es un alineamiento por pares cuya puntuación no puede quedar muy por debajo del óptimo de ese par: si lo hiciera, la suma total sería inferior a la de un MSA heurístico ya conocido. Esta desigualdad define una región alrededor de la diagonal del hipercubo fuera de la cual el camino óptimo no puede pasar, y permite alinear exactamente una decena de proteínas cortas. Más allá, incluso la región podada crece de forma exponencial.

4.1.4 La heurística progresiva

La salida práctica se debe a Feng y Doolittle (1987), que propusieron construir el MSA *de manera progresiva*, siguiendo el orden de parentesco de las secuencias. El procedimiento, que con variaciones usan casi todos los programas actuales, tiene tres etapas:

1. **Distancias.** Se alinean todos los pares de secuencias (o se estima su parecido con un método rápido) y se convierte cada similitud en una distancia d_{ij} , por ejemplo $d_{ij} = 1 - \text{identidad}$.
2. **Árbol guía.** Con la matriz de distancias se construye un árbol mediante un método de agrupamiento (UPGMA o *neighbor-joining*, que estudiaremos a fondo en el capítulo de filogenia).
3. **Alineamiento progresivo.** Se recorren los nodos internos del árbol desde las hojas hacia la raíz; en cada nodo se alinean los dos subalineamientos hijos mediante programación dinámica, y el resultado se congela y se pasa al nodo padre.

UPGMA, el método de agrupamiento más simple, une en cada paso los dos grupos A y B más cercanos y actualiza las distancias como promedio ponderado por tamaño:

$$d(A \cup B, C) = \frac{|A|d(A, C) + |B|d(B, C)}{|A| + |B|}, \quad h(A \cup B) = \frac{1}{2}d(A, B). \quad (4.5)$$

DONDE

A, B, C	Grupos (conjuntos de secuencias) en el paso actual.
$ A $	Número de secuencias del grupo A .
$h(\cdot)$	Altura del nodo nuevo en el dendrograma.

El árbol guía no pretende ser una filogenia correcta; solo es un *orden de trabajo* razonable: alinear primero lo fácil (lo más parecido) y dejar para el final lo difícil.

Alinear perfiles. El paso tres exige alinear no dos secuencias, sino dos *alineamientos*. Para ello cada subalineamiento se resume en un *perfil*: para cada columna u , el vector de frecuencias $f_u(a)$ de cada símbolo. La programación dinámica es idéntica a la de Needleman-Wunsch o Gotoh; solo cambia la puntuación de “alinear la columna u del perfil P con la columna v del perfil Q ”, que se toma como la SP media entre ambas:

$$S(P_u, Q_v) = \sum_{a \in \mathcal{A}} \sum_{b \in \mathcal{A}} f_u^P(a) f_v^Q(b) s(a, b). \quad (4.6)$$

DONDE

P_u, Q_v	Columna u del perfil P y columna v del perfil Q .
$f_u^P(a)$	Fracción (posiblemente ponderada) de secuencias de P con el símbolo a en la columna u .

Alinear una secuencia con otra es el caso particular en que ambos perfiles tienen una sola fila y f es una función indicadora. Como una columna de perfil es una distribución y no una letra, la ecuación (4.6) ya anticipa la idea central de la sección 4.3: puntuar *posiciones* con distribuciones propias.

Ejemplo 4.2. (El árbol guía de cinco globinas)

Las distancias de la figura 4.2 muestran dos parejas muy cercanas: las mioglobinas de humano y cachalote ($d = 0,156$) y las β -globinas de humano y ratón ($d = 0,197$). UPGMA procede así:

1. Une las mioglobinas, a altura $0,156/2 = 0,078$.
2. Une las β -globinas, a altura $0,197/2 \approx 0,099$.

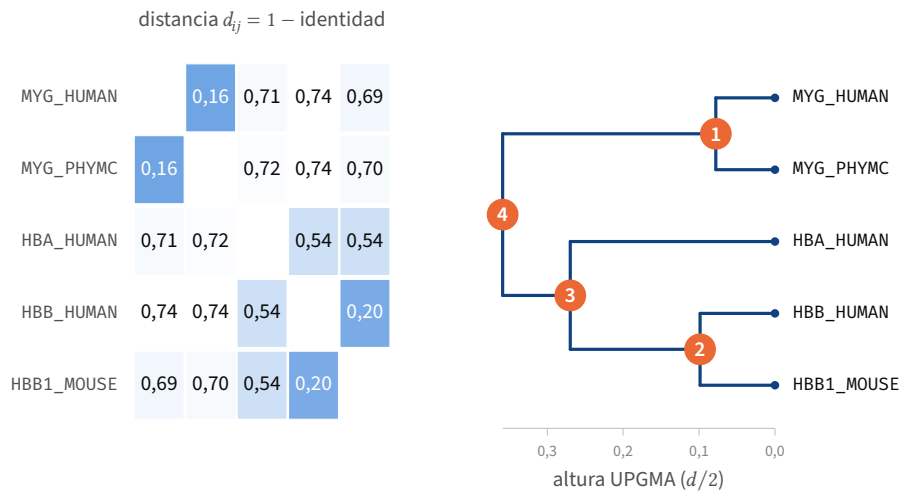


Figura 4.2. Las dos primeras etapas del alineamiento progresivo aplicadas a cinco globinas reales de UniProt. **Izquierda:** distancias $d_{ij} = 1 - \text{identidad}$ calculadas a partir de alineamientos globales por pares (BLOSUM62, huecos $-10/-1$); cuanto más oscuro, más parecidas. **Derecha:** árbol guía UPGMA; los círculos naranjas indican el *orden* en que se alinean los perfiles. Primero se unen las dos mioglobinas (1) y las dos cadenas β (2); luego la cadena α se incorpora a las β (3) y, por último, se alinean entre sí los dos grandes perfiles (4). Calculado con `figuras/cap04/generar.py`.

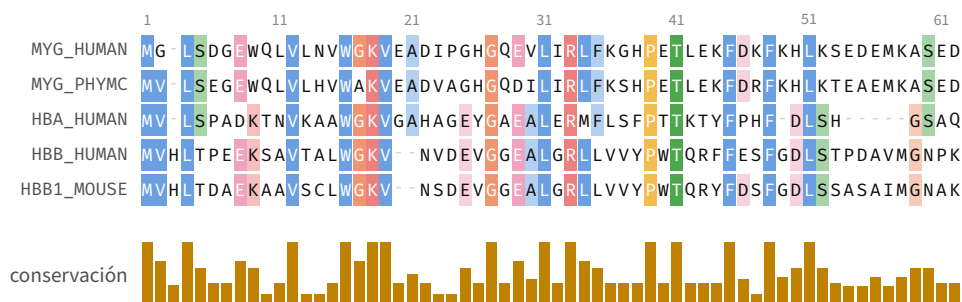


Figura 4.3. Primeras 62 columnas del alineamiento progresivo de las cinco globinas, calculado con perfiles y huecos afines. Los residuos se colorean por clase fisicoquímica (azul, hidrofóbicos; rojo, básicos; magenta, ácidos; verde, polares; naranja, glicina; amarillo, prolina; aqua, histidina y tirosina) cuando al menos el 60 % (claro) u 80 % (intenso) de la columna comparte el residuo. Las barras inferiores miden la conservación, $1 - H_c / \log_2 5$, penalizada por los huecos. Destacan el triptófano invariante de la columna 16 y el hueco de cinco posiciones de la cadena α (columnas 54–58), una delección real de la hemoglobina α respecto a la β .

3. Actualiza la distancia de HBA_HUMAN al grupo β con la ecuación (4.5): $\frac{1}{2}(0,536 + 0,543) = 0,539$. Es la menor, así que la cadena α se une a las β a altura 0,270.
4. Quedan dos grupos (mioglobinas y hemoglobinas); su distancia es la media de los seis pares cruzados, $4,305/6 \approx 0,718$, y la raíz queda a altura 0,359.

El orden de alineamiento reproduce la biología: las cadenas α y β de la hemoglobina proceden de una duplicación génica más reciente que la que separó a la mioglobina. El MSA resultante, de 155 columnas, tiene una puntuación SP de 2473 (BLOSUM62, huecos afines $-10/-1$ por par).

De Feng-Doolittle a la familia Clustal

El artículo de Feng y Doolittle (1987) tenía un objetivo filogenético: obtener alineamientos coherentes para construir árboles. Un año después, Higgins y Sharp (1988) publicaron CLUSTAL, un paquete diseñado para ejecutarse en los microordenadores personales de la época, que combinaba alineamientos rápidos por pares, un dendro-

grama y alineamiento progresivo de perfiles. Su sucesor, CLUSTAL W (Thompson et al., 1994), añadió tres ideas que siguen vigentes: ponderar las secuencias para compensar la sobrerrepresentación, penalizar los huecos de forma distinta según la posición (más baratos donde ya hay huecos o en regiones hidrofílicas) y elegir la matriz de sustitución según la divergencia de las secuencias que se alinean en cada nodo.

4.1.5 “Una vez un hueco, siempre un hueco”

El alineamiento progresivo es *codicioso*: cada nodo toma la mejor decisión local y la congela. Feng y Doolittle (1987) lo convirtieron en regla explícita: una vez que un hueco se introduce en un subalineamiento, no se elimina en pasos posteriores. La regla es sensata (los huecos entre secuencias cercanas son los más fiables) pero tiene un costo evidente: un error cometido al alinear las dos primeras secuencias se propaga a todo el alineamiento final, y ninguna información aportada después por las demás secuencias puede corregirlo.

Errores que se heredan

Si en el paso 1 de la figura 4.2 la programación dinámica hubiera colocado un hueco de las mioglobinas un par de posiciones a la izquierda de su lugar correcto (un empate de puntuación basta), ese hueco seguiría allí al final, aunque las tres hemoglobinas “votaran” por otra ubicación. Conviene revisar los huecos y comparar varios programas.

La historia de los programas posteriores es, en buena medida, la historia de los intentos de mitigar este problema sin perder la velocidad del método progresivo (figura 4.4).

Refinamiento iterativo. Una vez obtenido un MSA, se divide el árbol guía quitando una rama, lo que parte las secuencias en dos grupos; se extrae el subalineamiento de cada grupo, se eliminan las columnas que hayan quedado solo con huecos y se vuelven a alinear los dos perfiles. Si la SP mejora, se acepta el cambio. MUSCLE (Edgar, 2004) aplica este refinamiento restringido por el árbol tras dos etapas progresivas: la primera usa distancias rápidas por conteo de k -meros; la segunda recalcula el árbol a partir del primer MSA. Su función de puntuación perfil-perfil, la *log-expectation*, sustituye a la SP media de la ecuación (4.6).

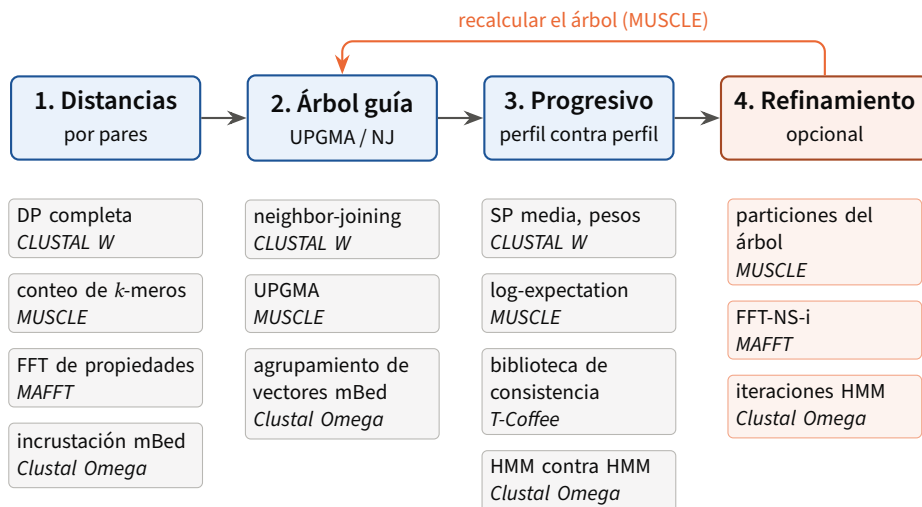


Figura 4.4. Casi todos los programas modernos de MSA comparten el esqueleto progresivo de Feng y Doolittle (1987) y se distinguen en cómo implementan cada etapa. Debajo de cada caja se indican, en cursiva, los programas que popularizaron cada variante. Las etapas 1 y 2 determinan la *velocidad*; las etapas 3 y 4, la *exactitud*. La flecha superior representa la segunda iteración de MUSCLE, que reconstruye el árbol a partir del primer alineamiento.

Consistencia. T-Coffee (Notredame et al., 2000) ataca el problema desde otro ángulo: antes de alinear nada, construye una *biblioteca* con todos los alineamientos por pares, globales y locales. El peso de alinear el residuo $x_i^{(1)}$ con $x_j^{(2)}$ aumenta si existe una tercera secuencia $x^{(3)}$ en la que ambos residuos se alinean con un mismo residuo $x_l^{(3)}$ (*extensión* de la biblioteca). Así, cuando el método progresivo alinea dos secuencias, “consulta” implícitamente a todas las demás, y la decisión codiciosa deja de serlo tanto.

Velocidad. MAFFT (Kato et al., 2002) convierte cada proteína en una señal numérica de volumen y polaridad de sus residuos y usa la transformada rápida de Fourier para localizar en segundos las regiones homólogas, que luego se alinean con programación dinámica restringida; ofrece un modo progresivo (FFT-NS-2) y uno con refinamiento iterativo (FFT-NS-i). Clustal Omega (Sievers et al., 2011) escala a conjuntos de decenas de miles de secuencias o más construyendo el árbol guía a partir de distancias a un subconjunto de secuencias de referencia (mBed) y alineando los perfiles como HMM contra HMM.

¿Cuál es el mejor? La pregunta solo tiene respuesta empírica. Los programas se comparan con *benchmarks* de alineamientos de referencia, casi siempre derivados de superposiciones de estructuras tridimensionales, como BALiBASE o PREFAB; la exactitud se mide como la fracción de pares de residuos (o de columnas) de la referencia que el programa reproduce. La revisión de Edgar y Batzoglou (2006) resume las lecciones de esas comparaciones: la consistencia y el refinamiento mejoran la exactitud sobre todo en familias divergentes (por debajo de un 25–30 % de identidad), donde ningún método es fiable, mientras que con secuencias cercanas todos los programas dan resultados muy parecidos.

```
msa_globinas.py

import subprocess
from Bio import AlignIO

# Clustal Omega: FASTA de entrada, formato Clustal de salida
subprocess.run(["clustalo", "-i", "globinas.fasta",
                "-o", "globinas.aln", "--outfmt=clu",
                "--force"], check=True)
aln = AlignIO.read("globinas.aln", "clustal")
print(aln.get_alignment_length(), "columnas")
for fila in aln:
    print(f"{fila.id:12s} {str(fila.seq[:60])}")

filas = [str(f.seq) for f in aln]
print("SP =", suma_de_pares(filas))          # ver ejercicio 4.2
print("pesos =", pesos_henikoff(filas))
```

Las funciones `suma_de_pares` y `pesos_henikoff` implementan la ecuación (4.1) con penalización lineal y la ecuación (4.2). Un ejercicio muy instructivo es comparar los MSA de Clustal Omega, MAFFT (`mafft --auto`) y MUSCLE sobre las mismas globinas y visualizar las columnas en las que discrepan.

4.2 Motivos, matrices de peso posicional y logos



Reconocer una firma

Nadie firma dos veces exactamente igual. Aun así, un empleado de banco experimentado reconoce la firma de un cliente: sabe que la inicial siempre tiene el mismo bucle, que el trazo final a veces sube y a veces no, y que el punto sobre la *i* es opcional. En su cabeza no hay una firma “correcta”, sino una idea de qué rasgos son obligatorios, cuáles son frecuentes y cuáles dan igual. Una proteína que se une al ADN hace algo muy parecido: tolera variación en algunas posiciones de su sitio de unión y es muy exigente en otras. Esta sección enseña a escribir esa “idea de la firma” como una tabla de números.

4.2.1 Motivos y secuencias consenso

Un *motivo* es un patrón corto y recurrente, de ADN o de proteína, asociado a una función: el sitio donde se une un factor de transcripción, una señal de corte y empalme, el sitio de unión al ribosoma o un segmento catalítico. A diferencia de los dominios completos de la sección 4.3, los motivos de ADN suelen medir entre 6 y 20 posiciones, no contienen huecos y se presentan como *sitios alineados* de igual longitud.

Tomemos un ejemplo clásico. La proteína de unión a la caja TATA (TBP) reconoce, unas 30 bases antes del inicio de la transcripción de muchos genes eucariotas, una secuencia rica en **A** y **T**. La base de datos JASPAR (Castro-Mondragon et al., 2022) guarda su perfil MA0108.2, construido a partir de 389 sitios alineados de 15 bases. La forma más antigua de resumir esos sitios es la *secuencia consenso*, que toma la base mayoritaria de cada columna:

GTATAAAA**GG**GGGG

El consenso es cómodo, pero pierde casi toda la información. No distingue una columna en la que el 96 % de los sitios tiene una **T** (la posición 4) de otra en la que la **G** gana con apenas un 39 % (la posición 1). Y un sitio real con una sola discrepancia respecto al consenso puede ser mucho mejor, o mucho peor, que otro con tres. Necesitamos una representación *cuantitativa*.

4.2.2 De sitios a frecuencias: conteos y pseudoconteos

Sea un conjunto de N sitios alineados de longitud W . La *matriz de conteos* (o de frecuencias de posición, PFM) registra cuántas veces aparece cada base en cada columna:

$$n_{b,i} = \sum_{s=1}^N \mathbb{1}[x_i^{(s)} = b], \quad b \in \{A, C, G, T\}, \quad i = 1, \dots, W. \quad (4.7)$$

DONDE

$x_i^{(s)}$ Base en la posición i del sitio s .

$n_{b,i}$ Número de sitios con la base b en la posición i ; $\sum_b n_{b,i} = N$.

La estimación de máxima verosimilitud de la probabilidad de la base b en la posición i es $n_{b,i}/N$. Con pocas observaciones, esta estimación es peligrosa: si una base no aparece en ninguno de los sitios conocidos, su probabilidad estimada es exactamente cero, y cualquier sitio futuro que la contenga será declarado *imposible*. La solución estándar es sumar *pseudoconteos*:

$$f_{b,i} = \frac{n_{b,i} + \alpha_b}{N + \sum_{b'} \alpha_{b'}}. \quad (4.8)$$

DONDE

$f_{b,i}$ Probabilidad estimada de la base b en la posición i .

α_b Pseudoconteo de la base b (p. ej., $\alpha_b = 0,5$, o $\alpha_b = \beta q_b$ proporcional al fondo).

Desde el punto de vista bayesiano, la ecuación (4.8) es la media posterior de una distribución multinomial con una distribución previa de Dirichlet de parámetros α_b : los pseudoconteos son “observaciones imaginarias” que expresan lo que creíamos antes de ver los datos. Cuando N es grande pesan poco; cuando N es pequeño, dominan. Para proteínas, en las que el alfabeto tiene 20 letras y los datos son siempre escasos, conviene usar mezclas de distribuciones de Dirichlet que codifican preferencias fisicoquímicas (Sjölander et al., 1996): si una columna contiene solo I y L, la previa “sabe” que una V es mucho más plausible que una D.

4.2.3 La matriz de peso posicional

La pregunta que queremos responder ante una ventana $y = y_1 \dots y_W$ de una secuencia es la misma del capítulo 3: ¿es más verosímil bajo el modelo del motivo o

bajo el modelo de fondo? Si suponemos que las posiciones son independientes, la razón de verosimilitudes se factoriza y su logaritmo es una suma:

$$W_{b,i} = \log_2 \frac{f_{b,i}}{q_b}, \quad S(y) = \log_2 \frac{\mathbb{P}(y \mid \text{motivo})}{\mathbb{P}(y \mid \text{fondo})} = \sum_{i=1}^W W_{y_i,i}. \quad (4.9)$$

DONDE	
$W_{b,i}$	Entrada de la matriz de peso posicional (PWM o PSSM), en bits.
q_b	Frecuencia de fondo de la base b (p. ej., 0,25, o la composición del genoma).
y	Ventana candidata de longitud W .
$S(y)$	Puntuación de la ventana: log-razón de verosimilitudes motivo/fondo.

Esta es exactamente la lógica de las matrices de sustitución (ecuación (3.9)), aplicada ahora a una sola secuencia contra un modelo *posicional*. Una entrada positiva indica que la base es más frecuente en los sitios que en el fondo; una negativa, lo contrario; y la suma de las entradas a lo largo de la ventana decide.

 **Un perceptrón para los ribosomas**

Las matrices de peso nacieron antes que su interpretación probabilística. Stormo et al. (1982) buscaban distinguir los verdaderos sitios de inicio de la traducción en *Escherichia coli* de los codones AUG que no se usan, y entrenaron para ello un *perceptrón*: una tabla de pesos, uno por base y posición, que suma sus valores sobre la ventana y compara el total con un umbral. Los pesos aprendidos resultaron ser un buen clasificador y, a la vez, una descripción legible del sitio. Dos décadas después, Stormo (2000) mostró que, bajo el supuesto de aditividad, la puntuación de una PWM es proporcional a la energía libre de unión de la proteína con el sitio, lo que da a la matriz un significado físico además del estadístico.

Ejemplo 4.3. (La columna 3 del perfil de TBP)

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	-0,7	-2,6	1,9	-4,8	1,9	1,5	1,9	1,2	0,7	-0,8	-0,2	-0,2	-0,2	-0,5	-0,3
C	0,6	-1,1	-7,6	-3,2	-7,6	-7,6	-4,8	-5,3	-1,1	0,5	0,6	0,4	0,3	0,1	0,1
G	0,6	-2,4	-5,3	-5,3	-4,2	-7,6	-2,3	-1,1	0,7	0,6	0,4	0,4	0,4	0,5	0,5
T	-1,6	1,7	-1,5	1,9	-1,7	0,3	-3,9	0,3	-1,5	-1,0	-1,6	-0,9	-0,7	-0,4	-0,5
R_i (bits)	0,23	0,97	1,49	1,69	1,49	1,08	1,51	0,61	0,28	0,16	0,17	0,08	0,06	0,06	0,05

Figura 4.5. Matriz de peso posicional $W_{b,i} = \log_2(f_{b,i}/0,25)$ del perfil de TBP (JASPAR MA0108.2, 389 sitios), con pseudoconteos $\alpha_b = 0,5$. Azul: bases favorecidas; rojo: desfavorecidas. El núcleo **TATAAA** (posiciones 2–7) concentra los pesos extremos; en la posición 3, la **C** vale $-7,6$ bits: verla en un sitio multiplica su verosimilitud por $2^{-7,6} \approx 1/190$. La fila inferior da el contenido de información R_i de cada columna (ecuación (4.11)); las posiciones 10–15 apenas aportan señal.

En la posición 3 del perfil MA0108.2 los conteos son $n_A = 352$, $n_C = 0$, $n_G = 2$, $n_T = 35$ ($N = 389$). Con $\alpha_b = 0,5$ el denominador de la ecuación (4.8) es $389 + 2 = 391$ y

$$\begin{aligned} f_{A,3} &= \frac{352,5}{391} = 0,9015, & f_{C,3} &= \frac{0,5}{391} = 0,0013, \\ f_{G,3} &= \frac{2,5}{391} = 0,0064, & f_{T,3} &= \frac{35,5}{391} = 0,0908. \end{aligned}$$

Con fondo uniforme, los pesos son $W_{A,3} = \log_2(0,9015/0,25) = 1,85$, $W_{C,3} = -7,61$, $W_{G,3} = -5,29$ y $W_{T,3} = -1,46$ bits. Sin pseudoconteos, $W_{C,3}$ sería $-\infty$ y un único error de secuenciación en esa posición anularía cualquier sitio. Sumando el máximo de cada columna se obtiene la puntuación del consenso, 16,21 bits; la peor secuencia posible puntúa $-49,13$.

4.2.4 Entropía, información y logos

¿Cuánta “señal” hay en cada columna? Claude Shannon definió la incertidumbre de una variable aleatoria discreta como su *entropía* (Shannon, 1948). Aplicada a la columna i de un motivo,

$$H_i = - \sum_b f_{b,i} \log_2 f_{b,i} \quad (\text{bits}). \quad (4.10)$$

DONDE

H_i Entropía de la columna i : número medio de preguntas de sí o no necesarias para adivinar la base.

Una columna uniforme ($f_{b,i} = 1/4$) tiene la entropía máxima, $\log_2 4 = 2$ bits; una columna invariante, entropía cero. El *contenido de información* de la columna es la reducción de incertidumbre que produce conocer que estamos en un sitio:

$$R_i = \log_2 4 - (H_i + e_N). \quad (4.11)$$

DONDE

R_i Contenido de información de la posición i , entre 0 y 2 bits para ADN.

e_N Corrección por muestra pequeña: con pocos sitios, H_i se subestima sistemáticamente; $e_N \rightarrow 0$ cuando $N \rightarrow \infty$.

Schneider y Stephens (1990) usaron R_i para inventar el *sequence logo*: en cada posición se apilan las cuatro letras, ordenadas de menor a mayor frecuencia, con altura

$$h_{b,i} = f_{b,i} R_i, \quad (4.12)$$

de modo que la altura total de la pila es R_i y la proporción de cada letra dentro de la pila es su frecuencia. De un vistazo, el logo muestra *qué* posiciones importan (pilas altas) y *qué* base se prefiere en ellas (la letra de arriba). WebLogo (Crooks et al., 2004) popularizó la herramienta en la web y es hoy el estándar de facto.

Con un fondo no uniforme (un genoma rico en A/T, por ejemplo), la medida adecuada es la *entropía relativa* o divergencia de Kullback-Leibler entre la columna y el fondo:

$$D_i = D_{\text{KL}}(f_{\cdot,i} \| q) = \sum_b f_{b,i} \log_2 \frac{f_{b,i}}{q_b} \geq 0, \quad (4.13)$$

que se reduce a R_i (sin corrección) cuando $q_b = 1/4$. Esta cantidad tiene una interpretación que conecta las dos mitades de la sección.

Teorema 4.2. (Puntuación esperada de una PWM)

Si los sitios se generan con las frecuencias $f_{b,i}$ del propio modelo, la puntuación esperada de la ecuación (4.9) es $\mathbb{E}_f[S] = \sum_i D_{\text{KL}}(f_{\cdot,i} \| q)$. Si las ventanas proceden del fondo, $\mathbb{E}_q[S] = -\sum_i D_{\text{KL}}(q \| f_{\cdot,i}) \leq 0$.

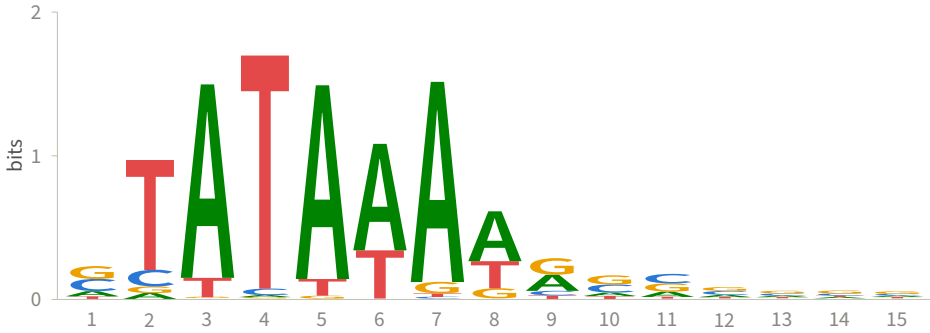


Figura 4.6. *Sequence logo* del perfil de TBP, dibujado en TikZ directamente a partir de las frecuencias de la figura 4.5: la altura de cada letra es $f_{b,i}R_i$ (ecuación (4.12)). Las posiciones 3–7 alternan A y T con más de 1,4 bits cada una; la posición 6 muestra que TBP tolera tanto A como T; la cola G/C de las posiciones 9–15 es casi ruido (menos de 0,3 bits). El contenido total del motivo es $\sum_i R_i \approx 9,9$ bits.

La demostración es una línea: por linealidad de la esperanza, $E_f[S] = \sum_i \sum_b f_{b,i} W_{b,i}$, que es la ecuación (4.13) sumada sobre las posiciones. El teorema dice que el contenido de información de un motivo es la *separación media*, en bits, entre sitios y fondo. Para TBP, $\sum_i D_i = 9,93$ bits, y una simulación de 20 000 sitios da una media de 9,95; las ventanas aleatorias promedian $-18,09$ bits, en perfecto acuerdo con $-\sum_i D_{\text{KL}}(q \| f_{\cdot,i}) = -18,09$ (figura 4.8). Un motivo con R bits aparece por azar, aproximadamente, una vez cada 2^R posiciones: el de TBP, una vez cada mil bases.

4.2.5 Buscar sitios en una secuencia

Con la PWM en la mano, buscar sitios es sencillo: se desliza una ventana de longitud W por la secuencia (y por su reverso complementario), se calcula $S(y)$ en cada posición y se reportan las que superan un umbral τ . El coste es $O(nW)$ para una secuencia de longitud n . La dificultad no es computacional, sino estadística: elegir τ .

La figura 4.8 muestra el dilema. Un umbral de 6 bits deja pasar solo un 0,19 % de las ventanas aleatorias, lo que parece poco; pero el genoma humano tiene unos $3,1 \times 10^9$ pares de bases y dos hebras, así que esperaríamos $2 \times 3,1 \times 10^9 \times 0,0019 \approx 1,2 \times 10^7$ “sitios” de TBP, la inmensa mayoría sin ninguna función. Esta abundancia de aciertos espurios no es un defecto de la PWM: refleja que la especificidad de un factor de transcripción *in vivo* depende también de la accesibilidad de la cromatina, de otros factores cercanos y de la posición respecto a los genes, información que la matriz no contiene.

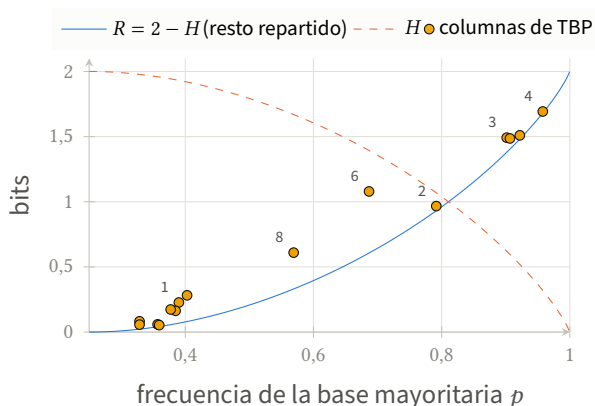


Figura 4.7. Entropía H (naranja) y contenido de información $R = 2 - H$ (azul) de una columna de ADN en la que la base mayoritaria tiene frecuencia p y las otras tres se reparten el resto por igual. Los puntos son las 15 columnas reales de TBP (se numeran algunas; las posiciones 9–15 se agrupan cerca de $p \approx 0,4$ y las 5 y 7 junto a la 3). Caen sobre la curva o por encima de ella, porque en ellas el resto no se reparte por igual (p. ej., la posición 6 mezcla A y T), y un reparto desigual *aumenta* la información. La relación no es lineal: pasar del 90 % al 99 % de conservación añade más información que pasar del 40 % al 60 %.

motivo_tbp.py

```
from Bio import motifs
from Bio.Seq import Seq

with open("MA0108.2.jaspar") as fh:           # descargado de JASPAR
    m = motifs.read(fh, "jaspar")
pwm = m.counts.normalize(pseudocounts=0.5)    # f con pseudoconteos
pssm = pwm.log_odds()                        # W = log2(f / q)
print(m.name, m.consensus)                   # TBP GTATAAAAGGCGGGG
print(f"máx={pssm.max:.2f} mín={pssm.min:.2f} "
      f"media en sitios={pssm.mean():.2f}")   # 16.21 -49.13 9.93

prom = Seq("GGCCGCTATAAAAGGCGCGGCATGCGCCGCCATG")
for pos, score in pssm.search(prom, threshold=6.0):
    hebra = "+" if pos >= 0 else "-"          # pos < 0: hebra reversa
    print(pos, hebra, f"{score:.2f}")        # 5 + 15.55
```

El módulo `Bio.motifs` de Biopython (Cock et al., 2009) implementa las ecuaciones (4.8) y (4.9); su método `mean()` devuelve precisamente la puntuación esperada

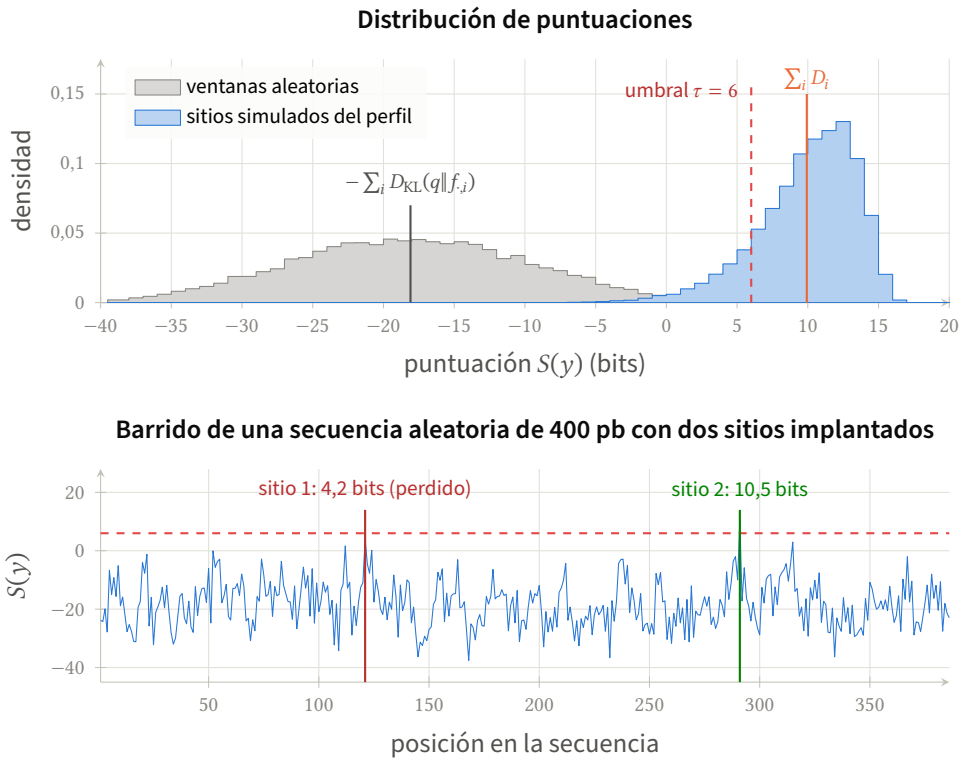


Figura 4.8. Arriba: distribución de $S(y)$ para 20 000 sitios generados a partir de las frecuencias de TBP (azul) y 20 000 ventanas de fondo uniforme (gris). Sus medias coinciden con las predicciones del teorema 4.2. Con $\tau = 6$ bits se recupera el 87 % de los sitios y solo pasa el 0,19 % de las ventanas aleatorias. Abajo: barrido de una secuencia aleatoria en la que se implantaron dos sitios muestreados del perfil. El sitio 2 (GTATAAGTAGCGAGG) destaca claramente; el sitio 1 (CCATAATAACTAGCG), también muestreado del modelo, puntúa solo 4,2 bits y cae bajo el umbral: un falso negativo tan legítimo como inevitable.

del teorema 4.2. La secuencia prom es un fragmento sintético con una caja TATA en la posición 5 (contando desde cero).

El supuesto de independencia

La ecuación (4.9) supone que cada posición contribuye de forma independiente a la unión. Es una aproximación sorprendentemente buena para muchos factores, pero falla cuando dos bases vecinas interactúan (por ejemplo, por la flexibilidad del ADN) o cuando el factor se une como dímero con espaciadores variables. En esos casos se usan modelos con dependencias entre posiciones, o los modelos de Márkov de la sección siguiente.

Descubrir motivos sin conocerlos

Hasta ahora partimos de sitios ya alineados. El problema inverso, encontrar un motivo desconocido compartido por un conjunto de secuencias (por ejemplo, promotores de genes coexpresados), es mucho más difícil, porque hay que estimar a la vez la PWM y la posición del sitio en cada secuencia. Lawrence et al. (1993) lo resolvieron con un muestreador de Gibbs: se elige al azar una posición en cada secuencia, se construye la PWM con todas menos una, se remuestra la posición de la excluida en proporción a $2^{S(y)}$ y se repite. El método converge hacia los motivos de alto contenido de información. Las alternativas deterministas se basan en el algoritmo EM (Dempster et al., 1977), el mismo principio que usaremos para entrenar HMM.

4.3 Modelos ocultos de Márkov y perfiles (HMMER)

 Colab · 4.3 · HMM y perfiles con HMMER

 repositorio del curso

Un partido por la radio

Imagine que escucha un partido de fútbol por una radio extranjera, en un idioma que no entiende. No ve el campo, pero oye el rugido del público: murmullo, silbidos, gritos. Cuando el equipo local ataca, los gritos son frecuentes; cuando defiende, predominan los silbidos. Además, la posesión no cambia a cada segundo: quien ataca suele seguir atacando un rato. Con esas dos regularidades (qué sonidos produce cada situación y cuánto duran

las situaciones) usted puede reconstruir, con bastante acierto, la historia del partido que no vio. Un modelo oculto de Márkov formaliza exactamente ese razonamiento: estados que no se observan, que persisten y cambian con ciertas probabilidades, y que emiten señales que sí se observan.

4.3.1 Estados, transiciones y emisiones

En bioinformática los “sonidos” son nucleótidos o aminoácidos, y los estados ocultos son categorías biológicas: región codificante o intergénica, isla CpG o ADN ordinario, hélice o lámina, posición 17 de un dominio o inserción. Eddy (2004) ofrece una introducción breve y muy clara; el tutorial de Rabiner (1989), escrito para el reconocimiento del habla, sigue siendo la referencia clásica.

Definición 4.2. (Modelo oculto de Márkov)

Un HMM con K estados sobre un alfabeto $\mathcal{A}$ es una terna $\theta = (\pi, \mathbf{A}, \mathbf{E})$ que genera un camino de estados $\pi_1 \pi_2 \dots \pi_L$ y una secuencia $x_1 \dots x_L$: el primer estado se elige con probabilidad π_k ; cada estado siguiente depende solo del anterior, con probabilidad a_{kl} ; y cada estado π_i emite el símbolo x_i con probabilidad $e_{\pi_i}(x_i)$. Se observa x ; el camino permanece oculto.

La probabilidad conjunta de una secuencia y un camino es el producto de las decisiones que los generaron:

$$P(x, \pi \mid \theta) = \pi_{\pi_1} e_{\pi_1}(x_1) \prod_{i=2}^L a_{\pi_{i-1}\pi_i} e_{\pi_i}(x_i). \quad (4.14)$$

DONDE

$\pi = \pi_1 \dots \pi_L$	Camino de estados ocultos (no confundir con π_k , la probabilidad inicial).
π_k	Probabilidad de empezar en el estado k .
a_{kl}	Probabilidad de transición del estado k al l ; $\sum_l a_{kl} = 1$.
$e_k(b)$	Probabilidad de que el estado k emita el símbolo b ; $\sum_b e_k(b) = 1$.
L	Longitud de la secuencia observada.

Usaremos a lo largo de la sección un modelo de juguete con dos estados para ADN: H, regiones ricas en G+C (como las islas CpG), y L, regiones pobres en G+C

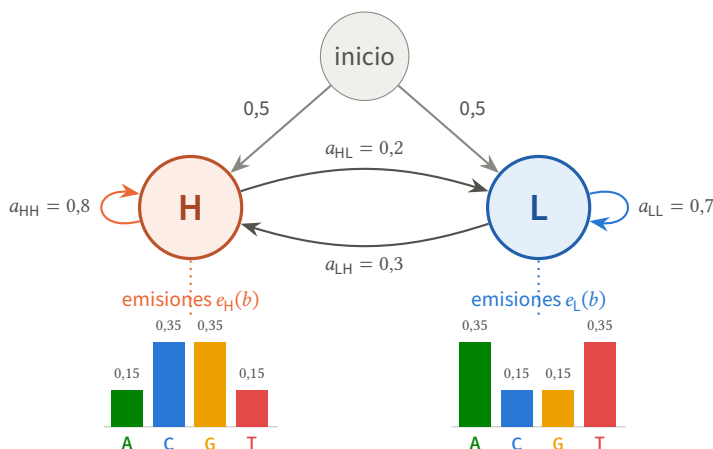


Figura 4.9. El modelo de juguete de esta sección. Dos estados ocultos, H (rico en G+C) y L (pobre), con transiciones que favorecen la permanencia en el mismo estado, y distribuciones de emisión opuestas (barras). Ninguna base individual revela el estado: una G es $0,35/0,15 \approx 2,3$ veces más probable en H que en L, pero puede proceder de ambos. La evidencia sobre el estado se acumula a lo largo de las posiciones consecutivas, porque los estados persisten.

(figura 4.9). El modelo plantea los tres problemas canónicos que Rabiner (1989) ordenó:

1. **Evaluación:** ¿cuál es la probabilidad $\mathbb{P}(x \mid \theta)$ de la secuencia, sumando sobre todos los caminos? (algoritmo *forward*).
2. **Decodificación:** ¿cuál es el camino de estados más probable? (algoritmo de Viterbi).
3. **Aprendizaje:** ¿qué parámetros θ hacen más probables las secuencias observadas? (algoritmo de Baum-Welch).

4.3.2 El algoritmo forward: sumar sobre todos los caminos

Hay K^L caminos posibles; para $K = 2$ y $L = 300$, unos 10^{90} . Por suerte, la estructura de Márkov permite reutilizar cálculos, igual que en la programación dinámica del capítulo 3, pero *sumando* en lugar de maximizar. Definimos la variable

forward como la probabilidad de haber emitido los primeros i símbolos y estar en el estado k :

$$f_k(1) = \pi_k e_k(x_1), \quad (4.15)$$

$$f_k(i+1) = e_l(x_{i+1}) \sum_{k=1}^K f_k(i) a_{kl}, \quad i = 1, \dots, L-1, \quad (4.16)$$

$$P(x | \theta) = \sum_{k=1}^K f_k(L). \quad (4.17)$$

DONDE

$f_k(i)$ $P(x_1 \dots x_i, \pi_i = k)$: probabilidad del prefijo terminando en el estado k .

K Número de estados.

El costo es $O(LK^2)$: en cada posición, cada estado suma las contribuciones de todos los estados anteriores. El algoritmo simétrico, *backward*, recorre la secuencia desde el final y calcula la probabilidad del sufijo dado el estado:

$$b_k(L) = 1, \quad b_k(i) = \sum_{l=1}^K a_{kl} e_l(x_{i+1}) b_l(i+1), \quad i = L-1, \dots, 1. \quad (4.18)$$

DONDE

$b_k(i)$ $P(x_{i+1} \dots x_L | \pi_i = k)$: probabilidad del sufijo dado que en i estamos en k .

Combinando ambos se obtiene la *probabilidad posterior* de cada estado en cada posición, la cantidad más útil en la práctica:

$$P(\pi_i = k | x, \theta) = \frac{f_k(i) b_k(i)}{P(x | \theta)}. \quad (4.19)$$

Ejemplo 4.4. (Forward a mano)

Con el modelo de la figura 4.9 y $x = \text{GCGATTAT}$:

$$f_H(1) = 0,5 \times 0,35 = 0,175, \quad f_L(1) = 0,5 \times 0,15 = 0,075.$$

Para la segunda base (C), según la ecuación (4.16),

$$f_H(2) = 0,35 (0,175 \times 0,8 + 0,075 \times 0,3) = 0,35 \times 0,1625 = 0,0569,$$

$$f_L(2) = 0,15 (0,175 \times 0,2 + 0,075 \times 0,7) = 0,15 \times 0,0875 = 0,0131.$$

Continuando hasta $i = 9$ y sumando (ecuación (4.17)) se obtiene $P(x | \theta) = 5,10 \times 10^{-6}$, es decir, $\log_2 P(x) = -17,58$ bits. Las probabilidades posteriores del estado H (ecuación (4.19)) son 0,80, 0,87, 0,86, 0,75 en las cuatro primeras posiciones (GCGC) y caen a 0,36, 0,22, 0,18, 0,18, 0,24 en el tramo ATTAT.

⚠ Desbordamiento por abajo

Las probabilidades forward decrecen exponencialmente con L . Para una secuencia de 5000 bases como la que usaremos para entrenar el modelo, $P(x) \approx 2^{-9712}$, un número que ningún formato de coma flotante puede representar (el menor double positivo es del orden de 2^{-1074}). Hay dos soluciones: trabajar en escala logarítmica, sustituyendo las sumas por $\log \sum \exp$ (p. ej., `numpy.logaddexp`), o *reescalar* $f_k(i)$ en cada posición dividiendo por $c_i = \sum_k f_k(i)$ y acumulando $\log P(x) = \sum_i \log c_i$. HMMER usa una variante de la segunda, el *sparse rescaling* (Eddy, 2011).

4.3.3 El algoritmo de Viterbi: el mejor camino

Si en lugar de sumar sobre caminos queremos el *más probable*, basta con cambiar la suma de la ecuación (4.16) por un máximo y recordar de dónde vino cada máximo. El algoritmo fue propuesto por Viterbi (1967) para decodificar códigos convolucionales en telecomunicaciones.

Teorema 4.3. (Algoritmo de Viterbi)

Sea $v_k(i)$ la probabilidad del camino más probable que emite $x_1 \cdots x_i$ y termina en el estado k . Entonces

$$v_k(1) = \pi_k e_k(x_1), \quad (4.20)$$

$$v_l(i+1) = e_l(x_{i+1}) \max_k [v_k(i) a_{kl}], \quad (4.21)$$

y $P(x, \pi^*) = \max_k v_k(L)$, donde π^* es el camino óptimo. El costo es $O(LK^2)$ en tiempo y $O(LK)$ en memoria.

DONDE

$v_k(i)$	Probabilidad del mejor camino parcial que termina en k en la posición i .
π^*	Camino de Viterbi: $\arg \max_{\pi} P(x, \pi)$.
$\text{ptr}_i(l)$	Puntero: el estado k que maximizó la ecuación (4.21) para llegar a l en i .

El camino se recupera, como en Needleman-Wunsch, con un *traceback*: $\pi_L^* = \arg \max_k v_k(L)$ y, hacia atrás, $\pi_{i-1}^* = \text{ptr}_i(\pi_i^*)$. En la práctica se trabaja con logaritmos, de modo que los productos se vuelven sumas y la ecuación (4.21) adopta la forma exacta de una recurrencia de programación dinámica: $V_l(i+1) = \log e_l(x_{i+1}) + \max_k [V_k(i) + \log a_{kl}]$.

Ejemplo 4.5. (Viterbi frente a posterior)

En la figura 4.10, $\log_2 v_L(9) = -19,98$ supera a $\log_2 v_H(9) = -22,42$, así que el camino termina en L. Su probabilidad conjunta es $2^{-19,98}$, y como $P(x) = 2^{-17,58}$, la probabilidad posterior del camino completo es

$$P(\pi^* | x) = \frac{2^{-19,98}}{2^{-17,58}} = 2^{-2,40} \approx 0,19.$$

El camino más probable acapara menos de una quinta parte de la probabilidad: el 81 % restante se reparte entre los otros 511 caminos. Por eso, cuando interesa la etiqueta de cada posición más que un camino global coherente, se prefiere la *decodificación posterior*, que asigna a cada i el estado con mayor ecuación (4.19).

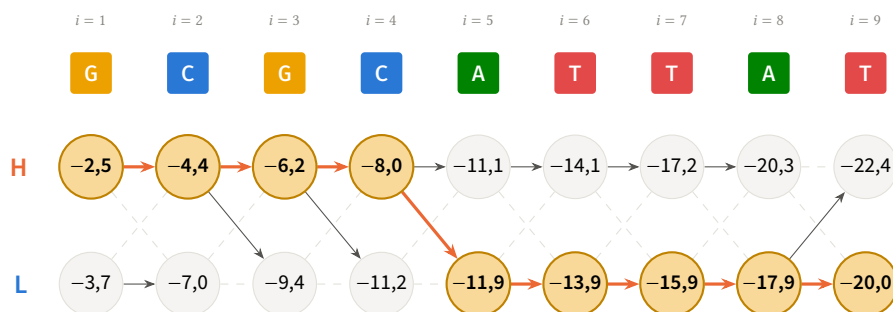


Figura 4.10. Enrejado (*trellis*) de Viterbi para $x = \text{GCGCATTAT}$ con el modelo de la figura 4.9. Cada nodo muestra $\log_2 v_k(i)$; las flechas grises indican el predecesor elegido por el máximo de la ecuación (4.21) y las discontinuas, las alternativas descartadas. El camino óptimo (naranja, nodos amarillos) es HHHHHLLLL: la región GCGC se asigna a H y el tramo ATTAT a L. Obsérvese que en $i = 5$ el mejor predecesor de L ya es H: el cambio de estado “se paga” una sola vez, con $\log_2 0,2$.

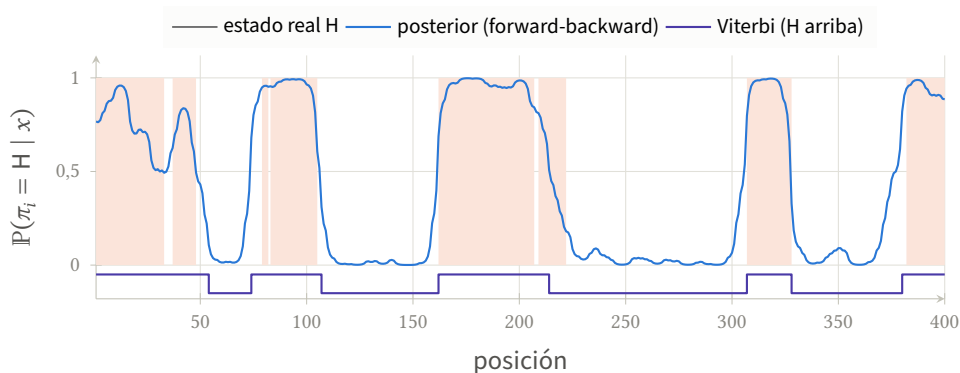


Figura 4.11. Decodificación de 400 bases simuladas con el modelo de la figura 4.9, pero con transiciones más persistentes ($a_{HH} = a_{LL} = 0,98$). Fondo naranja: tramos generados realmente por H. Curva azul: probabilidad posterior de H; su umbral en 0,5 acierta el estado en el 94 % de las posiciones. Escalón violeta (abajo): camino de Viterbi, con un 92,5 % de acierto. Ambos métodos detectan los bloques largos; los tramos cortos y los bordes son inciertos, y solo la posterior lo *dice*.

```

forward_viterbi.py

import numpy as np

def forward_viterbi(x, A, E, pi, alfabeto="ACGT"):
    """Log-verosimilitud (forward) y camino de Viterbi, en log2."""
    o = [alfabeto.index(c) for c in x]
    lA, lE = np.log2(A), np.log2(E)
    f = np.log2(pi) + lE[:, o[0]] # inicio forward (log)
    v, ptr = f.copy(), []
    for s in o[1:]:
        f = lE[:, s] + np.logaddexp2.reduce(f[:, None] + lA, axis=0)
        cand = v[:, None] + lA # K x K
        ptr.append(cand.argmax(axis=0))
        v = lE[:, s] + cand.max(axis=0)
    camino = [int(v.argmax())]
    for p in reversed(ptr): # traceback
        camino.append(int(p[camino[-1]]))
    return np.logaddexp2.reduce(f), v.max(), camino[::-1]

A = np.array([[0.8, 0.2], [0.3, 0.7]])
E = np.array([[.15, .35, .35, .15], [.35, .15, .15, .35]])
lp, lv, c = forward_viterbi("GCGCATTAT", A, E, np.array([.5, .5]))
print(f"{lp:.2f} {lv:.2f}", "".join("HL"[k] for k in c))
# -17.58 -19.98 HHHHLLLLL

```

4.3.4 Baum-Welch: aprender los parámetros

Hasta ahora supusimos conocidos A y E . Si dispusiéramos de secuencias con los estados anotados, estimarlos sería contar: la fracción de veces que H va seguido de L , la fracción de G emitidas por H , etc. Pero, por definición, los estados están ocultos. El algoritmo de Baum et al. (1970) resuelve el problema sustituyendo los conteos que no podemos observar por sus *valores esperados* bajo los parámetros actuales, y repitiendo.

Para cada posición i definimos la probabilidad posterior de una transición concreta y de un estado concreto:

$$\xi_i(k, l) = \mathbb{P}(\pi_i = k, \pi_{i+1} = l \mid x, \theta) = \frac{f_k(i) a_{kl} e_l(x_{i+1}) b_l(i+1)}{\mathbb{P}(x \mid \theta)}, \quad (4.22)$$

$$\gamma_i(k) = \mathbb{P}(\pi_i = k \mid x, \theta) = \frac{f_k(i) b_k(i)}{\mathbb{P}(x \mid \theta)}. \quad (4.23)$$

DONDE

$\xi_i(k, l)$	Probabilidad posterior de que la transición $k \rightarrow l$ ocurra entre las posiciones i e $i + 1$.
$\gamma_i(k)$	Probabilidad posterior de estar en k en la posición i (ecuación (4.19)).

Los conteos esperados se obtienen sumando sobre las posiciones (y sobre las secuencias de entrenamiento, si hay varias), y los nuevos parámetros son las frecuencias relativas de esos conteos:

$$\hat{a}_{kl} = \frac{\sum_{i=1}^{L-1} \xi_i(k, l)}{\sum_{l'} \sum_{i=1}^{L-1} \xi_i(k, l')}, \quad \hat{e}_k(b) = \frac{\sum_{i: x_i=b} \gamma_i(k)}{\sum_{i=1}^L \gamma_i(k)}, \quad \hat{\pi}_k = \gamma_1(k). \quad (4.24)$$

DONDE

$\hat{a}_{kl}, \hat{e}_k(b), \hat{\pi}_k$	Parámetros reestimados para la siguiente iteración.
$\sum_{i: x_i=b}$	Suma solo sobre las posiciones en las que se observó el símbolo b .

Se calculan forward y backward con los parámetros actuales (paso E), se reestiman los parámetros con la ecuación (4.24) (paso M) y se itera hasta que la verosimilitud deja de aumentar. Baum-Welch es un caso particular del algoritmo EM, formulado en general años después por Dempster et al. (1977), y hereda su propiedad fundamental.

Teorema 4.4. (Monotonía de Baum-Welch)

Cada iteración de la ecuación (4.24) cumple $P(x \mid \hat{\theta}) \geq P(x \mid \theta)$, con igualdad solo en un punto estacionario de la verosimilitud (Baum et al., 1970).

El teorema garantiza la convergencia a un máximo *local*, no al global. La verosimilitud de un HMM tiene en general muchos máximos locales, así que en la práctica se ejecuta el algoritmo desde varios puntos de partida, o se inicializa con una estimación razonable (por ejemplo, a partir de un MSA, como veremos con los perfiles).

4.3.5 HMM de pares: el alineamiento como modelo probabilístico

Estamos ahora en condiciones de cerrar un cabo que dejamos suelto en el capítulo 3. Las tres matrices de Gotoh (ecuación (3.6)) corresponden a tres estados de un HMM que, en lugar de emitir una secuencia, emite *dos* a la vez: un estado M emite

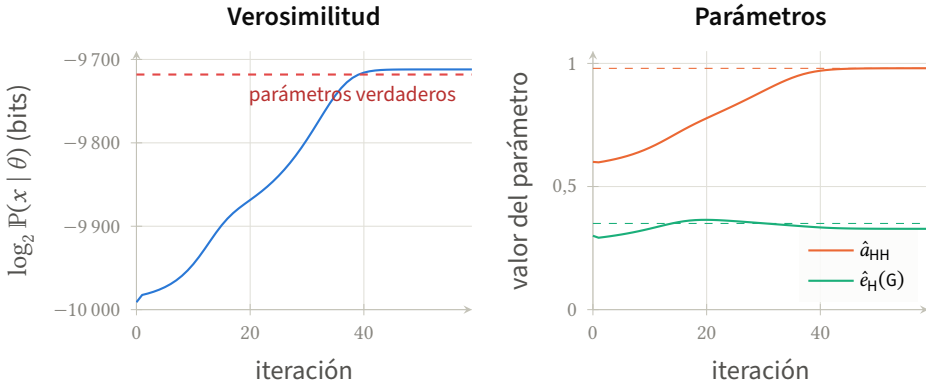


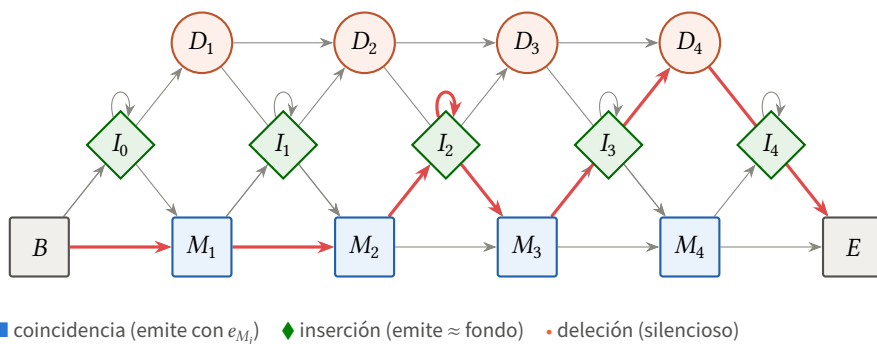
Figura 4.12. Baum-Welch aplicado a 5000 bases simuladas con $a_{HH} = a_{LL} = 0,98$, partiendo de parámetros deliberadamente erróneos ($a_{HH} = 0,6$; emisiones casi uniformes). **Izquierda:** la log-verosimilitud aumenta en cada iteración, como garantiza el teorema 4.4, desde -9991 hasta -9712 bits; tras 47 iteraciones queda a menos de 0,1 bits de su valor final. Termina *por encima* de la de los parámetros verdaderos (línea roja, -9718): el modelo se ajusta a las fluctuaciones de la muestra. **Derecha:** los parámetros convergen a $\hat{a}_{HH} = 0,981$ y $\hat{e}_H(G) = 0,328$, cerca de los valores verdaderos (discontinuas).

un par alineado (x_i, y_j) con probabilidad $p_{x_i y_j}$; un estado X emite x_i frente a un hueco; un estado Y emite y_j frente a un hueco. La transición $M \rightarrow X$ tiene probabilidad δ (abrir un hueco) y $X \rightarrow X$, probabilidad ε (extenderlo). Durbin et al. (1998) muestran que el algoritmo de Viterbi de este HMM de pares, escrito en log-razones frente a un modelo de fondo, es el algoritmo de Gotoh, con

$$s(a, b) = \log \frac{p_{ab}}{q_a q_b} + \text{cte.}, \quad d_o \approx -\log \delta, \quad d_e \approx -\log \varepsilon.$$

4.3.6 HMM de perfil: un modelo para cada familia

Un perfil (ecuación (4.6)) asigna una distribución propia a cada columna de un MSA; una PWM hace lo mismo para motivos sin huecos. Lo que les falta a ambos es una forma principada de modelar inserciones y deleciones que dependan de la posición: en una familia de proteínas, los bucles superficiales aceptan inserciones con facilidad, mientras que el núcleo hidrofóbico casi nunca las tolera. Krogh et al. (1994) resolvieron el problema con una arquitectura de HMM que hoy llamamos *HMM de perfil*, y Eddy (1998) la convirtió en una herramienta de uso general.



camino de CAACT -: $B M_1 M_2 I_2 I_3 M_3 D_4 E$

Figura 4.13. Arquitectura de un HMM de perfil de longitud 4, en la variante *Plan7* de HMMER, que no permite transiciones directas entre estados de inserción y delección. Los estados M_j (cuadrados) y D_j (círculos) forman la columna vertebral; los I_j (rombos) cuelgan entre posiciones y tienen un bucle que permite inserciones de cualquier longitud. En rojo, el camino que recorre la secuencia CAACT del ejemplo 4.6: dos residuos insertados tras la posición 2 y una delección de la posición 4. Toda secuencia alineada al MSA corresponde a un camino, y viceversa.

Definición 4.3. (HMM de perfil)

Un HMM de perfil de longitud m tiene, para cada posición consenso $j = 1, \dots, m$, tres estados: un estado de *coincidencia* M_j , que emite un residuo con distribución $e_{M_j}(\cdot)$ específica de la posición; un estado de *inserción* I_j , que emite residuos extra entre las posiciones j y $j + 1$ con una distribución cercana al fondo; y un estado *silencioso* de *delección* D_j , que salta la posición j sin emitir. Estados inicial (B) y final (E) completan el modelo.

La idea clave es que las penalizaciones de hueco, que en Needleman-Wunsch eran constantes, se convierten en probabilidades de transición *distintas en cada posición*: $a_{M_j D_{j+1}}$ puede ser minúscula en una hélice del núcleo y grande en un bucle. Y como el modelo es un HMM, todo lo que sabemos (forward, Viterbi, Baum-Welch) se aplica directamente.

Construcción a partir de un MSA. Dado un MSA de la familia, se decide primero qué columnas son *consenso*: una regla sencilla es declarar de coincidencia toda columna en la que al menos la mitad de las secuencias tienen un residuo, y de inserción las

demás. Después, cada secuencia del MSA define un camino por el modelo y basta con contar transiciones y emisiones, añadir pseudoconteos (o una previa de mezcla de Dirichlet, Sjölander et al., 1996) y normalizar:

$$e_{M_j}(a) = \frac{c_j(a) + \alpha_a}{\sum_{a'} (c_j(a') + \alpha_{a'})}, \quad a_{kl} = \frac{C_{kl} + \beta}{\sum_{l'} (C_{kl'} + \beta)}. \quad (4.25)$$

DONDE

$c_j(a)$	Número ponderado de secuencias con el residuo a en la columna consenso j .
C_{kl}	Número ponderado de veces que los caminos del MSA usan la transición $k \rightarrow l$.
α_a, β	Pseudoconteos de emisión y de transición.

Los conteos se ponderan (por ejemplo, con los pesos de la ecuación (4.2)) para que las subfamilias sobrerrepresentadas no dominen el modelo.

Ejemplo 4.6. (Un HMM de perfil de juguete)

Consideremos el MSA de cinco secuencias de ADN y seis columnas:

columna	1	2	3	4	5	6
s1	C	A	-	-	T	G
s2	C	A	A	-	T	G
s3	C	G	-	-	T	G
s4	C	A	A	C	T	-
s5	T	A	-	-	T	G
estado	M_1	M_2	I_2	I_2	M_3	M_4

Las columnas 3 y 4 tienen residuos en solo 2 y 1 de las 5 secuencias, así que son de inserción; las demás definen $M_1, \dots, M_4$. Con un pseudoconteo de 1 por base, las emisiones de M_1 (conteos C:4, T:1) son $e_{M_1} = (\frac{1}{9}, \frac{5}{9}, \frac{1}{9}, \frac{2}{9}) = (0,111; 0,556; 0,111; 0,222)$ para (A,C,G,T); la columna 5, invariante, da $e_{M_3}(T) = \frac{5+1}{5+4} = 0,667$; y M_4 , con solo cuatro residuos, $e_{M_4}(G) = \frac{4+1}{4+4} = 0,625$. Los caminos son $B M_1 M_2 M_3 M_4 E$ para s1, s3 y s5; $B M_1 M_2 I_2 M_3 M_4 E$ para s2; y $B M_1 M_2 I_2 M_3 D_4 E$ para s4 (figura 4.13). Desde M_2 hay tres transiciones a M_3 y dos a I_2 ; con $\beta = 1$ sobre las tres salidas posibles (M_3, I_2, D_3), $a_{M_2 M_3} = \frac{4}{8} = 0,5$, $a_{M_2 I_2} = \frac{3}{8} = 0,375$ y $a_{M_2 D_3} = \frac{1}{8} = 0,125$.

Alinear una secuencia al perfil. Para puntuar una secuencia x contra el modelo se usa Viterbi (o forward) en log-razones frente al fondo q . Las recurrencias tienen la misma forma que las de Gotoh, pero con parámetros que dependen de j :

$$V_j^M(i) = \log \frac{e_{M_j}(x_i)}{q_{x_i}} + \max \begin{cases} V_{j-1}^M(i-1) + \log a_{M_{j-1}M_j} \\ V_{j-1}^I(i-1) + \log a_{I_{j-1}M_j} \\ V_{j-1}^D(i-1) + \log a_{D_{j-1}M_j} \end{cases} \quad (4.26)$$

$$V_j^I(i) = \log \frac{e_{I_j}(x_i)}{q_{x_i}} + \max \begin{cases} V_j^M(i-1) + \log a_{M_jI_j} \\ V_j^I(i-1) + \log a_{I_jI_j} \end{cases} \quad (4.27)$$

$$V_j^D(i) = \max \begin{cases} V_{j-1}^M(i) + \log a_{M_{j-1}D_j} \\ V_{j-1}^D(i) + \log a_{D_{j-1}D_j} \end{cases} \quad (4.28)$$

DONDE

$V_j^M(i)$	Mejor log-razón de un camino que ha emitido $x_1 \dots x_i$ y termina en M_j (ídem para I_j, D_j).
q_{x_i}	Frecuencia de fondo del residuo x_i ; normalmente $e_{I_j} \approx q$, por lo que el primer término de la ecuación (4.27) es casi cero.
i, j	Posición en la secuencia y posición en el modelo, respectivamente.

Obsérvese que D_j no consume residuos (su índice i no cambia) y que, siguiendo la arquitectura Plan7, no hay transiciones $I \leftrightarrow D$. El costo es $O(mL)$, el mismo que alinear dos secuencias de longitudes m y L . Sustituyendo cada $\max$ por una suma (en espacio de probabilidades) se obtiene el algoritmo forward del perfil, que da la probabilidad de la secuencia bajo el modelo sumando sobre *todos* sus alineamientos posibles.

4.3.7 HMMER y Pfam

HMMER implementa este programa completo. Sus versiones iniciales puntuaban con Viterbi y eran unas cien veces más lentas que BLAST, lo que limitaba su uso. HMMER3 (Eddy, 2011) cambió la situación con dos ideas: puntuar con el algoritmo forward completo, que es más sensible porque integra la evidencia de todos los alineamientos, y colocar delante una cascada de filtros heurísticos que descartan pronto la inmensa mayoría de las secuencias no homólogas (figura 4.14). El primero, el filtro MSV (*multiple segment Viterbi*), calcula la mejor suma de varios segmentos sin huecos con instrucciones vectoriales SIMD, la misma técnica que acelera Smith-

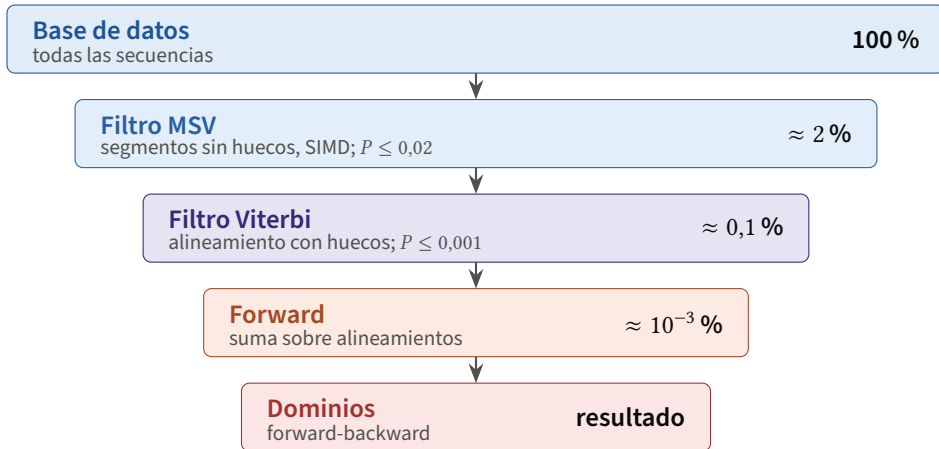


Figura 4.14. La cascada de aceleración de HMMER3 (Eddy, 2011). Cada etapa es más lenta y más sensible que la anterior, pero solo procesa las secuencias que la anterior dejó pasar. Los umbrales se expresan como valores P : el filtro MSV deja pasar, por defecto, el 2 % de las secuencias no homólogas, y el de Viterbi, el 0,1 %. La puntuación final, sobre la que se calculan los valores E , es siempre la del algoritmo forward completo; los filtros solo deciden a quién merece la pena aplicarlo.

Waterman. Según el propio artículo, HMMER3 es entre 100 y 1000 veces más rápido que HMMER2 y aproximadamente tan rápido como BLAST para proteínas.

HMMER expresa sus resultados igual que BLAST en el capítulo 3: una puntuación en bits (log-razón en base 2 entre el modelo y el fondo) y un valor E . Para asignar valores E sin simulaciones costosas, Eddy (2011) conjeturó, y comprobó empíricamente con miles de modelos, que las puntuaciones Viterbi en bits de secuencias no homólogas siguen una distribución de Gumbel de pendiente fija $\lambda = \ln 2$, y que la cola alta de las puntuaciones forward sigue una exponencial de la misma pendiente. Así, solo hay que estimar un parámetro de posición por modelo, lo que se hace una vez al construirlo.

Construir un HMM de perfil y buscar homólogos

```
# 1. HMM a partir de un MSA (Stockholm o FASTA alineado)
hmmbuild globinas.hmm globinas.sto
# 2. Buscar el perfil en una base de datos de proteínas
hmmsearch --tblout aciertos.tbl -E 1e-5 globinas.hmm uniprot_sprot.fasta
# 3. Anotar dominios de una proteína contra todo Pfam
```

```
hmmcompress Pfam-A.hmm # indexa la base (una vez)
hmmsearch --domtblout dominios.tbl Pfam-A.hmm consulta.fasta
```

```
leer_hmmer.py
```

```
from Bio import SearchIO

for q in SearchIO.parse("dominios.tbl", "hmmsearch3-domtbl"):
    for hit in q:
        for hsp in hit:
            print(f"{hit.id:15s} {hsp.bitscore:6.1f} bits "
                  f"E={hsp.evalue:.1e} "
                  f"{hsp.query_start + 1}-{hsp.query_end}")
```

La aplicación más influyente de los HMM de perfil es la anotación de dominios. Pfam (Mistry et al., 2021) es una colección de familias de proteínas, cada una definida por un *alineamiento semilla* curado a mano, un HMM de perfil construido con HMMER a partir de él y un *umbral de recolección* (*gathering threshold*) en bits, elegido por los curadores para que no incluya falsos positivos conocidos. El alineamiento completo de la familia se obtiene buscando el HMM en UniProt. La versión de 2021 añadió más de 350 familias nuevas, revisó las que cubren el proteoma del SARS-CoV-2 y reintrodujo Pfam-B, un suplemento de 136 730 agrupaciones automáticas de secuencias aún no cubiertas por ninguna familia.

De los ribosomas a las familias de proteínas

Los HMM llegaron a la biología desde el reconocimiento del habla, donde el tutorial de Rabiner (1989) los había popularizado. El grupo de David Haussler, en Santa Cruz, los adaptó a las proteínas: Krogh et al. (1994) modelaron con la arquitectura de la figura 4.13 las globinas, las cinasas y las proteínas EF-hand, y mostraron que un modelo entrenado solo con secuencias no alineadas podía producir un MSA y detectar miembros lejanos de la familia. El libro de Durbin et al. (1998), escrito por cuatro protagonistas de esta historia, sigue siendo la mejor introducción a todo el tema.

PWM: una distribución por posición, sin huecos.
HMM de perfil: una distribución por posición, con huecos que también dependen de la posición.

En resumen

- Un MSA se puntúa típicamente con la suma de pares, idealmente ponderada para compensar secuencias redundantes. Su optimización exacta cuesta $O(k^2 2^k n^k)$ y es NP-difícil.
- El alineamiento progresivo (distancias, árbol guía, alineamiento de perfiles) es rápido pero codicioso: “una vez un hueco, siempre un hueco”. El refinamiento iterativo (MUSCLE), la consistencia (T-Coffee), la FFT (MAFFT) y los árboles mBed con alineamiento HMM–HMM (Clustal Omega) mitigan sus errores o lo escalan.
- Una PWM resume sitios alineados como log-razones $\log_2(f_{b,i}/q_b)$; los pseudo-conteos evitan probabilidades nulas. Su puntuación esperada en sitios verdaderos es la entropía relativa total del motivo.
- El contenido de información $R_i = 2 - H_i$ mide la conservación de cada columna en bits; el *sequence logo* lo muestra apilando letras de altura $f_{b,i}R_i$.
- Un HMM se evalúa con forward, se decodifica con Viterbi o por probabilidades posteriores y se entrena con Baum-Welch (EM), todos en $O(LK^2)$.
- Un HMM de perfil añade a cada posición estados de coincidencia, inserción y delección, con parámetros propios. HMMER3 lo busca en bases de datos con filtros acelerados y puntuación forward; Pfam lo usa para definir y anotar familias de dominios.

Ejercicios

Dificultad:    básica    intermedia    avanzada

- 4.1**    Calcule a mano la puntuación SP (BLOSUM62, $d = 8$) de la columna K, K, R, -, E. ¿Cuánto cambia si se asigna peso 0,5 a cada una de las dos primeras secuencias y peso 1 a las demás?
- 4.2**    Escriba la función `suma_de_pares(filas, d)` que implementa la ecuación (4.1) con penalización lineal y compruebe que las cuatro primeras columnas del ejemplo 4.1 suman 62.
- 4.3**    Una columna de un motivo tiene frecuencias (0,5; 0,5; 0; 0) y otra (0,7; 0,1; 0,1; 0,1). ¿Cuál tiene más contenido de información? Dibuje a mano sus pilas en un *logo*.

- 4.4 ●●● Con la matriz de distancias de la figura 4.2, construya el árbol por *neighbor-joining* en lugar de UPGMA. ¿Cambia el orden en que se alinean los perfiles? Alinee las globinas con Clustal Omega y MAFFT y cuente las columnas en que difieren.
- 4.5 ●●● Descargue de JASPAR la matriz de conteos de otro factor (por ejemplo, CTCF, MA0139) y calcule su contenido total de información. ¿Cada cuántas bases esperaría un sitio por azar? Compare con el número de sitios de CTCF que se conocen en el genoma humano.
- 4.6 ●●● Verifique numéricamente, con el código de `forward_viterbi.py`, que $\sum_k f_k(i) b_k(i) = P(x)$ para *todas* las posiciones i . ¿Por qué debe cumplirse esta identidad?
- 4.7 ●●● Implemente Baum-Welch con reescalado para el modelo de dos estados y reproduzca la figura 4.12. Repita desde 20 puntos de partida aleatorios: ¿llegan todos al mismo máximo? ¿Qué ocurre si inicializa con $e_H = e_L$ exactamente?
- 4.8 ●●● Construya con `hmmbuild` un HMM de perfil a partir del MSA de globinas y búsquelo con `hmmsearch` en Swiss-Prot. Compare el número de aciertos con $E < 10^{-5}$ con el que obtiene `blastp` usando como consulta solo la hemoglobina β humana. Explique la diferencia en términos de la ecuación (4.26).

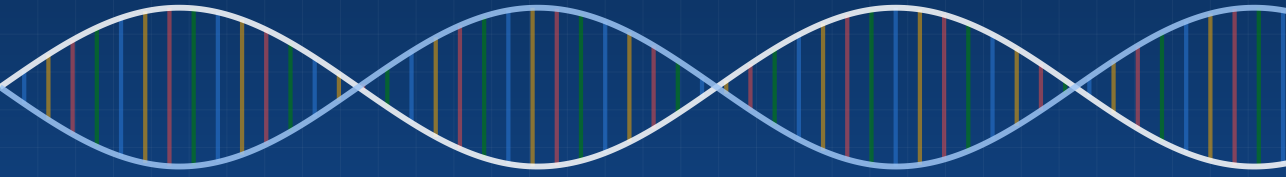
Referencias del capítulo

- Baum, L. E., Petrie, T., Soules, G., & Weiss, N. (1970). A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics*, 41(1), 164-171. <https://doi.org/10.1214/aoms/1177697196>
- Carrillo, H., & Lipman, D. (1988). The multiple sequence alignment problem in biology. *SIAM Journal on Applied Mathematics*, 48(5), 1073-1082. <https://doi.org/10.1137/0148063>
- Castro-Mondragon, J. A., Riudavets-Puig, R., Rauluseviciute, I., Berhanu Lemma, R., Turchi, L., Blanc-Mathieu, R., Lucas, J., Boddie, P., Khan, A., Manosalva Pérez, N., Fornes, O., Leung, T. Y., Aguirre, A., Hammal, F., Schmelter, D., Baranasic, D., Ballester, B., Sandelin, A., Lenhard, B., ... Mathelier, A. (2022). JASPAR 2022: The 9th release of the open-access database of transcription factor binding profiles. *Nucleic Acids Research*, 50(D1), D165-D173. <https://doi.org/10.1093/nar/gkab1113>

- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: Freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Crooks, G. E., Hon, G., Chandonia, J.-M., & Brenner, S. E. (2004). WebLogo: A sequence logo generator. *Genome Research*, 14(6), 1188-1190. <https://doi.org/10.1101/gr.849004>
- Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1), 1-22. <https://doi.org/10.1111/j.2517-6161.1977.tb01600.x>
- Durbin, R., Eddy, S. R., Krogh, A., & Mitchison, G. (1998). *Biological sequence analysis: Probabilistic models of proteins and nucleic acids*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511790492>
- Eddy, S. R. (1998). Profile hidden Markov models. *Bioinformatics*, 14(9), 755-763. <https://doi.org/10.1093/bioinformatics/14.9.755>
- Eddy, S. R. (2004). What is a hidden Markov model? *Nature Biotechnology*, 22(10), 1315-1316. <https://doi.org/10.1038/nbt1004-1315>
- Eddy, S. R. (2011). Accelerated profile HMM searches. *PLoS Computational Biology*, 7(10), Artículo e1002195. <https://doi.org/10.1371/journal.pcbi.1002195>
- Edgar, R. C. (2004). MUSCLE: Multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*, 32(5), 1792-1797. <https://doi.org/10.1093/nar/gkh340>
- Edgar, R. C., & Batzoglou, S. (2006). Multiple sequence alignment. *Current Opinion in Structural Biology*, 16(3), 368-373. <https://doi.org/10.1016/j.sbi.2006.04.004>
- Feng, D.-F., & Doolittle, R. F. (1987). Progressive sequence alignment as a prerequisite to correct phylogenetic trees. *Journal of Molecular Evolution*, 25(4), 351-360. <https://doi.org/10.1007/BF02603120>
- Henikoff, S., & Henikoff, J. G. (1994). Position-based sequence weights. *Journal of Molecular Biology*, 243(4), 574-578. [https://doi.org/10.1016/0022-2836\(94\)90032-9](https://doi.org/10.1016/0022-2836(94)90032-9)
- Higgins, D. G., & Sharp, P. M. (1988). CLUSTAL: A package for performing multiple sequence alignment on a microcomputer. *Gene*, 73(1), 237-244. [https://doi.org/10.1016/0378-1119\(88\)90330-7](https://doi.org/10.1016/0378-1119(88)90330-7)
- Katoh, K., Misawa, K., Kuma, K.-i., & Miyata, T. (2002). MAFFT: A novel method for rapid multiple sequence alignment based on fast Fourier transform. *Nucleic Acids Research*, 30(14), 3059-3066. <https://doi.org/10.1093/nar/gkf436>
- Krogh, A., Brown, M., Mian, I. S., Sjölander, K., & Haussler, D. (1994). Hidden Markov models in computational biology: Applications to protein modeling. *Journal of Molecular Biology*, 235(5), 1501-1531. <https://doi.org/10.1006/jmbi.1994.1104>

- Lawrence, C. E., Altschul, S. F., Boguski, M. S., Liu, J. S., Neuwald, A. F., & Wootton, J. C. (1993). Detecting subtle sequence signals: A Gibbs sampling strategy for multiple alignment. *Science*, 262(5131), 208-214. <https://doi.org/10.1126/science.8211139>
- Mistry, J., Chuguransky, S., Williams, L., Qureshi, M., Salazar, G. A., Sonnhammer, E. L. L., Tosatto, S. C. E., Paladin, L., Raj, S., Richardson, L. J., Finn, R. D., & Bateman, A. (2021). Pfam: The protein families database in 2021. *Nucleic Acids Research*, 49(D1), D412-D419. <https://doi.org/10.1093/nar/gkaa913>
- Notredame, C., Higgins, D. G., & Heringa, J. (2000). T-Coffee: A novel method for fast and accurate multiple sequence alignment. *Journal of Molecular Biology*, 302(1), 205-217. <https://doi.org/10.1006/jmbi.2000.4042>
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257-286. <https://doi.org/10.1109/5.18626>
- Schneider, T. D., & Stephens, R. M. (1990). Sequence logos: A new way to display consensus sequences. *Nucleic Acids Research*, 18(20), 6097-6100. <https://doi.org/10.1093/nar/18.20.6097>
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379-423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Sievers, F., Wilm, A., Dineen, D., Gibson, T. J., Karplus, K., Li, W., Lopez, R., McWilliam, H., Remmert, M., Söding, J., Thompson, J. D., & Higgins, D. G. (2011). Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. *Molecular Systems Biology*, 7(1), Artículo 539. <https://doi.org/10.1038/msb.2011.75>
- Sjölander, K., Karplus, K., Brown, M., Hughey, R., Krogh, A., Mian, I. S., & Haussler, D. (1996). Dirichlet mixtures: A method for improved detection of weak but significant protein sequence homology. *Bioinformatics*, 12(4), 327-345. <https://doi.org/10.1093/bioinformatics/12.4.327>
- Stormo, G. D. (2000). DNA binding sites: Representation and discovery. *Bioinformatics*, 16(1), 16-23. <https://doi.org/10.1093/bioinformatics/16.1.16>
- Stormo, G. D., Schneider, T. D., Gold, L., & Ehrenfeucht, A. (1982). Use of the «Perceptron» algorithm to distinguish translational initiation sites in *E. coli*. *Nucleic Acids Research*, 10(9), 2997-3011. <https://doi.org/10.1093/nar/10.9.2997>
- Thompson, J. D., Higgins, D. G., & Gibson, T. J. (1994). CLUSTAL W: Improving the sensitivity of progressive multiple sequence alignment through sequence weighting, position-specific gap penalties and weight matrix choice. *Nucleic Acids Research*, 22(22), 4673-4680. <https://doi.org/10.1093/nar/22.22.4673>
- Viterbi, A. J. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2), 260-269. <https://doi.org/10.1109/TIT.1967.1054010>
- Wang, L., & Jiang, T. (1994). On the complexity of multiple sequence alignment. *Journal of Computational Biology*, 1(4), 337-348. <https://doi.org/10.1089/cmb.1994.1.337>

Filogenética y evolución molecular



Las afinidades de todos los seres de la misma clase se han representado a veces por un gran árbol. Creo que este símil expresa en gran parte la verdad.

— Darwin (1859/2009)

● Al terminar este capítulo usted podrá

- Describir un árbol filogenético con precisión (raíz, nodos, ramas, clados, biparticiones) y calcular cuántas topologías posibles existen para n taxones.
- Formular la evolución de una secuencia como una cadena de Márkov de tiempo continuo, derivar la distancia de Jukes-Cantor y explicar qué añaden K80, F81, HKY85, GTR y la heterogeneidad gamma.
- Reconstruir árboles a partir de matrices de distancias con UPGMA y Neighbor-Joining, y reconocer cuándo cada método es consistente mediante las condiciones de ultrametricidad y de cuatro puntos.
- Calcular la verosimilitud de un árbol con el algoritmo de poda de Felsenstein, seleccionar modelos con AIC y BIC, interpretar el soporte bootstrap y UFBoot, y explicar la atracción de ramas largas.

En el capítulo 3 aprendimos a alinear secuencias y a decidir si su parecido es mayor que el esperado por azar. Ese parecido, dijimos, delata un ancestro común. Este capítulo da el paso siguiente: si varias secuencias descienden de ancestros comunes, ¿en qué orden se separaron sus linajes, y cuánto cambió cada uno desde entonces? La respuesta es un *árbol filogenético*, y reconstruirlo es uno de los problemas estadísticos más ricos de la biología.

El capítulo sigue ese orden. Primero construiremos el lenguaje probabilístico de la evolución molecular: cadenas de Márkov, matrices de tasas y los modelos de sustitución, desde el sencillo Jukes-Cantor hasta el modelo general reversible con tasas gamma (sección 5.1). Después usaremos esas distancias para reconstruir árboles con dos algoritmos clásicos y rapidísimos, UPGMA y Neighbor-Joining, y entenderemos con precisión matemática cuándo funcionan (sección 5.2). Por último, pasaremos al método que domina la práctica actual, la máxima verosimilitud, junto con la selección de modelos y el bootstrap, y lo contrastaremos con la parsimonia para ver por qué un buen modelo puede evitar errores sistemáticos (sección 5.3).

5.1 Distancias evolutivas y modelos de sustitución



Colab · 5.1 · Distancias y modelos de sustitución



repositorio del curso

La pared repintada

Imagine dos paredes que hace años se pintaron del mismo color. Desde entonces, cada cierto tiempo, alguien elige al azar uno de cuatro colores y repinta una de ellas. Hoy usted solo ve el color final de cada pared. Si los colores difieren, sabe que hubo al menos un repintado; pero si coinciden, no puede descartar que hayan pasado por rojo, azul y verde para volver, por casualidad, al mismo tono. Tras muchísimos repintados, las paredes coinciden una de cada cuatro veces, sin importar si pasaron diez o mil años. Contar diferencias visibles subestima la historia, y la subestima cada vez más cuanto más larga es. Para recuperar el número real de repintados hace falta un *modelo* de cómo se repinta.

Cada posición de una secuencia de ADN se comporta como esa pared: acumula sustituciones que se superponen, se revierten y se ocultan unas a otras. Esta sección

construye los modelos que permiten «ver a través» de esas superposiciones. Antes, sin embargo, necesitamos un vocabulario preciso para hablar de árboles.

5.1.1 Árboles: el vocabulario mínimo

Definición 5.1. (Árbol filogenético)

Un *árbol filogenético* es un grafo conexo y sin ciclos cuyas *hojas* (nodos de grado 1) representan los taxones o secuencias observadas y cuyos *nodos internos* representan ancestros hipotéticos. Cada *rama* (arista) tiene una longitud $b \geq 0$, medida en sustituciones esperadas por sitio. El árbol es *binario* si todo nodo interno tiene grado 3; es *enraizado* si uno de sus nodos, la *raíz*, se distingue como el ancestro común de todos los demás y define así la dirección del tiempo.

La figura 5.1 resume los términos. Dos más serán imprescindibles. Un *clado* es un ancestro junto con *todos* sus descendientes, y solo tiene sentido en árboles enraizados. En un árbol no enraizado, el concepto equivalente es la *bipartición* (*split*): al cortar una rama, el conjunto de hojas se divide en dos, por ejemplo $\{A, B\} | \{C, D, E\}$. Un árbol no enraizado queda completamente determinado por el conjunto de sus biparticiones, un hecho que usaremos para comparar árboles y para calcular el soporte bootstrap. Finalmente, la *topología* es la forma del árbol (quién está emparentado con quién) sin atender a las longitudes de las ramas.

¿Cuántos árboles hay?

Antes de reconstruir un árbol conviene saber entre cuántos candidatos debemos elegir. Consideremos árboles binarios no enraizados. Con tres hojas hay uno solo: una estrella de tres puntas. Un árbol binario no enraizado con k hojas tiene $2k - 3$ ramas; la $(k + 1)$ -ésima hoja puede insertarse en el punto medio de cualquiera de ellas, y cada elección produce una topología distinta. Multiplicando las opciones desde $k = 3$ hasta $k = n - 1$ se obtiene el resultado clásico, que se encuentra, por ejemplo, en Felsenstein (2004).

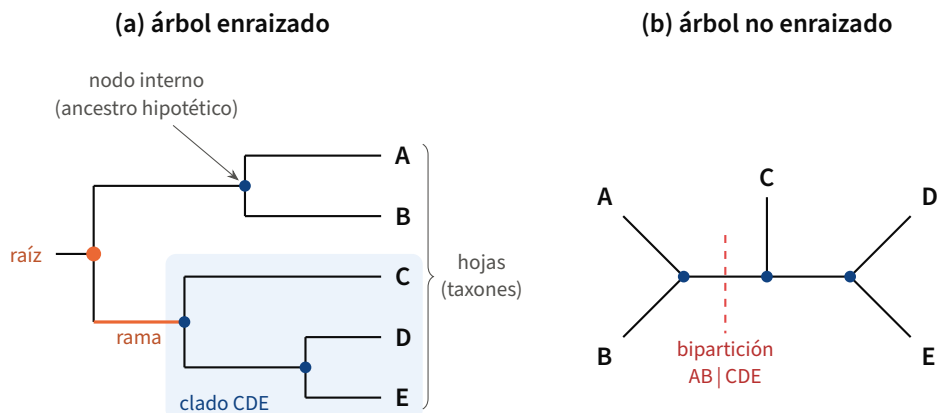


Figura 5.1. Anatomía de un árbol filogenético con cinco taxones. **(a)** En el árbol enraizado, la raíz (naranja) fija la dirección del tiempo: cada nodo interno (azul) es un ancestro hipotético, y el recuadro sombreado marca el clado formado por C, D, E y su ancestro común. **(b)** El mismo árbol sin raíz: las relaciones de parentesco se expresan como biparticiones; la línea roja corta la rama interna que separa $\{A, B\}$ de $\{C, D, E\}$. Los métodos de este capítulo producen árboles no enraizados; la raíz se añade después, por ejemplo con un grupo externo.

Teorema 5.1. (Número de topologías)

El número de topologías binarias distintas con $n \geq 3$ hojas etiquetadas es

$$U(n) = \prod_{k=3}^{n-1} (2k - 3) = (2n - 5)!!, \quad R(n) = (2n - 3)!! = U(n + 1), \quad (5.1)$$

para árboles no enraizados y enraizados, respectivamente.

DONDE

$U(n)$, $R(n)$	Número de árboles binarios no enraizados y enraizados con n hojas.
$m!!$	Doble factorial de un impar: $m!! = m \cdot (m - 2) \cdots 3 \cdot 1$.
$2k - 3$	Número de ramas de un árbol no enraizado con k hojas: lugares donde insertar la siguiente.

La igualdad $R(n) = U(n + 1)$ tiene una lectura sencilla: enraizar un árbol equivale a añadirle una hoja ficticia (la raíz) en alguna de sus $2n - 3$ ramas. La tabla 5.1 muestra lo rápido que crece el problema. Con diez taxones hay unos dos millones de

Tabla 5.1. Número de topologías binarias para n taxones (ecuación (5.1)). Valores calculados de forma exacta en `figuras/cap05/generar.py`.

n	no enraizados $(2n - 5)!!$	enraizados $(2n - 3)!!$
3	1	3
4	3	15
5	15	105
6	105	945
7	945	10 395
8	10 395	135 135
10	$2,03 \times 10^6$	$3,45 \times 10^7$
15	$7,91 \times 10^{12}$	$2,13 \times 10^{14}$
20	$2,22 \times 10^{20}$	$8,20 \times 10^{21}$
50	$2,84 \times 10^{74}$	$2,75 \times 10^{76}$

árboles no enraizados; con cincuenta, alrededor de 10^{74} , una cifra que se acerca al número de átomos del universo observable. Ningún computador puede evaluarlos todos, y por eso los métodos prácticos son, o bien algoritmos constructivos que edifican un único árbol paso a paso (sección 5.2), o bien búsquedas heurísticas que exploran una pequeña fracción del espacio (sección 5.3).

5.1.2 El reloj molecular y la distancia p

El reloj de las proteínas

A comienzos de los años sesenta, al comparar las secuencias de hemoglobinas y citocromos de distintos animales, Emile Zuckerkandl y Linus Pauling notaron que el número de diferencias entre dos proteínas homólogas crecía de forma aproximadamente proporcional al tiempo transcurrido desde la separación de las especies, estimado a partir de los fósiles. Propusieron entonces que las proteínas funcionan como un *reloj evolutivo molecular* (Zuckerkandl & Pauling, 1965). La idea transformó la biología: si las moléculas cambian a ritmo aproximadamente constante, sus diferencias no solo revelan parentescos, sino también fechas.

La medida más directa de divergencia entre dos secuencias alineadas es la proporción de sitios en que difieren.

Definición 5.2. (Distancia p)

Sean x e y dos secuencias alineadas con L sitios comparables (sin huecos). La distancia p es

$$p = \frac{1}{L} \sum_{h=1}^L 1 [x_h \neq y_h]. \quad (5.2)$$

DONDE

p	Proporción observada de sitios diferentes (distancia p o distancia de Hamming normalizada).
L	Número de sitios alineados comparables.
x_h, y_h	Nucleótidos de las dos secuencias en el sitio h .

La distancia p es un buen estimador del número de sustituciones solo cuando la divergencia es pequeña. A medida que el tiempo avanza, un sitio puede cambiar dos veces ($A \rightarrow G \rightarrow C$), revertir a su estado original ($A \rightarrow G \rightarrow A$) o cambiar en paralelo en ambos linajes hacia la misma base. Todos estos *impactos múltiples* (*multiple hits*) quedan ocultos. El resultado es la *saturación*: p crece cada vez más despacio y, para ADN con bases equiprobables, se estanca en $3/4$, la probabilidad de que dos bases elegidas al azar difieran. Para corregir la saturación necesitamos un modelo explícito del proceso de sustitución.

5.1.3 La evolución como cadena de Márkov de tiempo continuo

Modelaremos cada sitio de la secuencia como un proceso aleatorio $X(t)$ que toma valores en $\{A, C, G, T\}$ y salta de un estado a otro a lo largo del tiempo. Los modelos estándar hacen cuatro supuestos:

- 1. Independencia:** los sitios evolucionan de forma independiente (y, salvo el modelo gamma, idéntica).
- 2. Propiedad de Márkov:** la probabilidad de un cambio futuro depende solo del estado actual, no de la historia.
- 3. Homogeneidad:** las tasas de cambio no varían a lo largo del tiempo ni entre ramas.
- 4. Estacionariedad y reversibilidad:** la composición de bases está en equilibrio y el proceso «se ve igual» hacia adelante y hacia atrás en el tiempo.

Definición 5.3. (Matriz de tasas instantáneas)

Una cadena de Márkov de tiempo continuo sobre s estados se describe mediante una matriz $Q = (q_{ij})$ de tamaño $s \times s$ tal que, para un intervalo de tiempo Δt muy pequeño,

$$\mathbb{P}(X(t + \Delta t) = j | X(t) = i) = q_{ij} \Delta t + o(\Delta t), \quad i \neq j; \quad q_{ii} = - \sum_{j \neq i} q_{ij}. \quad (5.3)$$

DONDE

Q	Matriz de tasas instantáneas de sustitución (4×4 para ADN, 20×20 para proteínas).
q_{ij}	Tasa a la que el estado i se sustituye por j ($q_{ij} \geq 0$ si $i \neq j$).
q_{ii}	Menos la tasa total de salida del estado i ; hace que cada fila sume cero.
$o(\Delta t)$	Términos que se anulan más rápido que Δt (p. ej., dos cambios en el mismo intervalo).

La matriz Q describe lo que ocurre en un instante; lo que observamos, sin embargo, son los resultados después de un tiempo finito. Sea $P(t)$ la matriz de probabilidades de transición, con $P_{ij}(t) = \mathbb{P}(X(t) = j | X(0) = i)$. Para llegar de i a j en el tiempo $t + \Delta t$, el proceso debe estar en algún estado k en el tiempo t y luego pasar de k a j en el breve intervalo final:

$$P_{ij}(t + \Delta t) = \sum_k P_{ik}(t) (\delta_{kj} + q_{kj} \Delta t) + o(\Delta t) \implies \frac{P(t + \Delta t) - P(t)}{\Delta t} \rightarrow P(t) Q.$$

Esta es la *ecuación de Kolmogorov hacia adelante*, $P'(t) = P(t) Q$ con $P(0) = I$. Su solución es la exponencial de una matriz:

Teorema 5.2. (Probabilidades de transición)

Para una cadena de Márkov homogénea con matriz de tasas Q ,

$$P(t) = e^{Qt} = \sum_{k=0}^{\infty} \frac{(Qt)^k}{k!} = U \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_s t}) U^{-1}, \quad (5.4)$$

y se cumple la propiedad de Chapman-Kolmogorov $P(t + u) = P(t) P(u)$.

DONDE

$P(t)$	Matriz de probabilidades de transición: $P_{ij}(t)$ es la probabilidad de ver j tras un tiempo t partiendo de i .
e^{Qt}	Exponencial matricial, definida por la serie de potencias.
U, λ_k	Matriz de vectores propios de Q y sus valores propios ($\lambda_1 = 0$, los demás negativos).
I	Matriz identidad: con $t = 0$ nada ha cambiado.

La descomposición espectral de la ecuación (5.4) es la forma en que los programas calculan $P(t)$: se diagonaliza Q una sola vez y, para cada rama, basta con exponenciar s números. El valor propio nulo corresponde a la distribución estacionaria π , que satisface $\pi Q = 0$ y, por tanto, $\pi P(t) = \pi$ para todo t : si la composición de bases está en equilibrio, la evolución no la altera.

Dos condiciones adicionales hacen que los modelos sean manejables. La primera es la *reversibilidad temporal*, o condición de balance detallado:

$$\pi_i q_{ij} = \pi_j q_{ji} \quad \text{para todo } i, j. \quad (5.5)$$

DONDE

π_i	Frecuencia de equilibrio del nucleótido i ($\sum_i \pi_i = 1$).
$\pi_i q_{ij}$	Flujo de sustituciones $i \rightarrow j$ en equilibrio; la reversibilidad exige que iguale al flujo $j \rightarrow i$.

Con reversibilidad, la probabilidad de observar x en un extremo de una rama e y en el otro no depende de cuál extremo sea el «antiguo». Por eso, la distancia entre dos secuencias actuales separadas por dos ramas de longitudes b_1 y b_2 que se unen en su ancestro común se comporta como una sola rama de longitud $b_1 + b_2$, y por eso los métodos de inferencia pueden trabajar con árboles no enraizados. La segunda condición es una *normalización*: escalamos Q de modo que la tasa media de sustitución en equilibrio sea uno,

$$-\sum_i \pi_i q_{ii} = 1. \quad (5.6)$$

DONDE

$-\sum_i \pi_i q_{ii}$	Número esperado de sustituciones por sitio y por unidad de tiempo, promediado sobre la composición de equilibrio.
------------------------	---

Con esta convención, el tiempo t se mide directamente en *sustituciones esperadas por sitio*, y la longitud de una rama coincide con la distancia evolutiva d que queremos estimar. Tiempo y tasa quedan confundidos en un único parámetro: sin un reloj externo (fósiles, fechas de muestreo) no es posible separarlos.

5.1.4 Jukes-Cantor: la derivación completa

El modelo más sencillo, propuesto por Jukes y Cantor (1969), supone que las cuatro bases son equiprobables ($\pi_i = 1/4$) y que cualquier base se sustituye por cualquier otra con la misma tasa α :

$$Q_{JC} = \begin{pmatrix} -3\alpha & \alpha & \alpha & \alpha \\ \alpha & -3\alpha & \alpha & \alpha \\ \alpha & \alpha & -3\alpha & \alpha \\ \alpha & \alpha & \alpha & -3\alpha \end{pmatrix} \quad (\text{orden A, C, G, T}). \quad (5.7)$$

DONDE

α	Tasa de sustitución hacia cada una de las otras tres bases.
3α	Tasa total de salida de cualquier base; con la ecuación (5.6), $3\alpha = 1$.

Por simetría, todas las probabilidades de permanecer son iguales, $P_{ii}(t) \equiv a(t)$, y todas las de cambio también, $P_{ij}(t) \equiv (1 - a(t))/3$. La ecuación de Kolmogorov para un elemento diagonal da

$$a'(t) = \underbrace{-3\alpha a(t)}_{\text{salir de } i} + \underbrace{3 \cdot \alpha \frac{1 - a(t)}{3}}_{\text{volver a } i} = \alpha - 4\alpha a(t), \quad a(0) = 1.$$

Es una ecuación lineal de primer orden, cuya solución es $a(t) = \frac{1}{4} + \frac{3}{4}e^{-4\alpha t}$. Como el número esperado de sustituciones es $d = 3\alpha t$, podemos escribir $4\alpha t = 4d/3$ y obtenemos el resultado central de esta sección.

Teorema 5.3. (Distancia de Jukes-Cantor)

Bajo el modelo JC69, dos secuencias separadas por d sustituciones esperadas por sitio difieren en una proporción esperada de sitios

$$p(d) = \frac{3}{4} \left(1 - e^{-4d/3}\right). \quad (5.8)$$

Invirtiendo esta relación con el valor observado $\hat{p}$ se obtiene el estimador de la distancia y , por el método delta, su varianza aproximada:

$$\hat{d}_{JC} = -\frac{3}{4} \ln\left(1 - \frac{4}{3} \hat{p}\right), \quad \text{Var}(\hat{d}_{JC}) \approx \frac{\hat{p}(1 - \hat{p})}{L \left(1 - \frac{4}{3} \hat{p}\right)^2}. \quad (5.9)$$

DONDE

d	Distancia evolutiva: sustituciones esperadas por sitio entre las dos secuencias (suma de las ramas que las separan).
$p(d), \hat{p}$	Proporción de diferencias esperada según el modelo y observada en los datos.
$\hat{d}_{JC}$	Estimador de máxima verosimilitud de d bajo JC69.
L	Longitud del alineamiento; la varianza decrece como $1/L$.

La ecuación (5.8) captura la saturación con exactitud: para d pequeño, $p \approx d$ (casi no hay impactos múltiples), pero cuando $d \rightarrow \infty$, $p \rightarrow 3/4$. La ecuación (5.9) tiene dos consecuencias prácticas. Primero, $\hat{d}_{JC}$ no está definida si $\hat{p} \geq 3/4$: los datos son compatibles con cualquier distancia grande. Segundo, el denominador de la varianza se acerca a cero cuando $\hat{p}$ se acerca a $3/4$, de modo que las distancias grandes son muy imprecisas. La figura 5.2 muestra ambas cosas: la curva teórica, los puntos simulados y lo mucho que se aplana la relación a partir de $d \approx 1$.

5.1.5 Más allá de Jukes-Cantor: K80, F81, HKY85 y GTR

JC69 es elegante, pero dos de sus supuestos son claramente falsos en datos reales. El primero es que todas las sustituciones son igual de probables. Las bases se dividen químicamente en *purinas* (A, G, de dos anillos) y *pirimidinas* (C, T, de un anillo). Las *transiciones* intercambian bases dentro de un mismo grupo (A↔G, C↔T); las *transversiones*, entre grupos. Aunque hay el doble de tipos de transversión que de transición, en la mayoría de los genomas las transiciones son bastante más

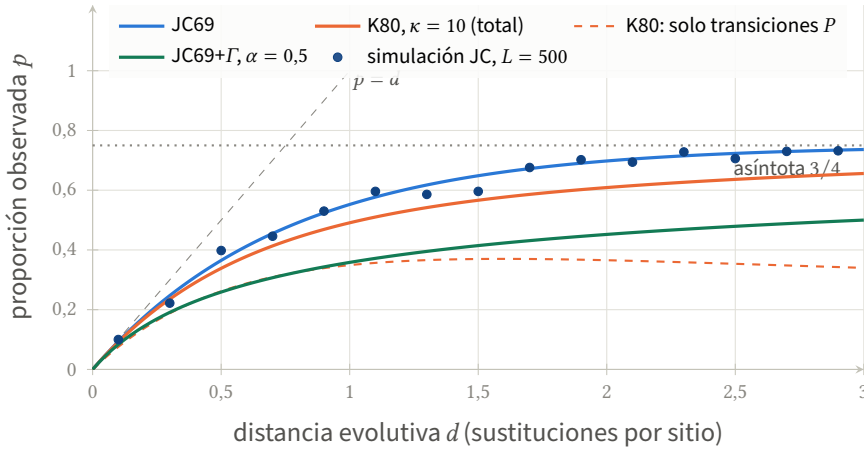


Figura 5.2. Saturación de la distancia observada. La proporción de sitios diferentes crece casi como d al principio, pero se curva y se estanca en $3/4$ (JC69, azul). Los puntos son pares de secuencias de 500 pb simulados bajo JC69: su dispersión alrededor de la curva recuerda que $\hat{p}$ es una estimación ruidosa. Con un sesgo fuerte hacia transiciones (K80 con $\kappa = 10$, naranja), las transiciones saturan mucho antes (línea discontinua, $P \rightarrow 1/4$) y el total avanza más lentamente. Con heterogeneidad de tasas (aqua), los sitios rápidos se saturan pronto y los lentos casi no cambian, de modo que la curva se aplana más despacio y engaña más: un mismo $\hat{p}$ corresponde a un d mucho mayor.

frecuentes, porque los errores de replicación y la desaminación de bases favorecen cambios que conservan el tamaño del anillo. Kimura (1980) incorporó esta asimetría con dos tasas: α para transiciones y β para transversiones (figura 5.3). Su parámetro clave es el cociente $\kappa = \alpha/\beta$, y la distancia correspondiente es

$$\hat{d}_{K80} = -\frac{1}{2} \ln(1 - 2P - Q) - \frac{1}{4} \ln(1 - 2Q). \quad (5.10)$$

DONDE

P, Q	Proporciones observadas de sitios que difieren por una transición y por una transversión ($p = P + Q$).
$\kappa = \alpha/\beta$	Cociente de tasas transición/transversión (en mamíferos suele estar entre 2 y 10).

(Aquí Q es la proporción de transversiones, no la matriz de tasas.) El segundo supuesto discutible es la composición equilibrada. Muchos genomas son ricos en **A+T** o en **G+C**. En el modelo F81 de Felsenstein (1981), la tasa hacia una base es

proporcional a su frecuencia de equilibrio, $q_{ij} = \mu \pi_j$, y la distancia generaliza la de Jukes-Cantor sustituyendo el 3/4 por $B = 1 - \sum_i \pi_i^2$, la probabilidad de que dos bases elegidas al azar según π difieran:

$$\hat{d}_{F81} = -B \ln\left(1 - \frac{\hat{p}}{B}\right), \quad B = 1 - \sum_i \pi_i^2. \quad (5.11)$$

DONDE

μ	Tasa global de sustitución (fijada por la normalización).
B	Nivel de saturación del modelo: la p esperada entre secuencias no relacionadas.

Hasegawa et al. (1985) combinaron ambas ideas en el modelo HKY85, que tiene frecuencias desiguales y un cociente κ . Finalmente, el *modelo general reversible* (GTR), cuya forma general analizaron Rodríguez et al. (1990), permite una tasa distinta para cada uno de los seis pares de bases. Todos ellos pueden escribirse con una sola fórmula,

$$q_{ij} = \mu r_{ij} \pi_j \quad (i \neq j), \quad r_{ij} = r_{ji}, \quad (5.12)$$

DONDE

r_{ij}	Parámetro de intercambiabilidad simétrico entre las bases i y j (uno de ellos se fija en 1).
π_j	Frecuencia de equilibrio de la base de llegada.
μ	Factor de escala que impone la ecuación (5.6).

y la reversibilidad (ecuación (5.5)) se cumple automáticamente, porque $\pi_i q_{ij} = \mu r_{ij} \pi_i \pi_j$ es simétrico en i, j . Los modelos más sencillos son casos particulares que restringen los r_{ij} y los π_j (tabla 5.2). Esta estructura *anidada* será importante al compararlos mediante pruebas de razón de verosimilitudes en la sección 5.3.

5.1.6 Heterogeneidad de tasas entre sitios: el modelo Γ

El supuesto más dañino de todos los modelos anteriores es que todos los sitios evolucionan a la misma velocidad. En un gen codificante, las terceras posiciones de los codones, a menudo sinónimas, cambian mucho más deprisa que las segundas; en un ARN ribosómico, los bucles varían mientras los tallos apareados se conservan. Si ignoramos esta variación, los sitios rápidos saturan pronto, los lentos no aportan diferencias, y la distancia resultante se subestima sistemáticamente.

Tabla 5.2. Modelos reversibles de sustitución de nucleótidos como casos particulares de GTR (ecuación (5.12)). Los parámetros libres no incluyen las longitudes de las ramas. Cualquiera de ellos puede combinarse con heterogeneidad gamma (+ Γ , un parámetro más) o con una proporción de sitios invariables (+I).

Modelo	Frecuencias π	Intercambiabilidades r_{ij}	Referencia	Libres
JC69	iguales	todas iguales	Jukes y Cantor (1969)	0
K80	iguales	transición κ , transversión 1	Kimura (1980)	1
F81	libres	todas iguales	Felsenstein (1981)	3
HKY85	libres	transición κ , transversión 1	Hasegawa et al. (1985)	4
GTR	libres	seis valores (uno fijo)	Rodríguez et al. (1990)	8

Yang (1994) popularizó la solución estándar: suponer que la tasa relativa r de cada sitio es una variable aleatoria con distribución gamma de media 1,

$$g(r; \alpha) = \frac{\alpha^\alpha}{\Gamma(\alpha)} r^{\alpha-1} e^{-\alpha r}, \quad \mathbb{E}[r] = 1, \quad \text{Var}(r) = \frac{1}{\alpha}, \quad (5.13)$$

DONDE

r	Tasa relativa de un sitio: multiplica todas las longitudes de rama para ese sitio.
α	Parámetro de forma; α pequeño significa tasas muy desiguales, $\alpha \rightarrow \infty$ recupera tasas iguales.
$\Gamma(\alpha)$	Función gamma de Euler, que normaliza la densidad.

La probabilidad de un sitio se calcula entonces promediando sobre r . Como la integral no tiene forma cerrada para árboles, Yang propuso aproximarla con K categorías de igual probabilidad, cada una representada por la tasa media de su intervalo. Con $K = 4$ y $\alpha = 0,5$, las cuatro tasas son 0,033, 0,252, 0,820 y 2,894: una cuarta parte de los sitios es casi invariable y otra cuarta parte evoluciona casi tres veces más rápido que el promedio. Para distancias entre pares la integral sí es explícita y da la distancia JC+ Γ ,

$$\hat{d}_{\text{JC}+\Gamma} = \frac{3}{4} \alpha \left[\left(1 - \frac{4}{3} \hat{p} \right)^{-1/\alpha} - 1 \right], \quad (5.14)$$

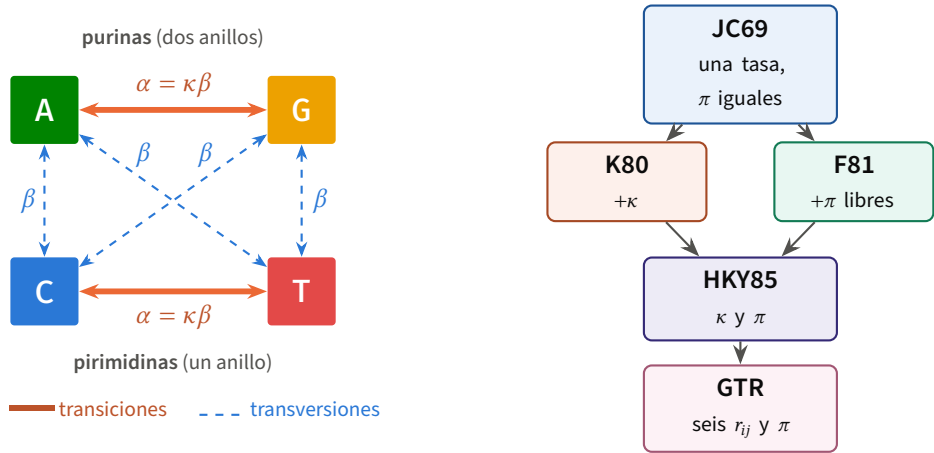


Figura 5.3. **Izquierda:** las seis clases de sustitución entre nucleótidos. Las transiciones (naranja, trazo grueso) conservan el tipo de anillo y ocurren a una tasa α típicamente varias veces mayor que la de las transversiones (azul, β). JC69 hace iguales todas las flechas; K80 distingue los dos colores; GTR permite un valor distinto para cada una de las seis. **Derecha:** jerarquía de modelos anidados; cada flecha añade parámetros y cada modelo inferior contiene al superior como caso particular.

DONDE

$\hat{d}_{JC+\Gamma}$ Distancia de Jukes-Cantor corregida por heterogeneidad gamma; tiende a $\hat{d}_{JC}$ cuando $\alpha \rightarrow \infty$.

que siempre es mayor que la de JC para el mismo $\hat{p}$. La figura 5.4 ilustra cómo cambia la forma de la distribución con α . En la práctica, además de $+\Gamma$, suele añadirse una proporción de sitios invariables ($+I$), y los programas modernos ofrecen modelos de tasas libres (*FreeRate*) que no imponen la forma gamma.

Ejemplo 5.1. (Tres distancias para el mismo par de secuencias)

Considere el siguiente par de fragmentos alineados de 40 pb (las marcas indican i para transición y v para transversión):

```
ATGGCTAAGCTTACGGATCCAGTACTGAAACGTTGCAAGT
|||i|||i|||v|||v|||i|||i||v||
ATGACTAACTTACTGATCCTGTACTGGAACGTCGCCAGT
```

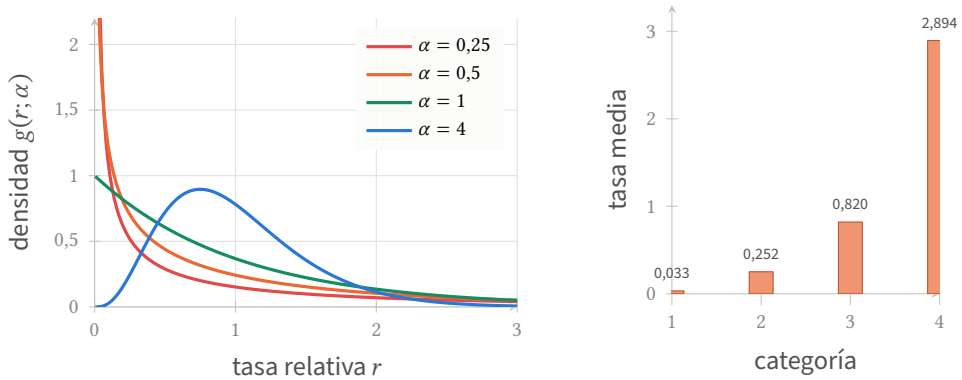


Figura 5.4. **Izquierda:** densidad gamma de media 1 para varios valores del parámetro de forma. Con $\alpha < 1$ la distribución tiene forma de L: muchos sitios casi invariables y unos pocos muy rápidos, la situación habitual en genes codificantes. Con $\alpha = 4$ las tasas se concentran alrededor de 1. **Derecha:** las cuatro categorías discretas de igual probabilidad para $\alpha = 0,5$ (Yang, 1994): la categoría más rápida evoluciona unas 90 veces más deprisa que la más lenta.

Hay 7 diferencias: 4 transiciones y 3 transversiones. Por lo tanto, $\hat{p} = 7/40 = 0,175$, $P = 0,100$ y $Q = 0,075$.

- **JC69** (ecuación (5.9)): $\hat{d} = -\frac{3}{4} \ln(1 - \frac{4}{3} \cdot 0,175) = -\frac{3}{4} \ln(0,7667) = 0,199$, con error estándar $\sqrt{\text{Var}} \approx 0,078$.
- **K80** (ecuación (5.10)): $\hat{d} = -\frac{1}{2} \ln(1 - 0,2 - 0,075) - \frac{1}{4} \ln(1 - 0,15) = 0,161 + 0,041 = 0,201$.
- **JC+ Γ** (ecuación (5.14)): con $\alpha = 1$, $\hat{d} = 0,228$; con $\alpha = 0,5$, $\hat{d} = 0,263$.

Tres lecciones se desprenden de estas cifras. Primera: todas las correcciones superan a $\hat{p}$, porque añaden los cambios ocultos. Segunda: K80 apenas difiere de JC aquí, porque la proporción de transiciones observada (4 de 7) no está muy sesgada; la diferencia crece con κ y con la divergencia. Tercera: la corrección gamma es la que más pesa (un 32 % más que JC con $\alpha = 0,5$), y el error estándar ($\pm 0,078$ para JC) es casi tan grande como las diferencias entre modelos: con 40 sitios, la incertidumbre de muestreo domina. Todas las cifras se reproducen con `figuras/cap05/generar.py`.

distancias.py

```
import numpy as np
from scipy.linalg import expm

TS = {("A", "G"), ("G", "A"), ("C", "T"), ("T", "C")}

def distancias(x: str, y: str) -> dict:
    """Distancias p, JC69 y K80 entre dos secuencias alineadas."""
    pares = [(a, b) for a, b in zip(x, y) if "-" not in (a, b)]
    L = len(pares)
    P = sum((a, b) in TS for a, b in pares) / L      # transiciones
    Q = sum(a != b and (a, b) not in TS for a, b in pares) / L
    p = P + Q
    d_jc = -0.75 * np.log(1 - 4 * p / 3)
    d_k80 = -0.5 * np.log(1 - 2 * P - Q) - 0.25 * np.log(1 - 2 * Q)
    return dict(p=p, jc=d_jc, k80=d_k80)

def matriz_P_k80(d: float, kappa: float) -> np.ndarray:
    """P(t) = exp(Qt) para K80 normalizado a d sustituciones."""
    Q = np.array([[0, 1, kappa, 1], [1, 0, 1, kappa],
                  [kappa, 1, 0, 1], [1, kappa, 1, 0]], float)
    np.fill_diagonal(Q, -Q.sum(1))
    Q /= -np.mean(np.diag(Q))      # tasa media = 1
    return expm(Q * d)            # orden A, C, G, T

print(distancias("ATGGCTAAGCTTACGG", "ATGACTAACTTACTG"))
print(matriz_P_k80(0.3, kappa=4).round(4)[0])  # fila de A
```

La segunda función muestra la ecuación (5.4) en acción. Con $d = 0,3$ y $\kappa = 4$, la fila de **A** de $P(t)$ es (0,758; 0,045; 0,151; 0,045): la probabilidad de terminar en **G** (transición) es más del triple que la de terminar en **C** o en **T**.

Distancias que mienten

1. **Saturación:** si $\hat{p}$ se acerca al nivel de saturación ($3/4$ en JC, B en F81), la distancia corregida explota o no existe. Para genes muy divergentes conviene usar proteínas en lugar de ADN, o eliminar las terceras posiciones de los codones.

2. *Modelo inadecuado*: ignorar la heterogeneidad de tasas subestima sobre todo las distancias grandes y deforma el árbol entero, no solo las ramas largas.
3. *Huecos*: la ecuación (5.2) supone sitios comparables; los programas eliminan los huecos par a par o en todo el alineamiento, y la elección cambia el resultado.

p mide lo que vemos; d mide lo que ocurrió.
El modelo de sustitución es el puente entre ambos.

5.2 UPGMA y Neighbor-Joining

 Colab · 5.2 · UPGMA y Neighbor-Joining

 repositorio del curso

Reconstruir las carreteras desde la tabla de kilómetros

En la guantera de un coche viejo aparece una tabla con las distancias por carretera entre cinco pueblos, pero no hay mapa. ¿Es posible dibujar la red de carreteras? Si las carreteras forman un árbol, sin circuitos, y las distancias se midieron recorriéndolas, la respuesta es sí, y además de forma exacta: cada distancia es la suma de los tramos que la componen, y esa redundancia basta para deducir dónde están los cruces y cuánto mide cada tramo. Pero si alguien anotó las distancias en línea recta, o se equivocó al copiarlas, la tabla deja de ser coherente con ningún árbol, y hay que buscar el que mejor se le ajuste.

Los métodos de distancias siguen exactamente ese programa. Primero reducen el alineamiento a una matriz D de distancias evolutivas entre pares, calculadas con los modelos de la sección 5.1; luego buscan un árbol cuyas distancias de camino reproduzcan D . Son rápidos, intuitivos y todavía se usan mucho, tanto como árboles finales en análisis exploratorios como árboles iniciales para métodos más costosos. Para entender cuándo funcionan necesitamos dos definiciones.

5.2.1 Árboles aditivos y ultramétricos

Definición 5.4. (Distancias aditivas y ultramétricas)

Una matriz de distancias D sobre n taxones es *aditiva* si existe un árbol con longitudes de rama no negativas tal que, para todo par i, j , D_{ij} es la suma de las longitudes de las ramas del camino entre i y j . Es *ultramétrica* si, además, el árbol puede enraizarse de modo que todas las hojas estén a la misma distancia de la raíz.

Un árbol ultramétrico es un árbol con *reloj molecular*: todos los linajes han acumulado el mismo número de sustituciones desde la raíz, de modo que la profundidad de cada nodo es proporcional a su edad. Un árbol aditivo no exige nada de eso: las ramas pueden tener longitudes arbitrarias, como ocurre cuando unos linajes evolucionan más deprisa que otros (figura 5.5). Ambas propiedades pueden comprobarse directamente sobre la matriz, sin conocer el árbol.

Teorema 5.4. (Condiciones de tres y cuatro puntos)

(i) D es ultramétrica si y solo si, para toda terna i, j, k ,

$$D_{ij} \leq \max\{D_{ik}, D_{jk}\}, \quad (5.15)$$

es decir, los dos valores mayores de cada terna son iguales. (ii) D es aditiva si y solo si, para todo cuarteto i, j, k, l , de las tres sumas

$$D_{ij} + D_{kl}, \quad D_{ik} + D_{jl}, \quad D_{il} + D_{jk} \quad (5.16)$$

las dos mayores son iguales (Buneman, 1974).

DONDE

i, j, k, l	Cuatro taxones cualesquiera, todos distintos.
D_{ij}	Distancia entre los taxones i y j .

La condición de cuatro puntos tiene una explicación gráfica muy clara (figura 5.6). En un árbol, cuatro hojas siempre se agrupan en dos pares separados por un camino interno de longitud $e \geq 0$. Si los pares son $\{i, j\}$ y $\{k, l\}$, la suma $D_{ij} + D_{kl}$ no recorre ese camino, mientras que las otras dos sumas lo recorren dos veces cada una. Así, las dos sumas «cruzadas» son iguales y superan a la primera en exactamente $2e$.

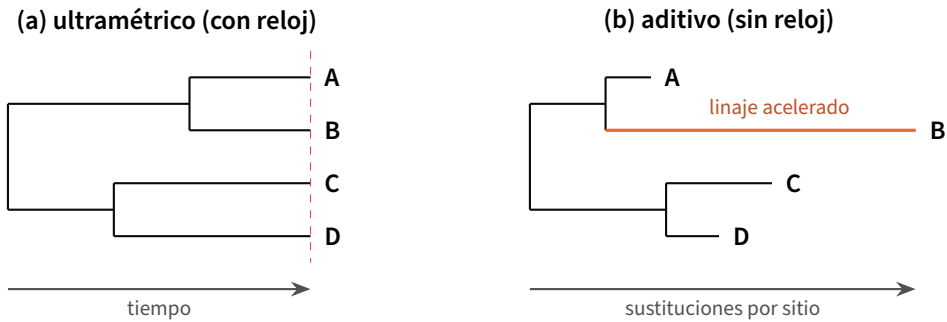


Figura 5.5. (a) En un árbol ultramétrico todas las hojas quedan alineadas (línea roja): la distancia raíz-hoja es la misma para todos los taxones, como exige un reloj molecular estricto, y la altura de cada nodo es proporcional a su edad. (b) En un árbol aditivo las ramas tienen longitudes arbitrarias; el linaje de B (naranja) ha acumulado muchas más sustituciones que su hermano A. Las distancias entre hojas siguen siendo sumas de ramas, pero ya no reflejan directamente los tiempos. UPGMA supone el caso (a); Neighbor-Joining solo necesita el caso (b).

Esto no solo prueba la condición: dice también cómo leer la topología del cuarteto (el par con la suma *menor* son vecinos) y cuánto mide su rama interna.

5.2.2 UPGMA: agrupar por promedios

El método de agrupamiento más antiguo que todavía se enseña es UPGMA (*unweighted pair group method with arithmetic mean*), introducido a finales de los años cincuenta en el contexto de la taxonomía numérica (Felsenstein, 2004). Su lógica es la de cualquier agrupamiento jerárquico aglomerativo:

1. Comience con cada taxón como un grupo de tamaño 1, situado a altura 0.
2. Encuentre el par de grupos (A, B) con la menor distancia D_{AB} .
3. Únalos en un nuevo nodo situado a altura $h = D_{AB}/2$ (cada rama mide h menos la altura del grupo hijo).
4. Calcule la distancia del nuevo grupo a cualquier otro grupo K como el promedio ponderado por tamaño:

$$D_{(AB),K} = \frac{|A| D_{AK} + |B| D_{BK}}{|A| + |B|}. \quad (5.17)$$

5. Repita desde el paso 2 hasta que quede un solo grupo, que será la raíz.

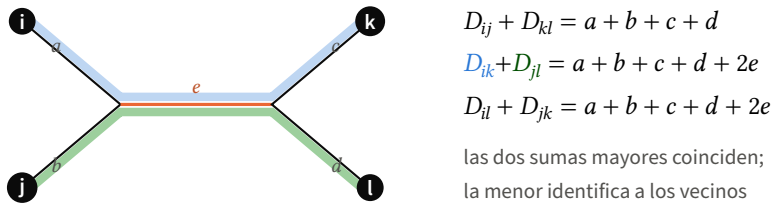


Figura 5.6. La condición de cuatro puntos. En un cuarteto con topología $ij|kl$, los caminos $i \rightarrow k$ (azul) y $j \rightarrow l$ (verde) atraviesan ambos la rama interna e (naranja), mientras que los caminos $i \rightarrow j$ y $k \rightarrow l$ no lo hacen. Por eso las dos sumas «cruzadas» son iguales y exceden a la tercera en $2e$. En la matriz del ejemplo 5.2, para el cuarteto A, B, C, D, las sumas son 1,1, 1,3 y 1,3: A y B son vecinos y la rama interna mide 0,1.

DONDE	
A, B, K	Grupos (clados) formados hasta el momento.
$ A $	Número de taxones del grupo A .
$D_{(AB),K}$	Distancia media entre los taxones de $A \cup B$ y los de K .

UPGMA produce siempre un árbol enraizado y ultramétrico, y su coste es $O(n^2)$ en memoria y $O(n^3)$ en tiempo en la versión directa. Su gran virtud es también su gran defecto: si D es ultramétrica, UPGMA recupera exactamente el árbol verdadero, pero si no lo es, porque los linajes evolucionan a ritmos distintos, puede unir taxones que no son vecinos. La razón es simple. UPGMA une el par más cercano, y en un árbol sin reloj el par más cercano no tiene por qué ser un par de hermanos: un taxón con una rama muy larga puede estar más lejos de su hermano que de un primo con ramas cortas. El ejemplo 5.2 y la figura 5.8 muestran exactamente esa situación.

5.2.3 Neighbor-Joining: corregir por la divergencia neta

Saitou y Nei (1987) propusieron una solución ingeniosa. En lugar de unir el par con la menor distancia, Neighbor-Joining (NJ) une el par con la menor distancia *corregida por lo lejos que está cada uno de todos los demás*. Un taxón con una rama larga está lejos de todo el mundo; restando su divergencia total se elimina ese efecto.

Teorema 5.5. (Criterio y actualización de Neighbor-Joining)

Con n nodos activos, defina la divergencia neta $r_i = \sum_k D_{ik}$ y la matriz

$$Q_{ij} = (n - 2) D_{ij} - r_i - r_j. \quad (5.18)$$

Se unen los nodos i, j que minimizan Q_{ij} en un nuevo nodo u , con ramas

$$b_{iu} = \frac{D_{ij}}{2} + \frac{r_i - r_j}{2(n - 2)}, \quad b_{ju} = D_{ij} - b_{iu}, \quad (5.19)$$

y las distancias del nuevo nodo a los demás se actualizan como

$$D_{uk} = \frac{1}{2} (D_{ik} + D_{jk} - D_{ij}). \quad (5.20)$$

El proceso se repite hasta que quedan tres nodos, que se unen en una estrella. Si D es aditiva, Nj recupera el árbol verdadero con sus longitudes de rama exactas.

DONDE

n	Número de nodos activos (hojas o nodos ya formados) en el paso actual.
r_i	Divergencia neta de i : suma de sus distancias a todos los nodos activos.
Q_{ij}	Criterio de selección; su mínimo identifica un par de vecinos.
b_{iu}	Longitud de la rama que une i con el nuevo nodo u .
D_{uk}	Distancia del nuevo nodo a otro nodo k : lo que queda del camino $i \rightarrow k$ al descontar la rama b_{iu} , promediado con el de j .

¿De dónde sale la ecuación (5.18)? Saitou y Nei la derivaron desde el principio de *evolución mínima*: partiendo de un árbol en estrella, unir i y j produce un árbol cuya longitud total S_{ij} puede calcularse a partir de D , y el par elegido es el que minimiza esa longitud. Studier y Keppler (1988) mostraron que minimizar S_{ij} equivale a minimizar la expresión más simple de la ecuación (5.18) (ambas difieren en un factor positivo y una constante) y que el algoritmo completo corre en tiempo $O(n^3)$. La fórmula de las ramas (ecuación (5.19)) también se entiende: la rama de i es la mitad de D_{ij} más una corrección proporcional a cuánto más lejos de todos está i que j . La ecuación (5.20), por su parte, es la condición de aditividad despejada: si u está en el camino entre i y k , entonces $D_{ik} = b_{iu} + D_{uk}$.

Lo que NJ optimiza realmente

Durante dos décadas se discutió qué criterio optimiza NJ, si es que optimiza alguno. Gascuel y Steel (2006) resumieron la respuesta: NJ es un algoritmo voraz (*greedy*) para el criterio de *evolución mínima balanceada*, que estima la longitud total del árbol con una ponderación particular de las distancias. Esto explica tanto su robustez (tolera bastante bien el ruido en D) como sus límites: al ser voraz, una decisión temprana errónea no se corrige después. Programas como FastME aplican búsquedas locales sobre ese mismo criterio para mejorar el árbol de NJ.

Ejemplo 5.2. (Neighbor-Joining paso a paso)

La tabla 5.3 contiene las distancias de camino de un árbol *aditivo pero no ultramétrico* con cinco taxones, en el que B tiene una rama larga (figura 5.8, izquierda). Puede comprobarse que satisface la condición de cuatro puntos: para A, B, C, D, las sumas son $D_{AB} + D_{CD} = 1,1$, $D_{AC} + D_{BD} = 1,3$ y $D_{AD} + D_{BC} = 1,3$.

Paso 1 ($n = 5$). Las divergencias netas son $r = (2,0; 2,9; 2,2; 2,6; 2,3)$. Por ejemplo, $Q_{AB} = 3 \cdot 0,5 - 2,0 - 2,9 = -3,4$ y $Q_{AC} = 3 \cdot 0,4 - 2,0 - 2,2 = -3,0$. El mínimo, $-3,4$, se alcanza en dos pares, A-B y D-E, ambos vecinos genuinos; tomamos D-E (resaltado en la tabla). Observe que A-C, el par más cercano (0,4), *no* gana. Las ramas son $b_D = 0,25 + (2,6 - 2,3)/6 = 0,30$ y $b_E = 0,20$. El nuevo nodo u_1 queda a distancias $D_{u_1A} = \frac{1}{2}(0,6 + 0,5 - 0,5) = 0,3$, $D_{u_1B} = 0,6$ y $D_{u_1C} = 0,3$.

Paso 2 ($n = 4$). Con los nodos A, B, C, u_1 resulta $r = (1,2; 1,8; 1,4; 1,2)$ y $Q_{AB} = 2 \cdot 0,5 - 1,2 - 1,8 = -2,0$, que empata con Q_{Cu_1} (en un cuarteto, los dos pares vecinos siempre empatan). Unimos A y B: $b_A = 0,25 + (1,2 - 1,8)/4 = 0,10$ y $b_B = 0,40$.

Paso 3 ($n = 3$). Quedan C, u_1 , u_2 con $D_{Cu_1} = 0,3$, $D_{Cu_2} = 0,3$ y $D_{u_1u_2} = 0,2$; la estrella final da ramas 0,2, 0,1 y 0,1.

El árbol obtenido coincide exactamente, topología y longitudes, con el verdadero (figura 5.7).

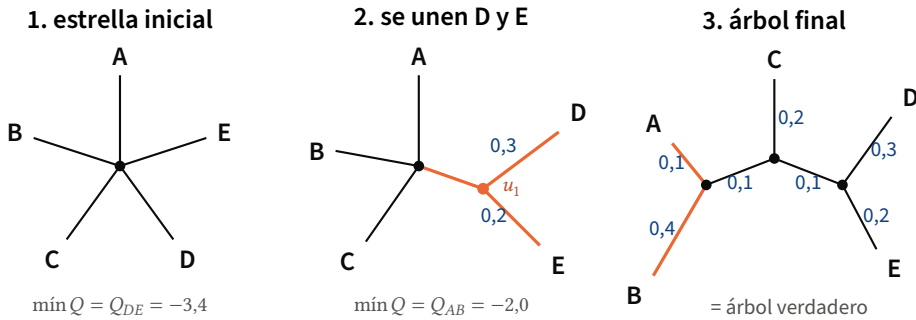


Figura 5.7. Neighbor-Joining sobre la matriz de la tabla 5.3. El algoritmo parte de una estrella en la que todos los taxones cuelgan de un único nodo y, en cada paso, «arranca» de la estrella el par que minimiza Q_{ij} (ecuación (5.18)), creando un nodo nuevo (naranja). Con cinco taxones bastan dos uniones; los tres nodos restantes forman la estrella final. Las longitudes, calculadas con la ecuación (5.19), reproducen exactamente las del árbol generador, incluida la rama larga de B.

D	A	B	C	D	E	Q	A	B	C	D	E	r_i
A	0	0,5	0,4	0,6	0,5	A	-	-3,4	-3,0	-2,8	-2,8	2,0
B	0,5	0	0,7	0,9	0,8	B	-3,4	-	-3,0	-2,8	-2,8	2,9
C	0,4	0,7	0	0,6	0,5	C	-3,0	-3,0	-	-3,0	-3,0	2,2
D	0,6	0,9	0,6	0	0,5	D	-2,8	-2,8	-3,0	-	-3,4	2,6
E	0,5	0,8	0,5	0,5	0	E	-2,8	-2,8	-3,0	-3,4	-	2,3

Tabla 5.3. Matriz de distancias aditiva del ejemplo (izquierda) y matriz Q del primer paso de NJ (derecha).

Biopython implementa ambos métodos en su módulo de construcción de árboles, `Bio.Phylo.TreeConstruction`. El siguiente fragmento reproduce el ejemplo; la matriz se pasa en forma triangular inferior. Al ejecutarlo, UPGMA devuelve las ramas A y C de longitud 0,200 unidas en un clado, mientras que NJ devuelve las longitudes verdaderas (0,1 para A, 0,4 para B, etc.).

arboles_distancia.py

[illegible]

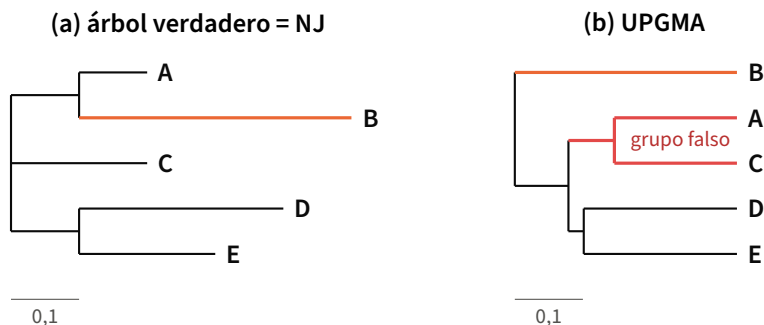


Figura 5.8. Cuando falla el reloj. (a) Árbol generador del ejemplo 5.2, dibujado desde su nodo central: B pertenece al linaje de A, pero su rama (naranja) es cuatro veces más larga. NJ lo recupera exactamente. (b) UPGMA sobre la misma matriz. Como $D_{AC} = 0.4$ es la menor distancia, UPGMA une primero A con C (rojo), un grupo que no existe en el árbol verdadero, y relega a B a la posición más externa, porque su rama larga lo aleja de todos. La hipótesis de ultrametricidad fuerza además a que todas las hojas terminen alineadas.

```
nombres = ["A", "B", "C", "D", "E"]
tri = [[0],
        [0.5, 0],
        [0.4, 0.7, 0],
        [0.6, 0.9, 0.6, 0],
        [0.5, 0.8, 0.5, 0.5, 0]] # triangular inferior
dm = DistanceMatrix(nombres, tri)
cons = DistanceTreeConstructor()
for metodo in (cons.upgma, cons.nj):
    arbol = metodo(dm)
    print(metodo.__name__)
    for clado in arbol.find_clades():
        if clado.branch_length:
            print(f" {clado.name:8s} {clado.branch_length:.3f}")
Phylo.draw_ascii(arbol)
```

⚠ Tres advertencias sobre los métodos de distancias

1. *Ramas negativas*: con datos reales, D nunca es exactamente aditiva, y la ecuación (5.19) puede dar longitudes negativas. Se suelen truncar a cero, pero son un síntoma de ruido o de un modelo de distancia inadecuado.

2. *La raíz no es un resultado*: el árbol de NJ es no enraizado. Dibujarlo con la raíz en el punto donde lo deja el programa sugiere relaciones ancestrales que el método no ha inferido.
3. *Información perdida*: al resumir el alineamiento en $\binom{n}{2}$ números, se descarta qué sitios concretos difieren. Los métodos basados en caracteres (sección 5.3) usan toda esa información, y por eso son estadísticamente más eficientes.

UPGMA supone un reloj; NJ solo supone aditividad.
 Cuando el reloj falla, el par más cercano no es necesariamente
 un par de hermanos.

5.3 Máxima verosimilitud y bootstrap



Colab · 5.3 · Máxima verosimilitud y bootstrap



repositorio del curso

El detective que mide la sorpresa

Un buen detective no se pregunta primero «¿cuál es la historia más probable?», porque no tiene forma de asignar probabilidades a historias que nunca ha visto. Se pregunta algo más modesto y más poderoso: «si esta historia fuera cierta, ¿qué tan probable sería encontrar exactamente las huellas, las colillas y las coartadas que tengo delante?». La historia bajo la cual las pruebas resultan *menos sorprendentes* es su mejor candidata. Y si varias historias explican las pruebas casi igual de bien, un detective honesto lo dice: su confianza depende de cuánto cambiaría la conclusión con un poco menos de evidencia.

Esa es, en esencia, la filosofía de la máxima verosimilitud (ML, *maximum likelihood*), llevada a la filogenética por Felsenstein (1981). Las «historias» son árboles con longitudes de rama y un modelo de sustitución; las «pruebas» son las columnas del alineamiento. A diferencia de los métodos de distancias, ML usa todos los sitios individualmente y dispone de un marco estadístico completo para estimar

parámetros, comparar modelos y medir la incertidumbre. Los libros de Felsenstein (2004) y Yang (2014) desarrollan esta teoría con gran detalle.

5.3.1 La verosimilitud de un árbol

Sea X un alineamiento de n secuencias y N sitios, y sea x_h la columna h . Dado un árbol T con longitudes de rama $\mathbf{b}$ y un modelo con parámetros θ (por ejemplo, κ , π y α), la independencia entre sitios permite escribir:

Definición 5.5. (Verosimilitud filogenética)

$$L(T, \mathbf{b}, \theta) = \mathbb{P}(X \mid T, \mathbf{b}, \theta) = \prod_{h=1}^N \mathbb{P}(x_h \mid T, \mathbf{b}, \theta), \quad (5.21)$$

$$\ell = \ln L = \sum_{h=1}^N \ln \mathbb{P}(x_h \mid T, \mathbf{b}, \theta).$$

El estimador de máxima verosimilitud es la terna $(\hat{T}, \hat{\mathbf{b}}, \hat{\theta})$ que maximiza ℓ .

DONDE

X, x_h	Alineamiento completo y su columna h (el estado de cada hoja en ese sitio).
N	Número de sitios del alineamiento.
$T, \mathbf{b}$	Topología y vector de longitudes de rama ($2n - 3$ ramas en un árbol no enraizado).
θ	Parámetros del modelo de sustitución.
ℓ	Logaritmo de la verosimilitud, que se usa porque L es un número diminuto.

Observe que la verosimilitud *no* es la probabilidad de que el árbol sea correcto: es la probabilidad de los datos suponiendo que lo es. El trabajo consiste en calcular $\mathbb{P}(x_h \mid \cdot)$ para una columna. Tomemos el árbol de cuatro hojas de la figura 5.9, enraizado (arbitrariamente) en el nodo interno 5. Las hojas muestran los estados observados; los nodos internos 5 y 6 son ancestros cuyos estados desconocemos, así que sumamos sobre todas sus posibilidades:

$$\mathbb{P}(x_h) = \sum_{x_5} \sum_{x_6} \pi_{x_5} P_{x_5 x_1}(b_1) P_{x_5 x_2}(b_2) P_{x_5 x_6}(b_5) P_{x_6 x_3}(b_3) P_{x_6 x_4}(b_4). \quad (5.22)$$

DONDE

$x_1, \dots, x_4$	Estados observados en las hojas para el sitio h .
x_5, x_6	Estados desconocidos de los nodos internos; se suman los 16 casos.
π_{x_5}	Probabilidad del estado en la raíz (frecuencia de equilibrio).
$P_{xy}(b)$	Probabilidad de transición de x a y a lo largo de una rama de longitud b (ecuación (5.4)).

Cada término es la probabilidad de una historia completa del sitio; sumarlos todos da la probabilidad de lo observado. El problema es que, con n hojas, hay $n - 2$ nodos internos en un árbol no enraizado y 4^{n-2} historias por sitio: para 50 secuencias, unas 10^{29} . Necesitamos, otra vez, una idea más astuta que la fuerza bruta.

5.3.2 El algoritmo de poda de Felsenstein

La idea es la misma que en la programación dinámica del capítulo 3: reordenar las sumas para no repetir cálculos. En la ecuación (5.22), los factores que dependen de x_6 pueden agruparse en una suma interior, que se calcula una sola vez para cada valor de x_6 . Generalizando este reordenamiento a cualquier árbol, Felsenstein (1981) obtuvo un recorrido desde las hojas hacia la raíz, denominado *poda* (*pruning*) porque cada subárbol ya procesado se «corta» y se sustituye por un vector de cuatro números.

Teorema 5.6. (Algoritmo de poda)

Para cada nodo k y estado x , sea $L_k(x)$ la probabilidad de los estados observados en las hojas que descienden de k , condicionada a que k esté en el estado x . Entonces

$$L_k(x) = \begin{cases} \mathbb{1}[x = x_k] & \text{si } k \text{ es una hoja,} \\ \prod_{c \in \text{hijos}(k)} \sum_y P_{xy}(b_c) L_c(y) & \text{si } k \text{ es interno,} \end{cases} \quad (5.23)$$

y la probabilidad del sitio es $\mathbb{P}(x_h) = \sum_x \pi_x L_{\text{raíz}}(x)$. El coste por sitio es $O(ns^2)$ en lugar de $O(s^{n-2})$.

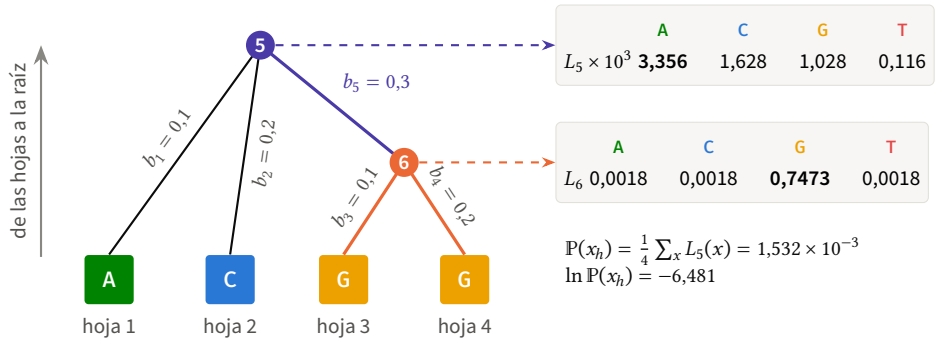


Figura 5.9. El algoritmo de poda para un sitio con patrón **ACGG** bajo JC69. El vector L_6 (naranja) resume el subárbol de las hojas 3 y 4: como ambas muestran **G**, el estado **G** en el nodo 6 es unas 400 veces más verosímil que cualquier otro. El vector L_5 (violeta) combina las hojas 1 y 2 con el subárbol ya podado, y su suma ponderada por $\pi = 1/4$ da la probabilidad del sitio. Todos los valores se calculan en `figuras/cap05/generar.py` y coinciden con la suma directa de las 16 historias de la ecuación (5.22).

DONDE	
$L_k(x)$	Verosimilitud parcial (o condicional) del subárbol que cuelga del nodo k .
b_c	Longitud de la rama que une k con su hijo c .
s	Número de estados: 4 para ADN, 20 para proteínas, 61 para codones.
$1[\cdot]$	Indicador; para una base ambigua (p. ej., R = A o G) o un hueco se usa 1 en todos los estados compatibles.

Dos observaciones completan el cuadro. Primero, bajo un modelo reversible la verosimilitud no depende de dónde se coloque la raíz: es el llamado *principio de la polea* de Felsenstein, que justifica trabajar con árboles no enraizados. Segundo, los sitios con el mismo patrón de estados tienen la misma probabilidad, así que basta con calcularla una vez por patrón distinto y multiplicarla por su frecuencia. En el alineamiento simulado de 600 sitios que usaremos más adelante solo hay 118 patrones distintos.

Ejemplo 5.3. (Verosimilitud de un sitio, paso a paso)

Usemos JC69 con las ramas de la figura 5.9. Para JC, $P_{xx}(b) = \frac{1}{4} + \frac{3}{4}e^{-4b/3}$ y $P_{xy}(b) = \frac{1}{4} - \frac{1}{4}e^{-4b/3}$, de modo que la probabilidad de permanecer y la de cambiar a una base concreta valen 0,9064 y 0,0312 para $b = 0,1$; 0,8245 y 0,0585 para $b = 0,2$; y 0,7527 y 0,0824 para $b = 0,3$.

Nodo 6. Sus hijos son hojas con **G**, de modo que $L_6(x) = P_{xG}(0,1)P_{xG}(0,2)$. Para $x = \mathbf{G}$: $0,9064 \times 0,8245 = 0,7473$; para cualquier otra base: $0,0312 \times 0,0585 = 0,0018$.

Nodo 5. Para $x = \mathbf{A}$, el factor de la rama hacia 6 es $\sum_y P_{Ay}(0,3)L_6(y) = 0,7527 \cdot 0,0018 + 0,0824 \cdot (0,0018 + 0,7473 + 0,0018) = 0,0633$. Multiplicando por $P_{AA}(0,1) = 0,9064$ (hoja 1) y por $P_{AC}(0,2) = 0,0585$ (hoja 2) se obtiene $L_5(\mathbf{A}) = 3,356 \times 10^{-3}$. Análogamente, $L_5 = (3,356; 1,628; 1,028; 0,116) \times 10^{-3}$.

Raíz. $P(x_h) = \frac{1}{4} \sum_x L_5(x) = 1,532 \times 10^{-3}$, y $\ln P(x_h) = -6,481$. El estado más verosímil del nodo 5 es **A**, apoyado por la hoja 1, que cuelga de la rama más corta: la verosimilitud «pondera» la evidencia de cada hoja según la longitud de su rama, algo que ningún conteo de diferencias hace.

poda.py

```
import numpy as np
```

```
def P_jc(t: float) -> np.ndarray:
```

```
    """Matriz de transición de Jukes-Cantor para una rama t."""
```

```
    e = np.exp(-4 * t / 3)
```

```
    return np.full((4, 4), 0.25 - 0.25 * e) + e * np.eye(4)
```

```
def parcial(nodo, sitio: dict) -> np.ndarray:
```

```
    """Vector  $L_k(x)$  de verosimilitudes parciales (poda)."""
```

```
    nombre, hijos = nodo
```

```
    if not hijos: # hoja: indicador
```

```
        v = np.zeros(4)
```

```
        v["ACGT".index(sitio[nombre])] = 1.0
```

```
        return v
```

```
    v = np.ones(4)
```

```
    for hijo, t in hijos: # producto sobre hijos
```

```
        v *= P_jc(t) @ parcial(hijo, sitio)
```

```
    return v
```

```
hoja = lambda n: (n, [])
```

```

arbol = ("5", [(hoja("1"), 0.1), (hoja("2"), 0.2),
              (("6", [(hoja("3"), 0.1), (hoja("4"), 0.2)]), 0.3)])
sitio = {"1": "A", "2": "C", "3": "G", "4": "G"}
L = 0.25 * parcial(arbol, sitio).sum() # pi = 1/4
print(f"L = {L:.4e}    lnL = {np.log(L):.4f}") # 1.5317e-03 -6.4814

```

5.3.3 Optimizar ramas, parámetros y topologías

Calcular ℓ para un árbol dado es la parte fácil. Maximizarla exige resolver tres problemas anidados. Las **longitudes de rama** se optimizan de una en una: al variar solo b_c , la verosimilitud es una función suave de una variable, y unas pocas iteraciones de Newton-Raphson bastan. Los **parámetros del modelo** (κ , α , r_{ij}) se optimizan de forma similar, alternando con las ramas. La **topología**, en cambio, es un objeto discreto que vive en el espacio astronómico de la tabla 5.1, y ahí solo cabe la búsqueda heurística: partir de un árbol razonable (a menudo, uno de NJ o de parsimonia) y mejorarlo con reordenamientos locales mientras la verosimilitud suba.

Los reordenamientos más comunes se muestran en la figura 5.10. El **intercambio de vecinos más cercanos** (NNI, *nearest neighbor interchange*) toma una rama interna, que separa cuatro subárboles, y prueba las dos formas alternativas de conectarlos; un árbol con n hojas tiene $n - 3$ ramas internas y, por lo tanto, $2(n - 3)$ vecinos NNI. La **poda y reinjerto** (SPR, *subtree pruning and regrafting*) corta un subárbol y lo vuelve a unir en cualquier otra rama, lo que permite saltos mucho mayores; la **bisección y reconexión** (TBR) generaliza SPR probando además todas las raíces posibles del subárbol cortado.

Los programas modernos combinan estas piezas con astucia. PhyML (Guindon & Gascuel, 2003) mostró que optimizar a la vez la topología (mediante NNI) y las longitudes de rama permitía analizar cientos de secuencias en minutos. RAXML (Stamatakis, 2014) se apoya en movimientos SPR con radio limitado y en implementaciones muy optimizadas para escalar a miles de taxones. IQ-TREE (Nguyen et al., 2015) introdujo una búsqueda estocástica que perturba aleatoriamente el mejor árbol encontrado y vuelve a optimizarlo con NNI, lo que ayuda a escapar de óptimos locales; su versión 2 (Minh et al., 2020) añadió modelos nuevos y métodos eficientes para datos genómicos. Ninguno garantiza encontrar el árbol de máxima verosimilitud global, y por eso se recomienda realizar varias búsquedas independientes y comparar sus resultados.

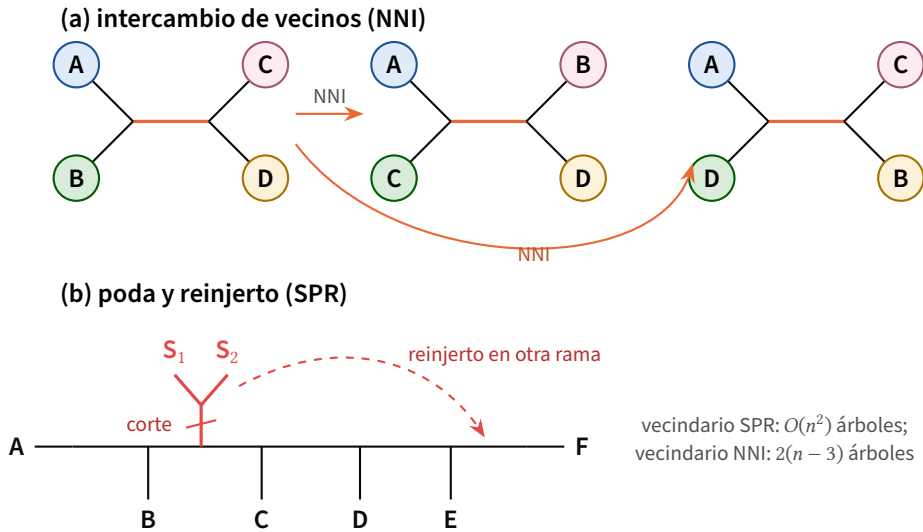


Figura 5.10. Movimientos de búsqueda en el espacio de topologías. (a) Una rama interna (naranja) separa cuatro subárboles; NNI prueba las dos formas alternativas de agruparlos (AC | BD y AD | BC). Es un movimiento pequeño y barato, pero puede quedar atrapado en óptimos locales. (b) SPR corta un subárbol (rojo) y lo reinjerta en otra rama del árbol; su vecindario es mucho mayor y permite escapar de óptimos locales, a cambio de más evaluaciones de la verosimilitud.

5.3.4 Selección de modelos: razón de verosimilitudes, AIC y BIC

¿Qué modelo usar? Un modelo con más parámetros siempre alcanza una verosimilitud igual o mayor, porque contiene a los más sencillos como casos particulares; la pregunta es si la mejora compensa la complejidad añadida. Para modelos anidados, como JC69 dentro de K80, la *prueba de razón de verosimilitudes* compara $2\Delta\ell = 2(\ell_1 - \ell_0)$ con una distribución χ^2 con tantos grados de libertad como parámetros adicionales. Para comparar muchos modelos, anidados o no, se usan criterios de información:

$$\text{AIC} = 2k - 2\hat{\ell}, \quad \text{BIC} = k \ln N - 2\hat{\ell}, \quad (5.24)$$

Tabla 5.4. Selección de modelos para un alineamiento de $N = 600$ sitios simulado bajo HKY85+ Γ ($\kappa = 4$, $\alpha = 0,5$) en el árbol de la figura 5.13. Todos los modelos se ajustaron sobre la misma topología (la de NJ); k incluye las 9 ramas. Δ es la diferencia con el mejor valor de cada criterio. Calculado con el motor de verosimilitud de `figuras/cap05/generar.py`.

Modelo	k	$\hat{\ell}$	AIC	Δ AIC	BIC	Δ BIC
JC	9	-2095,09	4208,2	135,0	4247,8	113,0
K8o	10	-2055,63	4131,3	58,1	4175,2	40,5
F81	12	-2084,13	4192,3	119,1	4245,0	110,3
HKY	13	-2042,54	4111,1	37,9	4168,2	33,5
HKY+ Γ	14	-2022,59	4073,2	0,0	4134,7	0,0
GTR+ Γ	18	-2020,78	4077,6	4,4	4156,7	22,0

DONDE	
$\hat{\ell}$	Logaritmo de la verosimilitud maximizada del modelo.
k	Número de parámetros libres, <i>incluidas</i> las $2n - 3$ longitudes de rama.
N	Número de sitios del alineamiento (tamaño de la muestra).

y se elige el modelo con el valor más bajo. El AIC de Akaike (1974) estima la capacidad predictiva del modelo sobre datos nuevos; el BIC de Schwarz (1978) aproxima la probabilidad posterior del modelo y, como su penalización crece con $\ln N$, favorece modelos más sencillos cuando hay muchos sitios. ModelFinder (Kalyanamoorthy et al., 2017), integrado en IQ-TREE, automatiza este proceso sobre cientos de combinaciones de matrices, frecuencias y modelos de heterogeneidad (incluidos los de tasas libres) y, por defecto, elige según el BIC.

Ejemplo 5.4. (Elegir el modelo con datos simulados)

La tabla 5.4 permite seguir el razonamiento completo. Pasar de JC69 a K8o añade un parámetro (κ) y aumenta $\hat{\ell}$ en 39,5 unidades: $2\Delta\ell = 78,9$, con $p \approx 6 \times 10^{-19}$ para una χ^2_1 . La relación transición/transversión es real. Pasar de HKY85 a HKY85+ Γ añade α y gana 19,95 unidades ($2\Delta\ell = 39,9$, $p \approx 3 \times 10^{-10}$): la heterogeneidad de tasas también es real. En cambio, pasar de HKY85+ Γ a GTR+ Γ añade cuatro parámetros y apenas gana 1,81 unidades ($2\Delta\ell = 3,6$,

$p = 0,46$ para una χ^2_4): las seis tasas libres no aportan nada que κ no explique ya. Los dos criterios de información coinciden y eligen HKY85+ Γ , precisamente el modelo con que se simulaban los datos; el BIC penaliza a GTR+ Γ con más dureza ($\Delta\text{BIC} = 22,0$) que el AIC ($\Delta\text{AIC} = 4,4$), porque $\ln 600 \approx 6,4 > 2$. Las estimaciones, $\hat{\kappa} = 3,71$ y $\hat{\alpha} = 0,57$, están cerca de los valores verdaderos (4 y 0,5).

5.3.5 Un contraste: parsimonia y atracción de ramas largas

Antes de la verosimilitud, el criterio dominante para datos de caracteres era la *máxima parsimonia*: preferir el árbol que explica los datos con el menor número de cambios. Fitch (1971) dio un algoritmo eficiente para contar ese mínimo en un árbol dado. En cada nodo interno se calcula un conjunto de estados posibles: si los conjuntos de los dos hijos se intersecan, el nodo recibe la intersección; si no, recibe su unión y se contabiliza un cambio. La parsimonia no necesita modelo, es rápida y funciona bien cuando las ramas son cortas. Pero precisamente por no tener modelo, no puede corregir los impactos múltiples.

Felsenstein (1978) demostró que existen árboles para los que la parsimonia es *estadísticamente inconsistente*: converge al árbol equivocado a medida que se añaden datos. El caso típico es el de la figura 5.11: dos ramas largas no hermanas (1 y 3) y ramas cortas en el resto. En las ramas largas se acumulan tantos cambios que, por puro azar, 1 y 3 coinciden a menudo en el mismo estado derivado. La parsimonia interpreta esas coincidencias como un único cambio compartido y agrupa a 1 con 3. Este fenómeno, la *atracción de ramas largas*, sigue siendo uno de los errores sistemáticos más importantes de la filogenética. La verosimilitud con un modelo adecuado es consistente en esa misma región, porque «sabe» que en ramas largas son esperables las convergencias; pero un modelo inadecuado (por ejemplo, ignorar la heterogeneidad de tasas) puede reintroducir el problema.

5.3.6 ¿Cuánta confianza? El bootstrap

Un árbol de máxima verosimilitud es una estimación puntual; necesitamos saber qué partes de él son sólidas. La herramienta más usada es el *bootstrap no paramétrico* de Efron (1979), cuya idea es simular la variabilidad de muestreo usando los propios datos como sustituto de la población. Felsenstein (1985) lo llevó a la filogenética tratando las columnas del alineamiento como la muestra:

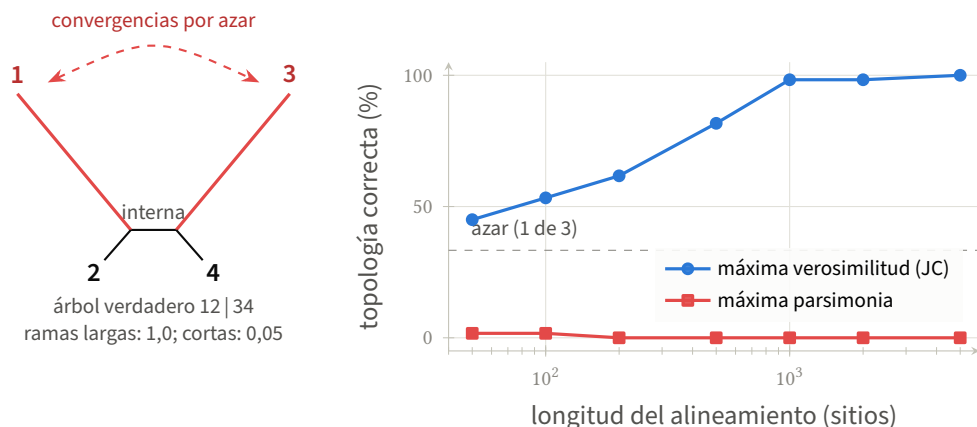


Figura 5.11. La atracción de ramas largas en la «zona de Felsenstein». **Izquierda:** árbol generador con dos ramas largas no hermanas (rojo). **Derecha:** proporción de 60 réplicas simuladas bajo JC69 en que cada método recupera la topología correcta. Con más datos, la máxima verosimilitud converge al árbol verdadero (100 % con 5000 sitios), mientras que la parsimonia converge con seguridad creciente al árbol *equivocado* 13 | 24, y lo hace por debajo incluso del 33 % que daría elegir al azar.

1. Construya un alineamiento réplica eligiendo N columnas *con reemplazo* del original (algunas aparecerán repetidas y otras faltarán; figura 5.12).
2. Infera un árbol a partir de la réplica con el mismo método que usó para los datos originales.
3. Repita B veces (típicamente entre 100 y 1000) y registre, para cada bipartición del árbol original, la fracción de réplicas en que aparece.

Definición 5.6. (Soporte bootstrap)

El soporte bootstrap de una bipartición S es

$$\text{BS}(S) = \frac{1}{B} \sum_{b=1}^B \mathbb{1}[S \in \hat{T}_b^*] \times 100 \%. \quad (5.25)$$

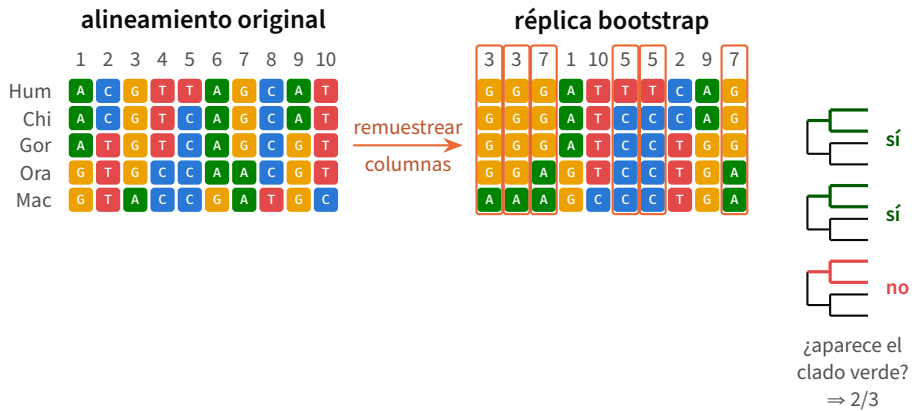


Figura 5.12. El bootstrap no paramétrico. Cada réplica se construye eligiendo columnas al azar, con reemplazo, del alineamiento original (los números indican qué columna se copió; las recuadradas en naranja aparecen más de una vez, y las columnas 4, 6 y 8 no aparecen). A partir de cada réplica se infiere un árbol, y el soporte de un clado es la fracción de réplicas en que ese clado reaparece (derecha: presente en dos de tres árboles). La variación entre réplicas imita la que habría entre alineamientos independientes del mismo tamaño.

DONDE

B	Número de réplicas bootstrap.
$\hat{T}_b^*$	Árbol estimado a partir de la réplica b .
$S \in \hat{T}_b^*$	La bipartición S está presente en ese árbol.

La figura 5.13 muestra el resultado para el alineamiento simulado de seis primates de la tabla 5.4. Los soportes deben leerse con cuidado. Un valor del 95 % no significa que el clado tenga un 95 % de probabilidad de ser verdadero: el bootstrap mide cuán estable es la inferencia frente a la variación de muestreo, y en la práctica suele ser conservador para clados verdaderos (Felsenstein, 2004). Además, el bootstrap estándar exige repetir la búsqueda completa B veces, lo que es prohibitivo para alineamientos grandes. El *bootstrap ultrarrápido* (UFBoot) de IQ-TREE reutiliza los árboles visitados durante la búsqueda y evalúa las réplicas por remuestreo de las verosimilitudes de cada sitio (RELL), con un coste muy inferior (Hoang et al., 2018). Sus valores no son equivalentes a los del bootstrap estándar: los autores recomiendan considerar fiables los clados con UFBoot ≥ 95 %, mientras que para el bootstrap clásico suele usarse un umbral más bajo, del orden del 70 %.

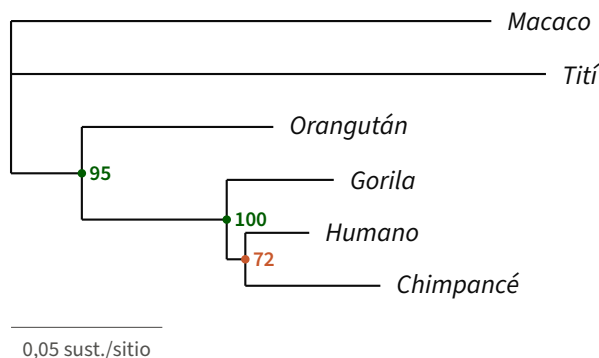


Figura 5.13. Árbol de seis primates inferido a partir de 600 sitios simulados bajo HKY85+ Γ sobre un árbol conocido. La topología es la de NJ (distancias JC) y coincide con la verdadera; las longitudes de rama se reoptimizaron por máxima verosimilitud con el modelo elegido (HKY85+ Γ). Los números son soportes bootstrap sobre 1000 réplicas (verde: ≥ 95 ; naranja: 70–94). El clado humano-chimpancé recibe solo un 72 %: la rama que lo define mide 0,008 sustituciones por sitio en el árbol verdadero, lo que equivale a unos 5 cambios esperados en todo el alineamiento, muy poca evidencia. El árbol se dibuja sin raíz explícita, con Macaco y Tití en la base.

Ejemplo 5.5. (Leer el soporte de la figura 5.13)

El clado humano-chimpancé-gorila aparece en el 100 % de las réplicas y el que añade al orangután, en el 95 %: sus ramas definitorias miden 0,03 y 0,06 sustituciones por sitio en el árbol generador, es decir, unas 18 y 36 sustituciones esperadas en 600 sitios. En cambio, el clado humano-chimpancé (72 %) se apoya en una rama de 0,008: en un 28 % de las réplicas, el azar del remuestreo reúne a gorila con uno de los dos. Sin embargo, la topología estimada es correcta. La lección es general: un soporte moderado no indica que el clado sea falso, sino que *estos datos* contienen poca información sobre él.

En la práctica, selección de modelo, búsqueda de ML y bootstrap se ejecutan con una sola orden: en IQ-TREE 2, -m MFP activa ModelFinder, -B pide réplicas UFBoot y -alrt añade la prueba SH-aLRT; en RAXML 8, -f a combina bootstrap rápido y búsqueda de ML.

Inferencia de máxima verosimilitud en la terminal

```
# IQ-TREE 2: selección de modelo + árbol ML + UFBoot + SH-aLRT
iqtree2 -s primates.fasta -m MFP -B 1000 -alrt 1000 -T AUTO
# salida: primates.fasta.iqtree (informe), .treefile (árbol)

# RAXML 8: 100 réplicas bootstrap rápidas + búsqueda ML (GTR+G)
raxmlHPC -f a -s primates.phy -n prim -m GTRGAMMA \
-p 12345 -x 12345 -N 100
```

⚠ Lo que el bootstrap no puede detectar

El bootstrap cuantifica el error *aleatorio*, el que desaparece con más datos. No detecta el error *sistemático*. En la zona de Felsenstein, la parsimonia agrupa las ramas largas con soporte cercano al 100 % (figura 5.11), y un modelo inadecuado puede hacer lo mismo con la verosimilitud.

La verosimilitud pregunta qué tan probable es lo observado bajo cada árbol.
El bootstrap pregunta cuánto cambiaría la respuesta con otra muestra de sitios.

☰ En resumen

- Un árbol filogenético no enraizado con n hojas es una de $(2n - 5)!!$ topologías posibles; para 50 taxones hay unas 10^{74} , por lo que la inferencia exige algoritmos constructivos o búsquedas heurísticas.
- La distancia p se satura por los impactos múltiples. Los modelos de sustitución, cadenas de Márkov con $P(t) = e^{Qt}$, corrigen ese sesgo: JC69 da $d = -\frac{3}{4} \ln(1 - \frac{4}{3}p)$; K80, F81, HKY85 y GTR añaden sesgo transición/transversión y composición desigual; + Γ modela la variación de tasas entre sitios.
- UPGMA supone un reloj molecular (distancias ultramétricas) y falla cuando los linajes evolucionan a ritmos distintos. Neighbor-Joining solo requiere aditividad (condición de cuatro puntos) y recupera exactamente el árbol cuando las distancias son aditivas.

- La máxima verosimilitud evalúa cada árbol por la probabilidad de todo el alineamiento; el algoritmo de poda de Felsenstein la calcula en tiempo lineal en el número de taxones, y la búsqueda de topologías usa NNI, SPR o TBR.
- Los modelos se comparan con razón de verosimilitudes, AIC y BIC. La parsimonia puede ser inconsistente (atracción de ramas largas); ML con un modelo adecuado no lo es. El bootstrap y UFBoot miden la estabilidad de cada clado, no los errores sistemáticos.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 5.1** ●●● Dibuje las 15 topologías no enraizadas posibles para cinco taxones. Compruebe que su número coincide con la ecuación (5.1) y explique por qué cada una puede enraizarse de 7 formas.
- 5.2** ●●● Dos secuencias de 300 pb difieren en 90 sitios. Calcule $\hat{p}$, $\hat{d}_{JC}$ y su error estándar. ¿Qué ocurre con la distancia si difieren en 230 sitios?
- 5.3** ●●● Compruebe que la matriz de distancias de la tabla 5.3 *no* es ultramétrica encontrando una terna que viole la ecuación (5.15).
- 5.4** ●●● A partir de la ecuación de Kolmogorov $P'(t) = P(t)Q$, derive $P_{ij}(t)$ para el modelo K80 y demuestre la ecuación (5.10). (Pista: considere por separado la probabilidad de terminar con una transición y con una transversión.)
- 5.5** ●●● Verifique numéricamente con `scipy.linalg.expm` que, para una matriz GTR construida con la ecuación (5.12), se cumplen $\pi P(t) = \pi$ y $\pi_i P_{ij}(t) = \pi_j P_{ji}(t)$ para cualquier t .
- 5.6** ●●● Amplíe el programa `poda.py` para calcular el logaritmo de la verosimilitud de un alineamiento completo sumando sobre sitios, y optimice la longitud de una rama con `scipy.optimize.minimize_scalar`.
- 5.7** ●●● Implemente el algoritmo de Fitch y úselo, junto con su función de verosimilitud del ejercicio anterior, para reproducir la figura 5.11. ¿A partir de qué cociente entre ramas largas y cortas empieza a fallar la parsimonia con 1000 sitios?

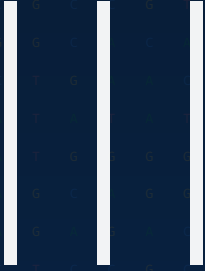
- 5.8** ●●● Descargue 10–15 secuencias del gen mitocondrial *COI* de especies de un mismo orden, alinéelas y ejecute IQ-TREE con `-m MFP -B 1000`. Compare el modelo elegido por el BIC con el elegido por el AIC y discuta qué clados reciben UFBoot < 95 % y por qué.

Referencias del capítulo

- Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716-723. <https://doi.org/10.1109/TAC.1974.1100705>
- Buneman, P. (1974). A note on the metric properties of trees. *Journal of Combinatorial Theory, Series B*, 17(1), 48-50. [https://doi.org/10.1016/0095-8956\(74\)90047-1](https://doi.org/10.1016/0095-8956(74)90047-1)
- Darwin, C. (2009). *The Origin of Species*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511694295> (Fecha inicial de publicación 1859)
- Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1-26. <https://doi.org/10.1214/aos/1176344552>
- Felsenstein, J. (1978). Cases in which parsimony or compatibility methods will be positively misleading. *Systematic Zoology*, 27(4), 401-410. <https://doi.org/10.2307/2412923>
- Felsenstein, J. (1981). Evolutionary trees from DNA sequences: A maximum likelihood approach. *Journal of Molecular Evolution*, 17(6), 368-376. <https://doi.org/10.1007/BF01734359>
- Felsenstein, J. (1985). Confidence limits on phylogenies: An approach using the bootstrap. *Evolution*, 39(4), 783-791. <https://doi.org/10.1111/j.1558-5646.1985.tb00420.x>
- Felsenstein, J. (2004). *Inferring Phylogenies*. Sinauer Associates.
- Fitch, W. M. (1971). Toward defining the course of evolution: Minimum change for a specific tree topology. *Systematic Zoology*, 20(4), 406-416. <https://doi.org/10.2307/2412116>
- Gascuel, O., & Steel, M. (2006). Neighbor-joining revealed. *Molecular Biology and Evolution*, 23(11), 1997-2000. <https://doi.org/10.1093/molbev/msl072>
- Guindon, S., & Gascuel, O. (2003). A simple, fast, and accurate algorithm to estimate large phylogenies by maximum likelihood. *Systematic Biology*, 52(5), 696-704. <https://doi.org/10.1080/10635150390235520>
- Hasegawa, M., Kishino, H., & Yano, T.-a. (1985). Dating of the human-ape splitting by a molecular clock of mitochondrial DNA. *Journal of Molecular Evolution*, 22(2), 160-174. <https://doi.org/10.1007/BF02101694>
- Hoang, D. T., Chernomor, O., von Haeseler, A., Minh, B. Q., & Vinh, L. S. (2018). UFBoot2: Improving the ultrafast bootstrap approximation. *Molecular Biology and Evolution*, 35(2), 518-522. <https://doi.org/10.1093/molbev/msx281>

- Jukes, T. H., & Cantor, C. R. (1969). Evolution of protein molecules. En H. N. Munro (Ed.), *Mammalian Protein Metabolism* (pp. 21-132). Academic Press. <https://doi.org/10.1016/B978-1-4832-3211-9.50009-7>
- Kalyaanamoorthy, S., Minh, B. Q., Wong, T. K. F., von Haeseler, A., & Jermini, L. S. (2017). ModelFinder: Fast model selection for accurate phylogenetic estimates. *Nature Methods*, 14(6), 587-589. <https://doi.org/10.1038/nmeth.4285>
- Kimura, M. (1980). A simple method for estimating evolutionary rates of base substitutions through comparative studies of nucleotide sequences. *Journal of Molecular Evolution*, 16(2), 111-120. <https://doi.org/10.1007/BF01731581>
- Minh, B. Q., Schmidt, H. A., Chernomor, O., Schrempf, D., Woodhams, M. D., von Haeseler, A., & Lanfear, R. (2020). IQ-TREE 2: New models and efficient methods for phylogenetic inference in the genomic era. *Molecular Biology and Evolution*, 37(5), 1530-1534. <https://doi.org/10.1093/molbev/msaa015>
- Nguyen, L.-T., Schmidt, H. A., von Haeseler, A., & Minh, B. Q. (2015). IQ-TREE: A fast and effective stochastic algorithm for estimating maximum-likelihood phylogenies. *Molecular Biology and Evolution*, 32(1), 268-274. <https://doi.org/10.1093/molbev/msu300>
- Rodríguez, F., Oliver, J. L., Marín, A., & Medina, J. R. (1990). The general stochastic model of nucleotide substitution. *Journal of Theoretical Biology*, 142(4), 485-501. [https://doi.org/10.1016/S0022-5193\(05\)80104-3](https://doi.org/10.1016/S0022-5193(05)80104-3)
- Saitou, N., & Nei, M. (1987). The neighbor-joining method: A new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4), 406-425. <https://doi.org/10.1093/oxfordjournals.molbev.a040454>
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461-464. <https://doi.org/10.1214/aos/1176344136>
- Stamatakis, A. (2014). RAxML version 8: A tool for phylogenetic analysis and post-analysis of large phylogenies. *Bioinformatics*, 30(9), 1312-1313. <https://doi.org/10.1093/bioinformatics/btu033>
- Studier, J. A., & Keppler, K. J. (1988). A note on the neighbor-joining algorithm of Saitou and Nei. *Molecular Biology and Evolution*, 5(6), 729-731. <https://doi.org/10.1093/oxfordjournals.molbev.a040527>
- Yang, Z. (1994). Maximum likelihood phylogenetic estimation from DNA sequences with variable rates over sites: Approximate methods. *Journal of Molecular Evolution*, 39(3), 306-314. <https://doi.org/10.1007/BF00160154>
- Yang, Z. (2014). *Molecular Evolution: A Statistical Approach*. Oxford University Press. <https://doi.org/10.1093/acprof:oso/9780199602605.001.0001>
- Zuckerkandl, E., & Pauling, L. (1965). Evolutionary divergence and convergence in proteins. En V. Bryson & H. J. Vogel (Eds.), *Evolving Genes and Proteins* (pp. 97-166). Academic Press. <https://doi.org/10.1016/B978-1-4832-2734-4.50017-6>

PARTE

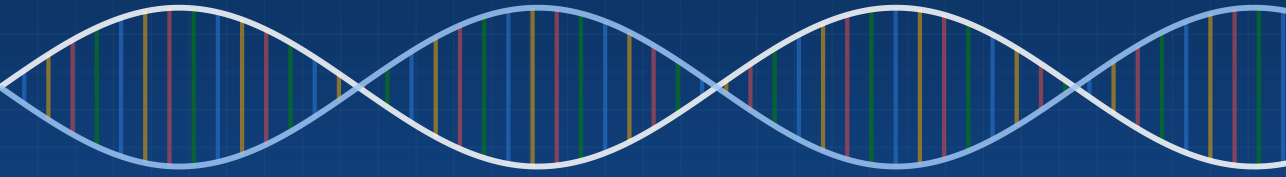


Del secuenciador al genoma

Lecturas, mapeo, ensamblaje y variantes

Los secuenciadores producen millones de fragmentos imperfectos. Esta parte sigue su recorrido completo: la calidad de cada base, su ubicación en una referencia, la reconstrucción de genomas desconocidos y la detección de las diferencias que nos hacen únicos.

Secuenciación de nueva generación (NGS)



Predecimos que, en la perspectiva larga de la historia, el impacto de la secuenciación de ADN estará a la par con el del microscopio.

— Shendure et al. (2017)

● Al terminar este capítulo usted podrá

- Explicar la química y la física de las principales plataformas de secuenciación (Sanger, 454, Illumina, PacBio y Oxford Nanopore) y relacionar cada mecanismo con su perfil de errores característico.
- Traducir una puntuación de calidad Phred en una probabilidad de error, decodificar la línea de calidades de un archivo FASTQ y calcular el número esperado de errores de una lectura.
- Leer un informe de control de calidad (FastQC/MultiQC) y decidir, con criterio, cómo recortar adaptadores y extremos de baja calidad.
- Derivar la teoría de Lander-Waterman (cobertura de Poisson, fracción no cubierta, número de *contigs*) y usarla, junto con modelos sobredispersos, para diseñar la profundidad de un experimento.

En 2001, leer un genoma humano costaba cerca de cien millones de dólares; dos décadas después, bastan unos pocos cientos (National Human Genome Research Institute, 2022). Ninguna otra tecnología de la biología ha abaratado su producto en cinco órdenes de magnitud en tan poco tiempo, y esa caída explica por qué hoy la secuenciación es el instrumento de medida universal del laboratorio: cuenta transcritos, localiza proteínas unidas al ADN, detecta patógenos en una muestra clínica o reconstruye la historia de poblaciones enteras. Pero las máquinas no entregan genomas: entregan millones de fragmentos cortos, o miles de fragmentos largos, cada uno con su propia incertidumbre. Todo lo que haremos en los capítulos siguientes (mapeo, llamado de variantes, ensamblaje, cuantificación de expresión) parte de ese material imperfecto, y conocer sus defectos es la mejor defensa contra conclusiones falsas.

El capítulo recorre el camino que va de la molécula al archivo. Primero veremos cómo funcionan las plataformas y por qué cada una se equivoca a su manera (sección 6.1). Luego aprenderemos el lenguaje con que la máquina expresa su confianza, la calidad Phred, y cómo usarlo para inspeccionar y limpiar las lecturas (sección 6.2). Por último, preguntaremos cuántas lecturas hacen falta para “ver” un genoma completo, una pregunta que Lander y Waterman (1988) respondieron con una elegante teoría probabilística que sigue guiando el diseño de cada experimento (sección 6.3).

6.1 Tecnologías de secuenciación



Colab · 6.1 · Tecnologías de secuenciación



repositorio del curso

Un estadio que se fotografía a sí mismo

Piense en esas coreografías de las tribunas en las que cada espectador levanta una cartulina de color a una señal. Suponga ahora que en cada asiento hay una persona con un mazo de cuatro colores y una lista secreta, y que desde el techo del estadio una cámara toma una foto después de cada señal. Tras cien señales tendremos cien fotografías; si seguimos un mismo asiento a través de todas ellas, leeremos su lista secreta, color por color. Nadie tuvo que leer las listas una por una: bastó sincronizar a miles de personas y fotografiarlas a la vez. Esta es, en esencia, la idea de la secuenciación masiva en paralelo. Y ya se adivinan sus defectos: si alguien se retrasa una señal, sus colores se

desfasan; si la foto está borrosa, los asientos vecinos se confunden; y cuanto más dura el espectáculo, más espectadores se distraen.

6.1.1 Leer una molécula: el método de Sanger

Todas las tecnologías de secuenciación resuelven el mismo problema: convertir una sucesión de bases químicamente casi idénticas en una señal física medible (una banda en un gel, un destello de luz, una variación de corriente) que pueda asignarse a una posición. La primera solución práctica, y la que dominó durante casi treinta años, fue la de Sanger et al. (1977). Su idea fue usar a la propia ADN polimerasa como “lectora”. La enzima copia una hebra molde a partir de un cebador, pero en la mezcla se añade, junto a los desoxinucleótidos normales (dNTP), una pequeña proporción de *didesoxinucleótidos* (ddNTP), que carecen del grupo hidroxilo 3' necesario para unir la siguiente base. Cada vez que la polimerasa incorpora un ddNTP, la síntesis se detiene. El resultado es una colección de fragmentos que terminan en cada posición donde aparece la base correspondiente; separándolos por tamaño con resolución de una base, se lee la secuencia.

Definición 6.1. (Probabilidad de terminación)

Sea r la probabilidad de que, frente a una posición del molde que admite la base b , la polimerasa incorpore el ddNTP en lugar del dNTP. Si las incorporaciones son independientes, la probabilidad de que la síntesis termine exactamente en la k -ésima aparición de b es geométrica:

$$\mathbb{P}(K = k) = (1 - r)^{k-1} r, \quad k = 1, 2, \dots \quad (6.1)$$

DONDE

r	Probabilidad de terminación por aparición; depende del cociente de concentraciones ddNTP/dNTP y de la preferencia de la enzima.
K	Número de apariciones de la base b que la polimerasa “atravesará” hasta terminar (incluida la última).
k	Valor concreto de K .

La ecuación (6.1) explica de inmediato la limitación más conocida del método. Si una base aparece, en promedio, una vez cada cuatro posiciones, la señal de los fragmentos que terminan cerca de la posición ℓ decae aproximadamente como

$(1 - r)^{l/4}$. Con $r = 0,005$, a las 400 bases queda el 61 % de la intensidad inicial y a las 800 bases solo el 37 %; con $r = 0,01$, la señal a 800 bases cae al 13 %. Aumentar r produce más fragmentos cortos (más señal al principio); disminuirlo alarga la lectura pero debilita todas las bandas. A esto se suma que la electroforesis separa cada vez peor un fragmento de n bases de otro de $n + 1$ a medida que n crece. Por ambas razones, las lecturas de Sanger se quedaron en el rango de 400 a 800 pares de bases (Bentley et al., 2008), largas para los estándares de la época pero imposibles de paralelizar a gran escala: cada capilar leía un único fragmento.

El genoma que se leyó a mano

Frederick Sanger recibió dos premios Nobel de Química: uno por secuenciar la insulina y otro, en 1980, por el método de terminación de cadena. En su artículo de 1977 lo aplicó al bacteriófago $\phi X174$ y señaló que era más rápido y exacto que sus métodos anteriores (Sanger et al., 1977). Automatizado con fluorocromos y electroforesis capilar, el método de Sanger produjo la primera secuencia del genoma humano. Para dar a cada base una medida objetiva de confianza nació el programa **phred** (Ewing et al., 1998), cuya escala de calidad estudiaremos en la sección 6.2. Shendure et al. (2017) ofrecen una crónica de estos cuarenta años.

6.1.2 Paralelizar: pirosecuenciación y la plataforma 454

La ruptura conceptual llegó al abandonar la electroforesis. En lugar de separar fragmentos, se puede *observar la síntesis mientras ocurre*. Ronaghi et al. (1998) describieron la *pirosecuenciación*: cada vez que la polimerasa incorpora un nucleótido, libera una molécula de pirofosfato (PP_i); una cascada enzimática convierte ese pirofosfato en ATP (ATP sulfurilasa) y el ATP en un destello de luz (luciferasa). Si se inyecta en la cámara de reacción un solo tipo de nucleótido a la vez, un destello indica que esa base es la siguiente del molde.

Margulies et al. (2005) llevaron esta química a una escala sin precedentes. Cada fragmento de ADN se amplificaba clonalmente sobre una microesfera dentro de una gota de emulsión agua-aceite (PCR en emulsión), y las esferas se depositaban en una placa de fibra óptica con cientos de miles de pocillos de picolitros. En un único experimento de cuatro horas el sistema leía 25 millones de bases con una exactitud del 99 % o superior, y con él se ensambló *de novo* el genoma de *Mycoplasma genitalium*. Había nacido la “nueva generación”.

La pirosecuenciación tiene un talón de Aquiles que se deduce de su propio principio. Si el molde contiene un homopolímero, por ejemplo **AAAAA**, la polimerasa incorpora las seis bases en un único flujo y la luz emitida es, idealmente, proporcional

a seis. Distinguir seis de siete a partir de una intensidad luminosa es mucho más difícil que distinguir una de dos, porque el ruido de la medición crece con la señal. Por eso el error dominante de 454 (y de otras químicas sin terminadores, como Ion Torrent) no son las sustituciones sino las *inserciones y deleciones en homopolímeros* (Metzker, 2010).

6.1.3 Illumina: síntesis con terminadores reversibles

La tecnología que terminó por dominar el mercado combina dos ideas: amplificar cada molécula *en su sitio*, sobre una superficie de vidrio, y leerla una base por ciclo con nucleótidos que se detienen *de forma reversible* (Bentley et al., 2008).

Preparación de la biblioteca. El ADN se fragmenta (por ultrasonido o enzimáticamente) hasta tamaños de unos cientos de pares de bases, y a cada fragmento se le ligan *adaptadores*: secuencias sintéticas conocidas que contienen los sitios de unión a la celda de flujo, los sitios de los cebadores de secuenciación y, a menudo, *índices* (*barcodes*) que identifican la muestra y permiten mezclar decenas de muestras en una misma corrida (*multiplexing*). La parte biológica del fragmento, entre los dos adaptadores, se llama *inserto*.

Amplificación en puente. La celda de flujo está tapizada de dos tipos de oligonucleótidos complementarios a los adaptadores. Un fragmento monocatenario se hibrida a uno de ellos, la polimerasa lo copia, el original se lava, y la copia anclada se dobla para hibridarse con un oligonucleótido vecino del otro tipo, formando un “puente”. Repetido decenas de veces, este ciclo produce un *clúster*: un grupo compacto de unas mil copias idénticas de la molécula original, lo bastante brillante para ser fotografiado (figura 6.1). La química de anclaje permite densidades de unos diez millones de colonias por centímetro cuadrado (Fedurco et al., 2006).

El ciclo de secuenciación. Cada clúster se secuencia por síntesis (SBS) con cuatro nucleótidos modificados de dos maneras: llevan un fluorocromo distinto por base y un *grupo bloqueador* en el extremo 3' que impide que la polimerasa añada una segunda base. Un ciclo consta de cuatro pasos (figura 6.2): (1) se inyectan los cuatro nucleótidos y la polimerasa incorpora exactamente uno en cada hebra; (2) se lava el exceso y se excita la celda con láser; una cámara registra el color de cada clúster; (3) se escinden químicamente el fluorocromo y el bloqueador, restaurando un 3'-OH libre; (4) se repite. Como los cuatro nucleótidos compiten a la vez, los homopolímeros ya no son un problema: el error típico de Illumina es la *sustitución*, no el *indel*.

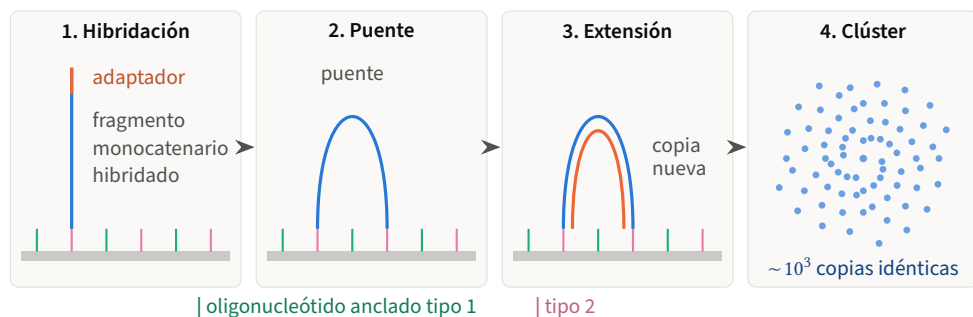


Figura 6.1. Amplificación en puente sobre la celda de flujo. (1) Un fragmento de la biblioteca, flanqueado por adaptadores, se hibrida con un oligonucleótido anclado. (2) La copia sintetizada se dobla y se hibrida con un oligonucleótido vecino del otro tipo. (3) La polimerasa la extiende y forma un puente de doble cadena, que se desnaturaliza para empezar otra ronda. (4) Tras unas treinta rondas, cada molécula original ha dado lugar a un clúster de alrededor de mil copias, visto aquí desde arriba. La amplificación convierte una molécula invisible en un punto luminoso, pero también introduce sesgos (sección 6.3).

6.1.4 Por qué la calidad cae a lo largo de la lectura: fase y desfase

Si todas las hebras de un clúster avanzaran en perfecta sincronía, la calidad sería la misma en el ciclo 1 que en el 150. No es así, y la razón es instructiva. En cada ciclo, una pequeña fracción de las hebras no incorpora ninguna base (por ejemplo, porque la escisión del bloqueador falló o la polimerasa no actuó): quedan *retrasadas* una posición (*phasing*). Otra fracción incorpora dos bases, porque algún nucleótido carecía de bloqueador: quedan *adelantadas* (*pre-phasing*). Las hebras desfasadas siguen emitiendo luz, pero del color equivocado, y contaminan la señal del clúster.

Teorema 6.1. (Decaimiento de la fracción en fase)

Sea $\varepsilon = p + q$ la probabilidad, por ciclo y por hebra, de un evento de desfase (retraso con probabilidad p , adelanto con probabilidad q), independiente entre ciclos. La fracción de hebras que siguen exactamente en fase tras n ciclos es

$$\phi(n) = (1 - \varepsilon)^n \approx e^{-\varepsilon n}. \quad (6.2)$$

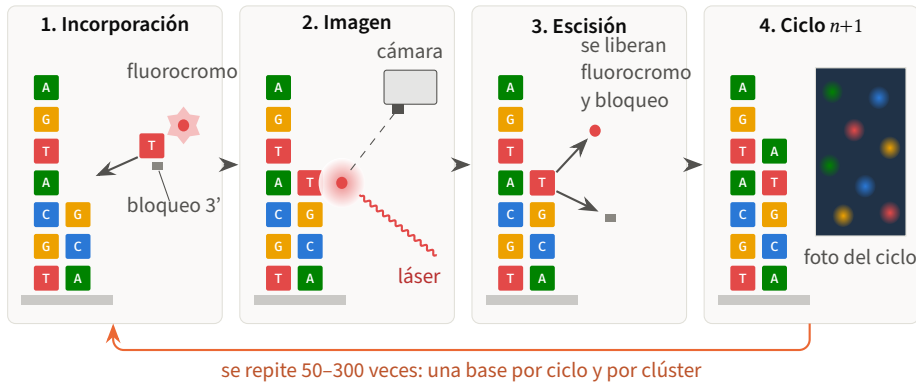


Figura 6.2. Un ciclo de secuenciación por síntesis con terminadores reversibles. (1) La polimerasa incorpora un único nucleótido marcado (aquí T, complementario de la A del molde); el grupo bloqueador impide añadir otro. (2) Un láser excita el fluorocromo y la cámara registra el color de cada clúster de la celda. (3) Se escinden el fluorocromo y el bloqueador, dejando un 3'-OH libre. (4) Comienza el ciclo siguiente; la secuencia de un clúster es la sucesión de colores que emite, foto tras foto. Una lectura de 150 bases requiere 150 ciclos y otras tantas imágenes de toda la celda.

DONDE

p	Probabilidad por ciclo de no incorporar ninguna base (<i>phasing</i>).
q	Probabilidad por ciclo de incorporar dos bases (<i>pre-phasing</i>).
ε	Tasa total de desfase por ciclo, $p + q$.
n	Número de ciclos completados (posición en la lectura).
$\phi(n)$	Fracción de hebras del clúster que emiten la base correcta en el ciclo n .

La demostración es inmediata: una hebra está en fase tras n ciclos si y solo si no sufrió ningún evento en ninguno de ellos, y la probabilidad de ese suceso es el producto de n factores $(1 - \varepsilon)$. La aproximación exponencial usa $\ln(1 - \varepsilon) \approx -\varepsilon$ para ε pequeño. Aunque ε sea diminuto, su efecto se *acumula*. Con $\varepsilon = 0,002$, en el ciclo 150 solo el 74 % de las hebras emite el color correcto, y en el ciclo 300 apenas el 55 % (figura 6.3). El resto forma un fondo de colores de las posiciones vecinas que, ciclo a ciclo, hace más difícil decidir cuál es la base dominante. El programa de llamado de bases (*basecaller*) corrige parte de este efecto modelando el desfase, pero no todo, y por eso la *calidad de las lecturas de Illumina disminuye hacia el extremo*

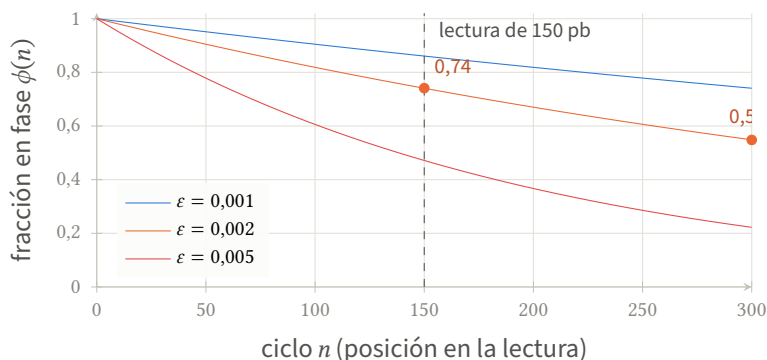


Figura 6.3. Fracción de hebras de un clúster que permanecen en fase según la ecuación (6.2), para tres tasas de desfase por ciclo. Tasas del orden de una milésima, que parecen despreciables, hacen que tras 300 ciclos casi la mitad de la señal provenga de hebras desfasadas. Este mecanismo, junto con el desgaste de los reactivos y el fotoblanqueo, fija el límite práctico de longitud de las lecturas cortas y explica la caída de calidad hacia el extremo 3'.

3'. Este patrón es el que veremos en todos los informes de control de calidad de la sección 6.2.

Lecturas pareadas y química de dos colores. Tras leer un extremo del inserto, la hebra puede regenerarse para leer el extremo opuesto: se obtienen *lecturas pareadas* (*paired-end*), dos lecturas cuya distancia y orientación relativas son conocidas aproximadamente. Esa restricción geométrica será valiosísima para el mapeo y el ensamblaje. Bentley et al. (2008) secuenciaron así el genoma de un varón yoruba con pares de lecturas de 35 bases y una profundidad media superior a 30×. Los instrumentos más recientes usan una química de *dos colores*, en la que una de las cuatro bases (G) se reconoce por la *ausencia* de fluorescencia; la consecuencia práctica es que un clúster que se apaga produce falsas colas de GGGGG, un artefacto que herramientas como *fastp* recortan de forma específica (Chen et al., 2018).

6.1.5 Tercera generación: molécula única y lecturas largas

Las plataformas de lecturas cortas leen clústeres, no moléculas; necesitan amplificar, y el desfase limita su longitud a unos cientos de bases. Las llamadas tecnologías de *tercera generación* observan una sola molécula en tiempo real y producen lecturas de decenas o cientos de kilobases.

PacBio SMRT. Eid et al. (2009) fijaron una única polimerasa en el fondo de un *guía de onda de modo cero* (*zero-mode waveguide*), un pozo de dimensiones inferiores a la longitud de onda de la luz que limita la observación a un volumen de zeptolitros. Los fluorocromos están unidos al fosfato terminal del nucleótido, de modo que se liberan con el pirofosfato al incorporarse la base: la hebra sintetizada queda sin modificar y la polimerasa puede seguir durante miles de bases. Cada incorporación se ve como un pulso de luz del color correspondiente. Los errores de esta lectura continua (*continuous long read*) son frecuentes, del orden del 10 al 15 %, dominados por inserciones y deleciones, pero tienen una propiedad preciosa: son *aproximadamente aleatorios*, sin sesgo sistemático fuerte. Eid et al. (2009) ya mostraron que el consenso de lecturas a 15× alcanzaba una exactitud mediana del 99,3 %.

Esa aleatoriedad es la base de las lecturas HiFi. Si el inserto se circulariza con adaptadores en horquilla, la polimerasa da muchas vueltas a la misma molécula y cada vuelta (*pass*) es una lectura independiente del mismo molde. La secuencia consenso circular (CCS) combina las pasadas.

Ejemplo 6.1. (Cuántas pasadas hacen una lectura HiFi)

Modelemos cada pasada como un voto independiente que acierta la base con probabilidad $1 - e$ y supongamos, de forma pesimista, que basta que la mayoría de las pasadas se equivoque para que el consenso falle. Con k pasadas (impar),

$$e_{\text{cons}}(k) = \sum_{j=\lfloor k/2 \rfloor}^k \binom{k}{j} e^j (1 - e)^{k-j}. \quad (6.3)$$

Con $e = 0,10$ por pasada: una pasada da $Q10$; tres pasadas, $e_{\text{cons}} = 0,028$ ($Q15,5$); cinco, $0,0086$ ($Q20,7$); nueve, $8,9 \times 10^{-4}$ ($Q30,5$), y quince, $3,4 \times 10^{-5}$ ($Q44,7$). La cota es pesimista porque, en la realidad, varias pasadas erróneas raramente coinciden en el *mismo* error. El resultado ilustra un principio general: los errores aleatorios se corrigen con redundancia; los sistemáticos, no. Wenger et al. (2019) optimizaron este procedimiento y obtuvieron lecturas HiFi con un 99,8 % de exactitud y una longitud media de 13,5 kb, capaces de ensamblar un genoma humano con *contigs* N50 superiores a 15 Mb.

Oxford Nanopore. La idea más radical prescinde por completo de la óptica y de la polimerasa como sensor. Una proteína motora hace pasar una hebra de ADN, base a base, a través de un nanoporo proteico insertado en una membrana con un voltaje aplicado; la corriente iónica que atraviesa el poro depende de las *varias* bases que lo

ocupan en cada instante (un k -mer), y la señal resultante (el *squiggle*) se traduce a bases con redes neuronales (Jain et al., 2016; Rang et al., 2018). No hay límite químico intrínseco a la longitud de lectura: Jain et al. (2018) secuenciaron un genoma humano con lecturas ultralargas de N50 superior a 100 kb y una lectura máxima de 882 kb, que permitieron ensamblar completo el complejo principal de histocompatibilidad. El precio ha sido una tasa de error alta, entre el 5 y el 15 % en las químicas R9 según la revisión de Rang et al. (2018), con una fracción *sistemática* importante en homopolímeros (la corriente cambia poco mientras el poro “ve” siempre la misma base). Las químicas posteriores han reducido mucho ese problema: con los poros R10.4 ya es posible obtener genomas bacterianos casi terminados sin corrección con lecturas cortas (Sereika et al., 2022).

6.1.6 Comparación y perfiles de error

La tabla 6.1 y la figura 6.4 resumen el panorama. La lección fundamental no es qué plataforma es “mejor”, sino que *cada mecanismo físico deja una huella característica en los errores*, y los algoritmos posteriores deben conocerla: un alineador pensado para sustituciones aisladas sufrirá con lecturas cuyos errores son *indels*, y un ensamblador que confía en k -mers exactos fracasará con lecturas al 10 % de error. Las revisiones de Metzker (2010) y Goodwin et al. (2016) describen estas diferencias con detalle.

⚠ Aleatorio no es lo mismo que sistemático

Un error aleatorio se diluye al aumentar la profundidad: con 30 lecturas independientes, el consenso será correcto aunque cada lectura falle una vez de cada diez. Un error *sistemático*, en cambio, se repite en todas las lecturas que cubren el mismo contexto (un homopolímero largo, una región muy rica en GC) y ninguna profundidad lo corrige. Antes de confiar en una variante observada en muchas lecturas, pregúntese si todas ellas pudieron equivocarse *de la misma manera*.

6.1.7 El costo de un genoma

La figura 6.5 muestra la serie histórica que mantiene el Instituto Nacional de Investigación del Genoma Humano de EE. UU. (National Human Genome Research Institute, 2022). Hasta 2007 el costo cayó al ritmo de la ley de Moore, que describe la mejora de los microprocesadores (una reducción a la mitad cada dos años). A partir de 2008, con la llegada de las plataformas masivamente paralelas, la curva

Tabla 6.1. Características aproximadas de las principales plataformas. Las cifras son órdenes de magnitud típicos, que cambian con cada química y versión de instrumento; véanse las fuentes citadas en el texto.

Plataforma	Señal	Longitud	Error	Error dominante
Sanger	fluorescencia, electroforesis	400–800 pb	bajo	caída de señal al final
454 / Ion Torrent	luz / pH por flujo	100–700 pb	~1 %	<i>indels</i> en homopolímeros
Illumina	fluorescencia por ciclo	50–300 pb	~0,1 %	sustituciones; peor en 3'
PacBio CLR	pulso de luz en ZMW	10–50 kb	10–15 %	<i>indels</i> aleatorios
PacBio HiFi	consenso circular	10–25 kb	≤0,5 %	residual en homopolímeros
Nanopore	corriente iónica	1–>100 kb	1–15 %	<i>indels</i> sistemáticos en homopolímeros

se desplomó: si la secuenciación hubiera seguido a los procesadores, un genoma costaría hoy unos 70 000 dólares; cuesta del orden de 500. Esa divergencia tiene una consecuencia que se repite a lo largo del libro: hoy es mucho más barato *generar* datos que *analizarlos*, y el cuello de botella de la genómica se ha desplazado del laboratorio a la computadora.

Cada plataforma convierte bases en una señal física distinta, y su perfil de errores es la huella de esa física: sustituciones en Illumina, *indels* en homopolímeros en 454 y Nanopore, *indels* aleatorios corregibles por consenso en PacBio.

6.2 Calidad Phred, control de calidad y *trimming*



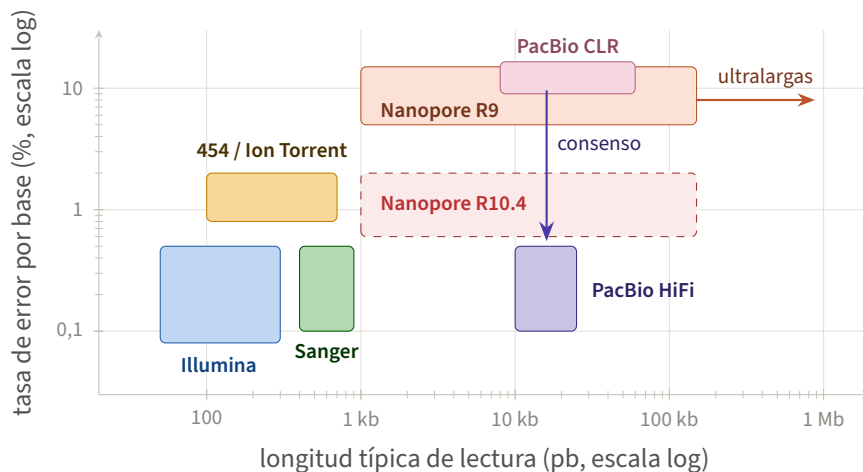


Figura 6.4. Mapa aproximado de las plataformas según longitud de lectura y tasa de error por base (ambos ejes logarítmicos). Los rectángulos indican órdenes de magnitud típicos, no especificaciones exactas (Goodwin et al., 2016; Jain et al., 2018; Rang et al., 2018; Wenger et al., 2019). La flecha violeta muestra la estrategia HiFi: sacrificar longitud y rendimiento para convertir errores aleatorios en exactitud por consenso. Durante una década existió un compromiso entre longitud y exactitud; las químicas recientes de lecturas largas lo han ido borrando.

El pronóstico del tiempo de cada base

Un buen servicio meteorológico no dice “mañana lloverá”: dice “mañana hay un 70 % de probabilidad de lluvia”. Y lo que lo hace bueno no es acertar siempre, sino estar *calibrado*: de todos los días en que anunció un 70 %, llovió aproximadamente en siete de cada diez. Un secuenciador moderno hace lo mismo con cada base que entrega. Junto a la letra **A** escribe, en clave, algo como “me equivoco en una de cada mil veces que digo esto”. Aprender a leer esa clave, y a desconfiar cuando el pronóstico empeora, es el primer paso de cualquier análisis.

6.2.1 La escala Phred

Cuando el programa **phred** asignaba bases a partir de las curvas de fluorescencia de los secuenciadores capilares, calculaba para cada pico varios parámetros de la traza (espaciado entre picos, cociente entre la señal llamada y la no llamada, etc.) y, mediante una tabla calibrada con lecturas de secuencia conocida, los convertía en

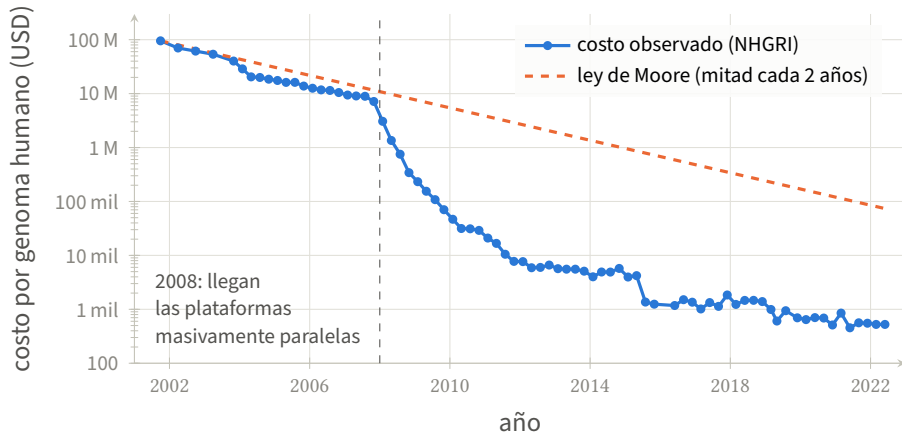


Figura 6.5. Costo de secuenciar un genoma humano, 2001–2022, según los datos del NHGRI (National Human Genome Research Institute, 2022), en escala logarítmica. La línea naranja muestra lo que habría costado si hubiera seguido la ley de Moore desde 2001. La brecha de dos órdenes de magnitud que se abre a partir de 2008 corresponde a la transición de la electroforesis capilar a la secuenciación masiva en paralelo. Datos: `figuras/cap06/nhgri_costos.csv`.

una probabilidad de error (Ewing & Green, 1998; Ewing et al., 1998). Para expresar esa probabilidad de forma compacta, los autores propusieron una escala logarítmica que hoy es universal.

Definición 6.2. (Calidad Phred)

Si p es la probabilidad de que la base asignada sea incorrecta, su calidad Phred es

$$Q = -10 \log_{10} p \quad \Longleftrightarrow \quad p = 10^{-Q/10}. \quad (6.4)$$

DONDE

p	Probabilidad de error de la base, estimada por el <i>basecaller</i> .
Q	Calidad Phred, en “decibelios” de probabilidad; normalmente se redondea a un entero.

La escala tiene dos virtudes. Primero, es *intuitiva en órdenes de magnitud*: cada 10 unidades de Q dividen el error por diez. $Q10$ es un error de cada 10 bases (90 % de exactitud), $Q20$ uno de cada 100, $Q30$ uno de cada mil y $Q40$ uno de cada diez mil

(99,99 %). Segundo, convierte productos en sumas: la probabilidad de que dos bases independientes sean ambas incorrectas, $p_1 p_2$, corresponde a una calidad $Q_1 + Q_2$. La cifra “Q30” que aparece en las especificaciones de los instrumentos es, por lo tanto, una promesa concreta: como mucho un error por cada mil bases.

6.2.2 Del registro a la probabilidad: el formato FASTQ

Las lecturas se almacenan en archivos FASTQ, que Cock et al. (2010) documentaron formalmente junto con sus variantes históricas. Cada lectura ocupa cuatro líneas: un identificador que empieza con @, la secuencia, una línea separadora que empieza con +, y una cadena de calidades *de la misma longitud que la secuencia*, en la que cada carácter codifica el Q de la base correspondiente.

Definición 6.3. (Codificación ASCII de las calidades)

En la codificación estándar (llamada *Sanger* o Phred+33), el carácter κ de la línea de calidades representa

$$Q = \text{ord}(\kappa) - 33, \quad (6.5)$$

de modo que ! (ASCII 33) es Q_0 , + (43) es Q_{10} , 5 (53) es Q_{20} , ? (63) es Q_{30} e I (73) es Q_{40} .

DONDE

κ	Carácter imprimible de la línea de calidades.
$\text{ord}(\kappa)$	Código ASCII del carácter (por ejemplo, $\text{ord}(\text{I}) = 73$).
33	Desplazamiento (<i>offset</i>); se elige 33 porque es el primer carácter ASCII imprimible después del espacio.

Por desgracia, no siempre fue así. Los primeros instrumentos de Solexa/Illumina usaron un desplazamiento de 64 y, además, una escala distinta, basada en el cociente de probabilidades (*odds*),

$$Q_{\text{Solexa}} = -10 \log_{10} \frac{p}{1-p}, \quad Q_{\text{Phred}} = 10 \log_{10} (10^{Q_{\text{Solexa}}/10} + 1), \quad (6.6)$$

que admite valores negativos y coincide con la escala Phred solo para calidades altas (Cock et al., 2010). Versiones posteriores pasaron a Phred+64 y, finalmente, a la codificación Sanger que hoy es el estándar. La consecuencia práctica es que un

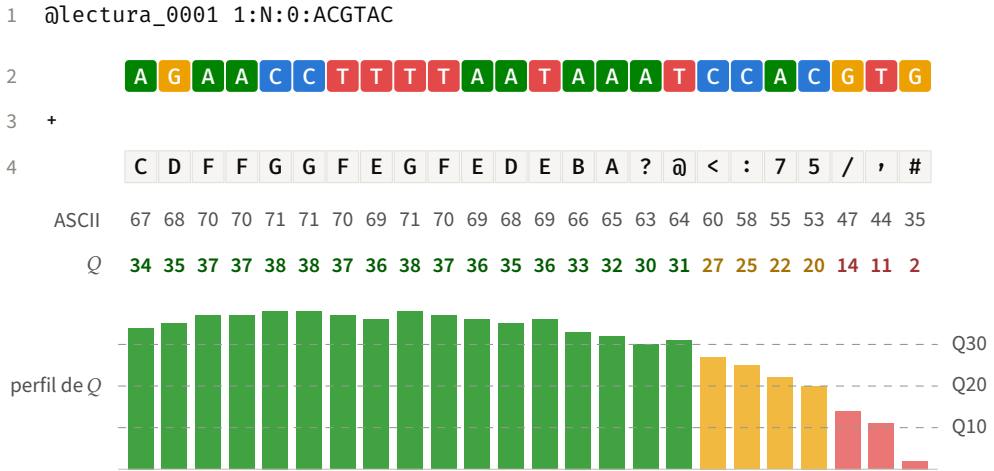


Figura 6.6. Anatomía de un registro FASTQ. Las líneas 1 a 4 son el identificador, la secuencia, el separador y la cadena de calidades. Debajo de cada carácter de calidad se muestra su código ASCII y la calidad $Q = \text{ord} - 33$ (ecuación (6.5)); las barras reproducen el perfil de calidad con las referencias Q_{10} , Q_{20} y Q_{30} . Obsérvese la caída hacia el extremo 3' y el último carácter, # (Q_2), que algunos *basecallers* usan como marca de “base no fiable”. Lectura simulada con `figuras/cap06/generar.py`.

archivo antiguo leído con el desplazamiento equivocado tendrá todas sus calidades desplazadas en 31 unidades: un error silencioso que convierte bases malas en excelentes, o al revés.

DONDE

Q_{Solexa}	Calidad en la escala de <i>odds</i> logarítmicos de Solexa.
Q_{Phred}	Calidad equivalente en la escala Phred de la ecuación (6.4).

6.2.3 Errores esperados en una lectura

Si conocemos la probabilidad de error de cada base, podemos resumir la calidad de toda una lectura. Considerando las bases independientes (una aproximación razonable), el número de errores de una lectura de longitud L es una suma de variables de Bernoulli.

Teorema 6.2. (Errores esperados y probabilidad de lectura perfecta)

Para una lectura con probabilidades de error $p_1, \dots, p_L$ independientes, el número de errores X cumple

$$\mathbb{E}[X] = \sum_{i=1}^L p_i = \sum_{i=1}^L 10^{-Q_i/10}, \quad \mathbb{P}(X = 0) = \prod_{i=1}^L (1 - p_i). \quad (6.7)$$

DONDE

X	Número (aleatorio) de bases incorrectas en la lectura.
p_i, Q_i	Probabilidad de error y calidad Phred de la base i .
L	Longitud de la lectura.

La esperanza de una suma es la suma de las esperanzas, sin necesidad siquiera de independencia; la segunda fórmula, en cambio, sí la requiere. Un detalle importante es que el *promedio de las calidades* no es un buen resumen: como p es exponencial en Q , una sola base de $Q2$ aporta más errores esperados que treinta bases de $Q30$. Por eso algunos filtros (por ejemplo, en el análisis de amplicones) descartan lecturas según su $\mathbb{E}[X]$, no según su Q medio.

Ejemplo 6.2. (La lectura de la figura 6.6)

Las 24 calidades de la figura suman $\mathbb{E}[X] = 0,78$ errores esperados. Sin embargo, 0,63 de ellos proceden de la última base ($Q2$, $p = 10^{-0,2} \approx 0,63$); las 20 primeras, todas con $Q \geq 20$, aportan en conjunto menos de 0,02. Recortar solo las cuatro últimas bases elimina más del 95 % de los errores esperados.

A mayor escala, comparemos dos lecturas de 150 bases con calidad uniforme. A $Q30$, $\mathbb{E}[X] = 150 \times 10^{-3} = 0,15$ y $\mathbb{P}(X = 0) = 0,999^{150} \approx 0,86$: casi nueve de cada diez lecturas no tienen ningún error. A $Q20$, $\mathbb{E}[X] = 1,5$ y $\mathbb{P}(X = 0) = 0,99^{150} \approx 0,22$: casi cuatro de cada cinco lecturas llevan al menos un error. La diferencia entre $Q20$ y $Q30$, que parece modesta en la escala logarítmica, cambia por completo la naturaleza de los datos (figura 6.7).

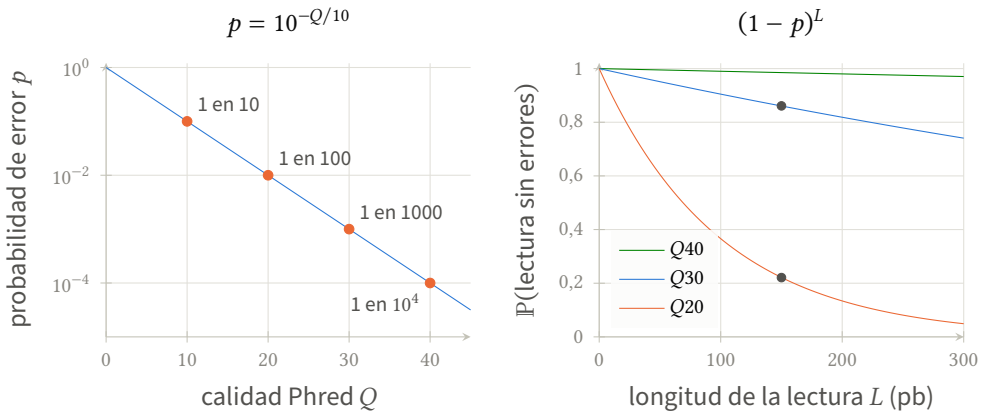


Figura 6.7. **Izquierda:** la escala Phred es logarítmica; cada 10 unidades de Q reducen el error en un factor 10. **Derecha:** probabilidad de que una lectura no contenga ningún error (ecuación (6.7)) si todas sus bases tienen la misma calidad. Los puntos grises marcan las lecturas de 150 pb del ejemplo 6.2: 0,86 a $Q30$ frente a 0,22 a $Q20$.

⚠ Las calidades son estimaciones, no verdades

Un Q es tan bueno como la calibración del *basecaller* que lo produjo. Las calidades pueden estar sistemáticamente infladas o desinfladas según el instrumento, la química y el contexto de secuencia; por eso los flujos de llamado de variantes suelen *recalibrarlas* empíricamente usando posiciones conocidas. Además, muchos instrumentos actuales agrupan las calidades en unos pocos niveles para comprimir los archivos, de modo que un Q individual tiene una resolución limitada.

6.2.4 Control de calidad: FastQC y MultiQC

Antes de cualquier análisis conviene mirar los datos en conjunto. **FastQC** (Andrews, 2010) es la herramienta de referencia: recorre un archivo FASTQ y produce un informe con una serie de módulos, cada uno marcado como aprobado, dudoso o fallido. Los más útiles son:

- **Calidad por posición** (*per base sequence quality*): diagramas de caja de Q en cada posición de la lectura (figura 6.8). Revela la caída hacia 3' que predice la ecuación (6.2), o problemas puntuales en ciclos concretos (una burbuja en la celda, un fallo de reactivo).

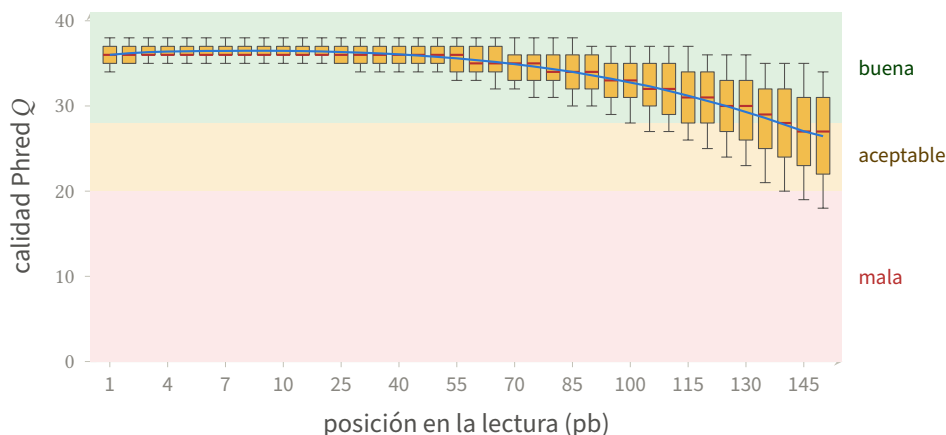


Figura 6.8. Calidad por posición de 20 000 lecturas simuladas de 150 pb, al estilo del módulo *per base sequence quality* de FastQC. Como en FastQC, las primeras nueve posiciones se muestran individualmente y el resto en bloques de cinco. Cada caja abarca del cuartil 1 al 3, con la mediana en rojo y bigotes entre los percentiles 10 y 90; la línea azul es la media. El patrón es el típico de Illumina: calidad alta y estable al principio, una ligera caída en las primeras bases y un deterioro creciente, con mayor dispersión, hacia el extremo 3'. En las últimas posiciones el percentil 10 cae por debajo de Q_{20} : candidatas a recorte.

- **Contenido de bases por posición:** en una biblioteca aleatoria, las proporciones de A, C, G y T deberían ser constantes a lo largo de la lectura. Las oscilaciones en las primeras posiciones son habituales en RNA-seq por los sesgos de los cebadores aleatorios, y no justifican por sí solas un recorte.
- **Contenido de GC por lectura:** debería parecerse a una distribución unimodal centrada en el GC del organismo; una segunda moda sugiere contaminación.
- **Niveles de duplicación y secuencias sobrerrepresentadas:** detectan bibliotecas de baja complejidad, dímeros de adaptadores y contaminantes abundantes.
- **Contenido de adaptadores:** fracción acumulada de lecturas en las que aparece una secuencia de adaptador conocida a partir de cada posición.

Un proyecto real tiene decenas o cientos de muestras, y abrir cientos de informes es inviable. **MultiQC** (Ewels et al., 2016) resuelve este problema: busca en un directorio los resultados de FastQC y de muchas otras herramientas (recortadores, alineadores, cuantificadores) y los agrega en un único informe interactivo, donde las muestras atípicas y los efectos de lote saltan a la vista.

6.2.5 Duplicados y complejidad de la biblioteca

Un duplicado es un par de lecturas que proceden de la *misma molécula original*: copias creadas por la PCR de la preparación de la biblioteca, o duplicados “ópticos” en los que un mismo clúster se registró dos veces. Los duplicados no aportan información independiente y, si no se identifican, inflan artificialmente la confianza en lo que sea que esas copias contengan, incluido un error de la PCR temprana. Pero no todos los duplicados son artefactos: si la biblioteca tiene pocas moléculas distintas, incluso un muestreo perfecto las repetirá.

Teorema 6.3. (Moléculas distintas observadas)

Si una biblioteca contiene M moléculas distintas igualmente representadas y se secuencian N lecturas elegidas al azar, el número esperado de moléculas distintas observadas es

$$E[D] = M \left[1 - \left(1 - \frac{1}{M} \right)^N \right] \approx M \left(1 - e^{-N/M} \right), \quad (6.8)$$

y la tasa de duplicación esperada es $1 - E[D]/N$.

DONDE

M	Complejidad de la biblioteca: número de moléculas únicas disponibles.
N	Número de lecturas secuenciadas.
D	Número de moléculas distintas observadas al menos una vez.

La demostración usa un argumento que reaparecerá, casi idéntico, en la sección 6.3: una molécula concreta no es elegida por ninguna de las N lecturas con probabilidad $(1 - 1/M)^N \approx e^{-N/M}$, y la esperanza de D es M veces la probabilidad de lo contrario. Con $M = 10^7$ moléculas y $N = 2 \times 10^7$ lecturas, $E[D] \approx 8,65 \times 10^6$ y la tasa de duplicación es del 57 %: la mitad del presupuesto de secuenciación se gasta en releer lo mismo. Con una biblioteca diez veces más compleja ($M = 10^8$), la misma cantidad de lecturas produce solo un 9 % de duplicados. La lección práctica es que *la complejidad de la biblioteca, no el número de lecturas, limita la información*; secuenciar más una biblioteca pobre solo produce más duplicados.

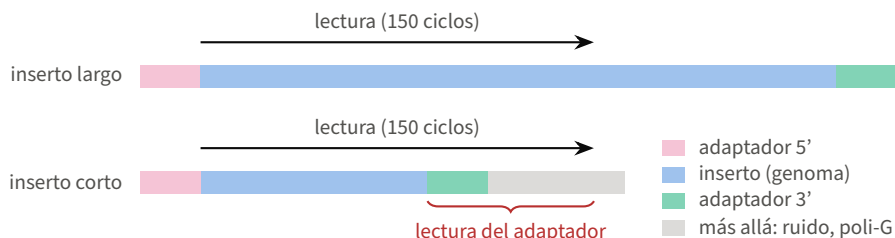


Figura 6.9. Cuando el inserto (azul) es más largo que el número de ciclos, la lectura termina dentro del genoma. Cuando es más corto, la lectura lo atraviesa y continúa en el adaptador 3' (aqua) y, más allá, en una región sin molde que produce ruido o, en la química de dos colores, colas de GGG. Las bases desde el inicio del adaptador deben recortarse. En lecturas pareadas, los dos miembros del par se solapan por completo en este caso, lo que permite localizar el adaptador sin conocer su secuencia.

6.2.6 Recorte de adaptadores

Si el inserto es más corto que la longitud de lectura, la polimerasa lo atraviesa por completo y sigue leyendo el adaptador del otro extremo (figura 6.9). Esas bases no pertenecen al organismo: si no se eliminan, impedirán el alineamiento o, peor, producirán falsas discrepancias en el extremo 3'. El fenómeno es inevitable en bibliotecas de fragmentos cortos, como las de ARN pequeños o las de ADN antiguo y degradado.

Detectar el adaptador parece trivial, porque su secuencia es conocida (por ejemplo, el prefijo **AGATCGGAAGAGC**, común a los adaptadores TruSeq de Illumina). La dificultad está en el extremo: cuando el adaptador empieza a solo k bases del final de la lectura, apenas se ven sus primeras k letras, y una coincidencia tan corta puede ser casual.

Definición 6.4. (Coincidencia casual con el adaptador)

Si las bases de la lectura son independientes y equiprobables, la probabilidad de que sus últimas k bases coincidan por azar con las primeras k del adaptador es

$$P(\text{coincidencia casual de longitud } k) = 4^{-k}. \quad (6.9)$$

DONDE

k Longitud del solapamiento entre el extremo 3' de la lectura y el comienzo del adaptador.

Con $k = 3$, uno de cada 64 extremos (1,6 %) coincide por azar; con $k = 10$, menos de uno por millón. Las herramientas, por lo tanto, deben aceptar un compromiso: exigir un solapamiento mínimo largo deja adaptadores residuales cortos; aceptar solapamientos cortos recorta de más unas pocas bases genuinas en una fracción de las lecturas, un daño normalmente menor. [cutadapt](#) (Martin, 2011) busca el adaptador mediante un alineamiento semiglobal que tolera una fracción de errores, y recorta desde el punto de coincidencia. [Trimmomatic](#) (Bolger et al., 2014) añade un modo “palíndromo” para lecturas pareadas, que aprovecha que ambos miembros del par, cuando el inserto es corto, son complementarios inversos entre sí. [fastp](#) (Chen et al., 2018) usa la misma idea de solapamiento para detectar adaptadores automáticamente y, en un único recorrido del archivo, realiza también el control de calidad, el recorte por calidad y el de colas de poli-G.

6.2.7 Recorte por calidad

Una vez eliminados los adaptadores, queda decidir qué hacer con los extremos de baja calidad. Hay dos estrategias clásicas.

Ventana deslizante. [Trimmomatic](#) recorre la lectura desde el extremo 5' con una ventana de w bases y la corta cuando la calidad media de la ventana cae por debajo de un umbral t (opción `SLIDINGWINDOW:w:t`) (Bolger et al., 2014). Formalmente, la lectura se trunca en

$$x^* = \min \left\{ x : \frac{1}{w} \sum_{i=x}^{x+w-1} Q_i < t \right\}, \quad (6.10)$$

conservando las bases $1, \dots, x^* - 1$. Promediar sobre una ventana evita que una única base mala, aislada en medio de una lectura excelente, provoque un corte prematuro.

DONDE

w, t Anchura de la ventana y umbral de calidad media (p. ej., $w = 4$, $t = 20$).

x^* Primera posición en la que empieza una ventana con calidad media inferior al umbral.

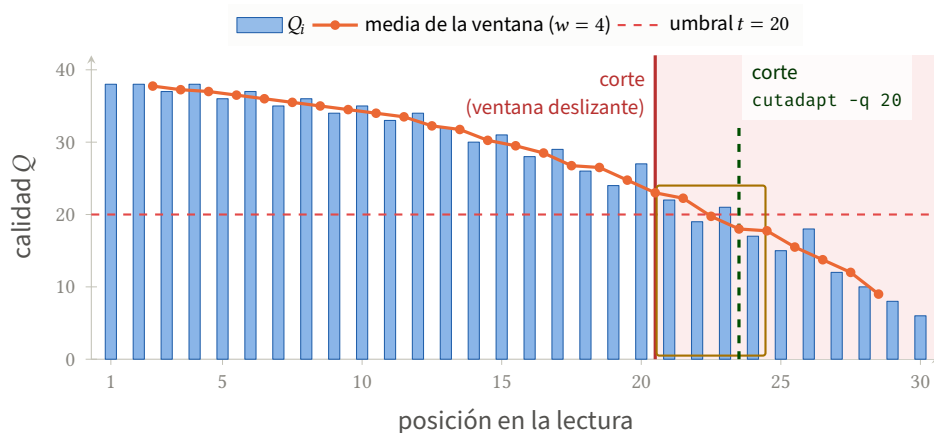


Figura 6.10. Recorte por calidad de una lectura de 30 bases. Las barras son las calidades individuales; la curva naranja, la media de cada ventana de cuatro bases, dibujada en su centro. La primera ventana con media inferior a 20 empieza en la posición 21 (recuadro amarillo, media 19,75), así que la ventana deslizante conserva 20 bases (ecuación (6.10)). El criterio de suma acumulada (ecuación (6.11)) es menos conservador y conserva 23: las bases 21 a 23 ($Q = 22, 19, 21$) tienen, en conjunto, una calidad neta por encima del umbral. Ninguno de los dos es “el correcto”; son reglas distintas para el mismo compromiso.

Suma acumulada desde 3’. La alternativa, heredada del alineador BWA y usada por la opción `-q` de `cutadapt`, recorre la lectura desde el extremo 3’ acumulando $t - Q_i$ y corta donde esa suma alcanza su máximo:

$$x^* = \arg \max_x \sum_{i=x}^L (t - Q_i). \quad (6.11)$$

Es exactamente el problema del segmento de suma máxima que vimos en el capítulo 3 con Smith-Waterman, restringido a sufijos: las bases con $Q < t$ suman a favor del recorte y las buenas en contra, de modo que se elimina el sufijo cuya “mala calidad neta” es máxima.

Ejemplo 6.3. (Comparar los dos criterios)

La figura 6.10 aplica ambos criterios a la misma lectura con $t = 20$. Las medias de ventana de cuatro bases descienden lentamente (37,75, 37,25, ..., 22,25) hasta que la ventana que empieza en la posición 21, con calidades

(22, 19, 21, 17), da $19,75 < 20$: *Trimmomatic* corta allí y conserva 20 bases. El criterio de BWA acumula desde el final: $20 - 6 = 14$, $+12 = 26$, $+10 = 36$, $+8 = 44$, $+2 = 46$, $+5 = 51$, $+3 = 54$ (posición 24), $-1 = 53$, $+1 = 54$, $-2 = 52$... El máximo, 54, se alcanza por primera vez en la posición 24, así que se conservan las 23 primeras bases. La diferencia (tres bases) es pequeña aquí, pero ilustra por qué dos herramientas con el “mismo” umbral no producen los mismos archivos.

```
recorte.py

from Bio import SeqIO

def errores_esperados(q):
    return sum(10 ** (-qi / 10) for qi in q)

def ventana_deslizante(q, w=4, t=20):
    """Devuelve cuántas bases conservar (criterio de Trimmomatic)."""
    for x in range(len(q) - w + 1):
        if sum(q[x:x + w]) / w < t:
            return x
    return len(q)

with open("recortadas.fastq", "w") as out:
    for rec in SeqIO.parse("muestra_R1.fastq", "fastq"):
        q = rec.letter_annotations["phred_quality"]      # ya sin +33
        corte = ventana_deslizante(q)
        rec = rec[:corte]                                # corta seq y Q
        if len(rec) >= 50 and errores_esperados(q[:corte]) < 1:
            SeqIO.write(rec, out, "fastq")
```

En la práctica no escribiremos nuestro propio recortador: usaremos herramientas probadas, rápidas y que manejan correctamente las lecturas pareadas (si un miembro del par se descarta, el otro debe ir a un archivo aparte, o los archivos quedarán desincronizados). Un flujo típico de control de calidad es el siguiente:

Control de calidad y recorte de un par de archivos

```
# 1. Diagnóstico inicial
fastqc -t 4 -o qc_crudo/ muestra_R1.fastq.gz muestra_R2.fastq.gz

# 2. Adaptadores + calidad + longitud mínima en un solo paso
```

```

fastp -i muestra_R1.fastq.gz -I muestra_R2.fastq.gz \
-o limpia_R1.fastq.gz -O limpia_R2.fastq.gz \
--detect_adapter_for_pe --trim_poly_g \
--cut_right --cut_right_window_size 4 \
--cut_right_mean_quality 20 --length_required 50 \
-h fastp.html -j fastp.json -w 8

# 3. Diagnóstico después del recorte y resumen de todo el proyecto
fastqc -t 4 -o qc_limpio/ limpia_R1.fastq.gz limpia_R2.fastq.gz
multiqc qc_crudo/ qc_limpio/ fastp.json -o informe_qc/

```

⚠ No recorte por reflejo

El recorte agresivo tiene costos: acorta lecturas (lo que empeora el mapeo en regiones repetidas), descarta pares enteros y puede introducir sesgos si la calidad depende de la secuencia. Muchos alineadores modernos realizan alineamientos locales que ignoran (*soft-clip*) los extremos que no alinean, y los llamadores de variantes ya ponderan cada base por su calidad. Recortar adaptadores es casi siempre necesario; recortar por calidad debe hacerse con umbrales moderados (Q_{15} – Q_{20}) y comprobando su efecto con un segundo informe de FastQC, no como un ritual. Schirmer et al. (2015) mostraron, para amplicones de MiSeq, que la preparación de la biblioteca y los cebadores influyen en los patrones de error tanto como el propio instrumento: ningún recorte arregla un problema del laboratorio.

$Q = -10 \log_{10} p$. Resuma la calidad de una lectura con sus errores esperados $\sum p_i$, no con el promedio de Q ; recorte adaptadores siempre y calidad con moderación.

6.3 Cobertura y la teoría de Lander-Waterman



Gotas sobre la acera

Empieza a llover sobre una acera seca. Las primeras gotas dejan manchas aisladas; al cabo de un rato las manchas se tocan y forman islas mojadas; mucho después, la acera está completamente oscura. Pero incluso cuando ya ha caído agua más que suficiente para cubrir la acera dos o tres veces, si mira con atención encontrará todavía algún rincón seco: las gotas caen al azar y el azar no reparte con justicia. La secuenciación de un genoma por fragmentos aleatorios se parece mucho a esa lluvia. ¿Cuántas “gotas” (lecturas) necesitamos para que no queden rincones secos? ¿Cuántas islas habrá mientras tanto? La respuesta es una de las aplicaciones más bellas de la distribución de Poisson.

6.3.1 Cobertura y profundidad

Supongamos que secuenciamos N lecturas de longitud L de un genoma de tamaño G . La cantidad total de bases leídas es NL , y la definimos en relación con el tamaño del genoma.

Definición 6.5. (Cobertura media)

La *cobertura* (o profundidad media) de un experimento es

$$c = \frac{NL}{G}, \quad (6.12)$$

y se expresa como “ $c\times$ ” (por ejemplo, $30\times$). La *profundidad* D_x en una posición x es el número de lecturas que la contienen.

DONDE

N	Número de lecturas (o de pares de lecturas, si se cuenta el inserto).
L	Longitud de cada lectura, en pares de bases.
G	Tamaño del genoma (haploide), en pares de bases.
c	Cobertura media: cuántas veces, en promedio, se lee cada base.
D_x	Profundidad (aleatoria) en la posición x .

Por construcción, la profundidad media sobre todas las posiciones es c ; la pregunta interesante es cómo se *distribuye* alrededor de ese valor. La figura 6.11 muestra una

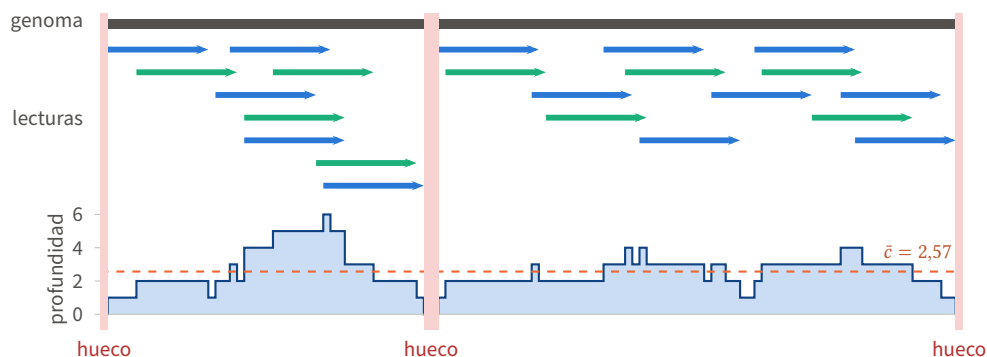


Figura 6.11. Veintidós lecturas de 14 posiciones colocadas al azar sobre un genoma de 120 posiciones ($c = 22 \cdot 14/120 \approx 2,57$). **Arriba:** las lecturas, apiladas para que no se solapen visualmente. **Abajo:** la profundidad en cada posición. Pese a que en promedio cada base se leyó más de dos veces y media, la profundidad varía entre 0 y más del doble de la media, y aparecen huecos (rojo) que ninguna lectura cubre; dos de ellos están en los extremos, donde un genoma lineal es más difícil de cubrir. La teoría predice $Ge^{-c} \approx 9$ posiciones descubiertas; en esta realización hay 4. Generado con `figuras/cap06/generar.py`.

realización: aunque la cobertura media es de 2,6×, hay posiciones leídas seis veces y pequeños huecos sin ninguna lectura.

6.3.2 La distribución de Poisson de la profundidad

Lander y Waterman (1988) formularon el modelo que usamos hasta hoy. Aunque lo propusieron para el mapeo físico con clones, sus supuestos se trasladan sin cambios a las lecturas: (i) las lecturas comienzan en posiciones independientes y uniformemente distribuidas a lo largo del genoma; (ii) todas tienen la misma longitud L ; (iii) el genoma es mucho mayor que las lecturas, $G \gg L$. Waterman desarrolla el modelo con detalle en su libro de texto (Waterman, 1995).

Teorema 6.4. (Profundidad de Poisson)

Bajo los supuestos (i)–(iii), la profundidad en una posición fija sigue, aproximadamente, una distribución de Poisson de media c :

$$\mathbb{P}(D_x = k) = \frac{e^{-c} c^k}{k!}, \quad k = 0, 1, 2, \dots \quad (6.13)$$

En particular, la fracción esperada del genoma no cubierta es $\mathbb{P}(D_x = 0) = e^{-c}$, y la fracción cubierta, $1 - e^{-c}$.

Derivación paso a paso. Fijemos una posición x . Una lectura cubre x si y solo si empieza en alguna de las L posiciones $x - L + 1, \dots, x$. Como hay (aproximadamente) G posiciones de inicio posibles y todas son equiprobables, la probabilidad de que una lectura dada cubra x es

$$\pi = \frac{L}{G}.$$

Las N lecturas son independientes, de modo que el número de ellas que cubren x es binomial:

$$\mathbb{P}(D_x = k) = \binom{N}{k} \pi^k (1 - \pi)^{N-k}, \quad \mathbb{E}[D_x] = N\pi = \frac{NL}{G} = c.$$

En un experimento real N es enorme (millones) y π minúscula ($L/G \sim 10^{-7}$ en un genoma humano), mientras que su producto c es moderado. Es el régimen clásico en que la binomial converge a la Poisson. Explícitamente, con $\pi = c/N$,

$$\begin{aligned} \binom{N}{k} \left(\frac{c}{N}\right)^k \left(1 - \frac{c}{N}\right)^{N-k} &= \underbrace{\frac{N(N-1)\dots(N-k+1)}{N^k}}_{\rightarrow 1} \frac{c^k}{k!} \underbrace{\left(1 - \frac{c}{N}\right)^N}_{\rightarrow e^{-c}} \underbrace{\left(1 - \frac{c}{N}\right)^{-k}}_{\rightarrow 1} \\ &\xrightarrow{N \rightarrow \infty} \frac{e^{-c} c^k}{k!}. \end{aligned}$$

Con $k = 0$ obtenemos la fracción no cubierta, e^{-c} . $\square$

DONDE

π	Probabilidad de que una lectura concreta cubra la posición x .
k	Valor de la profundidad cuya probabilidad se calcula.
e^{-c}	Fracción esperada del genoma sin ninguna lectura.

La fórmula es implacable. Con $c = 1$, el 36,8 % del genoma queda sin leer, aunque “hayamos leído el genoma una vez”. Con $c = 3$ falta el 5 %; con $c = 5$, el 0,67 %; con $c = 10$, el 0,0045 %. Como la fracción descubierta decrece exponencialmente, cada unidad adicional de cobertura divide lo que falta por $e \approx 2,72$, pero nunca lo anula. Observe también el paralelismo formal con la ecuación (6.8): en ambos casos se pregunta por los objetos (posiciones o moléculas) que un muestreo aleatorio no ha tocado, y en ambos la respuesta es $e^{-(\text{número medio de toques})}$.

6.3.3 Islas, contigs y huecos

Para el ensamblaje, más que las bases no cubiertas importa cuántos fragmentos contiguos (*islas* o *contigs*) podremos reconstruir. Dos lecturas solo pueden unirse si se solapan lo suficiente para detectarlo; llamemos T al solapamiento mínimo detectable y $\sigma = 1 - T/L$ a la fracción de la lectura que *no* necesita solaparse.

Teorema 6.5. (*Lander-Waterman: número y tamaño de las islas*)

Bajo los supuestos anteriores y con solapamiento mínimo detectable $T = (1 - \sigma)L$:

$$E[\text{número de islas}] = Ne^{-c\sigma}, \quad (6.14)$$

$$E[\text{lecturas por isla}] = e^{c\sigma}, \quad (6.15)$$

$$E[\text{longitud de una isla}] = L \left[\frac{e^{c\sigma} - 1}{c} + 1 - \sigma \right]. \quad (6.16)$$

DONDE

T	Solapamiento mínimo (en pb) necesario para reconocer que dos lecturas son vecinas.
σ	Fracción $1 - T/L$; con $T = 0$ vale 1.
$Ne^{-c\sigma}$	Número esperado de islas; en un genoma lineal, los huecos son una menos que las islas.

Derivación. Cada isla tiene exactamente una lectura “final”, la situada más a la derecha. Una lectura que empieza en la posición s es final si ninguna otra lectura empieza en el intervalo $(s, s + L - T]$, de longitud $L\sigma$: si alguna empezara allí, se solaparía con ella en al menos T bases y la isla continuaría. Para cada una de las

otras $N - 1$ lecturas, la probabilidad de no empezar en ese intervalo es $1 - L\sigma/G$, luego

$$P(\text{una lectura es final}) = \left(1 - \frac{L\sigma}{G}\right)^{N-1} \approx e^{-NL\sigma/G} = e^{-c\sigma}.$$

Contar islas es contar lecturas finales, y por linealidad de la esperanza se obtiene la ecuación (6.14). Si cada lectura, recorriendo de izquierda a derecha, “cierra” la isla con probabilidad $e^{-c\sigma}$, el número de lecturas por isla es geométrico con esa probabilidad de éxito, de media $e^{c\sigma}$ (ecuación (6.15)). La longitud de la isla (ecuación (6.16)) se obtiene sumando la longitud de la última lectura a las distancias entre inicios consecutivos dentro de la isla, que son exponenciales truncadas en $L\sigma$; el cálculo detallado está en Lander y Waterman (1988). $\square$

La ecuación (6.14) esconde un comportamiento no monótono. Midamos el número de islas en unidades de G/L (el número de lecturas que “cabrían” en el genoma sin solaparse): como $N = cG/L$,

$$\frac{E[\text{islas}]}{G/L} = ce^{-c\sigma}, \quad \frac{d}{dc}(ce^{-c\sigma}) = e^{-c\sigma}(1 - c\sigma) = 0 \implies c^* = \frac{1}{\sigma}. \quad (6.17)$$

Al principio, cada lectura nueva suele caer en territorio virgen y *crea* una isla; más tarde, las lecturas nuevas *unen* islas existentes. El número de islas alcanza su máximo, $(G/L)/(e\sigma)$, cuando la cobertura vale $1/\sigma$, y después decae exponencialmente (figura 6.12). Un ensamblaje con cobertura baja no es simplemente “un poco peor”: está en la zona de máxima fragmentación.

Ejemplo 6.4. (Planificar el ensamblaje de una bacteria)

Queremos ensamblar un genoma bacteriano de $G = 5 \times 10^6$ pb con lecturas de $L = 150$ pb, y nuestro ensamblador necesita un solapamiento de al menos $T = 30$ pb, de modo que $\sigma = 0,8$. Aplicando las ecuaciones (6.13), (6.14) y (6.16):

c	N (lecturas)	bases sin cubrir, Ge^{-c}	islas, $Ne^{-c\sigma}$	longitud media de isla
1	33 333	1 839 397	14 978	214 pb
5	166 667	33 690	3053	1638 pb
10	333 333	227	112	44,7 kb

Con $5\times$ tendríamos más de tres mil fragmentos; con $10\times$, alrededor de un centenar. Con $30\times$ la teoría predice $Ne^{-24} \approx 4 \times 10^{-5}$ islas adicionales, es decir, esencialmente un único *contig*. ¿Por qué entonces los ensamblajes reales de lecturas cortas a $30\times$ o $50\times$ terminan en decenas o cientos de *contigs*?

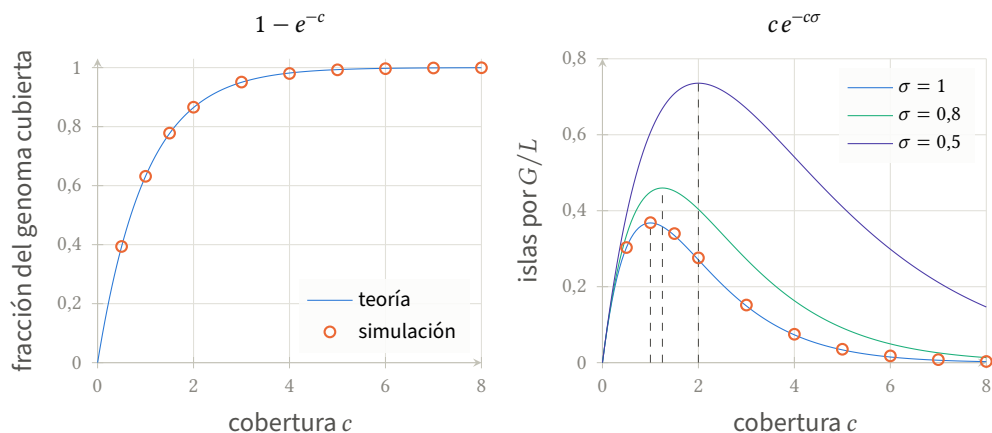


Figura 6.12. Las dos curvas de Lander-Waterman. **Izquierda:** fracción del genoma cubierta por al menos una lectura. **Derecha:** número esperado de islas (*contigs*), en unidades de G/L , para tres valores de $\sigma = 1 - T/L$; las líneas discontinuas marcan los máximos en $c^* = 1/\sigma$ (ecuación (6.17)). Los círculos naranjas son el promedio de cinco simulaciones con $G = 10^6$ y $L = 500$ (con $\sigma = 1$) y coinciden con la teoría. Exigir más solapamiento (menor σ) retrasa la fusión de islas: para la misma cobertura hay más *contigs*.

Porque el modelo ignora dos cosas: las *repeticiones* y el *sesgo de cobertura*. Las abordamos a continuación.

🧪 Lo que Lander-Waterman no ve: las repeticiones

El modelo supone que dos lecturas que se solapan *siempre* pueden unirse sin ambigüedad. Si el genoma contiene una repetición más larga que las lecturas (un elemento transponible, un operón de ARN ribosómico), las lecturas que caen dentro de ella son compatibles con varias posiciones y el ensamblador no puede decidir cómo continuar: la isla se rompe *independientemente de la cobertura*. Ninguna profundidad de lecturas cortas resuelve una repetición de 6 kb; una sola lectura de 20 kb que la atraviese de lado a lado, sí. Esta es la razón de fondo por la que las lecturas largas han transformado el ensamblaje: las lecturas ultralargas de Nanopore permitieron cerrar huecos de la referencia humana GRCh38 (Jain et al., 2018), y las HiFi producen *contigs* de decenas de megabases (Wenger et al., 2019). Lo retomaremos al estudiar el ensamblaje *de novo*.

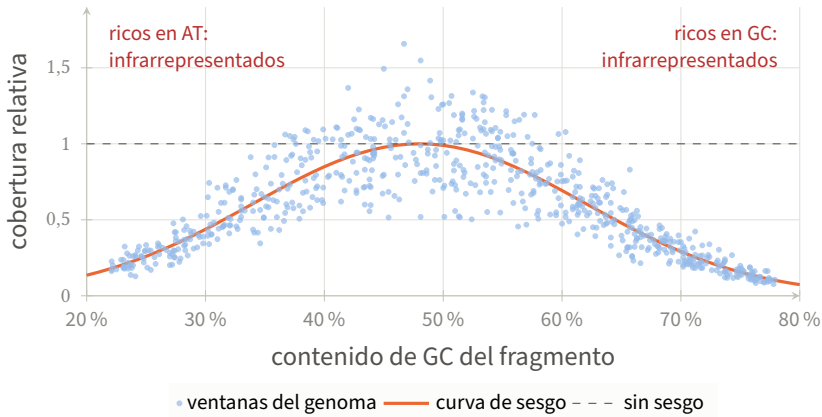


Figura 6.13. Esquema del sesgo de GC (datos simulados). Cada punto es una ventana del genoma; su cobertura relativa depende del contenido de GC según una curva unimodal, como la descrita por Benjamini y Speed (2012), más ruido de muestreo. Si el sesgo no se corrige, las regiones de GC extremo parecen tener menos copias de ADN de las que realmente tienen, un error grave al estimar variaciones en el número de copias. La forma exacta de la curva cambia entre bibliotecas, por lo que debe estimarse en cada muestra.

6.3.4 Más allá de Poisson: sesgos y sobredispersión

La distribución de Poisson tiene varianza igual a su media. Los datos reales casi nunca la cumplen: la varianza observada de la profundidad es mayor, a veces mucho mayor. Se dice entonces que la profundidad está *sobredispersa*. La causa principal es que el supuesto (i), inicio uniforme de las lecturas, es falso: la probabilidad de que un fragmento acabe secuenciado depende de su secuencia.

El sesgo mejor estudiado es el del **contenido de GC**. Benjamini y Speed (2012) analizaron datos de Illumina y encontraron que la relación entre el número de fragmentos y su contenido de GC tiene forma *unimodal*: tanto los fragmentos muy ricos en GC como los muy ricos en AT están *infrarrepresentados*. Mostraron además que lo que importa es el GC del *fragmento completo*, no solo el de la lectura, lo que apunta a la PCR de la biblioteca como causa principal, y que la forma de la curva varía de una muestra a otra (figura 6.13). Ross et al. (2013) compararon varias plataformas y concluyeron que PacBio presenta la cobertura menos sesgada, seguida de Illumina, pero que todas las tecnologías muestran sesgos en regiones extremas de GC y en homopolímeros largos; en un genoma humano secuenciado a 120× con Illumina, el 0,20 % del genoma autosómico quedó cubierto a menos del 10 % de la media, entre ellas muchas regiones promotoras ricas en GC.

Un modelo para la sobredispersión. Si la tasa de muestreo varía de una posición a otra (por GC, mapeabilidad u otros factores), podemos modelarla como una variable aleatoria. La elección clásica es una tasa con distribución gamma.

Teorema 6.6. (Mezcla Poisson-gamma)

Si, en cada posición, $D \mid \lambda \sim \text{Poisson}(\lambda)$ y la tasa local sigue $\lambda \sim \text{Gamma}(r, c/r)$ (forma r , escala c/r), la profundidad marginal es binomial negativa:

$$\mathbb{P}(D = k) = \frac{\Gamma(k+r)}{k! \Gamma(r)} \left(\frac{r}{r+c} \right)^r \left(\frac{c}{r+c} \right)^k, \quad \mathbb{E}[D] = c, \quad \text{Var}(D) = c + \frac{c^2}{r}. \quad (6.18)$$

DONDE

λ	Tasa de muestreo local (profundidad esperada en esa posición).
r	Parámetro de forma; mide la homogeneidad: $r \rightarrow \infty$ recupera la Poisson.
$\Gamma(\cdot)$	Función gamma, extensión continua del factorial: $\Gamma(n) = (n-1)!$.

La derivación es un cálculo directo: se integra la Poisson contra la densidad gamma,

$$\mathbb{P}(D = k) = \int_0^\infty \frac{e^{-\lambda} \lambda^k}{k!} \frac{\lambda^{r-1} e^{-\lambda r/c}}{\Gamma(r) (c/r)^r} d\lambda = \frac{1}{k! \Gamma(r) (c/r)^r} \int_0^\infty \lambda^{k+r-1} e^{-\lambda(1+r/c)} d\lambda,$$

y la última integral vale $\Gamma(k+r)/(1+r/c)^{k+r}$; reordenando se llega a la ecuación (6.18). La varianza se obtiene por la ley de la varianza total: $\text{Var}(D) = \mathbb{E}[\text{Var}(D \mid \lambda)] + \text{Var}(\mathbb{E}[D \mid \lambda]) = \mathbb{E}[\lambda] + \text{Var}(\lambda) = c + c^2/r$. El término c^2/r es la sobredispersión: crece con el cuadrado de la cobertura, así que *secuenciar más no la elimina*.

6.3.5 ¿Cuánta profundidad necesito?

Con estos modelos podemos responder la pregunta de diseño más frecuente. Suponga que, para llamar con confianza una variante en una posición, necesitamos al menos k lecturas, y que queremos que eso ocurra en una fracción $1 - \alpha$ del genoma. Buscamos el menor c tal que

$$\mathbb{P}(D \geq k \mid c) = 1 - F(k-1 \mid c) \geq 1 - \alpha, \quad (6.19)$$

donde F es la función de distribución acumulada del modelo elegido (Poisson o binomial negativa). No hay fórmula cerrada, pero la búsqueda numérica es trivial.

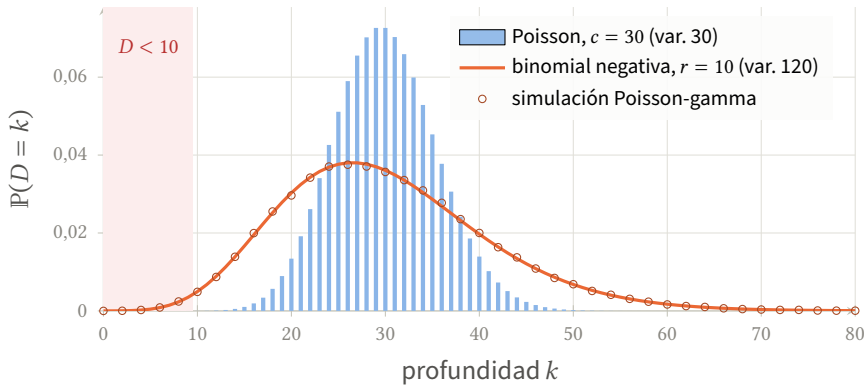


Figura 6.14. Distribución de la profundidad para una cobertura media de $30\times$. Las barras azules son el modelo ideal de Poisson; la curva naranja, una binomial negativa con la misma media y cuatro veces su varianza ($r = 10$), y los círculos, una simulación de 200 000 posiciones con tasas gamma (ecuación (6.18)). Bajo Poisson, la fracción de posiciones con menos de 10 lecturas (zona roja) es 7×10^{-6} ; con sobredispersión es del 0,9 %, más de mil veces mayor. En un genoma humano eso son decenas de millones de bases donde no se podrán llamar variantes con confianza.

DONDE

k	Profundidad mínima necesaria en una posición (p. ej., 10 lecturas).
α	Fracción tolerada del genoma por debajo de ese mínimo (p. ej., 0,01).
$F(\cdot c)$	Función de distribución acumulada de la profundidad con cobertura media c .

Ejemplo 6.5. (Diseño de la profundidad de un genoma humano)

Queremos que el 99 % del genoma tenga al menos 10 lecturas. Resolviendo la ecuación (6.19) numéricamente:

- con el modelo de Poisson basta $c \geq 18,8\times$;
- con una binomial negativa de $r = 10$ hace falta $c \geq 29,4\times$;
- con un sesgo mayor ($r = 5$), $c \geq 44,1\times$.

Si exigimos 20 lecturas, los valores suben a $31,9\times$, $53,9\times$ y $83,3\times$. El modelo ideal subestima la profundidad necesaria en un factor de 1,5 a 2. No es casual que la recomendación habitual para la resecuenciación de genomas humanos

con lecturas cortas se sitúe en torno a 30× (Bentley et al., 2008; Sims et al., 2014): es aproximadamente lo que pide un modelo realista para tener unas diez lecturas en casi todo el genoma. Sims et al. (2014) revisan cómo cambian estas consideraciones para otros diseños, como exomas, RNA-seq o ChIP-seq, donde la “cobertura” relevante no es uniforme por naturaleza.

profundidad.py

```
import numpy as np
from scipy import stats
from scipy.optimize import brentq

def lander_waterman(G, L, c, T=0):
    s = 1 - T / L
    N = c * G / L
    return {"lecturas": N,
            "sin_cubrir_pb": G * np.exp(-c),
            "islas": N * np.exp(-c * s),
            "long_isla": L * ((np.exp(c * s) - 1) / c + 1 - s)}

def cobertura_necesaria(k, alfa=0.01, r=None):
    """Menor c con  $P(D \geq k) \geq 1 - \text{alfa}$  (Poisson o NB)."""
    def f(c):
        if r is None:
            return stats.poisson.sf(k - 1, c) - (1 - alfa)
        return stats.nbinom.sf(k - 1, r, r / (r + c)) - (1 - alfa)
    return brentq(f, k / 10, 50 * k)

print(lander_waterman(5e6, 150, c=5, T=30))    # ~3053 islas
print(round(cobertura_necesaria(10), 2))        # 18.78 (Poisson)
print(round(cobertura_necesaria(10, r=10), 2))  # 29.38 (NB)
```

⚠ La cobertura “bruta” engaña

La cobertura que importa es la *efectiva*: la que queda después de descartar duplicados, lecturas de baja calidad, lecturas que no mapean o que mapean en varios sitios, y bases recortadas. Un experimento anunciado como 30× puede quedarse en 22× útiles tras estas pérdidas. Planifique con la cobertura efectiva y verifique siempre la distribución observada de la profundidad, no solo su media.

Con cobertura c , falta una fracción e^{-c} del genoma y hay unas $Ne^{-c\sigma}$ islas. Los datos reales son sobredispersos: diseñe con un modelo binomial negativo y con la cobertura efectiva, no con la bruta.

En resumen

- El método de Sanger lee fragmentos terminados por ddNTP; su longitud está limitada por la estadística geométrica de la terminación y por la resolución de la electroforesis. La pirosecuenciación (454) paralelizó la lectura, pero comete *indels* en homopolímeros.
- Illumina amplifica cada molécula en un clúster (amplificación en puente) y la lee con terminadores reversibles, una base por ciclo. El desfase acumulado, $\phi(n) = (1 - \varepsilon)^n$, explica la caída de calidad hacia 3'; su error típico es la sustitución.
- PacBio y Oxford Nanopore leen moléculas únicas de decenas a cientos de kilobases. Los errores aleatorios de PacBio se corrigen por consenso circular (HiFi); los de Nanopore tienen un componente sistemático en homopolímeros que las químicas recientes han reducido.
- La calidad Phred es $Q = -10 \log_{10} p$, codificada en FASTQ como $\text{ord}(\kappa) - 33$. El número esperado de errores de una lectura es $\sum_i p_i$.
- FastQC y MultiQC diagnostican calidad, contenido, duplicación y adaptadores. La tasa de duplicados depende de la complejidad de la biblioteca: $E[D] \approx M(1 - e^{-N/M})$. Los adaptadores se recortan siempre; la calidad, con ventanas deslizantes o sumas acumuladas y umbrales moderados.
- Con cobertura $c = NL/G$, la profundidad es aproximadamente Poisson(c): falta una fracción e^{-c} del genoma y se esperan $Ne^{-c\sigma}$ islas, con un máximo en $c = 1/\sigma$. El sesgo de GC produce sobredispersión (binomial negativa), que eleva la profundidad necesaria hacia 30× para genomas humanos.

Ejercicios

Dificultad: ●●● básica ●●● intermedia ●●● avanzada

- 6.1 ●●● Decodifique a mano la línea de calidades `II?5+#` en Phred+33: obtenga Q , p y el número esperado de errores de esas seis bases. ¿Qué calidades obtendría si, por error, leyera el archivo como Phred+64?
- 6.2 ●●● Un genoma de levadura mide 12 Mb. ¿Cuántas lecturas de 150 pb necesita para una cobertura de 40×? Si cada lectura pareada aporta dos lecturas, ¿cuántos pares?
- 6.3 ●●● Explique, a partir del mecanismo físico, por qué 454 y Nanopore cometen *indels* en homopolímeros mientras que Illumina comete sobre todo sustituciones.
- 6.4 ●●● Con la ecuación (6.2), estime la tasa de desfase ε que haría que en el ciclo 250 solo la mitad de las hebras siguieran en fase. ¿Cómo cambiaría la longitud máxima útil si ε se redujera a la mitad?
- 6.5 ●●● Usando la ecuación (6.3), calcule cuántas pasadas se necesitan para alcanzar Q_{30} si el error por pasada es del 15 %. Discuta por qué el modelo es pesimista.
- 6.6 ●●● Una biblioteca de ChIP-seq muestra un 60 % de duplicados con 3×10^7 lecturas. Estime su complejidad M con la ecuación (6.8) y prediga la tasa de duplicados si se secuencian 6×10^7 lecturas. ¿Vale la pena?
- 6.7 ●●● Implemente el criterio de la ecuación (6.11) y compárelo con la ventana deslizante sobre un archivo FASTQ real: ¿qué fracción de bases elimina cada uno con $t = 20$? Visualice el resultado con FastQC antes y después.
- 6.8 ●●● Simule el experimento de Lander-Waterman con $G = 10^6$, $L = 100$ y $T = 20$, para $c \in \{0, 5, 1, \dots, 8\}$, y compare el número de islas y su longitud media con las ecuaciones (6.14) y (6.16). ¿Dónde se desvía la simulación de la teoría y por qué?
- 6.9 ●●● Demuestre que, bajo la mezcla Poisson-gamma, la fracción del genoma no cubierta es $(1 + c/r)^{-r}$ y compárela con e^{-c} para $c = 10$ y $r = 5$. Interprete: ¿qué nos dice sobre la eficacia de “secuenciar más” para cerrar huecos cuando hay sesgo?
- 6.10 ●●● Descargue un archivo BAM público de un genoma bacteriano, calcule la profundidad por posición con `samtools depth` y ajuste una Poisson y una

binomial negativa. Estime r y relacione las posiciones de baja cobertura con su contenido de GC en ventanas de 500 pb.

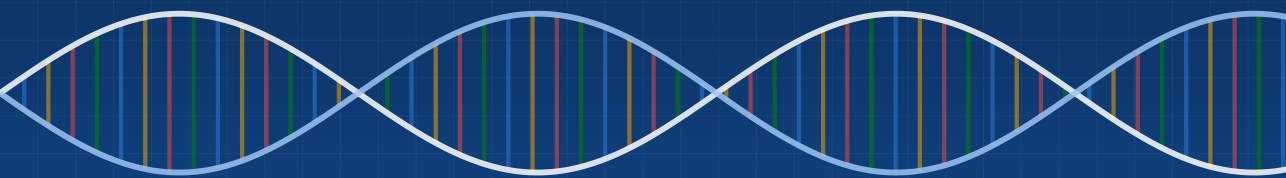
Referencias del capítulo

- Andrews, S. (2010). *FastQC: A quality control tool for high throughput sequence data*. <https://www.bioinformatics.babraham.ac.uk/projects/fastqc/>
- Benjamini, Y., & Speed, T. P. (2012). Summarizing and correcting the GC content bias in high-throughput sequencing. *Nucleic Acids Research*, 40(10), e72. <https://doi.org/10.1093/nar/gks001>
- Bentley, D. R., Balasubramanian, S., Swerdlow, H. P., Smith, G. P., Milton, J., et al. (2008). Accurate whole human genome sequencing using reversible terminator chemistry. *Nature*, 456(7218), 53-59. <https://doi.org/10.1038/nature07517>
- Bolger, A. M., Lohse, M., & Usadel, B. (2014). Trimmomatic: a flexible trimmer for Illumina sequence data. *Bioinformatics*, 30(15), 2114-2120. <https://doi.org/10.1093/bioinformatics/btu170>
- Chen, S., Zhou, Y., Chen, Y., & Gu, J. (2018). fastp: an ultra-fast all-in-one FASTQ preprocessor. *Bioinformatics*, 34(17), i884-i890. <https://doi.org/10.1093/bioinformatics/bty560>
- Cock, P. J. A., Fields, C. J., Goto, N., Heuer, M. L., & Rice, P. M. (2010). The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 38(6), 1767-1771. <https://doi.org/10.1093/nar/gkp1137>
- Eid, J., Fehr, A., Gray, J., Luong, K., Lyle, J., et al. (2009). Real-time DNA sequencing from single polymerase molecules. *Science*, 323(5910), 133-138. <https://doi.org/10.1126/science.1162986>
- Ewels, P., Magnusson, M., Lundin, S., & Käller, M. (2016). MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics*, 32(19), 3047-3048. <https://doi.org/10.1093/bioinformatics/btw354>
- Ewing, B., & Green, P. (1998). Base-calling of automated sequencer traces using Phred. II. Error probabilities. *Genome Research*, 8(3), 186-194. <https://doi.org/10.1101/gr.8.3.186>
- Ewing, B., Hillier, L., Wendl, M. C., & Green, P. (1998). Base-calling of automated sequencer traces using Phred. I. Accuracy assessment. *Genome Research*, 8(3), 175-185. <https://doi.org/10.1101/gr.8.3.175>
- Fedurco, M., Romieu, A., Williams, S., Lawrence, I., & Turcatti, G. (2006). BTA, a novel reagent for DNA attachment on glass and efficient generation of solid-phase amplified DNA colonies. *Nucleic Acids Research*, 34(3), e22. <https://doi.org/10.1093/nar/gnj023>
- Goodwin, S., McPherson, J. D., & McCombie, W. R. (2016). Coming of age: ten years of next-generation sequencing technologies. *Nature Reviews Genetics*, 17(6), 333-351. <https://doi.org/10.1038/nrg.2016.49>

- Jain, M., Koren, S., Miga, K. H., Quick, J., Rand, A. C., et al. (2018). Nanopore sequencing and assembly of a human genome with ultra-long reads. *Nature Biotechnology*, 36(4), 338-345. <https://doi.org/10.1038/nbt.4060>
- Jain, M., Olsen, H. E., Paten, B., & Akeson, M. (2016). The Oxford Nanopore MinION: delivery of nanopore sequencing to the genomics community. *Genome Biology*, 17(1), Artículo 239. <https://doi.org/10.1186/s13059-016-1103-0>
- Lander, E. S., & Waterman, M. S. (1988). Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics*, 2(3), 231-239. [https://doi.org/10.1016/0888-7543\(88\)90007-9](https://doi.org/10.1016/0888-7543(88)90007-9)
- Margulies, M., Egholm, M., Altman, W. E., Attiya, S., Bader, J. S., et al. (2005). Genome sequencing in microfabricated high-density picolitre reactors. *Nature*, 437(7057), 376-380. <https://doi.org/10.1038/nature03959>
- Martin, M. (2011). Cutadapt removes adapter sequences from high-throughput sequencing reads. *EMBnet.journal*, 17(1), 10-12. <https://doi.org/10.14806/ej.17.1.200>
- Metzker, M. L. (2010). Sequencing technologies — the next generation. *Nature Reviews Genetics*, 11(1), 31-46. <https://doi.org/10.1038/nrg2626>
- National Human Genome Research Institute. (2022). *DNA sequencing costs: Data*. Consultado el 29 de septiembre de 2026, desde <https://www.genome.gov/about-genomics/fact-sheets/DNA-Sequencing-Costs-Data>
- Rang, F. J., Kloosterman, W. P., & de Ridder, J. (2018). From squiggle to basepair: computational approaches for improving nanopore sequencing read accuracy. *Genome Biology*, 19(1), Artículo 90. <https://doi.org/10.1186/s13059-018-1462-9>
- Ronaghi, M., Uhlén, M., & Nyrén, P. (1998). A sequencing method based on real-time pyrophosphate. *Science*, 281(5375), 363-365. <https://doi.org/10.1126/science.281.5375.363>
- Ross, M. G., Russ, C., Costello, M., Hollinger, A., Lennon, N. J., Hegarty, R., Nusbaum, C., & Jaffe, D. B. (2013). Characterizing and measuring bias in sequence data. *Genome Biology*, 14(5), Artículo R51. <https://doi.org/10.1186/gb-2013-14-5-r51>
- Sanger, F., Nicklen, S., & Coulson, A. R. (1977). DNA sequencing with chain-terminating inhibitors. *Proceedings of the National Academy of Sciences*, 74(12), 5463-5467. <https://doi.org/10.1073/pnas.74.12.5463>
- Schirmer, M., Ijaz, U. Z., D'Amore, R., Hall, N., Sloan, W. T., & Quince, C. (2015). Insight into biases and sequencing errors for amplicon sequencing with the Illumina MiSeq platform. *Nucleic Acids Research*, 43(6), e37. <https://doi.org/10.1093/nar/gku1341>
- Sereika, M., Kirkegaard, R. H., Karst, S. M., Michaelsen, T. Y., Sørensen, E. A., Wollenberg, R. D., & Albertsen, M. (2022). Oxford Nanopore R10.4 long-read sequencing enables the generation of near-finished bacterial genomes from pure cultures and metagenomes without short-read or reference polishing. *Nature Methods*, 19(7), 823-826. <https://doi.org/10.1038/s41592-022-01539-7>

- Shendure, J., Balasubramanian, S., Church, G. M., Gilbert, W., Rogers, J., Schloss, J. A., & Waterston, R. H. (2017). DNA sequencing at 40: past, present and future. *Nature*, 550(7676), 345-353. <https://doi.org/10.1038/nature24286>
- Sims, D., Sudbery, I., Illott, N. E., Heger, A., & Ponting, C. P. (2014). Sequencing depth and coverage: key considerations in genomic analyses. *Nature Reviews Genetics*, 15(2), 121-132. <https://doi.org/10.1038/nrg3642>
- Waterman, M. S. (1995). *Introduction to computational biology: Maps, sequences and genomes*. Chapman & Hall.
- Wenger, A. M., Peluso, P., Rowell, W. J., Chang, P.-C., Hall, R. J., et al. (2019). Accurate circular consensus long-read sequencing improves variant detection and assembly of a human genome. *Nature Biotechnology*, 37(10), 1155-1162. <https://doi.org/10.1038/s41587-019-0217-9>

Mapeo de lecturas



Ignorar sin más las repeticiones no es una opción: eso crea sus propios problemas y puede hacer que pasen inadvertidos fenómenos biológicos importantes.

— Treangen y Salzberg (2012)

● Al terminar este capítulo usted podrá

- Explicar por qué el alineamiento por programación dinámica no escala al mapeo de cientos de millones de lecturas, y comparar las dos familias de índices que lo resuelven: tablas de *hash* de semillas e índices de sufijos.
- Construir a mano el arreglo de sufijos, la transformada de Burrows-Wheeler y el índice FM de una secuencia corta, y ejecutar paso a paso la búsqueda hacia atrás con el *LF-mapping*.
- Describir el paradigma semilla-encadenamiento-extensión de BWA-MEM y minimap2, incluidos los minimizadores, la función de encadenamiento y la calidad de mapeo (MAPQ), y leer sin ayuda un registro SAM.
- Construir un flujo reproducible con `samtools` (ordenar, marcar duplicados, indexar), inspeccionar alineamientos en IGV y usar perfiles de cobertura para detectar deleciones y duplicaciones.

Al terminar el capítulo 6 teníamos en las manos un archivo FASTQ limpio: millones de fragmentos de 150 bases, cada uno con su calidad Phred, y una teoría (la de Lander-Waterman) que nos decía cuántos necesitábamos para cubrir un genoma. Pero una lectura suelta no significa nada; solo adquiere sentido cuando sabemos *de dónde* del genoma procede. Esa es la tarea del *mapeo*: colocar cada lectura en su lugar de origen dentro de una secuencia de referencia. Casi todo el análisis genómico moderno (el llamado de variantes del próximo capítulo, la cuantificación de expresión con RNA-seq, los picos de ChIP-seq, la metilación) empieza con este paso, y hereda sus errores.

El problema parece una aplicación directa del capítulo 3: basta con alinear cada lectura contra el genoma con Smith-Waterman. La dificultad es de escala. Un genoma humano a 30× son unos 620 millones de lecturas de 150 pares de bases, y la referencia mide $3,1 \times 10^9$ bases. Li y Homer (2010) resumieron la respuesta de la comunidad en una frase que hoy parece obvia pero que en 2008 no lo era: con los algoritmos adecuados, el alineamiento de lecturas cortas “ha dejado de ser el cuello de botella” del análisis. Este capítulo cuenta cómo se logró.

Empezaremos por la idea algorítmica más bella del área: una permutación de la secuencia, la transformada de Burrows-Wheeler, que permite buscar cualquier patrón en un genoma humano ocupando menos memoria que el propio genoma (sección 7.1). Después veremos cómo las herramientas reales (BWA, Bowtie, minimap2) combinan índices, semillas y programación dinámica, cómo expresan su confianza con la calidad de mapeo y cómo escriben el resultado en el formato SAM/BAM que manipularemos con [samtools](#) (sección 7.2). Terminaremos mirando los alineamientos con los ojos, en IGV, y convirtiéndolos en perfiles de cobertura capaces de revelar deleciones y duplicaciones (sección 7.3).

7.1 Transformada de Burrows-Wheeler y FM-index

 [Colab](#) · 7.1 · Transformada de Burrows-Wheeler y FM-index  [repositorio del curso](#)

Un diccionario de todos los finales

Imagine un diccionario muy extraño: en lugar de palabras, contiene *todos los finales* de un libro. Cada entrada empieza en una letra distinta del texto

y continúa hasta el punto final, y todas están ordenadas alfabéticamente. Si quiere saber dónde aparece la frase “había una vez”, no necesita leer el libro: abre el diccionario por la mitad, decide si su frase va antes o después, vuelve a partir por la mitad, y en una veintena de saltos llega al bloque de entradas que empiezan con esa frase. Cada entrada, además, recuerda en qué página empezaba. El problema es que el diccionario pesa muchísimo más que el libro. La transformada de Burrows-Wheeler es el truco que permite quedarse con una sola columna de ese diccionario, tan delgada como el propio texto, sin perder la capacidad de buscar.

7.1.1 El problema del mapeo

Formalicemos lo que pedimos a un mapeador. Sea $R = r_1 r_2 \dots r_G$ la secuencia de referencia (un genoma, con sus cromosomas concatenados) y $q = q_1 \dots q_m$ una lectura, con $m \ll G$.

Definición 7.1. (Mapeo de una lectura)

Dada una función de puntuación de alineamiento local $S(q, R_{u..v})$ (por ejemplo, la de Smith-Waterman con huecos afines del capítulo 3), mapear q consiste en encontrar el intervalo de la referencia

$$(\hat{u}, \hat{v}) = \arg \max_{1 \leq u \leq v \leq G} \max \{S(q, R_{u..v}), S(\bar{q}, R_{u..v})\}, \quad (7.1)$$

y en cuantificar la probabilidad de que $(\hat{u}, \hat{v})$ sea el origen verdadero de la lectura.

DONDE

R, G	Secuencia de referencia y su longitud en bases.
q, m	Lectura y su longitud.
$\bar{q}$	Reverso complementario de la lectura: el fragmento pudo secuenciarse desde cualquiera de las dos hebras.
$R_{u..v}$	Subcadena de la referencia entre las posiciones u y v .
$S(\cdot, \cdot)$	Puntuación del mejor alineamiento local entre la lectura y la subcadena.

La segunda mitad de la definición es tan importante como la primera, y a menudo se olvida: un mapeador no solo responde *dónde*, sino *con qué seguridad*. Volveremos a ello con la calidad de mapeo en la sección 7.2.

Resolver la ecuación (7.1) por fuerza bruta es imposible. Con Smith-Waterman, cada lectura exige llenar una matriz de $m \times G = 150 \times 3,1 \times 10^9 \approx 4,7 \times 10^{11}$ celdas; multiplicado por $6,2 \times 10^8$ lecturas, son unos $2,9 \times 10^{20}$ celdas. Aun a mil millones de celdas por segundo, el cálculo tardaría unos nueve mil años. Necesitamos *filtrar*: descartar en un instante casi todo el genoma y reservar la programación dinámica para unas pocas regiones candidatas.

7.1.2 Semillas y tablas de hash

La primera generación de mapeadores tomó prestada la idea de BLAST: si la lectura procede de la posición u , casi seguro comparte con la referencia algún k -mero exacto (una *semilla*). Basta con un índice que, para cada k -mero, liste las posiciones donde aparece. ¿Qué k elegir? Si el genoma fuera una secuencia aleatoria de bases equiprobables, el número esperado de apariciones *casuales* de un k -mero concreto sería

$$E[N_k] = \frac{G - k + 1}{4^k} \approx \frac{G}{4^k}.$$
 (7.2)

DONDE	
N_k	Número de posiciones de la referencia donde aparece, por azar, un k -mero dado.
$G - k + 1$	Número de k -meros (posiciones de inicio) en la referencia.
4^k	Número de k -meros distintos posibles sobre {A, C, G, T}.

Para el genoma humano, un 12-mero aparece por azar unas 185 veces, un 16-mero 0,72 veces y un 20-mero apenas 0,0028 veces (figura 7.1). Las semillas deben tener, pues, unas 20 bases para ser *informativas*. Pero una tabla directa indexada por k -mero necesita 4^k entradas: con enteros de 32 bits, 64 MiB para $k = 12$ y 16 GiB para $k = 16$, antes de guardar una sola posición. Por eso los mapeadores basados en *hash*, como MAQ (Li et al., 2008), indexaban semillas cortas, a menudo espaciadas, y a veces indexaban las *lecturas* en lugar del genoma. Funcionaban, pero eran lentos cuando había que escalar a cientos de individuos (Li & Durbin, 2009).

La alternativa, que acabó por imponerse para lecturas cortas, fue indexar *todos los sufijos* de la referencia. Un índice de sufijos no depende de un k fijo: permite preguntar por un patrón de cualquier longitud y, como veremos, extenderlo carácter a carácter hasta que deje de aparecer.

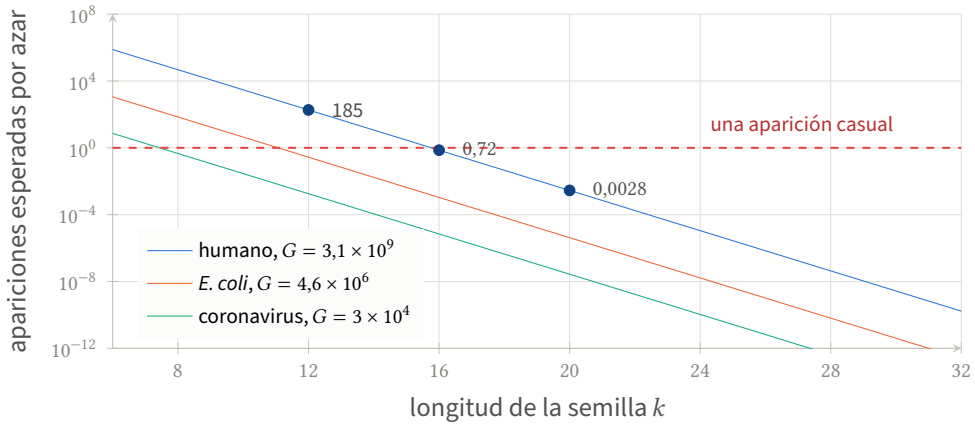


Figura 7.1. Número esperado de apariciones casuales de un k -mero en genomas aleatorios de distinto tamaño (ecuación (7.2)). En escala logarítmica cada base adicional divide la cifra por cuatro. En el genoma humano, una semilla necesita unas 20 bases para que una coincidencia exacta sea más probablemente verdadera que casual; en un virus basta con 8 o 9. Los genomas reales no son aleatorios: sus repeticiones hacen que muchos k -meros largos aparezcan miles de veces, como veremos en la sección 7.2.

7.1.3 El arreglo de sufijos

Añadimos al final de la referencia un símbolo centinela $\$$ que no pertenece al alfabeto y que, por convenio, es menor que cualquier base. Así, $T = R\$$ tiene longitud $n = G + 1$ y ningún sufijo es prefijo de otro.

Definición 7.2. (Arreglo de sufijos)

El arreglo de sufijos de T es la permutación $SA[0..n-1]$ de $\{0, \dots, n-1\}$ tal que

$$T[SA[0]..] < T[SA[1]..] < \dots < T[SA[n-1]..] \quad (7.3)$$

en orden lexicográfico, donde $T[i..]$ denota el sufijo que empieza en la posición i (numeramos desde cero).

DONDE

T, n	Texto indexado (referencia más centinela) y su longitud.
$T[i..]$	Sufijo de T que empieza en la posición i .
$SA[r]$	Posición en T del sufijo que ocupa el lugar r en el orden alfabético.

La propiedad clave es que todos los sufijos que empiezan por un patrón P quedan *contiguos* en el arreglo, porque comparten prefijo. Encontrar las apariciones de P se reduce a encontrar un *intervalo* $[sp, ep)$ de filas, lo que se hace con dos búsquedas binarias en $O(m \log n)$ comparaciones de caracteres. Manber y Myers (1993) propusieron esta estructura como alternativa compacta a los árboles de sufijos, que resuelven el mismo problema en tiempo $O(m)$ pero ocupan, en la práctica, más de diez bytes por base (Gusfield, 1997). Aun así, el arreglo de sufijos necesita $n \lceil \log_2 n \rceil$ bits: para el genoma humano, 32 bits por posición, unos 12 GB. En 2008, eso era más memoria de la que tenía un servidor típico.

7.1.4 La transformada de Burrows-Wheeler

Escribamos todas las rotaciones cíclicas de T , una debajo de otra, y ordenémoslas alfabéticamente. Obtenemos una matriz M de $n \times n$ caracteres (figura 7.2). Como T termina en el centinela único, ordenar las rotaciones equivale a ordenar los sufijos: la fila r de M empieza con el sufijo $T[SA[r]..]$.

Definición 7.3. (Transformada de Burrows-Wheeler)

La BWT de T es la última columna L de la matriz de rotaciones ordenadas. Equivalentemente, a partir del arreglo de sufijos,

$$L[r] = \begin{cases} T[SA[r] - 1] & \text{si } SA[r] > 0, \\ \$ & \text{si } SA[r] = 0. \end{cases} \quad (7.4)$$

Llamamos F a la primera columna de M : son los caracteres de T ordenados.

DONDE

M	Matriz cuyas filas son las n rotaciones de T en orden lexicográfico.
L	Última columna de M : la transformada de Burrows-Wheeler, BWT(T).
F	Primera columna de M ; se deduce de L ordenando sus caracteres.

Rotaciones de T		Matriz ordenada M	
i		$r(SA)$	
0	T A C A T A C A G \$	0 (9)	\$ T A C A T A C A G
1	A C A T A C A G \$ T	1 (5)	A C A G \$ T A C A T
2	C A T A C A G \$ T A	2 (1)	A C A T A C A G \$ T
3	A T A C A G \$ T A C	3 (7)	A G \$ T A C A T A C
4	T A C A G \$ T A C A	4 (3)	A T A C A G \$ T A C
5	A C A G \$ T A C A T	5 (6)	C A G \$ T A C A T A
6	C A G \$ T A C A T A	6 (2)	C A T A C A G \$ T A
7	A G \$ T A C A T A C	7 (8)	G \$ T A C A T A C A
8	G \$ T A C A T A C A	8 (4)	T A C A G \$ T A C A
9	\$ T A C A T A C A G	9 (0)	T A C A T A C A G \$
			F L

ordenar →

$BWT(T) = L = GTTCCAAAA\$$

Figura 7.2. Construcción de la BWT de $T = TACATACAG\$$. **Izquierda:** las diez rotaciones cíclicas de T , indexadas por su posición de inicio i . **Derecha:** las mismas rotaciones en orden alfabético ($\$$ precede a todas las bases). Junto a cada fila r aparece, entre paréntesis, $SA[r]$. La primera columna F (azul) contiene las letras de T ordenadas; la última, L (naranja), es la transformada. Observe cómo las cuatro A de L quedan juntas: los sufijos que empiezan por C, G o T tienden a estar precedidos por la misma letra, y la BWT agrupa esas repeticiones. Generada con `figuras/cap07/generar.py`.

En palabras: $L[r]$ es el carácter que *precede*, en el texto, al sufijo de la fila r . Para nuestro ejemplo de juguete, $T = TACATACAG\$$, el arreglo de sufijos es (9, 5, 1, 7, 3, 6, 2, 8, 4, 0) y la transformada es $L = GTTCCAAAA\$$.

🔗 Un algoritmo de compresión que nadie esperaba

Michael Burrows y David Wheeler describieron la transformada en 1994, en un reporte técnico del Centro de Investigación de Sistemas de Digital Equipment Corporation, con un propósito muy distinto al nuestro: comprimir texto. Su observación fue que L tiende a formar rachas largas de un mismo carácter (en inglés, las letras que preceden a “he” suelen ser t), lo que la hace muy fácil de comprimir, y que, sorprendentemente, la permutación es *reversible*. El compresor `bzip2` se basa en ella. Adjeroh et al. (2008) reconstruyen esta historia y su conexión con los arreglos de sufijos. Pasaron seis años hasta que Paolo Ferragina y Giovanni Manzini, en un congreso de 2000, se dieron cuenta de que la BWT comprimida no solo guardaba el texto, sino que permitía *buscar* en él; la versión extendida de su trabajo es Ferragina

y Manzini (2005). Y pasaron otros nueve años hasta que Bowtie y BWA llevaron la idea a la genómica (Langmead et al., 2009; Li & Durbin, 2009).

7.1.5 El *LF-mapping*: por qué la BWT es reversible

¿Cómo recuperar T a partir de L sola? La clave es una propiedad que, una vez vista, parece evidente.

Teorema 7.1. (Orden de las apariciones)

La j -ésima aparición de un carácter c en la columna L y la j -ésima aparición de c en la columna F corresponden a la misma posición del texto T .

Demostración. Considere dos filas de M cuya última letra es c , digamos cX rotada como Xc y Yc , con $X < Y$ porque las filas están ordenadas. Si rotamos ambas un carácter a la derecha obtenemos cX y cY , que son también filas de M (con c en la columna F), y como comparten el primer carácter, su orden relativo lo decide el resto: $cX < cY$. El orden entre las c se conserva al pasar de L a F . $\square$

Esta propiedad permite definir, para cada fila r , la fila $LF(r)$ en la que aparece como primera letra el mismo carácter que $L[r]$. Necesitamos dos tablas:

$$C[c] = \#\{i : T[i] < c\}, \quad (7.5)$$

$$\text{Occ}(c, r) = \#\{i < r : L[i] = c\}, \quad (7.6)$$

y con ellas

$$LF(r) = C[L[r]] + \text{Occ}(L[r], r). \quad (7.7)$$

DONDE

$C[c]$	Número de caracteres de T estrictamente menores que c : la fila donde empieza el bloque de c en la columna F .
$\text{Occ}(c, r)$	Número de apariciones de c en las primeras r filas de L (filas 0 a $r - 1$): el rango de $L[r]$ entre sus iguales.
$LF(r)$	Fila de M cuyo primer carácter es el mismo carácter del texto que $L[r]$; equivale a retroceder una posición en T .

La ecuación (7.7) es la traducción directa del teorema: el bloque de c en F empieza en la fila $C[c]$, y dentro del bloque las c están en el mismo orden que en L , así que la c de rango j ocupa la fila $C[c] + j$. Como la fila $LF(r)$ empieza justo con el carácter

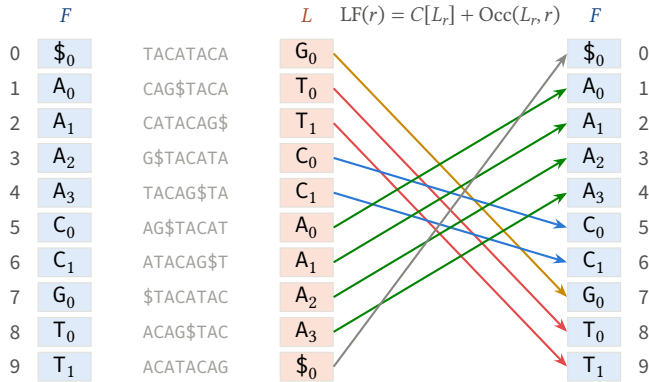


Figura 7.3. El LF-mapping para $T = \text{TACATACAG\$}$. Los subíndices numeran las apariciones de cada letra de arriba abajo. Cada flecha une la fila r de L con la fila $LF(r)$ de F (repetida a la derecha para mayor claridad): la j -ésima A de L va a la j -ésima A de F , y lo mismo para cada letra. Las flechas del mismo color nunca se cruzan; esa es la propiedad del teorema 7.1. En gris, el centro de cada fila, que el índice *no* almacena.

que precedía al sufijo de la fila r , aplicar LF equivale a *dar un paso hacia atrás* en el texto.

Ejemplo 7.1. (Reconstruir T desde su BWT)

Con $L = \text{GTTCCAAAA\$}$ y $F = \text{\$AAAACCGTT}$ tenemos $C[A] = 1$, $C[C] = 5$, $C[G] = 7$ y $C[T] = 8$. Empezamos en la fila 0, la que comienza con $\$$; su última letra, $L[0] = G$, es el último carácter de R . Aplicamos la ecuación (7.7) repetidamente:

$$0 \xrightarrow{G} 7 \xrightarrow{A} 3 \xrightarrow{C} 5 \xrightarrow{A} 1 \xrightarrow{T} 8 \xrightarrow{A} 4 \xrightarrow{C} 6 \xrightarrow{A} 2 \xrightarrow{T} 9,$$

donde sobre cada flecha escribimos $L[r]$ en la fila de partida. Por ejemplo, $LF(0) = C[G] + \text{Occ}(G, 0) = 7 + 0 = 7$ y $LF(7) = C[A] + \text{Occ}(A, 7) = 1 + 2 = 3$. Las letras recogidas, GACATACAT, leídas al revés dan TACATACAG: el texto original. En la fila 9 encontramos $L[9] = \$$ y paramos.

7.1.6 El índice FM y la búsqueda hacia atrás

El mismo mecanismo que reconstruye el texto sirve para *buscar* en él. La idea de Ferragina y Manzini (2005) es procesar el patrón $P = p_1 \cdots p_m$ de derecha a izquierda, manteniendo el intervalo de filas de M que empiezan con el sufijo de P ya procesado.

Teorema 7.2. (Búsqueda hacia atrás)

Sea $[sp, ep)$ el intervalo de filas de M que empiezan con la cadena W . Entonces las filas que empiezan con cW forman el intervalo

$$sp' = C[c] + \text{Occ}(c, sp), \quad ep' = C[c] + \text{Occ}(c, ep). \quad (7.8)$$

Empezando con W vacío, $[sp, ep) = [0, n)$, y aplicando la regla para $c = p_m, p_{m-1}, \dots, p_1$, el patrón aparece en T exactamente $ep - sp$ veces; si en algún paso $sp' \geq ep'$, no aparece.

DONDE

W	Sufijo del patrón ya procesado (al principio, la cadena vacía).
$[sp, ep)$	Intervalo semiabierto de filas de M (o del arreglo de sufijos) que empiezan con W .
c	Siguiente carácter del patrón, leído de derecha a izquierda.
$[sp', ep')$	Intervalo de las filas que empiezan con cW .

Por qué funciona. Las filas que empiezan con cW son exactamente las rotaciones $cW \dots$; al rotarlas un carácter a la izquierda se convierten en filas $W \dots c$, es decir, filas del intervalo $[sp, ep)$ cuya última letra es c . Por el teorema 7.1, esas c de L se mapean, en el mismo orden, a un bloque contiguo del bloque de c en F . Las c de L situadas antes de sp son $\text{Occ}(c, sp)$; por eso el nuevo bloque empieza en $C[c] + \text{Occ}(c, sp)$ y termina justo antes de $C[c] + \text{Occ}(c, ep)$. $\square$

Lo extraordinario es el coste: cada carácter del patrón cuesta dos consultas a Occ , sin importar el tamaño del genoma. Buscar una lectura de 150 bases en el genoma humano cuesta 300 consultas, lo mismo que buscarla en el genoma de un virus.

Ejemplo 7.2. (Buscar ACA en TACATACAG\$)

Usamos la tabla 7.1. Empezamos con $[0, 10)$ y leemos el patrón al revés:

Tabla 7.1. Tabla $\text{Occ}(c, r)$ para $L = \text{GTTCCAAAA}\$$: cada fila r cuenta las apariciones de cada base en $L[0..r-1]$. La tabla C es $C[A] = 1, C[C] = 5, C[G] = 7, C[T] = 8$.

r	$L[r]$	$\text{Occ}(A, r)$	$\text{Occ}(C, r)$	$\text{Occ}(G, r)$	$\text{Occ}(T, r)$
0	G	0	0	0	0
1	T	0	0	1	0
2	T	0	0	1	1
3	C	0	0	1	2
4	C	0	1	1	2
5	A	0	2	1	2
6	A	1	2	1	2
7	A	2	2	1	2
8	A	3	2	1	2
9	\$	4	2	1	2
10	–	4	2	1	2

1. $c = A$: $sp' = 1 + \text{Occ}(A, 0) = 1 + 0 = 1$; $ep' = 1 + \text{Occ}(A, 10) = 1 + 4 = 5$. Intervalo $[1, 5)$: las cuatro filas que empiezan con A.
2. $c = C$: $sp' = 5 + \text{Occ}(C, 1) = 5$; $ep' = 5 + \text{Occ}(C, 5) = 5 + 2 = 7$. Intervalo $[5, 7)$: filas que empiezan con CA.
3. $c = A$: $sp' = 1 + \text{Occ}(A, 5) = 1$; $ep' = 1 + \text{Occ}(A, 7) = 1 + 2 = 3$. Intervalo $[1, 3)$.

Hay $3 - 1 = 2$ apariciones, en las posiciones $SA[1] = 5$ y $SA[2] = 1$ del texto: TACATACAG. Si buscamos GAC, tras procesar AC tenemos $[1, 3)$ y el paso con G da $sp' = 7 + \text{Occ}(G, 1) = 8$ y $ep' = 7 + \text{Occ}(G, 3) = 8$: el intervalo se vacía y sabemos, sin mirar el texto, que GAC no aparece.

El siguiente programa construye el índice de forma ingenua (ordenando sufijos, lo que cuesta $O(n^2 \log n)$ y solo sirve para ejemplos) y busca por el método del teorema 7.2. Los programas reales construyen el arreglo de sufijos en tiempo lineal o casi lineal, con algoritmos especializados que no requieren tener en memoria todos los sufijos (Mäkinen et al., 2015).

	Paso 0: inicio	Paso 1: A	Paso 2: CA	Paso 3: ACA
0	\$TACATACAG	\$TACATACAG	\$TACATACAG	\$TACATACAG
1	ACAG\$TACAT	ACAG\$TACAT	ACAG\$TACAT	ACAG\$TACAT
2	ACATACAG\$T	ACATACAG\$T	ACATACAG\$T	ACATACAG\$T
3	AG\$TACATAC	AG\$TACATAC	AG\$TACATAC	AG\$TACATAC
4	ATACAG\$TAC	ATACAG\$TAC	ATACAG\$TAC	ATACAG\$TAC
5	CAG\$TACATA	CAG\$TACATA	CAG\$TACATA	CAG\$TACATA
6	CATACAG\$TA	CATACAG\$TA	CATACAG\$TA	CATACAG\$TA
7	G\$TACATACA	G\$TACATACA	G\$TACATACA	G\$TACATACA
8	TACAG\$TACA	TACAG\$TACA	TACAG\$TACA	TACAG\$TACA
9	TACATACAG\$	TACATACAG\$	TACATACAG\$	TACATACAG\$
	[0, 10) 10 filas	[1, 5) 4 filas	[5, 7) 2 filas	[1, 3) 2 filas

Figura 7.4. Búsqueda hacia atrás de $P = ACA$ en la matriz ordenada de $T = TACATACAG\$$, leída como una secuencia de fotogramas. En cada paso se antepone un carácter del patrón (de derecha a izquierda) y el intervalo amarillo se contrae a las filas que empiezan con el sufijo procesado, resaltado en rojo. El índice nunca almacena estas filas completas: solo necesita L , C y Occ para calcular los extremos del intervalo con la ecuación (7.8). El tamaño final del intervalo, dos filas, es el número de apariciones.

indice_fm.py

```
def indice_fm(T):
    """T debe terminar en '$'. Devuelve SA, L, C y Occ."""
    sa = sorted(range(len(T)), key=lambda i: T[i:])
    L = "".join(T[i - 1] for i in sa)          # T[-1] es '$'
    alf = sorted(set(T))
    C, tot = {}, 0
    for c in alf:
        C[c], tot = tot, tot + T.count(c)
    occ = {c: [0] for c in alf}
    for ch in L:                               # Occ(c, r) acumulado
        for c in alf:
            occ[c].append(occ[c][-1] + (ch == c))
    return sa, L, C, occ

def buscar(P, sa, C, occ):
    sp, ep = 0, len(sa)
    for c in reversed(P):                       # de derecha a izquierda
        if c not in C:
            return []
        sp, ep = C[c] + occ[c][sp], C[c] + occ[c][ep]
```

```

    if sp >= ep:
        return []
    return sorted(sa[sp:ep])

sa, L, C, occ = indice_fm("TACATACAG$")
print(L, buscar("ACA", sa, C, occ)) # GTTCCAAAA$ [1, 5]

```

7.1.7 Muestreo: cómo cabe el genoma humano en un portátil

El programa anterior guarda Occ completa: cuatro enteros por posición. Para el genoma humano, con enteros de 32 bits, eso son $4 \times 32 \times 3,1 \times 10^9$ bits, unos 50 GB, peor que el arreglo de sufijos. El índice FM resuelve el problema con dos muestreos (Ferragina & Manzini, 2005).

Puntos de control de Occ. Se guardan los valores de $\text{Occ}(c, r)$ solo cuando r es múltiplo de un intervalo d (los *checkpoints*). Para cualquier otra fila, se toma el punto de control anterior y se cuentan las apariciones de c en el trozo de L que falta, a lo sumo $d - 1$ caracteres. Con L codificada a 2 bits por base, ese recuento se hace con operaciones de bits sobre unas pocas palabras de máquina.

Muestreo del arreglo de sufijos. La búsqueda hacia atrás devuelve un intervalo de *filas*, pero necesitamos *posiciones* del genoma, es decir, valores de SA. Se guarda $\text{SA}[r]$ solo para las filas cuyo valor es múltiplo de s . Para una fila no muestreada, aplicamos LF (un paso atrás en el texto) hasta caer en una fila muestreada; si hicieron falta j pasos, $\text{SA}[r] = \text{SA}[\text{LF}^j(r)] + j$, con $j < s$.

La memoria total, en bits, es

$$\mathcal{M}(d, s) = \frac{2n}{\tilde{L}} + \underbrace{\frac{4bn}{d}}_{\text{Occ}} + \underbrace{\frac{bn}{s}}_{\text{SA}}, \quad (7.9)$$

mientras que el tiempo de localizar cada aparición crece como $O(s)$ pasos de LF, cada uno de coste $O(d)$.

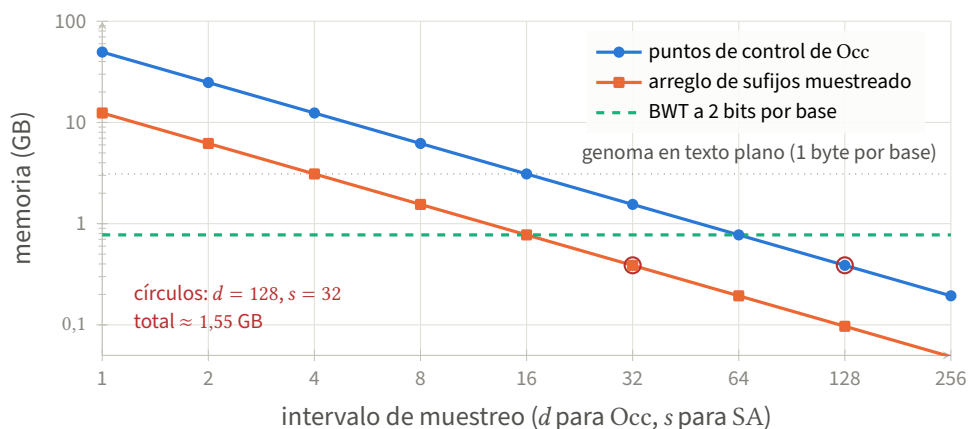


Figura 7.5. Memoria de cada componente del índice FM del genoma humano ($n = 3,1 \times 10^9$, enteros de 32 bits) en función del intervalo de muestreo (ecuación (7.9)). Sin muestreo, Occ ocuparía unos 50 GB y el arreglo de sufijos 12 GB. Con un punto de control cada 128 filas y una muestra del arreglo cada 32 posiciones (círculos), el índice completo de una hebra ocupa unos 1,55 GB, la mitad que el genoma escrito como texto. El precio es tiempo: cada posición recuperada exige, en promedio, varios pasos de *LF-mapping*.

DONDE

$\mathcal{M}(d, s)$	Memoria del índice, en bits.
n	Longitud del texto indexado.
b	Bits por entero almacenado ($b = \lceil \log_2 n \rceil = 32$ para el genoma humano).
d	Distancia entre puntos de control de Occ.
s	Intervalo de muestreo del arreglo de sufijos.

Con $d = 128$ y $s = 32$, la ecuación (7.9) da 0,775 GB para L , 0,388 GB para los puntos de control y otros 0,388 GB para el arreglo muestreado: unos 1,55 GB (figura 7.5). Estos son, de hecho, los intervalos que describen Li y Durbin (2009) para BWA, que además indexa el genoma invertido para poder acotar la búsqueda aproximada; Bowtie, con técnicas similares, alineaba más de 25 millones de lecturas por hora de CPU usando unos 1,3 GB de memoria (Langmead et al., 2009).

Buscar con errores: *backtracking* acotado

La búsqueda hacia atrás encuentra coincidencias *exactas*, pero una lectura real contiene errores de secuenciación y variantes. Bowtie y la primera versión de BWA exploran variantes del patrón: en cada paso, además de la base leída, prueban las otras tres (una sustitución) o saltan un carácter (un *indel*), y retroceden cuando el intervalo se vacía. El espacio de búsqueda crece exponencialmente con el número de diferencias permitidas, así que ambas herramientas lo podan. Bowtie prioriza sustituir primero las bases de baja calidad Phred (Langmead et al., 2009). BWA precalcula un arreglo $D(i)$, cota inferior del número de diferencias que necesitará el prefijo $P[0..i]$, y abandona una rama en cuanto las diferencias acumuladas más $D(i)$ superan el máximo permitido (Li & Durbin, 2009). Un detalle elegante: como los sufijos repetidos del genoma comparten intervalo, una repetición con mil copias se explora *una sola vez*.

Detalles que el ejemplo oculta

- *Las dos hebras*. Una lectura puede proceder de cualquier hebra: el mapeador busca q y su reverso complementario $\bar{q}$, o indexa ambas hebras de la referencia.
- *Las N*. Las bases ambiguas de la referencia (huecos del ensamblaje) se sustituyen por bases aleatorias o se excluyen al construir el índice; en la lectura se tratan como diferencias.
- *Varios cromosomas*. Se concatenan en un único texto; una tabla auxiliar traduce la posición global a (cromosoma, posición) y descarta las coincidencias que cruzan una frontera.

La BWT reordena el genoma de modo que buscar un patrón cuesta dos consultas por carácter, independientemente del tamaño del genoma, y el índice completo cabe en menos memoria que el propio texto.

7.2 BWA, minimap2 y samtools

Huellas digitales de un texto copiado

Un profesor sospecha que un ensayo fue copiado de alguno de los millones de documentos de internet. No puede compararlo palabra por palabra con todos. En cambio, toma de cada documento unas pocas “huellas”: por ejemplo, en cada tramo de diez frases, la que tenga el código más pequeño según una regla fija. Dos textos que compartan un párrafo largo compartirán, por fuerza, algunas huellas, porque la regla elige la misma frase en ambos. Con las huellas coincidentes en la mano, el profesor comprueba si aparecen en el mismo orden y a distancias parecidas; si es así, ha encontrado la fuente, y solo entonces lee con atención el pasaje sospechoso. Semilla, encadenamiento y extensión: así trabajan los mapeadores modernos.

7.2.1 El paradigma semilla-encadenamiento-extensión

La búsqueda exacta con retroceso funciona bien para lecturas de 36 bases con dos o tres diferencias, las de los primeros secuenciadores Illumina. Con lecturas de 100 a 250 bases, y más aún con lecturas largas de nanoporos, la búsqueda con errores se vuelve inviable y, además, innecesaria: una lectura larga contiene, casi con seguridad, varios tramos exactos suficientemente largos para ser únicos. Los mapeadores actuales combinan tres etapas:

1. **Semillas:** coincidencias exactas entre la lectura y la referencia, halladas con un índice (FM o *hash*).
2. **Encadenamiento:** agrupar las semillas que son *colineales* (mismo orden y distancias compatibles en lectura y referencia), porque proceden probablemente de un mismo origen.
3. **Extensión:** alinear con programación dinámica, restringida a una banda, las bases entre semillas y los extremos de la cadena.

La programación dinámica, cara, solo se ejecuta en un puñado de regiones candidatas por lectura.

BWA-MEM. La familia BWA pasó por tres algoritmos: BWA-*backtrack*, para lecturas cortas (Li & Durbin, 2009); BWA-SW, que alineaba lecturas largas recorriendo el

índice con programación dinámica (Li & Durbin, 2010); y BWA-MEM (Li, 2013), hoy el estándar para lecturas Illumina. BWA-MEM siembra con *coincidencias exactas supermaximales* (SMEM): para cada posición de la lectura, la coincidencia exacta más larga que la cubre y que no está contenida en otra. El índice FM es ideal para esto, porque extender una coincidencia un carácter cuesta una actualización del intervalo. Si una SMEM es muy larga (más de 28 bases por defecto), BWA-MEM vuelve a sembrar con coincidencias más cortas que aparecen más veces, para no perder el origen verdadero cuando la SMEM cae en una repetición. Después encadena semillas cercanas y colineales, descarta cadenas contenidas en otras mucho mejores, y extiende con programación dinámica de huecos afines en banda. Durante la extensión aplica una variante del criterio *X-drop* de BLAST que vimos en el capítulo 3, y decide automáticamente entre alineamiento local y de extremo a extremo.

Bowtie 2. Langmead y Salzberg (2012) siguieron un camino paralelo: el índice FM encuentra semillas sin huecos y la extensión se hace con programación dinámica vectorizada con instrucciones SIMD del procesador. Bowtie 2 sigue siendo muy usado en ChIP-seq y ATAC-seq.

7.2.2 Minimizadores

Para lecturas largas y ruidosas, Li (2018) optó en minimap2 por un índice de *hash*, pero sin guardar todos los k -meros del genoma: solo sus *minimizadores*, una idea de Roberts et al. (2004).

Definición 7.4. $((w, k)$ -minimizador)

Sea h una función que ordena los k -meros (por ejemplo, un *hash* pseudo-aleatorio). Para cada ventana de w k -meros consecutivos de una secuencia, el minimizador de la ventana es el k -mero de valor h mínimo:

$$\mu_j = \arg \min_{j \leq i < j+w} h(s[i..i+k-1]). \quad (7.10)$$

El conjunto de minimizadores de la secuencia es $\{\mu_j\}$ para todas las ventanas j .

DONDE

k	Longitud de cada k -mero.
w	Número de k -meros consecutivos por ventana; la ventana abarca $w + k - 1$ bases.
h	Orden total sobre los k -meros; el orden alfabético es el más simple, un <i>hash</i> aleatorio el más eficaz.
μ_j	Posición del k -mero elegido en la ventana que empieza en j .

La definición garantiza dos propiedades. La primera es la de *cobertura*: toda ventana de $w + k - 1$ bases aporta al menos un minimizador, así que dos secuencias que comparten un tramo exacto de esa longitud comparten al menos un minimizador; es la misma garantía que buscaba el algoritmo de *winnowing* para detectar plagio en documentos (Schleimer et al., 2003). La segunda es la de *escasez*: ventanas vecinas comparten casi todos sus k -meros y suelen elegir el mismo, de modo que el número de minimizadores distintos es mucho menor que el de k -meros.

Teorema 7.3. (*Densidad de los minimizadores*)

Si los k -meros de una secuencia aleatoria reciben valores h independientes y sin empates, la fracción esperada de posiciones elegidas como minimizador es

$$\delta \approx \frac{2}{w + 1}. \quad (7.11)$$

DONDE

δ	Densidad: número de minimizadores dividido por el número de k -meros.
----------	---

Esbozo. Compare dos ventanas consecutivas, j y $j + 1$. Juntas abarcan $w + 1$ k -meros, y por simetría cada uno tiene la misma probabilidad, $1/(w + 1)$, de ser el menor de todos. Si el menor es el primero (que solo pertenece a la ventana j), la ventana $j + 1$ tiene que elegir otro k -mero: aparece un minimizador nuevo. Si el menor es el último (el que acaba de entrar), la ventana $j + 1$ lo elige y también es nuevo. Si está en cualquier posición intermedia, ambas ventanas eligen el mismo y no se añade nada. La probabilidad de estrenar minimizador al avanzar una posición es, por tanto, $2/(w + 1)$ (Schleimer et al., 2003). $\square$

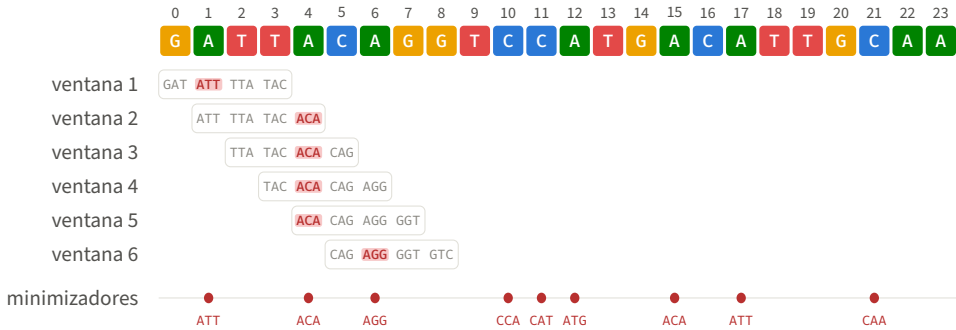


Figura 7.6. Minimizadores ($w = 4$, $k = 3$) de una secuencia de 24 bases, con orden alfabético para que puedan comprobarse a mano. Cada fila muestra una ventana de cuatro 3-meros consecutivos, cada uno escrito bajo su base inicial, con el menor resaltado en rojo. Las ventanas 2 a 5 eligen el mismo ACA: por eso se guardan muchos menos minimizadores que k -meros. La última fila marca los nueve minimizadores de toda la secuencia (de 22 k -meros; densidad 0,41, frente a $2/(w + 1) = 0,40$). Una lectura que contenga cualquier tramo de $w + k - 1 = 6$ bases de esta secuencia compartirá con ella al menos uno de estos minimizadores.

Con $w = 10$ se guarda solo un k -mero de cada 5,5: el índice se reduce a menos de una quinta parte. En una secuencia aleatoria de 200 000 bases con $k = 15$ y $w = 10$, nuestra simulación da una densidad de 0,188 con un *hash* aleatorio (la teoría predice 0,182) y de 0,199 con orden alfabético, que tiende a elegir k -meros ricos en A consecutivos y es menos eficiente.

minimizadores.py

```
import zlib

def minimizadores(s, k=15, w=10):
    """Posiciones de los (w,k)-minimizadores de s."""
    h = [zlib.crc32(s[i:i + k].encode())          # orden pseudoaleatorio
          for i in range(len(s) - k + 1)]
    elegidos = set()
    for j in range(len(h) - w + 1):
        v = h[j:j + w]
        elegidos.add(j + v.index(min(v)))
    return sorted(elegidos)
```

7.2.3 Encadenamiento

Cada minimizador de la lectura que también aparece en el índice produce un *ancla*: un par de posiciones (x, y) en referencia y lectura donde termina una coincidencia exacta de longitud w_i (aquí, $w_i = k$). Las anclas verdaderas se alinean a lo largo de una diagonal; las espurias, procedentes de repeticiones, quedan dispersas. Li (2018) define la puntuación $f(i)$ de la mejor cadena que termina en el ancla i mediante programación dinámica:

$$f(i) = \max \left\{ \max_{j < i} \{f(j) + \alpha(j, i) - \beta(j, i)\}, w_i \right\}, \quad (7.12)$$

con

$$\alpha(j, i) = \min \{ \min \{y_i - y_j, x_i - x_j\}, w_i \}, \quad \beta(j, i) = \gamma((y_i - y_j) - (x_i - x_j)), \quad (7.13)$$

$$\gamma(\ell) = \begin{cases} 0,01 \bar{w} |\ell| + 0,5 \log_2 |\ell| & \ell \neq 0, \\ 0 & \ell = 0. \end{cases} \quad (7.14)$$

DONDE

(x_i, y_i, w_i)	Ancla i : termina en la posición x_i de la referencia y y_i de la lectura, y cubre w_i bases idénticas.
$f(i)$	Puntuación máxima de una cadena de anclas que termina en i .
$\alpha(j, i)$	Bases coincidentes que el ancla i añade tras la j (no cuenta dos veces el solapamiento).
$\beta(j, i)$	Penalización por el desfase entre las distancias en lectura y referencia: un <i>indel</i> de longitud $ \ell $.
$\bar{w}$	Longitud media de las semillas.

β vale infinito si el ancla j no precede a la i en la lectura o si están demasiado lejos. La ecuación (7.12) es una recurrencia de tipo Smith-Waterman (compárela con la del capítulo 3) sobre *anclas* en lugar de bases: el término w_i permite empezar una cadena nueva, igual que el cero permitía empezar un alineamiento local. En la práctica se evalúan solo los 50 predecesores más cercanos, lo que reduce el coste de cuadrático a lineal en el número de anclas sin perder casi nunca la cadena óptima.

Tras encadenar, minimap2 rellena con programación dinámica los huecos entre anclas consecutivas y los extremos, con una penalización de huecos “afín en dos tramos” que permite *indels* largos. El mismo programa, con distintos parámetros preestablecidos (-x sr para lecturas cortas, map-ont y map-hifi para lecturas largas, splice para ARN, asm5 para ensamblajes), cubre casi todos los casos de uso.

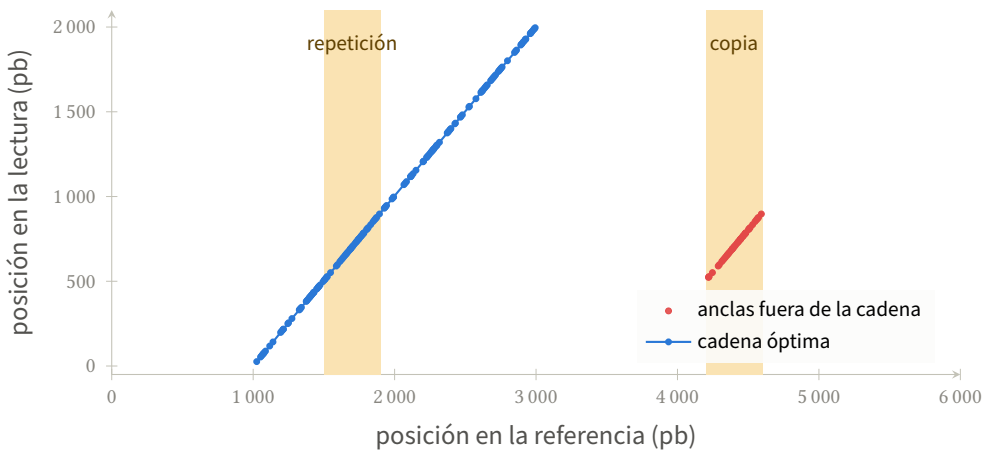


Figura 7.7. Encadenamiento de minimizadores ($k = 15$, $w = 10$) de una lectura simulada de 2000 pb, con un 4 % de errores, contra una referencia de 6000 pb que contiene una repetición de 400 pb (bandas amarillas). La ecuación (7.12) elige 187 de las 236 anclas (azul), alineadas sobre una diagonal casi perfecta: la lectura procede de las posiciones 1000 a 3000. Las anclas rojas del extremo derecho son minimizadores de la repetición que también aparecen en su copia; no pueden encadenarse con el resto sin pagar un *indel* de casi 2700 bases, y quedan descartadas. Si la lectura cayera *entera* dentro de la repetición, las dos cadenas empatarían: es el origen de la ambigüedad que mide la calidad de mapeo.

Índices y mapeo

```
# BWA-MEM: índice FM (una sola vez) y mapeo de lecturas pareadas
bwa index ref.fa
bwa mem -t 8 -R '@RG\tID:m1\tSM:m1' ref.fa r1.fq.gz r2.fq.gz \
> m1.sam
# minimap2: índice de minimizadores al vuelo (o guardado con -d)
minimap2 -ax sr ref.fa r1.fq.gz r2.fq.gz > m1_sr.sam
minimap2 -ax map-ont ref.fa nanoporo.fq.gz > m1_ont.sam
```

7.2.4 Calidad de mapeo

Supongamos que una lectura tiene dos ubicaciones candidatas. ¿Cuán seguros podemos estar de la mejor? Li et al. (2008) introdujeron en MAQ la *calidad de mapeo*, que expresa en escala Phred (la misma del capítulo 6) la probabilidad de que la posición asignada sea errónea.

Definición 7.5. (Calidad de mapeo)

Sean $u_1, \dots, u_K$ las ubicaciones candidatas de una lectura q y $p(q | u)$ la probabilidad de observar q si procede de u . Suponiendo un origen *a priori* uniforme, la probabilidad *a posteriori* de que la mejor ubicación $\hat{u}$ sea la verdadera es

$$\mathbb{P}(\hat{u} | q) = \frac{p(q | \hat{u})}{\sum_{k=1}^K p(q | u_k)}, \quad (7.15)$$

y la calidad de mapeo es

$$\text{MAPQ} = -10 \log_{10} (1 - \mathbb{P}(\hat{u} | q)). \quad (7.16)$$

DONDE

u_k	Posición candidata k (con su hebra).
$p(q u)$	Verosimilitud de la lectura dada la posición; si solo hay sustituciones, aproximadamente el producto de las probabilidades de error $\varepsilon_i = 10^{-Q_i/10}$ de las bases discordantes.
MAPQ	Calidad de mapeo en escala Phred: 20 significa 1 % de probabilidad de posición errónea, 30 significa 0,1 %.

Ejemplo 7.3. (Dos candidatas y un empate)

Una lectura coincide con la posición u_1 salvo por una base de calidad $Q = 35$, y con u_2 salvo por dos bases de calidades 25 y 30. Aproximando $p(q | u)$ por el producto de las probabilidades de error de las discordancias,

$$p(q | u_1) = 10^{-3,5} = 3,16 \times 10^{-4}, \quad p(q | u_2) = 10^{-2,5} \cdot 10^{-3} = 3,16 \times 10^{-6}.$$

La probabilidad de error es $3,16 \times 10^{-6} / (3,16 \times 10^{-4} + 3,16 \times 10^{-6}) = 0,0099$, y $\text{MAPQ} = -10 \log_{10} 0,0099 = 20,0$. Una sola base discordante de diferencia, pero de buena calidad, basta para preferir u_1 con un 99 % de confianza. Si en cambio u_1 y u_2 fueran idénticas, la ecuación (7.16) daría $-10 \log_{10} 0,5 = 3,01$; con tres copias idénticas, 1,76.

En la práctica, los programas no enumeran todas las candidatas: estiman el denominador a partir de las dos mejores y de cuántas subóptimas encontraron. BWA supone que la ubicación verdadera siempre está entre las halladas y comprueba por simulación que sus valores son razonables: de 1 569 108 lecturas simuladas con $\text{MAPQ} = 60$, solo 11 estaban mal colocadas (Li & Durbin, 2009). minimap2 usa una fórmula empírica basada en las puntuaciones de encadenamiento de la cadena primaria (f_1) y de la mejor secundaria (f_2):

$$\text{MAPQ} = 40 \left(1 - \frac{f_2}{f_1} \right) \min \left\{ 1, \frac{a}{10} \right\} \ln f_1, \quad (7.17)$$

donde a es el número de anclas de la cadena primaria (Li, 2018); el valor final se trunca en 60. Con $f_1 = 200$ y 40 anclas, una secundaria con $f_2 = 150$ da un MAPQ de 53, y una casi empatada con $f_2 = 195$ lo hunde hasta 5.

⚠ Lo que el MAPQ no es

- *No es una calidad del alineamiento.* Una lectura puede alinearse perfectamente, sin una sola discordancia, y tener $\text{MAPQ} = 0$ porque encaja igual de bien en dos copias de una repetición.
- *No es comparable entre programas.* BWA, Bowtie 2 y minimap2 calculan y truncan el MAPQ de maneras distintas. Un filtro “ $\text{MAPQ} \geq 20$ ” significa cosas diferentes en cada caso.

- *Depende de la referencia.* Si falta en la referencia la copia verdadera de una repetición, las lecturas se mapean con confianza al lugar equivocado.

7.2.5 Repeticiones y multimapado

Casi la mitad del genoma humano está formada por secuencias repetidas: elementos transponibles, satélites, duplicaciones segmentarias (Treangen & Salzberg, 2012). Una lectura de 150 bases que cae en el interior de una copia reciente de un elemento Alu o de una duplicación segmentaria no tiene un origen identificable: se *multimapea*. Los formatos distinguen tres tipos de registros: el alineamiento *primario*, uno por lectura, elegido al azar entre los empatados; los *secundarios*, que señalan otras ubicaciones plausibles; y los *suplementarios*, que no son alternativas sino *piezas* de una lectura quimérica, cuyo extremo izquierdo se alinea en un lugar y el derecho en otro, como ocurre en una translocación.

Cuánto de este problema es inevitable depende también de la referencia. GRCh38 dejó, pese a su calidad (Schneider et al., 2017), huecos en centrómeros y brazos cortos acrocéntricos. El genoma T2T-CHM13 completó el 8 % restante: 3055 millones de bases sin huecos, con casi 200 millones de bases nuevas, en buena parte satélites y duplicaciones segmentarias recientes (Nurk et al., 2022). Las lecturas que antes no tenían dónde caer, o que caían en una copia paráloga equivocada, ahora tienen un sitio.

7.2.6 Lecturas pareadas y tamaño de inserto

Cuando se secuencian ambos extremos de un fragmento, las dos lecturas no son independientes: deben mapearse en hebras opuestas, orientadas una hacia la otra, a una distancia compatible con el tamaño de los fragmentos de la biblioteca. Esa información resuelve muchas ambigüedades: si una lectura cae en una repetición pero su pareja cae en secuencia única, la pareja “ancla” a la primera. BWA-MEM estima la distribución del tamaño de inserto a partir de los pares mapeados sin ambigüedad y puntúa cada combinación (i, j) de ubicaciones de las dos lecturas con (Li, 2013)

$$S_{ij} = S_i + S_j - \min \{ -a \log_4 P(d_{ij}), U \}, \quad (7.18)$$

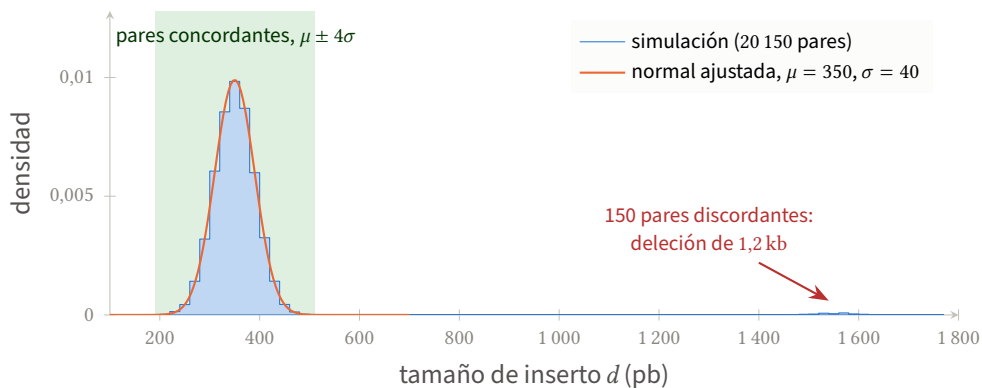


Figura 7.8. Tamaño de inserto de 20 000 pares simulados con $\mu = 350$ y $\sigma = 40$ pb, más 150 pares que atraviesan una deleción de 1200 pb en la muestra. Como la referencia conserva el segmento que la muestra perdió, esos pares parecen 1200 pb más largos. Bajo el modelo normal, la probabilidad de un inserto mayor que $\mu + 4\sigma = 510$ pb es 3×10^{-5} : esperaríamos menos de un par así por azar, y observamos 150 agrupados en la misma región. Los detectores de variantes estructurales combinan esta señal con la cobertura (sección 7.3) y con las lecturas partidas.

DONDE

S_i, S_j	Puntuaciones de Smith-Waterman de cada lectura en su ubicación.
d_{ij}	Distancia entre las dos ubicaciones (infinita si la orientación es incorrecta).
$P(d)$	Probabilidad de observar un inserto mayor que d según una distribución normal ajustada.
a	Puntuación de una coincidencia; convierte probabilidades en unidades de puntuación.
U	Penalización máxima: si el par es muy improbable, es preferible tratar las lecturas como no emparejadas.

El logaritmo en base 4 aparece al interpretar la puntuación de Smith-Waterman como una razón de verosimilitudes, como vimos en el capítulo 3. Los pares que el modelo no puede explicar, por estar demasiado separados, demasiado juntos o mal orientados, son *discordantes*, y son precisamente la señal que buscan los detectores de variantes estructurales (figura 7.8).

Tabla 7.2. Los once campos obligatorios de un registro SAM, ilustrados con el primer extremo de un par ficticio de 20 bases. Las posiciones son base 1.

#	Campo	Ejemplo	Significado
1	QNAME	r017	Nombre de la lectura (igual en ambos extremos del par).
2	FLAG	99	Suma de bits (tabla 7.3): 1 + 2 + 32 + 64.
3	RNAME	chr7	Secuencia de referencia.
4	POS	1021	Posición de la primera base <i>alineada</i> (no recortada).
5	MAPQ	60	Calidad de mapeo (ecuación (7.16)); 255 si no está disponible.
6	CIGAR	3S12M1D5M	Operaciones del alineamiento: 3 recortadas, 12 alineadas, 1 delección, 5 alineadas.
7	RNEXT	=	Referencia de la pareja (=: la misma).
8	PNEXT	1180	Posición de la pareja.
9	TLEN	179	Longitud observada del fragmento, de 1021 a 1199.
10	SEQ	GTA...	Bases de la lectura, en la hebra de la referencia.
11	QUAL	FFF...	Calidades Phred+33 (capítulo 6).

7.2.7 El formato SAM/BAM

Todos los mapeadores escriben sus resultados en el formato SAM (*Sequence Alignment/Map*), creado para el Proyecto 1000 Genomas (Li et al., 2009) y mantenido hoy como especificación formal (SAM/BAM Format Specification Working Group, 2024). Un archivo SAM es texto tabulado: una cabecera con líneas que empiezan por @ (secuencias de referencia @SQ, grupos de lectura @RG, programas @PG) y una línea por alineamiento con once campos obligatorios (tabla 7.2), seguidos de etiquetas opcionales TAG:TIPO:VALOR, como NM:i:2 (distancia de edición) o AS:i: (puntuación del alineamiento).

La cadena CIGAR. Describe el alineamiento como una sucesión de longitudes y operaciones: M (alineada, coincida o no; = y X distinguen ambos casos), I (inserción respecto a la referencia), D (delección), N (salto en la referencia, típico de un intrón en RNA-seq), S (recorte suave: las bases están en SEQ pero no alineadas) y H (recorte duro: las bases ni siquiera se guardan). En el ejemplo, las operaciones que consumen la lectura (S, M, I) suman 3 + 12 + 5 = 20, su longitud, y las que consumen referencia (M, D, N) suman 12 + 1 + 5 = 18: el alineamiento cubre las posiciones 1021 a 1038. Si la pareja se alinea como 20M a partir de 1180, termina en 1199 y el fragmento mide 1199 – 1021 + 1 = 179 pb.

Tabla 7.3. Bits del campo FLAG de SAM (SAM/BAM Format Specification Working Group, 2024). Un filtro frecuente es -F 0x904 (excluir no mapeadas, secundarias y suplementarias), que deja un registro primario por lectura mapeada.

Bit	Dec.	Significado	Bit	Dec.	Significado
0x1	1	lectura pareada	0x40	64	primera del par
0x2	2	par concordante (<i>proper pair</i>)	0x80	128	segunda del par
0x4	4	lectura no mapeada	0x100	256	alineamiento secundario
0x8	8	pareja no mapeada	0x200	512	no pasa filtros de calidad
0x10	16	lectura en hebra reversa	0x400	1024	duplicado de PCR u óptico
0x20	32	pareja en hebra reversa	0x800	2048	alineamiento suplementario

BAM es la versión binaria y comprimida de SAM, en bloques BGZF independientes; eso permite *indexarla* (archivo `.bai` o `.csi`) y saltar directamente a una región sin leer el resto del archivo, siempre que esté ordenada por coordenada. CRAM va más allá y guarda solo las diferencias respecto a la referencia, con archivos aún menores.

7.2.8 samtools: del SAM crudo al BAM analizable

El SAM que sale del mapeador está en el orden de las lecturas del FASTQ. Para analizarlo hacen falta varios pasos, que `samtools` resuelve en un flujo de tuberías (Danecek et al., 2021; Li et al., 2009). El orden importa (figura 7.9):

1. `samtools fixmate -m` rellena, con los dos extremos de cada par a la vista, la información sobre la pareja y añade las etiquetas que necesitará el marcado de duplicados. Requiere los pares contiguos, como los entrega BWA-MEM.
2. `samtools sort` ordena por coordenada.
3. `samtools markdup` marca con el bit 0x400 los *duplicados*: pares cuyos extremos 5' coinciden en posición y orientación, que probablemente son copias por PCR de un mismo fragmento original o duplicados ópticos del secuenciador (recuerde la complejidad de la biblioteca del capítulo 6). `samblaster` lo hace en flujo, sin ordenar, y además extrae las lecturas discordantes y partidas (Faust & Hall, 2014).
4. `samtools index` crea el índice para el acceso aleatorio.

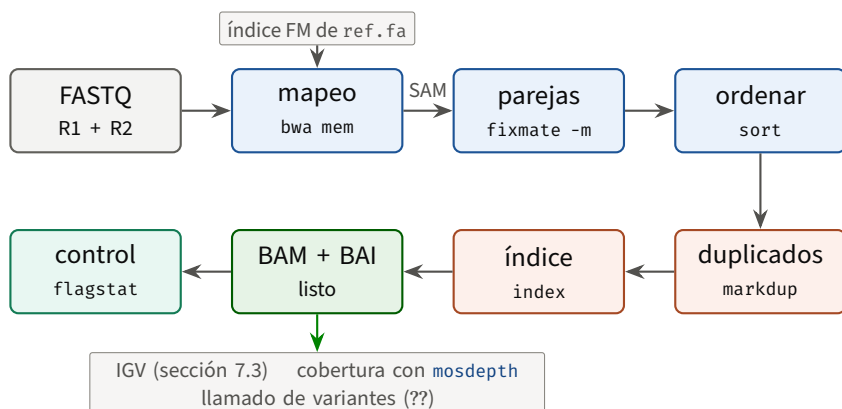


Figura 7.9. Flujo de trabajo típico de mapeo de lecturas Illumina pareadas. En azul, los pasos que se encadenan en una sola tubería sin archivos intermedios; en naranja, los que operan sobre el BAM ordenado. *fixmate* debe ir *antes* de ordenar, cuando los dos extremos de cada par aún están contiguos, y *markdup*, *después*. El BAM indexado es el punto de partida de la visualización, la cobertura y el llamado de variantes.

De SAM a BAM analizable

```

bwa mem -t 8 -R '@RG\tID:m1\tSM:m1' ref.fa r1.fq.gz r2.fq.gz \
| samtools fixmate -m - - \
| samtools sort -@ 4 -o m1.ordenado.bam -
samtools markdup -@ 4 m1.ordenado.bam m1.bam
samtools index m1.bam
samtools flagstat m1.bam # resumen de FLAG
samtools view -c -F 0x904 -q 20 m1.bam # primarias con MAPQ>=20
samtools view m1.bam chr7:55019017-55211628 | head # una región
  
```

⚠ Errores frecuentes en el flujo

- Olvidar el grupo de lectura (-R). Muchos programas posteriores lo exigen para distinguir muestras y carriles.
- Marcar duplicados en datos de amplicones o de RNA-seq sin pensarlo: allí los “duplicados” son la norma biológica o técnica, no un artefacto.

- Mezclar referencias: el BAM solo es válido junto con la misma versión exacta del FASTA (chr7 frente a 7, GRCh37 frente a GRCh38). Consulte las líneas @SQ de la cabecera con `samtools view -H`.

7.3 Visualización de alineamientos y cobertura



Colab · 7.3 · Visualización de alineamientos y cobertura



repositorio del

curso

Una autopista vista desde un helicóptero

Desde un helicóptero se ve el tráfico de una autopista de dos carriles como una densidad de coches más o menos uniforme. Si un tramo tiene un carril cerrado por obras, la densidad cae a la mitad; si el tramo entero está cortado, no pasa ningún coche; si se abre un tercer carril, la densidad sube un cincuenta por ciento. Y hay un tramo en que la autopista corre paralela a otra idéntica: desde el aire es imposible saber por cuál va cada coche, y el contador automático, prudente, no cuenta ninguno. Un perfil de cobertura es esa vista aérea del genoma: las lecturas son los coches, el número de copias son los carriles, y las repeticiones son las autopistas gemelas.

7.3.1 Mirar antes de calcular: IGV

Así como el *dot plot* del capítulo 3 precedía al alineamiento, mirar los alineamientos precede a cualquier análisis serio. El *Integrative Genomics Viewer* (IGV) permite navegar un BAM indexado a cualquier escala, desde un cromosoma entero hasta una base (Robinson et al., 2011). Su diseño carga solo la región visible, aprovechando el índice del BAM, y muestra la información en *pistas* apiladas: la anotación de genes, la cobertura, las lecturas individuales (Thorvaldsdóttir et al., 2013). Las convenciones visuales codifican casi todos los campos del SAM:

- las bases que *coinciden* con la referencia no se dibujan; solo se colorean las discordantes, de modo que el ojo va directo a las diferencias;
- una línea negra horizontal indica una delección, y una marca violeta, una inserción;

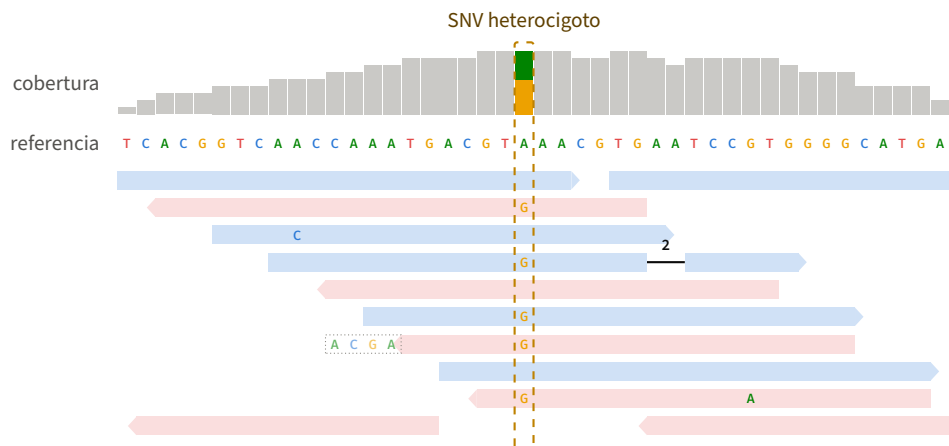


Figura 7.10. Apilamiento (*pileup*) de lecturas sobre 44 bases de referencia, dibujado con las convenciones de IGV. Las lecturas azules se alinean en la hebra directa y las rojas en la reversa (la punta indica el sentido). En la columna recuadrada, 5 de las 9 lecturas llevan una G donde la referencia tiene A: la pista de cobertura se colorea con esa proporción (0,56), la firma de un SNV heterocigoto. Las letras aisladas en otras columnas (una C y una A) son errores de secuenciación: aparecen en una sola lectura. La línea negra marca una delección de 2 pb en una lectura, y las letras tenues dentro del recuadro punteado son bases con recorte suave (4S), presentes en la lectura pero no alineadas.

- el color de fondo de cada lectura puede indicar su hebra, y las lecturas con MAPQ = 0 se dibujan transparentes;
- en la pista de cobertura, las posiciones donde una fracción notable de las lecturas difiere de la referencia se colorean con la proporción de cada base.

La figura 7.10 resume la lectura de un *pileup*. Distinguir una variante de un error es una cuestión de conteo. Si la tasa de error por base es $\varepsilon = 10^{-3}$ (calidad 30), la probabilidad de que 5 o más de 9 lecturas muestren la misma base errónea por azar es del orden de $\binom{9}{5}\varepsilon^5 \approx 1,3 \times 10^{-13}$; la de que una sola lectura muestre un error, $9\varepsilon \approx 0,009$. El siguiente capítulo convertirá este razonamiento en un modelo bayesiano de genotipos.

- *Sesgo de hebra*: una “variante” presente solo en lecturas de una hebra suele ser un artefacto químico o de alineamiento.
- *Variantes al final de las lecturas*: cerca de un *indel*, el alineador puede preferir varias sustituciones en el extremo de la lectura a abrir un hueco, lo que produce falsos SNV agrupados.
- *Pilas de lecturas con MAPQ bajo y muchas discordancias*: huella de una repetición o de una región que falta en la referencia; las lecturas de otro lugar “se refugian” allí.

7.3.2 La cobertura como señal

La definición de cobertura del capítulo 6 era un promedio; ahora la necesitamos posición a posición.

Definición 7.6. (Profundidad por posición)

Sea $\mathcal{A}$ el conjunto de alineamientos que superan los filtros (por ejemplo, primarios, no duplicados y con $\text{MAPQ} \geq 20$), y sea $B(a)$ el conjunto de bloques de referencia $[s, e)$ que el alineamiento a cubre, es decir, sus operaciones M , $=$ o X de la cadena CIGAR. La profundidad en la posición x es

$$c(x) = \sum_{a \in \mathcal{A}} \sum_{[s,e) \in B(a)} \mathbb{1}[s \leq x < e]. \quad (7.19)$$

DONDE

$c(x)$	Número de lecturas filtradas que cubren la posición x con una base alineada.
$\mathcal{A}$	Alineamientos que se cuentan; los filtros cambian mucho el resultado en repeticiones.
$[s, e)$	Bloque semiabierto de posiciones de referencia cubierto sin interrupción; una delección o un intrón parten un alineamiento en varios bloques.

Calcular la ecuación (7.19) de forma directa cuesta $O(\sum_a |a|)$: unos 10^{11} incrementos para un genoma a 30×. Pedersen y Quinlan (2018) observaron en `mosdepth`

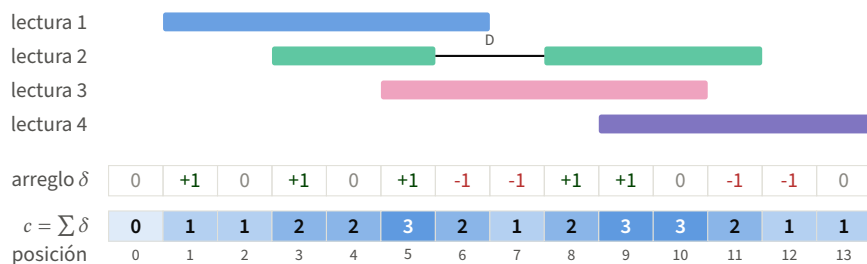


Figura 7.11. El algoritmo de `mosdepth` (ecuación (7.20)) sobre 14 posiciones. Cada bloque alineado suma 1 en su inicio y resta 1 en su final (posición siguiente a la última cubierta). La lectura 2 contiene una deleción (D) y aporta dos bloques. La suma acumulada del arreglo δ reproduce la profundidad de cada posición, coloreada de claro a oscuro. Con N bloques y un cromosoma de longitud G , el coste es $O(N + G)$, sin importar cuántas lecturas se apilen en cada base.

que basta con anotar dónde *empieza* y dónde *termina* cada bloque en un arreglo de diferencias δ del tamaño del cromosoma,

$$\delta(s) += 1, \quad \delta(e) -= 1 \quad \text{para cada bloque } [s, e), \quad c(x) = \sum_{y \leq x} \delta(y), \quad (7.20)$$

y reconstruir la profundidad con una suma acumulada. El coste pasa a ser proporcional al número de bloques más la longitud del cromosoma, y no a la profundidad (figura 7.11). El precio es memoria: un entero de 32 bits por base, cerca de 1 GB para el cromosoma 1 humano. `mosdepth` evita además contar dos veces la región donde los dos extremos de un par se solapan, que no aporta información independiente. Otras herramientas cubren usos distintos: `samtools depth`, `bedtools genomecov` (Quinlan & Hall, 2010) y `bamCoverage` de `deepTools`, que produce pistas normalizadas en formato `bigWig` para comparar muestras en un navegador (Ramírez et al., 2016).

profundidad.py

```
import numpy as np
import pysam

def profundidad(bam, crom, mapq=20):
    """Profundidad por base con el arreglo de diferencias."""
    with pysam.AlignmentFile(bam) as f:
        G = f.get_reference_length(crom)
        d = np.zeros(G + 1, dtype=np.int32)
```



```

for a in f.fetch(crom):
    if (a.is_unmapped or a.is_secondary or a.is_supplementary
        or a.is_duplicate or a.mapping_quality < mapq):
        continue
    for s, e in a.get_blocks():           # bloques M/=X
        d[s] += 1
        d[e] -= 1
return np.cumsum(d[:-1])

```

Cobertura con mosdepth

```

# profundidad media en ventanas de 500 pb, solo MAPQ >= 20
mosdepth -t 4 -n --by 500 -Q 20 m1 m1.bam
zcat m1.regions.bed.gz | head -3      # crom inicio fin media
cat m1.mosdepth.summary.txt          # media por cromosoma

```

7.3.3 Cobertura y número de copias

Si las lecturas se muestrean uniformemente, la profundidad esperada en una región es proporcional a su número de copias. Para un genoma diploide,

$$\mathbb{E}[c(x)] = \bar{c} \frac{\text{CN}(x)}{2} b(x), \quad (7.21)$$

y el estadístico natural para comparar una ventana W con el resto del genoma es el logaritmo del cociente

$$\ell_W = \log_2 \frac{\bar{c}_W}{\text{mediana}_V \bar{c}_V}. \quad (7.22)$$

DONDE

$\bar{c}$	Cobertura media del genoma (p. ej., 30×).
$\text{CN}(x)$	Número de copias de la región en la muestra: 2 normal, 1 deleción heterocigota, o homocigota, 3 duplicación heterocigota.
$b(x)$	Sesgo local multiplicativo (contenido de GC, mapeabilidad), idealmente 1.
$\bar{c}_W$	Profundidad media en la ventana W .
ℓ_W	Razón logarítmica: 0 normal, -1 deleción heterocigota, $\log_2 1,5 = 0,58$ duplicación, $-\infty$ deleción homocigota.

La figura 7.12 muestra estos patrones en un perfil simulado. Obsérvese la trampa de la repetición: con todas las lecturas, su cobertura es normal (unos 35); con el

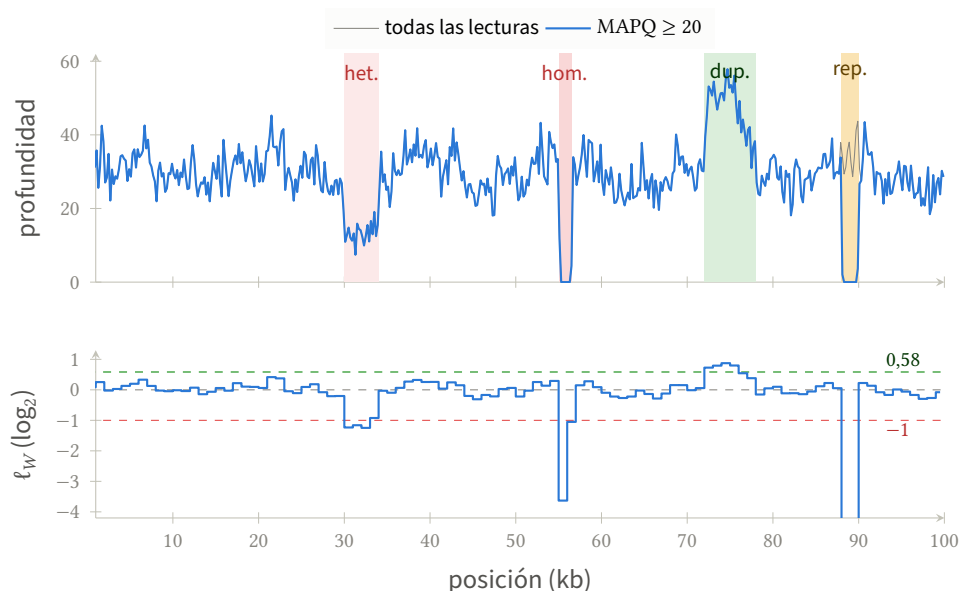


Figura 7.12. Perfil de cobertura simulado de 100 kb a 30× con lecturas de 150 pb. **Arriba:** profundidad media en bloques de 200 pb, con todas las lecturas (gris) y solo con $\text{MAPQ} \geq 20$ (azul). Se aprecian una deleción heterocigota de 4 kb (la profundidad cae a la mitad), una homocigota de 1,5 kb (cae a cero), una duplicación de 6 kb (sube un 50 %) y una repetición de 2 kb cuya otra copia está en el genoma: las lecturas se mapean pero con $\text{MAPQ} = 0$, así que la cobertura filtrada se desploma aunque no falte ADN. El sesgo ondulante imita el efecto del contenido de GC. **Abajo:** razón logarítmica en ventanas de 1 kb (ecuación (7.22)); las ventanas de la deleción heterocigota promedian $-1,14$ y las de la duplicación $0,69$, cerca de los valores teóricos (líneas discontinuas).

filtro de $\text{MAPQ} \geq 20$, cae a 0,6 y se confundiría con una delección homocigota. Un análisis de cobertura siempre debe acompañarse de una máscara de *mapeabilidad* que diga qué regiones del genoma pueden recibir lecturas únicas. CNVnator, uno de los métodos clásicos de profundidad de lectura, corrige el sesgo de GC, segmenta el perfil con un algoritmo de *mean-shift* y alcanza, con cobertura alta, una resolución de los puntos de corte de menos de 200 pb en el 90 % de los casos (Abyzov et al., 2011). Sus autores subrayan que la profundidad es complementaria a las lecturas discordantes y partidas: cada señal detecta variantes que las otras no ven.

7.3.4 ¿Qué delección es detectable?

Hagamos la cuenta que decide si una delección se ve o se pierde en el ruido. En una ventana de W bases con cobertura $\bar{c}$ y lecturas de longitud L , el número de lecturas que empiezan en la ventana es aproximadamente de Poisson con media $\lambda = \bar{c}W/L$, y su desviación típica es $\sqrt{\lambda}$. Una delección heterocigota reduce la media a $\lambda/2$; la diferencia, medida en desviaciones típicas de la hipótesis nula, es

$$z_{\text{het}} = \frac{\lambda - \lambda/2}{\sqrt{\lambda}} = \frac{\sqrt{\lambda}}{2}, \quad z_{\text{hom}} = \frac{\lambda}{\sqrt{\lambda}} = \sqrt{\lambda}. \quad (7.23)$$

Exigiendo $z \geq z^*$ y despejando W ,

$$W_{\min}^{\text{het}} = \frac{4 z^{*2} L}{\bar{c}}, \quad W_{\min}^{\text{hom}} = \frac{z^{*2} L}{\bar{c}}. \quad (7.24)$$

DONDE

λ	Número esperado de lecturas que empiezan en la ventana: $\bar{c}W/L$.
z^*	Umbral de significación en desviaciones típicas; con millones de ventanas en el genoma, $z^* \approx 5$ controla los falsos positivos.
$W_{\min}$	Tamaño mínimo de una delección detectable por profundidad.

Ejemplo 7.4. (Diseño para detectar delecciones)

Con $\bar{c} = 30$, $L = 150$ y $z^* = 5$, la ecuación (7.24) da $W_{\min}^{\text{het}} = 4 \cdot 25 \cdot 150/30 = 500$ pb y $W_{\min}^{\text{hom}} = 125$ pb. A $10\times$, los valores se triplican: 1500 y 375 pb; a $60\times$, se reducen a la mitad: 250 y 62 pb. Una duplicación heterocigota, que cambia la media de λ a $1,5\lambda$, produce la misma diferencia absoluta que una delección heterocigota y requiere, por tanto, ventanas similares. Por debajo de unos

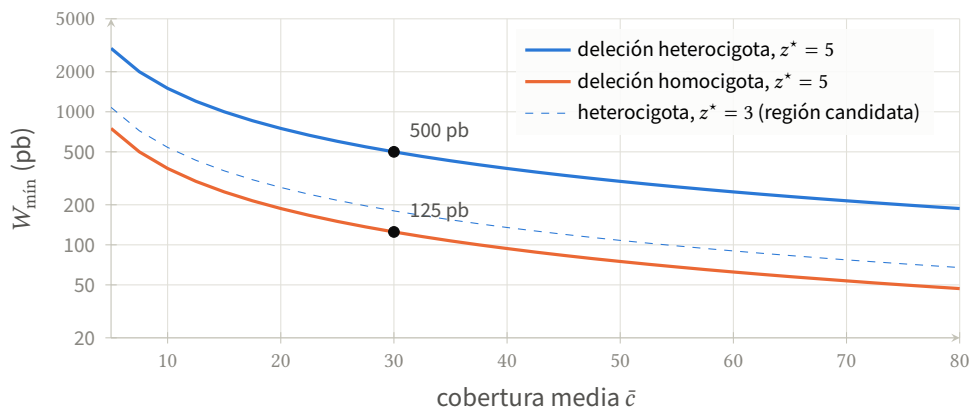


Figura 7.13. Tamaño mínimo de una deleción detectable solo por profundidad, según la ecuación (7.24), para lecturas de 150 pb. Duplicar la cobertura reduce a la mitad el tamaño detectable; pasar de una búsqueda en todo el genoma ($z^* = 5$) a la comprobación de una región candidata ($z^* = 3$) lo reduce casi a un tercio. El modelo es optimista: supone muestreo de Poisson perfecto, y el sesgo residual tras la corrección de GC aumenta la varianza.

cientos de bases, la profundidad no basta y hay que recurrir a lecturas partidas y a pares discordantes (figura 7.8).

El techo de la sobredispersión

Si el recuento de una ventana no es de Poisson sino binomial negativo, con varianza $\lambda + \lambda^2/r$ como en el capítulo 6, la ecuación (7.23) se convierte en

$$z_{\text{het}} = \frac{\lambda/2}{\sqrt{\lambda + \lambda^2/r}} \xrightarrow{\lambda \rightarrow \infty} \frac{\sqrt{r}}{2}.$$

La señal ya no crece indefinidamente con la ventana ni con la cobertura: tiene un techo fijado por la sobredispersión. Con $r = 50$, z_{het} nunca supera 3,5, por mucho que se secuencie. Esta es la razón matemática por la que los métodos de profundidad dedican tanto esfuerzo a corregir los sesgos de GC y de mapeabilidad: reducir la varianza sistemática rinde más que añadir lecturas.

La cobertura es una medida del número de copias, pero solo después de filtrar duplicados, controlar la calidad de mapeo y corregir los sesgos; una caída de cobertura puede ser una delección o una repetición.

En resumen

- Mapear es encontrar el origen de cada lectura y cuantificar la confianza en él. La programación dinámica exhaustiva es inviable a escala genómica; los mapeadores filtran con índices y reservan el alineamiento para unas pocas regiones candidatas.
- El arreglo de sufijos agrupa las apariciones de un patrón en un intervalo. La BWT es la columna de caracteres precedentes de ese arreglo; el *LF-mapping* ($LF(r) = C[L_r] + \text{Occ}(L_r, r)$) la hace reversible y la búsqueda hacia atrás encuentra un patrón con dos consultas por carácter.
- Los puntos de control de Occ y el muestreo del arreglo de sufijos reducen el índice FM del genoma humano a 1–2 GB, a cambio de más pasos de *LF-mapping* por posición localizada.
- BWA-MEM siembra con SMEM sobre el índice FM; minimap2 con minimizadores, cuya densidad es $\approx 2/(w + 1)$. Ambos encadenan anclas colineales y extienden con programación dinámica en banda.
- El MAPQ es la probabilidad, en escala Phred, de que la posición asignada sea errónea; es bajo en repeticiones aunque el alineamiento sea perfecto, y no es comparable entre programas.
- SAM/BAM guarda cada alineamiento con su FLAG, su CIGAR y la información de la pareja; *samtools* ordena, marca duplicados e indexa. Los pares discordantes y las lecturas partidas señalan variantes estructurales.
- La profundidad por base se calcula en tiempo lineal con un arreglo de diferencias; su razón logarítmica revela delecciones $(-1, -\infty)$ y duplicaciones $(0, 58)$, con un tamaño mínimo detectable inversamente proporcional a la cobertura.

Ejercicios

Dificultad:   básica   intermedia   avanzada

- 7.1 ●●● Construya a mano el arreglo de sufijos y la BWT de GATTACA\$. Compruebe con `indice_fm` su resultado y verifique que las letras iguales de L y F conservan su orden relativo.
- 7.2 ●●● Usando la tabla 7.1, ejecute la búsqueda hacia atrás de TAC y de CAT en TACATACAG\$. Dé los intervalos intermedios y las posiciones finales.
- 7.3 ●●● Decodifique los FLAG 83, 147, 163 y 2064. ¿Cuáles pertenecen a pares concordantes? ¿Cuál es un alineamiento suplementario y qué sugiere sobre la lectura?
- 7.4 ●●● Con la ecuación (7.9), calcule la memoria del índice FM para el genoma de trigo ($1,6 \times 10^{10}$ bases) con $d = 128$ y $s = 32$. ¿Cuántos bits necesita cada entero? Discuta qué intervalo s elegiría para que el índice quepa en 32 GB.
- 7.5 ●●● Implemente la reconstrucción de T a partir de L con el *LF-mapping* y compruebe que funciona con una secuencia aleatoria de 10 000 bases.
- 7.6 ●●● Simule secuencias aleatorias y mida la densidad de los minimizadores con orden alfabético y con un *hash* aleatorio para $w \in \{5, 10, 20\}$ y $k = 15$. Compare con la ecuación (7.11). ¿Por qué el orden alfabético es peor en secuencias con muchas A?
- 7.7 ●●● Una lectura tiene tres ubicaciones: una con una discordancia de calidad 30, y dos con dos discordancias de calidades 30 y 30. Calcule el MAPQ con la ecuación (7.16). ¿Cómo cambia si la calidad de la única discordancia de la primera es 10?
- 7.8 ●●● Descargue un BAM público de una muestra humana (por ejemplo, del Proyecto 1000 Genomas), calcule con `mosdepth` la profundidad en ventanas de 1 kb de un cromosoma y busque tramos con $\ell_W < -0,7$. Contraste sus candidatos con las lecturas discordantes vistas en IGV.
- 7.9 ●●● Demuestre la aproximación $2/(w+1)$ de la ecuación (7.11) con detalle, suponiendo que los valores de *hash* son variables continuas independientes. ¿Qué ocurre con la densidad en una región de baja complejidad como **A**T**A**T**A**
TA**T**A**T**?

Referencias del capítulo

- Abyzov, A., Urban, A. E., Snyder, M., & Gerstein, M. (2011). CNVnator: An approach to discover, genotype, and characterize typical and atypical CNVs from family and population genome sequencing. *Genome Research*, 21(6), 974-984. <https://doi.org/10.1101/gr.114876.110>
- Adjeroh, D., Bell, T., & Mukherjee, A. (2008). *The Burrows-Wheeler Transform: Data Compression, Suffix Arrays, and Pattern Matching*. Springer. <https://doi.org/10.1007/978-0-387-78909-5>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), Artículo giab008. <https://doi.org/10.1093/gigascience/giab008>
- Faust, G. G., & Hall, I. M. (2014). SAMBLASTER: fast duplicate marking and structural variant read extraction. *Bioinformatics*, 30(17), 2503-2505. <https://doi.org/10.1093/bioinformatics/btu314>
- Ferragina, P., & Manzini, G. (2005). Indexing compressed text. *Journal of the ACM*, 52(4), 552-581. <https://doi.org/10.1145/1082036.1082039>
- Gusfield, D. (1997). *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511574931>
- Langmead, B., & Salzberg, S. L. (2012). Fast gapped-read alignment with Bowtie 2. *Nature Methods*, 9(4), 357-359. <https://doi.org/10.1038/nmeth.1923>
- Langmead, B., Trapnell, C., Pop, M., & Salzberg, S. L. (2009). Ultrafast and memory-efficient alignment of short DNA sequences to the human genome. *Genome Biology*, 10(3), Artículo R25. <https://doi.org/10.1186/gb-2009-10-3-r25>
- Li, H. (2013). *Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM*. arXiv: 1303.3997 [q-bio.GN]. <https://doi.org/10.48550/arXiv.1303.3997>
- Li, H. (2018). Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics*, 34(18), 3094-3100. <https://doi.org/10.1093/bioinformatics/bty191>
- Li, H., & Durbin, R. (2009). Fast and accurate short read alignment with Burrows-Wheeler transform. *Bioinformatics*, 25(14), 1754-1760. <https://doi.org/10.1093/bioinformatics/btp324>
- Li, H., & Durbin, R. (2010). Fast and accurate long-read alignment with Burrows-Wheeler transform. *Bioinformatics*, 26(5), 589-595. <https://doi.org/10.1093/bioinformatics/btp698>
- Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R., & 1000 Genome Project Data Processing Subgroup. (2009). The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, 25(16), 2078-2079. <https://doi.org/10.1093/bioinformatics/btp352>

- Li, H., & Homer, N. (2010). A survey of sequence alignment algorithms for next-generation sequencing. *Briefings in Bioinformatics*, 11(5), 473-483. <https://doi.org/10.1093/bib/bbq015>
- Li, H., Ruan, J., & Durbin, R. (2008). Mapping short DNA sequencing reads and calling variants using mapping quality scores. *Genome Research*, 18(11), 1851-1858. <https://doi.org/10.1101/gr.078212.108>
- Mäkinen, V., Belazzougui, D., Cunial, F., & Tomescu, A. I. (2015). *Genome-Scale Algorithm Design: Biological Sequence Analysis in the Era of High-Throughput Sequencing*. Cambridge University Press. <https://doi.org/10.1017/CBO9781139940023>
- Manber, U., & Myers, G. (1993). Suffix arrays: A new method for on-line string searches. *SIAM Journal on Computing*, 22(5), 935-948. <https://doi.org/10.1137/0222058>
- Nurk, S., Koren, S., Rhie, A., Rautiainen, M., et al. (2022). The complete sequence of a human genome. *Science*, 376(6588), 44-53. <https://doi.org/10.1126/science.abj6987>
- Pedersen, B. S., & Quinlan, A. R. (2018). Mosdepth: quick coverage calculation for genomes and exomes. *Bioinformatics*, 34(5), 867-868. <https://doi.org/10.1093/bioinformatics/btx699>
- Quinlan, A. R., & Hall, I. M. (2010). BEDTools: a flexible suite of utilities for comparing genomic features. *Bioinformatics*, 26(6), 841-842. <https://doi.org/10.1093/bioinformatics/btq033>
- Ramírez, F., Ryan, D. P., Grüning, B., Bhardwaj, V., Kilpert, F., Richter, A. S., Heyne, S., Dündar, F., & Manke, T. (2016). deepTools2: a next generation web server for deep-sequencing data analysis. *Nucleic Acids Research*, 44(W1), W160-W165. <https://doi.org/10.1093/nar/gkw257>
- Roberts, M., Hayes, W., Hunt, B. R., Mount, S. M., & Yorke, J. A. (2004). Reducing storage requirements for biological sequence comparison. *Bioinformatics*, 20(18), 3363-3369. <https://doi.org/10.1093/bioinformatics/bth408>
- Robinson, J. T., Thorvaldsdóttir, H., Winckler, W., Guttman, M., Lander, E. S., Getz, G., & Mesirov, J. P. (2011). Integrative genomics viewer. *Nature Biotechnology*, 29(1), 24-26. <https://doi.org/10.1038/nbt.1754>
- SAM/BAM Format Specification Working Group. (2024). *Sequence Alignment/Map Format Specification*. GA4GH / hts-specs. <https://samtools.github.io/hts-specs/SAMv1.pdf>
- Schleimer, S., Wilkerson, D. S., & Aiken, A. (2003). Winnowing: Local algorithms for document fingerprinting. *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*, 76-85. <https://doi.org/10.1145/872757.872770>
- Schneider, V. A., Graves-Lindsay, T., Howe, K., Bouk, N., Chen, H.-C., Kitts, P. A., Murphy, T. D., Pruitt, K. D., Thibaud-Nissen, F., Albracht, D., et al. (2017). Evaluation of GRCh38 and de novo haploid genome assemblies demonstrates the enduring quality of the reference assembly. *Genome Research*, 27(5), 849-864. <https://doi.org/10.1101/gr.213611>

- Thorvaldsdóttir, H., Robinson, J. T., & Mesirov, J. P. (2013). Integrative Genomics Viewer (IGV): high-performance genomics data visualization and exploration. *Briefings in Bioinformatics*, 14(2), 178-192. <https://doi.org/10.1093/bib/bbs017>
- Treangen, T. J., & Salzberg, S. L. (2012). Repetitive DNA and next-generation sequencing: computational challenges and solutions. *Nature Reviews Genetics*, 13(1), 36-46. <https://doi.org/10.1038/nrg3117>

PARTE

IV

Genomas en acción

Poblaciones, transcriptomas, epigenomas y comunidades

Un genoma no es un texto estático: varía entre individuos, se expresa de forma distinta en cada célula, se regula químicamente y convive con miles de genomas microbianos. Aquí aprendemos a medir y a modelar esa dinámica.

PARTE



Estructura, sistemas e inteligencia

De la proteína a la red y al modelo

La última parte sube de escala: de la forma tridimensional de las proteínas a las redes que forman, de los modelos de aprendizaje automático a los flujos de trabajo que hacen todo esto reproducible.



Este libro se compuso con Lua^AT_EX y biblatex-apa.
Tipografías: Libertinus Serif, Source Sans Pro y Fira Mono.
Figuras en TikZ y PGFPlots generadas con Python.
Juvenal Yosa, PhD · Copiloto: Claude (Anthropic)
Licencia MIT · 2026